



Alliance with **FPT** Education

UNIVERSITY OF GREENWICH
COMP1786 - Mobile
Application Design and
Development
Coursework

Student name	Lieu Vinh Khoi
ID number (00xxxxxxx)	001391472
Lecturer/Tutor name	Doan Dinh Ho
Student submission date	29 November 2024

Table of Contents

COMP1786 – Mobile Application Design and Development Report	10
1. SECTION 1 - BRIEF STATEMENT OF FEATURES YOU HAVE IMPLEMENTED	10
2. SECTION 2 - REFLECTION	13
2.1 Reflections on Android and Cross-Platform App Development	13
2.2 Usecase	14
2.3 Entity-Relationship Diagram.....	21
2.4 Wireframe	22
3. SECTION 3 - EVALUATION	32
3.1 Admin App Evaluation	32
3.1.1 Human-Computer Interaction (HCI).....	32
3.1.2 Security	35
3.1.3 Adaptability to Screen Sizes	35
3.1.4 Real-World Deployment Considerations	35
3.1.5 Accessibility.....	36
3.2 Customer App Evaluation.....	36
3.2.1 Human-Computer Interaction (HCI).....	36
3.2.2 Security	39
3.2.3 Adaptability to Screen Sizes	40
3.2.4 Real-World Deployment Considerations	40
3.2.5 Accessibility.....	40
4. SECTION 4 - DESIGN	41
4.1 Admin Application:.....	41
4.2 Customer Application:	112
5. SECTION 4 - CODE.....	146
6. SECTION 6 - TESTING.....	250
7. REFERENCE	284

Table of Figures

Figure 1. Use case of customer role	14
Figure 2. Usecase of instructor role	15
Figure 3. General usecase of the Admin role.....	16
Figure 4. Detailed Use Case Diagram for Admin Yoga Class Management Functions.....	17
Figure 5. Detailed Use Case Diagram for Admin Class Instance Management Functions	18
Figure 6. Detailed Use Case Diagram for Admin Booking Management Functions	19
Figure 7. Detailed Use Case Diagram for Admin Account Management Functions.....	20
Figure 8. Detailed Use Case Diagram for Admin Class Type Management Functions	21
Figure 9. Entity-Relationship Diagram.....	21
Figure 10. Welcome page.....	22
Figure 11. Login page.....	22
Figure 12. Signup page.....	23
Figure 13. Homepage of Admin.....	23
Figure 14. Yoga class management.....	24
Figure 15. Add yoga class.....	24
Figure 16. Class instance management.....	25
Figure 17. Add class instance	25
Figure 18. Bookings management.....	26
Figure 19. Booking details	26
Figure 20. Account management	27
Figure 21. Add a new account.....	27
Figure 22. Class type management	28
Figure 23. Homepage of Customer.....	28
Figure 24. Yoga class list with search by day of week.....	29
Figure 25. Yoga class list with search by time of day	29
Figure 26. Yoga class details.....	30
Figure 27. Book yoga class.....	30
Figure 28. Shopping cart	31

Figure 29. My Bookings	31
Figure 30. Response on successful yoga class addition.....	32
Figure 31. Labels are clear and easy to understand	32
Figure 32. Confirm yoga class deletion	33
Figure 33. Similar color scheme across pages.....	33
Figure 34. Notice of mandatory request to add class price	33
Figure 35. Search bar for yoga classes.....	33
Figure 36. Combine search and filter for yoga classes.....	34
Figure 37. Simple admin homepage interface, easy to navigate.....	34
Figure 38. Detailed and clear error message	35
Figure 39. Responses after an event is completed	36
Figure 40. Yoga class details are displayed clearly	37
Figure 41. Delete class instances in the cart before creating a booking.....	37
Figure 42. Consistent interface in terms of color and elements	37
Figure 43. Notice of booking cannot be canceled.....	38
Figure 44. Latest Bookings in Homepage	38
Figure 45. Search by day of week or date.....	38
Figure 46. Simple interface, focusing on suggesting customers shopping yoga classes.....	39
Figure 47. Display account-related errors clearly and in detail	39
Figure 48. Login Interface.....	41
Figure 49. Sign Up Interface.....	42
Figure 50. Incomplete fields when logging in.....	43
Figure 51. Incorrect authentication information	44
Figure 52. Notification for incomplete registration fields.....	45
Figure 53. Notification for duplicate email in registration.	46
Figure 54. Notification for password length requirement in registration.....	47
Figure 55. Notification for password mismatch in registration.....	48
Figure 56. Home page of the admin app.	49
Figure 57. Settings menu on the admin home page.	50

Figure 58. Yoga Classes Management Page.	51
Figure 59. Options Menu in Yoga Classes Management.	52
Figure 60. Yoga class details.	53
Figure 61. Add Yoga Class Interface	54
Figure 62. Notification for Missing Class Capacity in Add Class Interface.	55
Figure 63. Notification for Missing Class Price in Add Class Interface.	56
Figure 64. Confirm yoga class information before saving.	57
Figure 65. Confirmation message for successful class creation.	58
Figure 66. Update class interface in the admin app.	59
Figure 67. Capacity requirement notification in the Update Class interface.	60
Figure 68. Price requirement notification in the Update Class interface.	61
Figure 69. Successful class update notification on the Class Management screen.	62
Figure 70. Confirmation dialog for deleting a class in the class management screen.	63
Figure 71. Search functionality for filtering classes by type.	64
Figure 72. Search functionality for filtering classes by day of the week.	65
Figure 73. Filtering Yoga Classes by Day of the Week.	66
Figure 74. Class Instance Management Screen.	67
Figure 75. Options Menu for Class Instance Management.	68
Figure 76. Add Class Instance Interface.	69
Figure 77. Notification for Required Fields in Add Class Instance.	70
Figure 78. Notification for Incorrect Day Selection in Add Class Instance.	71
Figure 79. Notification for Instructor Unavailability in Add Class Instance.	72
Figure 80. Update Class Instance Interface.	73
Figure 81. Error Handling in Update Class Instance Interface.	74
Figure 82. Date Validation Error in Update Class Instance Interface.	75
Figure 83. Instructor Availability Conflict in Update Class Instance Interface.	76
Figure 84. Options and confirmation dialog for deleting a class instance.	77
Figure 85. Comprehensive search function in Class Instances management.	78
Figure 86. Class instances filtered by selected day of the week for quick access.	79

Figure 87. Account Management interface for viewing and managing user accounts.....	80
Figure 88. Options for updating or deleting accounts in Account Management.....	81
Figure 89. Add New Account interface with required fields notification.	82
Figure 90. Add New Account interface with duplicate email notification.	83
Figure 91. Add New Account interface with password length requirement notification.	84
Figure 92. Add New Account interface with password mismatch notification.	85
Figure 93. Update Account interface without password field.	86
Figure 94. Update Account interface with disabled role field.	87
Figure 95. Update Account interface with empty required fields warning.....	88
Figure 96. Duplicate email warning in Update Account.....	89
Figure 97. Successful account update notification.	90
Figure 98. Delete Account Confirmation Dialog	91
Figure 99. Search Account Functionality.....	92
Figure 100. Role-based Filtering in Account Management.....	93
Figure 101. Booking Management Interface.....	94
Figure 102. View Booking Detail Screen.....	95
Figure 103. Booking Confirmation Prompt.....	96
Figure 104. Booking Rejection Prompt.....	97
Figure 105. Booking Details After Status Change	98
Figure 106. Search Booking Functionality.....	99
Figure 107. Filter Booking by Status.....	100
Figure 108. Class Type Management.....	101
Figure 109. Class Type Management Options	102
Figure 110. Add Class Type Dialog.....	103
Figure 111. Add Class Type Validation.....	104
Figure 112. Edit Class Type.....	105
Figure 113. Search Class Type Functionality.....	106
Figure 114. Home Screen Options and Data Reset Confirmation.....	107
Figure 115. Synchronizing Data from Firebase to SQLite.....	108

Figure 116. Update Yoga Class - Limited Field Update.....	109
Figure 117. Update Class Instance - Date Reset on Yoga Class Change	110
Figure 118. Teacher Page.....	111
Figure 119. Customer Login Interface.....	112
Figure 120. Login Error Message.....	113
Figure 121. Successful Login Notification	114
Figure 122. Password Reset Error Notification.....	115
Figure 123. OTP Verification Interface.....	116
Figure 124. Reset Password Dialog and Error Message.....	117
Figure 125. Reset Password Confirmation and Error Notification.....	118
Figure 126. Password Reset Success Notification	119
Figure 127. Account Registration Interface	120
Figure 128. Account Registration Error Notification	121
Figure 129. Account Registration Duplicate Email Error	122
Figure 130. Account Registration Phone Number Length Error	123
Figure 131. Registration Error Notification - Password Validation.....	124
Figure 132. OTP Verification During Sign-Up.....	125
Figure 133. Successful Registration Notification	126
Figure 134. Home Page Interface of Universal Yoga.....	127
Figure 135. Sidebar Navigation Menu of Universal Yoga	128
Figure 136. Yoga Class List Interface	129
Figure 137. Search Yoga Classes by Day of the Week.....	130
Figure 138. Search Yoga Class by Time of Day.....	131
Figure 139. Yoga Class Details	132
Figure 140. Book Yoga Class.....	133
Figure 141. Date-based class instance search feature.	134
Figure 142. Book Yoga Class - Course Selection	135
Figure 143. Book Yoga Class - Successful Addition to Cart.....	136
Figure 144. Book Yoga Class - Disabled Selection for Existing Class Instances.....	137

Figure 145. Shopping Cart - Selected Class Instances	138
Figure 146. Shopping Cart - Confirm Delete	139
Figure 147. Booking Confirmation Success.....	140
Figure 148. My Bookings - Pending Status	141
Figure 149. My Bookings - Page Reload Confirmation	142
Figure 150. Unable to Cancel Booking - Status Update Required	143
Figure 151. Display of Various Booking Statuses with Auto-Refresh Option.....	144
Figure 152. Pending Booking Details with Grouped Yoga Class Instances	145

Table of Tables

Table 1. Student Information	10
Table 2. Brief statement of features you have implemented	10
Table 3. Test case for admin application	250
Table 4. Test case for customer application.....	274

COMP1786 – Mobile Application Design and Development Report

Table 1. Student Information

Your name	Lieu Vinh Khoi	Your Student ID	001391472
-----------	----------------	-----------------	-----------

1. SECTION 1 - BRIEF STATEMENT OF FEATURES YOU HAVE IMPLEMENTED

Table 2. Brief statement of features you have implemented

Feature	Status	Your Comments
Functionality A	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I have fully implemented all the requirements for functionality A. In my application, users can enter the full details of a yoga class, including required fields like the day of the week, time, capacity, duration, price, and class type. I have also included an optional field for class descriptions, as specified. Additionally, a field like status information has been added to enhance functionality. Validation checks ensure all required fields are completed; otherwise, users receive an error prompt. Upon completion, the app displays these details for user confirmation, allowing them to review and modify entries before finalizing. This functionality has been thoroughly tested, and all specified features are working as expected.
Functionality B	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I have fully implemented the features to store, view, edit, and delete yoga class details in an SQLite database. The `ClassListActivity` displays all saved classes, and allows users to filter by type, and search by fields. Each class can be edited or deleted with confirmation prompts for safe management. Additionally, I included database reset functionality, meeting all requirements specified in the task.
Functionality C	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐	I have successfully implemented all required features for managing class instances. The `EnterClassInstanceDetailsActivity` allows users to add,

	☐ Not implemented ☐	edit, and delete instances for specific yoga courses. I ensured that the instance date aligned with the yoga class's scheduled day. Additionally, users can assign an instructor and provide optional comments. All class instances are stored in SQLite, and users can select a class instance to update or delete it in `ClassInstanceListActivity`. These implementations fulfill the requirements outlined in the task.
Functionality D	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I implemented a search feature that allows users to find classes by teacher name, supporting partial input to display all matching classes. Additionally, I added advanced options to search by `date` or `day of the week`, with a filter that enables users to refine results easily. Users can also select a class from the search results to view full details, fully meeting the requirements for this functionality.
Functionality E	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I implemented a comprehensive data synchronization feature that allows users to upload all details of yoga classes to Firebase, my cloud-based database. My approach ensures that the app checks for network availability and uploads data only when Wi-Fi is connected. If Wi-Fi is unavailable, the app stores the data locally in SQLite and syncs it automatically when Wi-Fi becomes available or at periodic intervals. Additionally, if any data is updated locally, these changes are marked with a status field and subsequently synchronized with Firebase upon reconnection. This setup ensures data consistency between the local and cloud databases, fully meeting the requirements for this functionality.
Functionality F	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I extended the core requirements by adding several features that improve user management and accessibility in the admin app. I implemented user authentication to secure access to the admin section, ensuring only authorized users can manage class and user information. Additionally, I introduced role-based access, where each user is assigned a role (e.g., admin,

		customer, or teacher), allowing specific views and permissions based on their role for each app. Teachers, for instance, can view only the class instances assigned to them. To enhance data handling, I added filter spinners on various pages to streamline data retrieval, making the interface more intuitive and user-friendly.
Functionality G	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I developed a hybrid app using .NET MAUI specifically for customers interested in booking yoga classes. This app is fully integrated with Firebase, providing real-time access to yoga classes and instances created by the Android Admin app. Customers can register, log in, browse available classes, view detailed information, and select specific class instances to add to their cart for booking. Additionally, I implemented a search function, allowing users to filter classes by day of the week or time of day, which enhances the user experience by helping them find sessions that best fit their schedules. This approach fulfills all requirements outlined for the hybrid customer app.
Functionality H	Fully completed ☒ Partially completed ☐ Having bugs/Not working ☐ Not implemented ☐	I implemented the booking feature in the hybrid app, allowing users to add class instances to a shopping cart and then submit it to Firebase with their ID for record-keeping. The booking model captures key details, including a list of selected class instances, total price, and user information, ensuring that each booking is associated with the correct user. Additionally, users can view all their booked classes in one place, streamlining their experience by providing access to all booked in the app. This setup meets all outlined requirements for the customer booking functionality.

Link to recorded video (if you record your application before submitting the report)

The recorded video will demonstrate the implemented product. It is roughly 10 minutes.

- Link Google Drive: https://drive.google.com/file/d/1Rgiv71_cTv7nlPP5l1vvIfA1TrJbJQ1R/view?usp=sharing
- Backup Link: <https://youtu.be/rOmptI2PjK4>

2. SECTION 2 - REFLECTION

2.1 Reflections on Android and Cross-Platform App Development

Working on the Universal Yoga project, where I developed both an Android admin app and a cross-platform customer app, was an in-depth journey filled with valuable lessons in app architecture, user security, data synchronization, and UI design.

I started with the admin app, prioritizing secure user authentication by implementing SHA-256 password hashing with salt to keep data protected. Setting up a local SQLite database was a must for offline functionality, and syncing it with Firebase whenever Wi-Fi became available became both a challenge and a deep learning experience. I researched different techniques for efficient data sync, ultimately setting up periodic and connectivity-based synchronization to keep the data updated automatically.

A significant milestone was syncing data from Firebase to SQLite, particularly on the first app launch, which gave me insight into Android database handling. With the data flow solidified, I built out essential features for managing yoga classes, bookings, and user accounts. Creating a functional, user-friendly UI was challenging but rewarding, requiring attention to usability and exhaustive testing, especially around critical actions like adding, updating, or deleting items.

Moving on to the customer app, which I built in .NET MAUI on Visual Studio 2022, I could rely on the sync structure I'd already set up. This app skipped the local database, syncing directly with Firebase for real-time updates and simplifying data handling. My focus here was to create a smooth experience, allowing users to browse and book classes effortlessly. Designing an engaging UI was a challenge—features like a “last booking list” and highlights for new classes made the app intuitive and enjoyable to use.

This project was a major learning experience, giving me hands-on skills with Firebase integration, secure data management, structured error handling, and UI design. Each obstacle pushed me to think creatively and build a stronger app. Looking back, I'm grateful for the challenges; they ultimately made the app better and deepened my understanding, giving me skills I'll carry forward into future projects.

2.2 Usecase

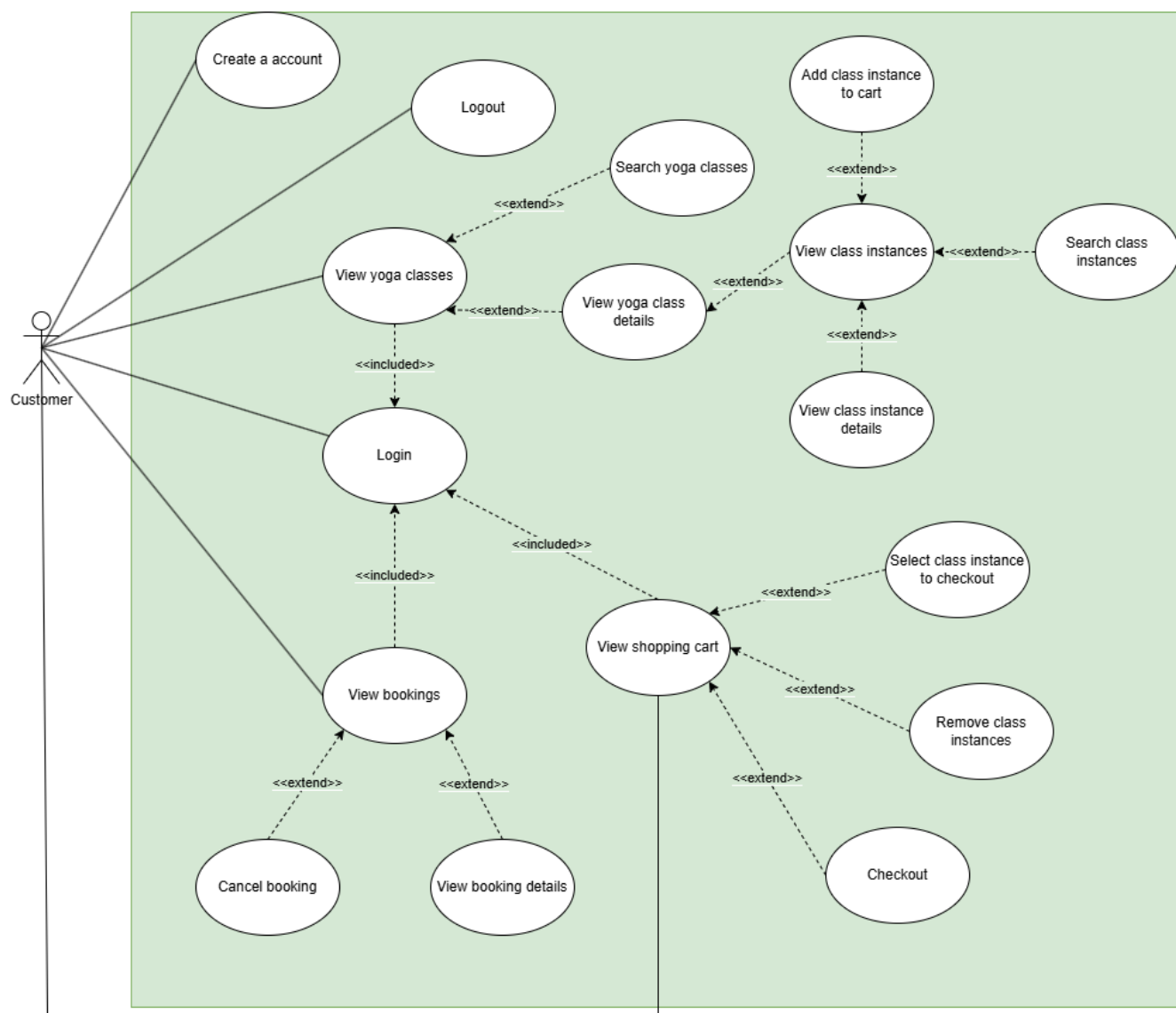


Figure 1. Use case of customer role

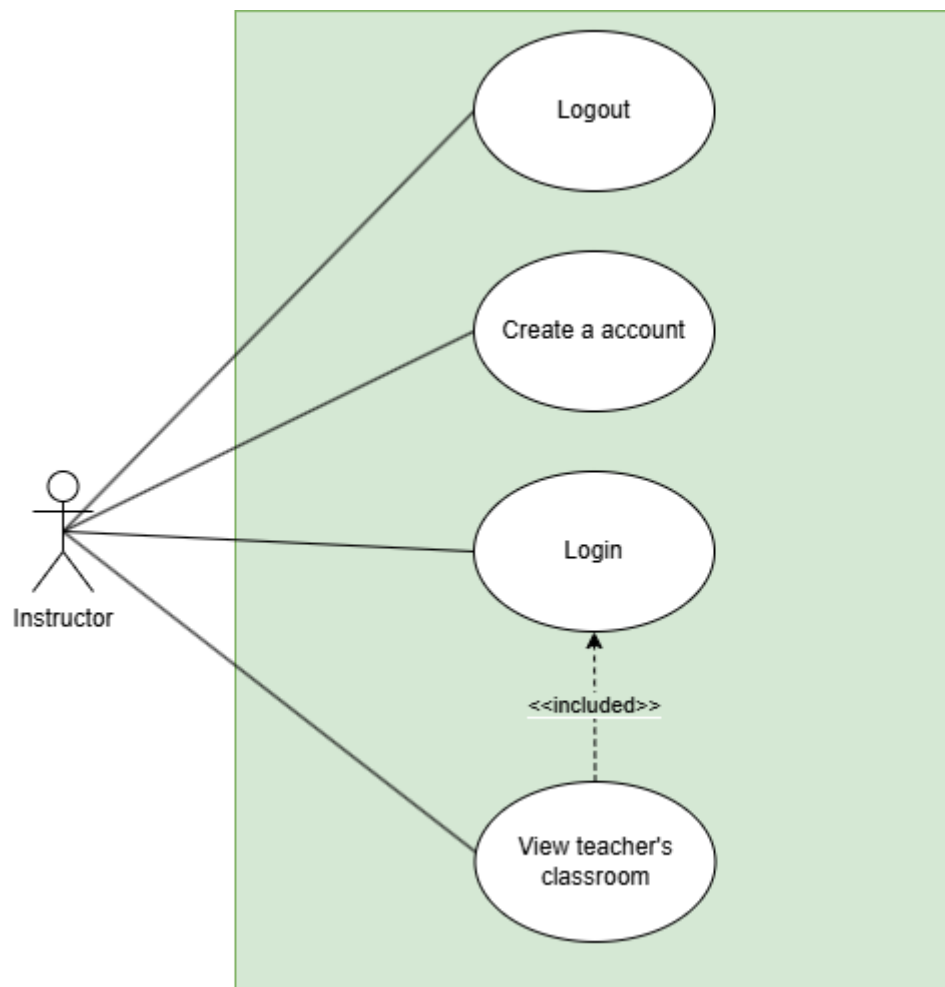


Figure 2. Usecase of instructor role



Figure 3. General usecase of the Admin role

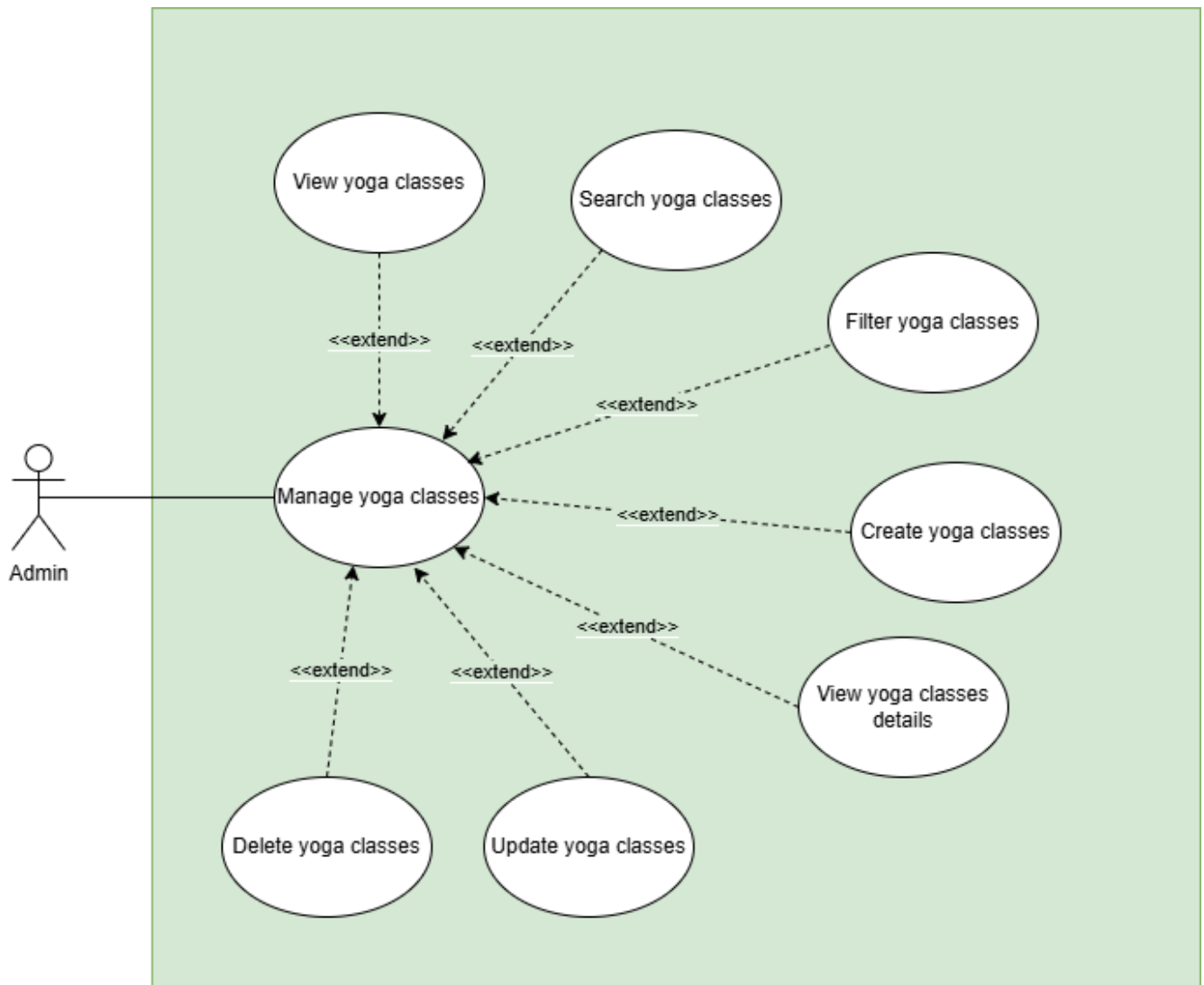


Figure 4. Detailed Use Case Diagram for Admin Yoga Class Management Functions

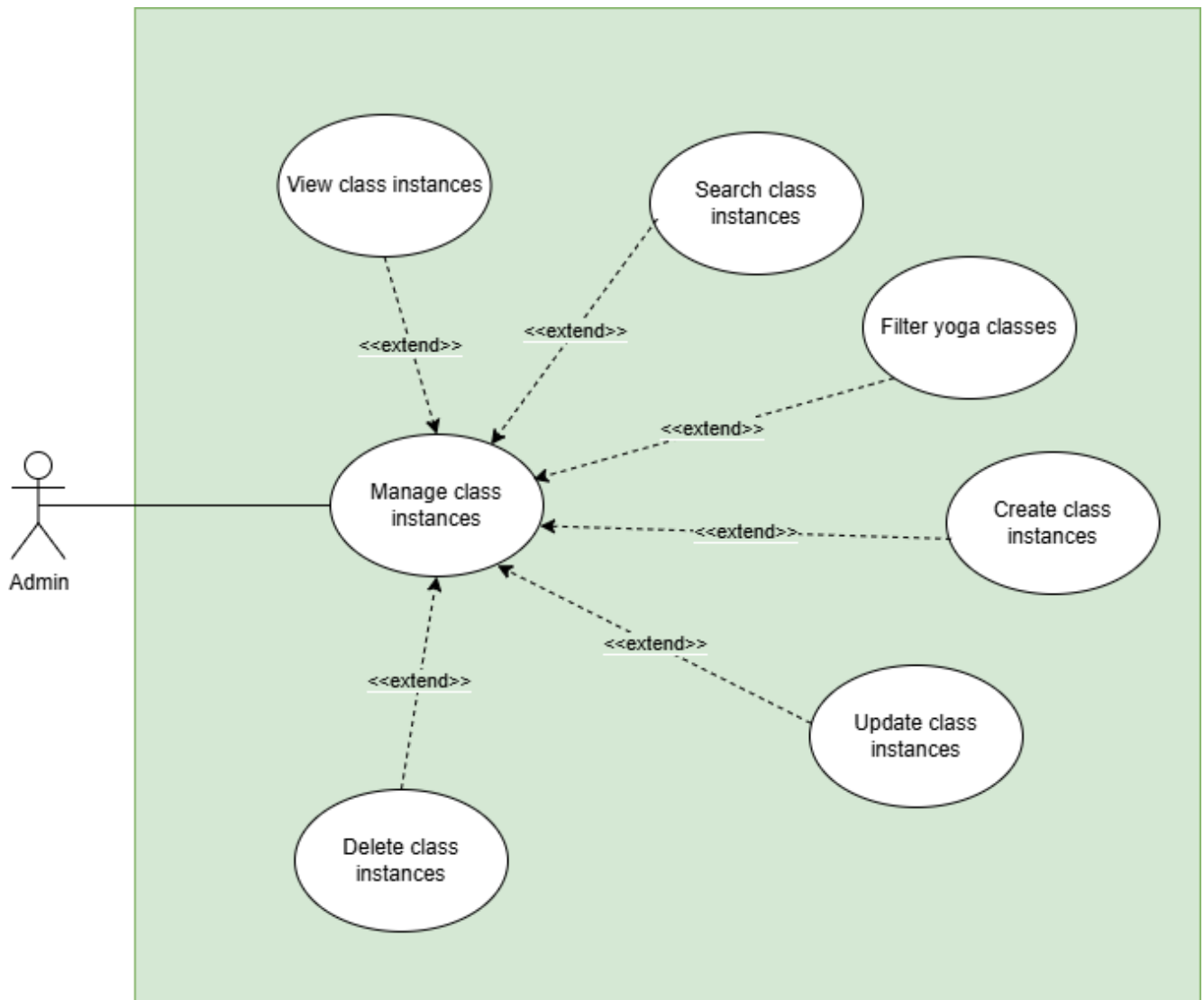


Figure 5. Detailed Use Case Diagram for Admin Class Instance Management Functions

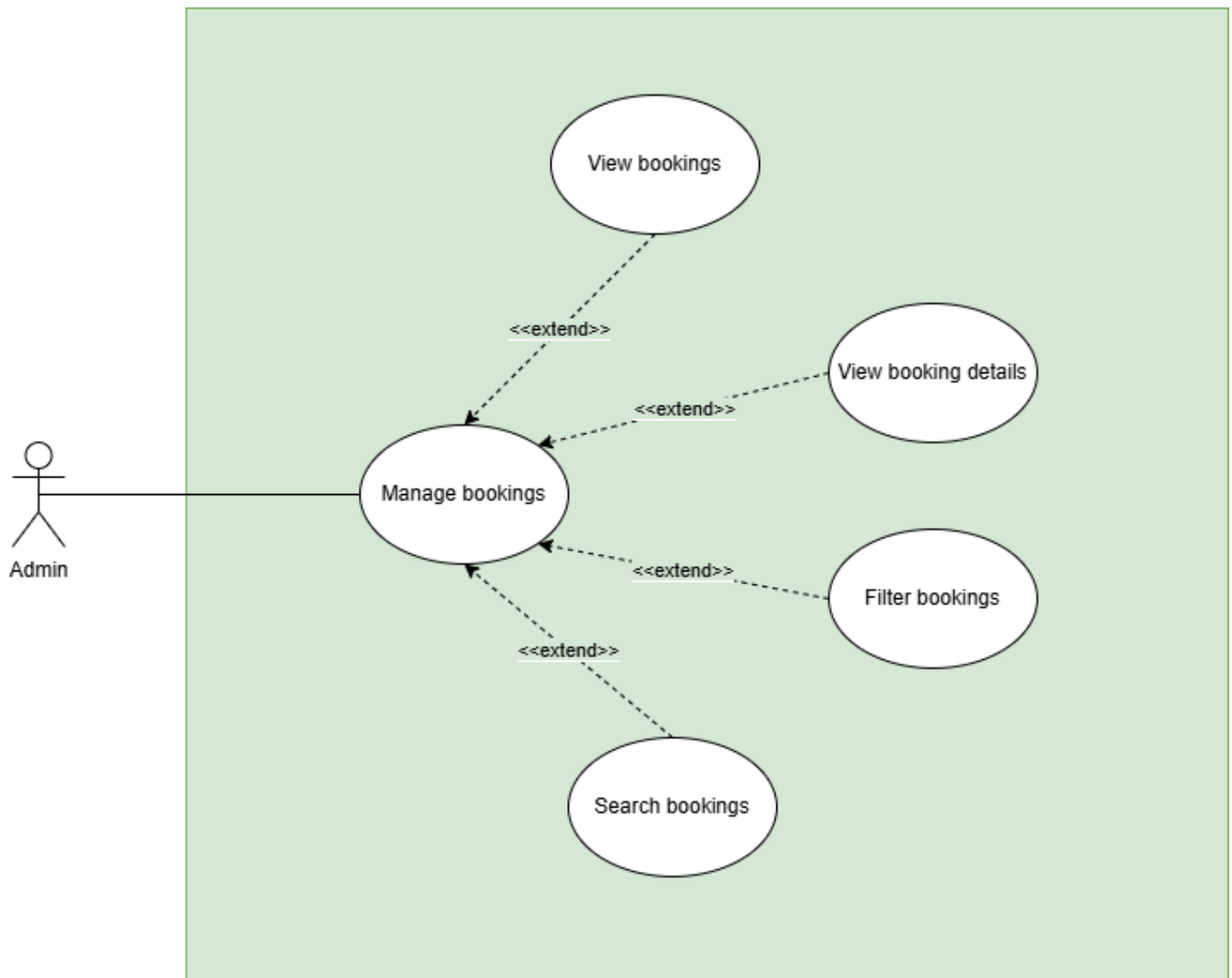


Figure 6. Detailed Use Case Diagram for Admin Booking Management Functions

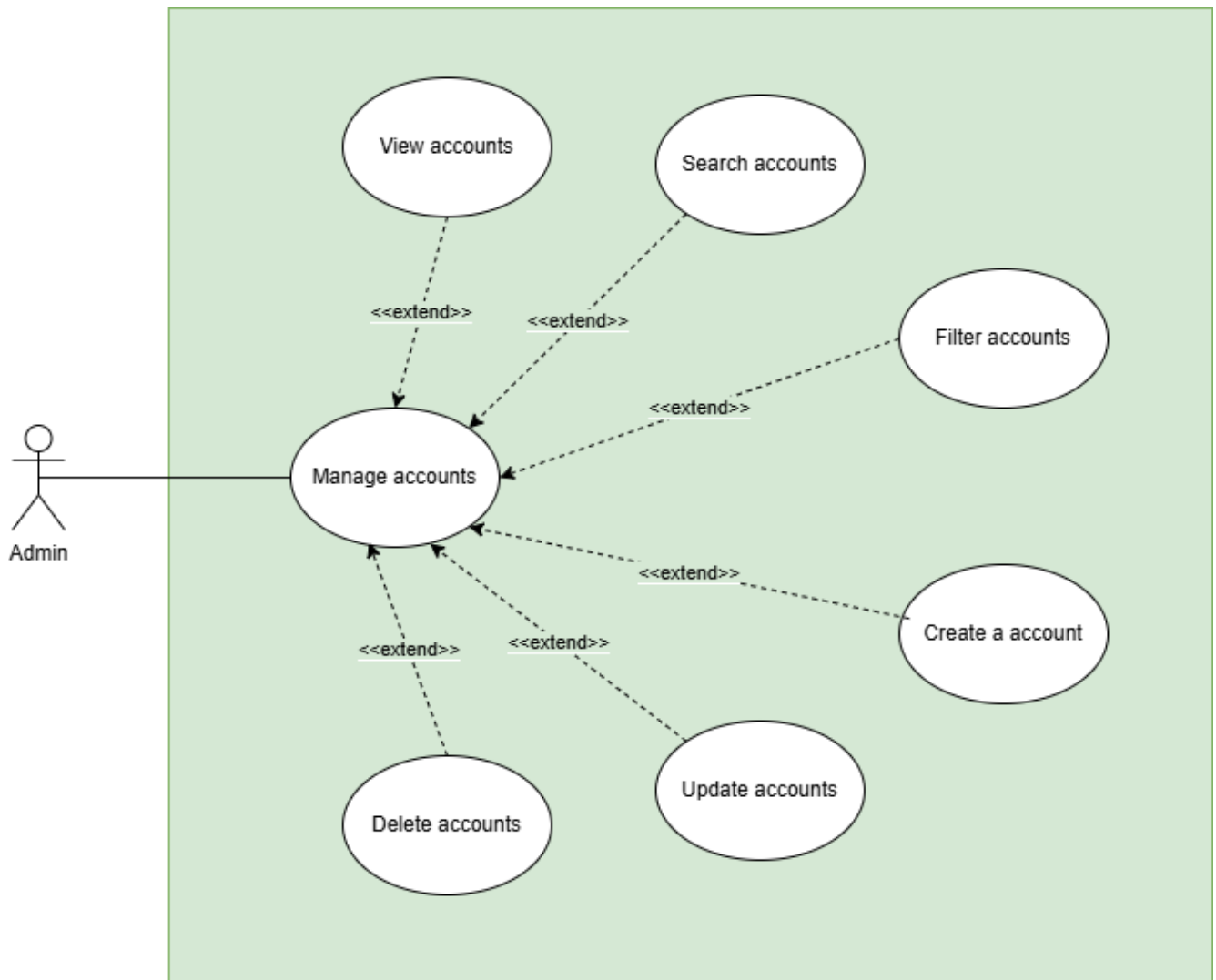


Figure 7. Detailed Use Case Diagram for Admin Account Management Functions

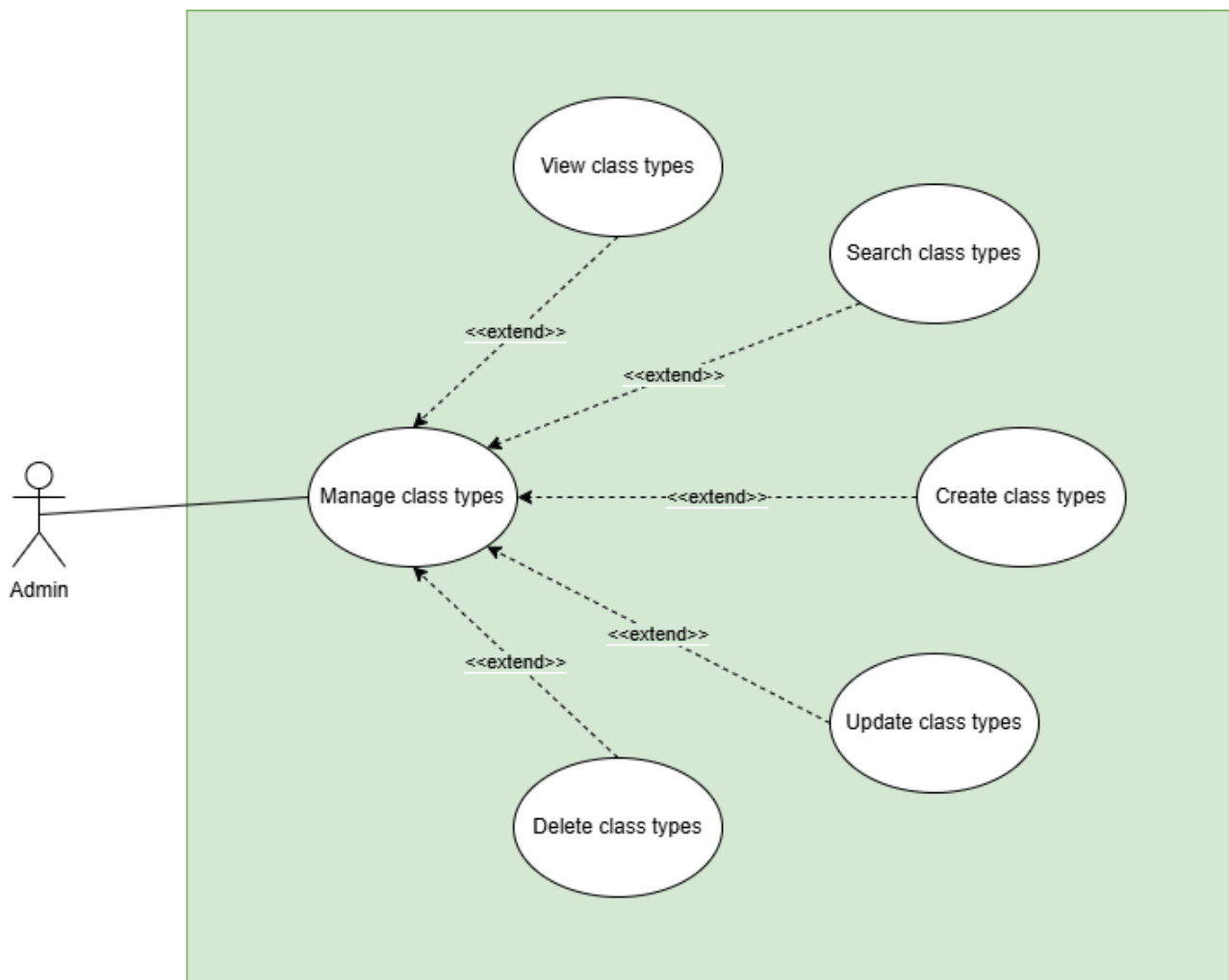


Figure 8. Detailed Use Case Diagram for Admin Class Type Management Functions

2.3 Entity-Relationship Diagram

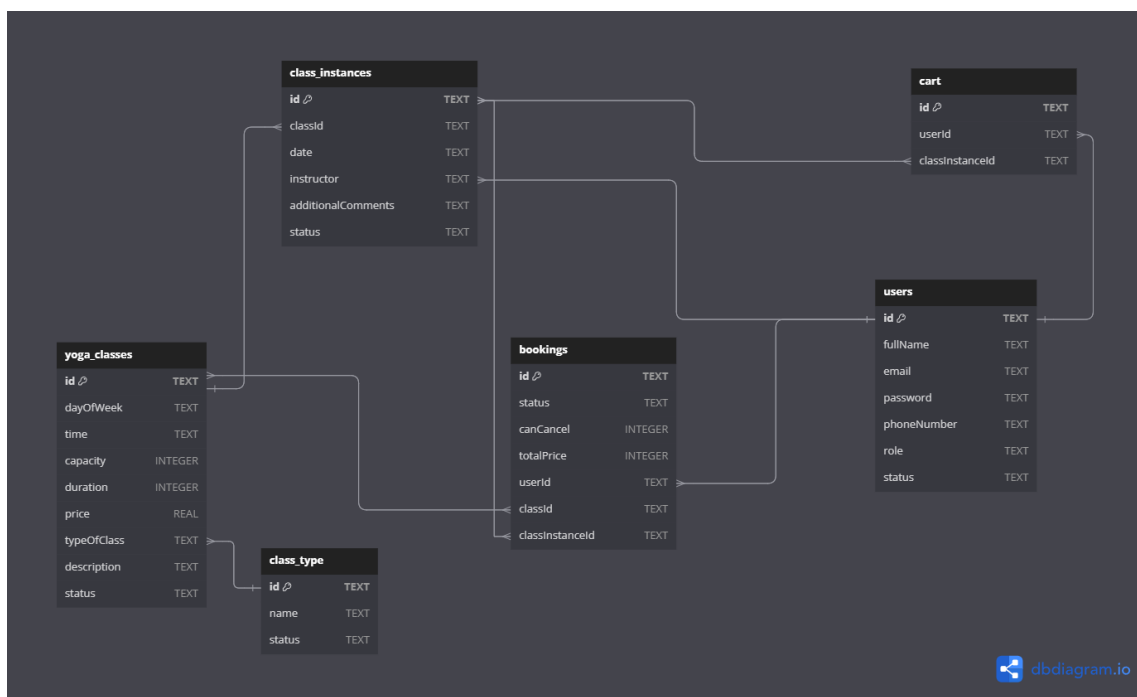


Figure 9. Entity-Relationship Diagram

2.4 Wireframe

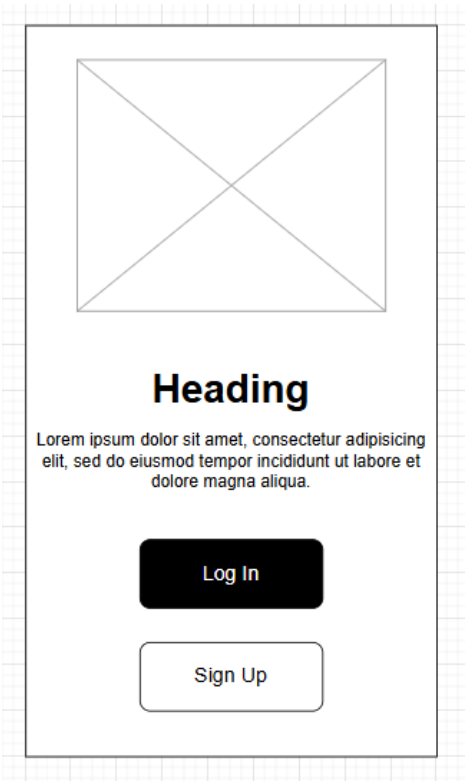
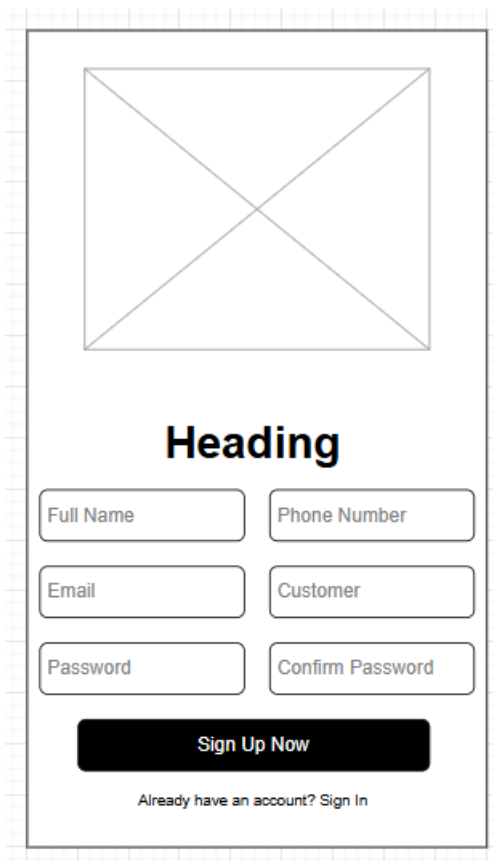


Figure 10. Welcome page

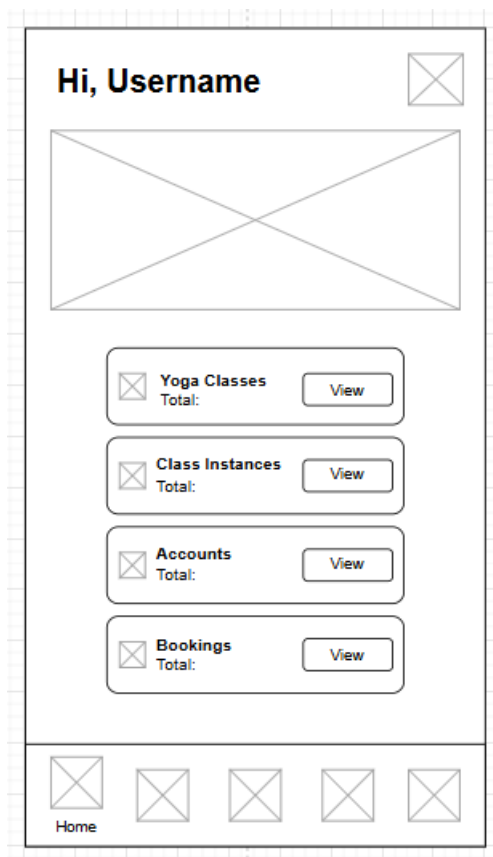


Figure 11. Login page



A wireframe of a signup page. At the top is a large rectangular placeholder with an 'X' inside. Below it is a bold heading 'Heading'. Under the heading are six input fields arranged in two columns: 'Full Name', 'Phone Number', 'Email', 'Customer', 'Password', and 'Confirm Password'. Below these fields is a black button with the text 'Sign Up Now'. At the bottom, there is a link that says 'Already have an account? Sign In'.

Figure 12. Signup page



A wireframe of an admin homepage. At the top left is a greeting 'Hi, Username' followed by a small square placeholder with an 'X'. Below this is a large rectangular placeholder with an 'X'. The main content area contains four summary cards, each with a checkbox icon, a title, a 'Total:' label, and a 'View' button. The cards are: 'Yoga Classes', 'Class Instances', 'Accounts', and 'Bookings'. At the bottom is a navigation bar with five square icons; the first one is labeled 'Home'.

Figure 13. Homepage of Admin

☐
Class

+ Add Class

Search for...

All ☐

☐ Type of class
 ☐ Price

☐ Day of week
 ☐ Time
 ☐ Duration

☐ Description
 ☐ Status
 ☐

☐ Type of class
 ☐ Price

☐ Day of week
 ☐ Time
 ☐ Duration

☐ Description
 ☐ Status
 ☐

☐ Type of class
 ☐ Price

☐ Day of week
 ☐ Time
 ☐ Duration

☐ Description
 ☐ Status
 ☐

☐
☐
☐
☐
☐

Class

Figure 14. Yoga class management

☐
Add class

Wifi:

Day of the Week
 Monday ☐

Time

00

45

Capacity
 Capacity

Duration (minutes)

45

Price
 Price (\$)

Type of Class
 Flow Yoga ☐

Description (Optional)
 Description (Optional)

Save

Figure 15. Add yoga class

Class Instances

+ Add Instance

Date

Instructor

Type of class - Day of week - Time

Status

Description

Date

Instructor

Type of class - Day of week - Time

Status

Description

Date

Instructor

Type of class - Day of week - Time

Status

Description

Class Instance

Figure 16. Class instance management

Class Instance

Wifi:

Select Yoga Class

Flow Yoga - Monday - (10:00 - 11:00)

Date

Select Date

Instructor

Carla Smith

Additional Comments

Enter Additional Comments

Save Class Instance

Figure 17. Add class instance

☐

Bookings

All ☐

☐ Full name
 ☐

☐ Total Price
 ☐ Status

☐ Yoga Classes
 ☐ Class

☐ Full name
 ☐

☐ Total Price
 ☐ Status

☐ Yoga Classes
 ☐ Class

☐ Full name
 ☐

☐ Total Price
 ☐ Status

☐ Yoga Classes
 ☐ Class

☐

☐

☐

☐ Booking

☐

Figure 18. Bookings management

☐

Booking Details

☐ Full Name:

☐ Total Price:

☐ Status:

Yoga Classes

Type of Class - Day of Week (Time)

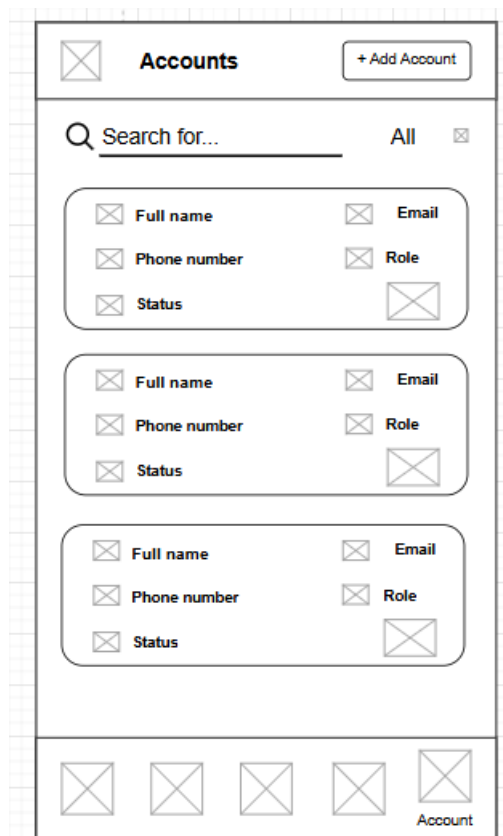
☐ Date - ☐ Instructor:

☐ Date - ☐ Instructor:

Confirm

Reject

Figure 19. Booking details



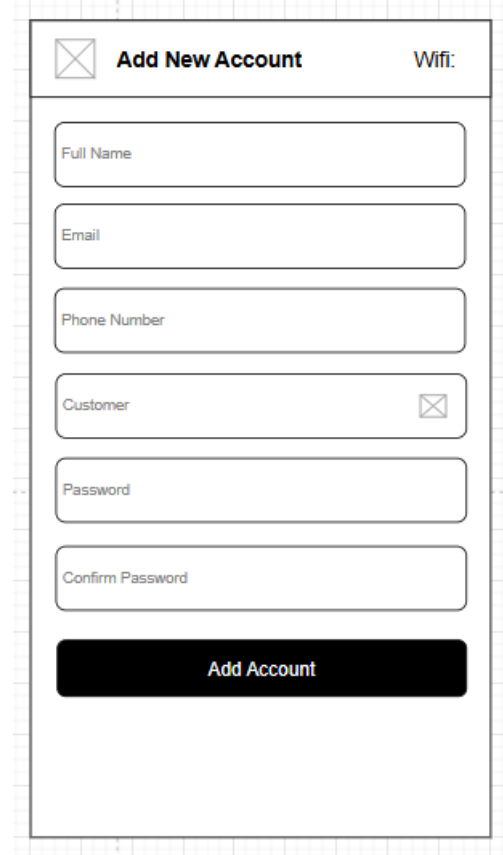
The interface is titled "Accounts" with a back arrow icon on the left and a "+ Add Account" button on the right. Below the title is a search bar with a magnifying glass icon and the text "Search for...". To the right of the search bar is a filter dropdown menu currently set to "All".

Below the search bar are three identical account cards. Each card contains a grid of fields with checkboxes to their left:

- Full name
- Phone number
- Status
- Email
- Role
- (An empty checkbox field)

At the bottom of the screen is a horizontal list of five account icons, each represented by a square with an 'X' inside. The last icon is labeled "Account".

Figure 20. Account management



The form is titled "Add New Account" with a back arrow icon on the left and a "Wifi:" label on the right. The form contains several input fields:

- Full Name
- Email
- Phone Number
- Customer (with a checkbox icon on the right)
- Password
- Confirm Password

At the bottom of the form is a large black button with the text "Add Account" in white.

Figure 21. Add a new account

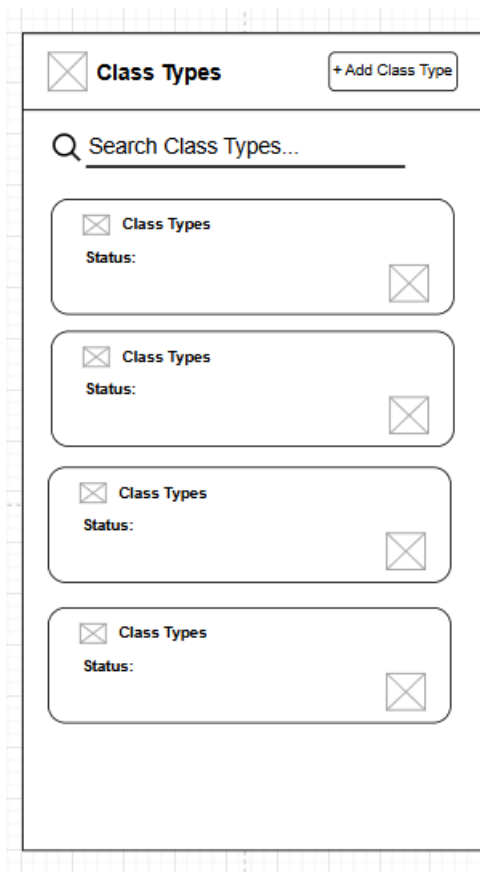


Figure 22. Class type management

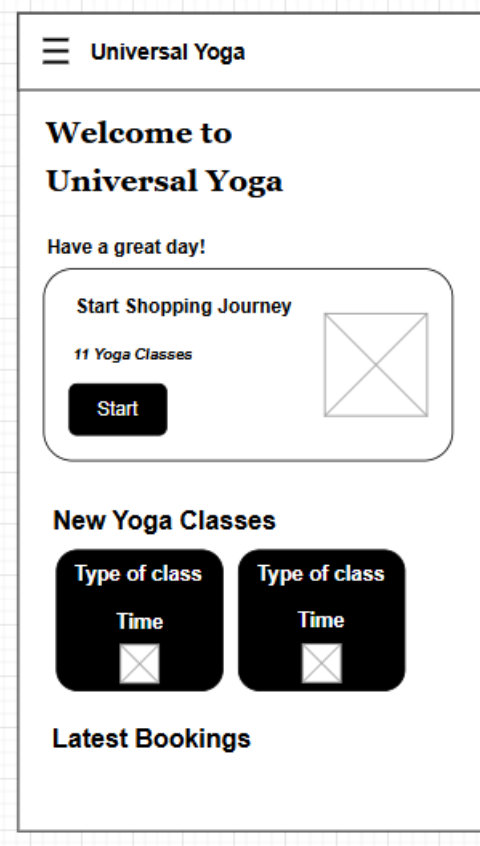


Figure 23. Homepage of Customer

Yoga Class List

Choose a search option:

☒

Search by Day of the Week

☒ Monday
☒ Tuesday
☒ Wednesday

☐

Search by Time of Day

Type of Class

Day of week

Time

Type of Class

Day of week

Time

Figure 24. Yoga class list with search by day of week

Yoga Class List

Choose a search option:

☐

Search by Day of the Week

☒

Search by Time of Day

Enter time of day

Type of Class

Day of week

Time

Type of Class

Day of week

Time

Figure 25. Yoga class list with search by time of day

< Yoga Class Details

Type of class

Description:

Price:

Time:

Capacity:

Day:

Book Yoga Class

Figure 26. Yoga class details

< Book Yoga Class

Select the course you want to join:

Search by date

Date:

☒ **Instructor:**

Comments:

Date:

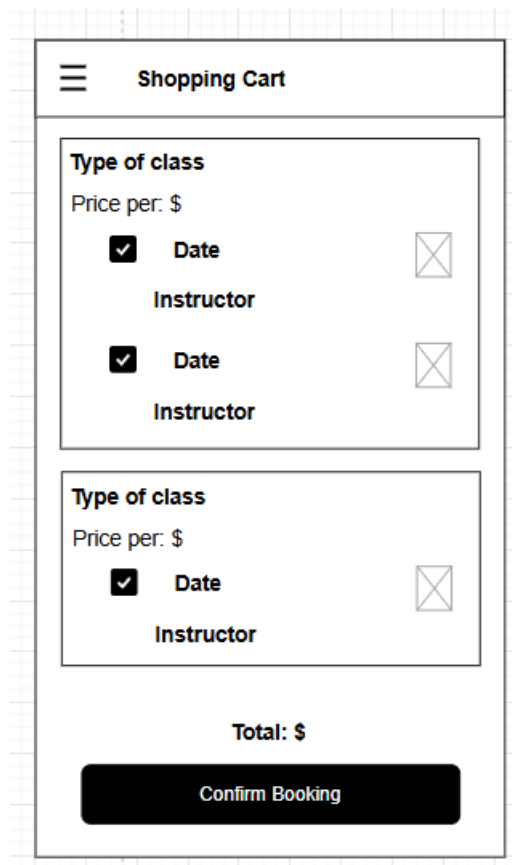
☒ **Instructor:**

Comments:

Total: \$

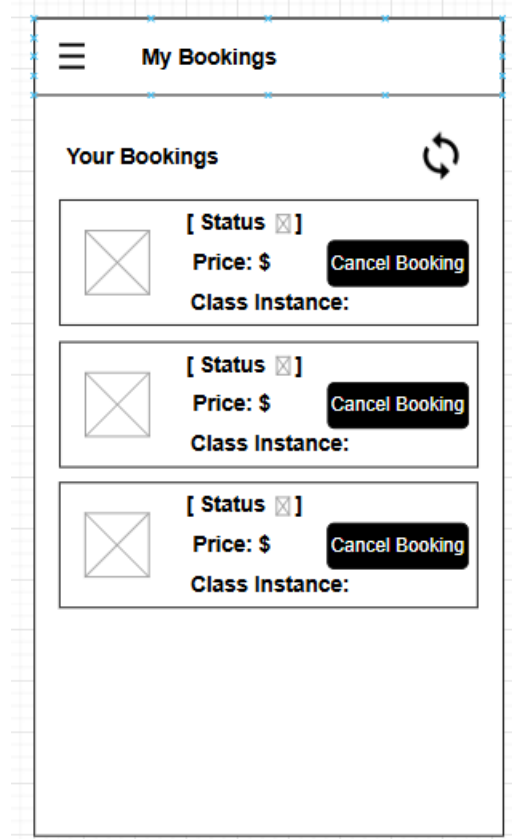
Add to Cart

Figure 27. Book yoga class



The 'Shopping Cart' screen features a hamburger menu icon on the left. The main content area contains two identical class selection boxes. Each box is titled 'Type of class' and includes a 'Price per: \$' label. Inside each box, there are two rows of options: a checked checkbox next to 'Date' and an unchecked checkbox next to 'Instructor'. Below these boxes, the text 'Total: \$' is displayed, followed by a large black button labeled 'Confirm Booking'.

Figure 28. Shopping cart



The 'My Bookings' screen features a hamburger menu icon on the left. The main content area is titled 'Your Bookings' and includes a refresh icon. Below this title, there are three identical booking entries. Each entry consists of a placeholder icon (a square with an 'X'), a status label '[Status]' with a refresh icon, a 'Price: \$' label, and a 'Class Instance:' label. To the right of each entry is a black button labeled 'Cancel Booking'.

Figure 29. My Bookings

3. SECTION 3 - EVALUATION

3.1 Admin App Evaluation

3.1.1 Human-Computer Interaction (HCI)

The Admin App is evaluated according to Jakob Nielsen's usability heuristics (Nielsen, J., 1994):

- **Visibility of System Status:** Provides immediate responses for actions like adding, editing, and deleting data.

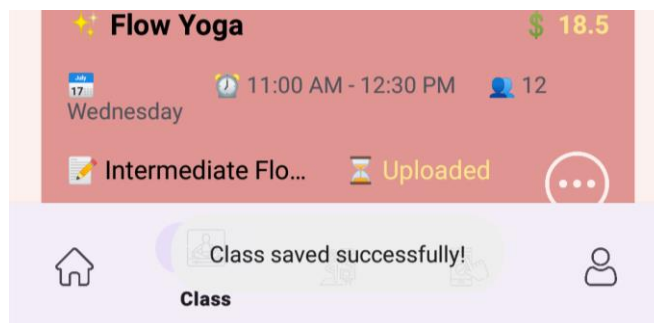


Figure 30. Response on successful yoga class addition

- **Match Between System and the Real World:** Uses simple, user-friendly language tailored for yoga class management.

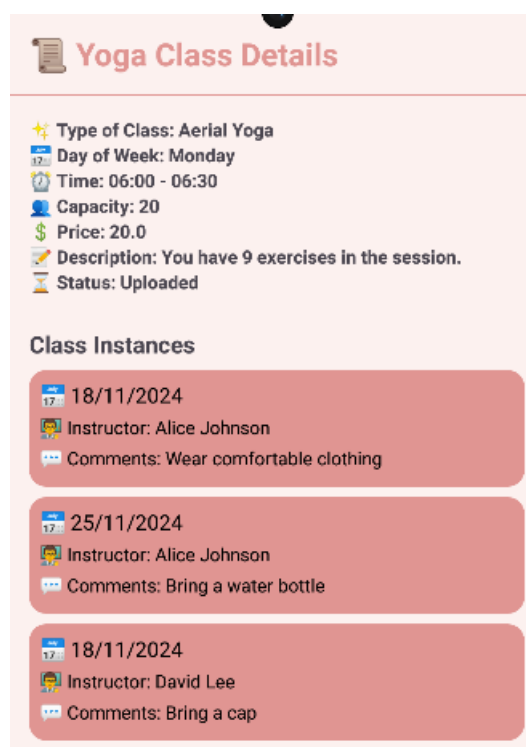


Figure 31. Labels are clear and easy to understand

- **User Control and Freedom:** Allows easy modification or cancellation of class deletion events using the "No" option.

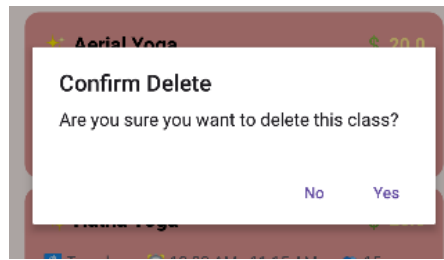


Figure 32. Confirm yoga class deletion

- **Consistency and Standards:** Maintains a consistent UI across CRUD functionalities.

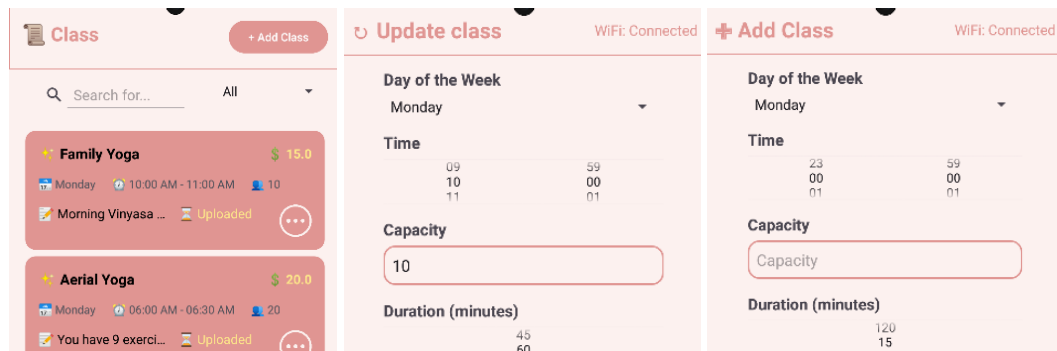


Figure 33. Similar color scheme across pages

- **Error Prevention:** Validates required fields to prevent errors.

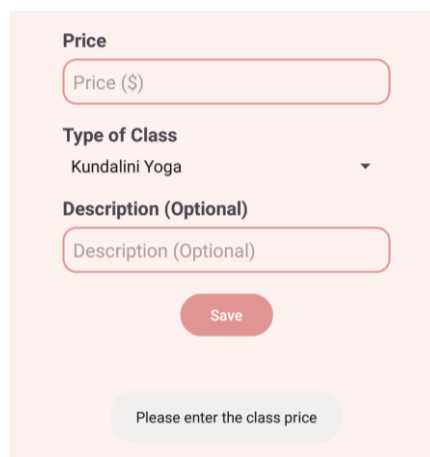


Figure 34. Notice of mandatory request to add class price

- **Recognition Rather Than Recall:** Features an easy-to-use search bar for quick access to information.

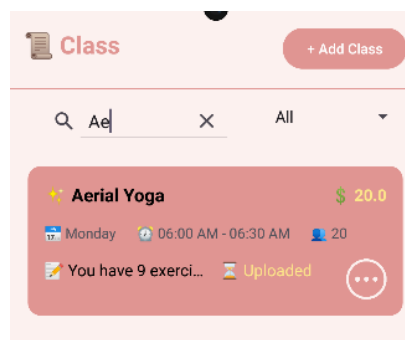


Figure 35. Search bar for yoga classes

- **Flexibility and Efficiency of Use:** Advanced search and quick data synchronization enhance efficiency.

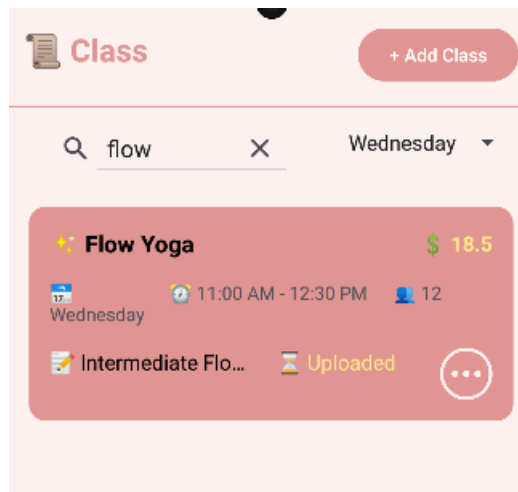


Figure 36. Combine search and filter for yoga classes

- **Aesthetic and Minimalist Design:** Clean, focused design with minimal distractions.

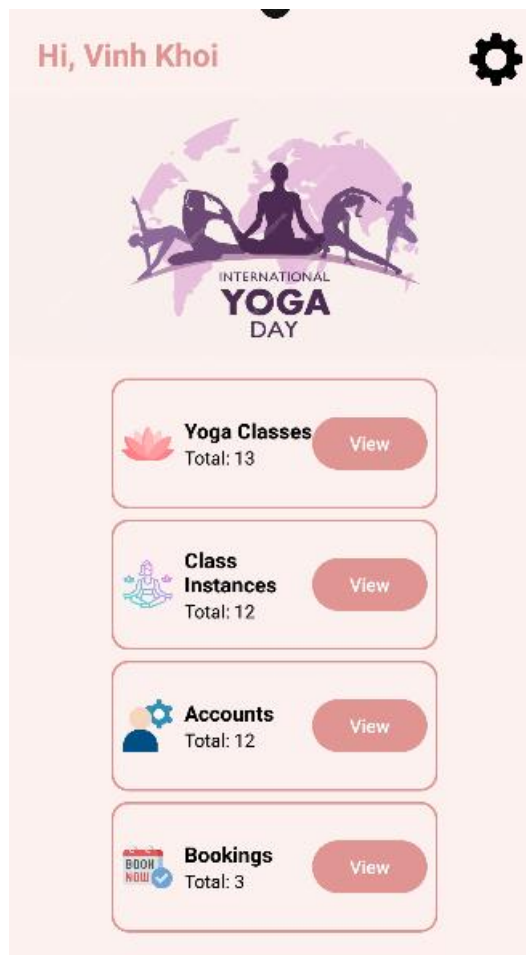


Figure 37. Simple admin homepage interface, easy to navigate

- **Help Users Recognize, Diagnose, and Recover from Errors:** Clear error messages guide users to resolution.

The screenshot shows a mobile app interface for updating a class instance. At the top, there's a header bar with the title 'Update Class Instance' and a status 'WiFi: Connected'. Below this, the form has several sections: 'Select Yoga Class' with a dropdown menu showing 'Ashtanga Yoga - Sunday - (09:30 -..'; 'Date' with a date picker showing '10/11/2024'; 'Instructor' with a dropdown menu showing 'Carla Smith'; and 'Additional Comments' with a text input field containing 'Stretch beforehand'. A 'Save Class Instance' button is at the bottom. A light blue error message box at the bottom of the form states: 'Please select a valid Sunday that is not in the past.'

Figure 38. Detailed and clear error message

3.1.2 Security

- **User Authentication and Authorization:** Admins must log in to access functionalities. Passwords are encrypted and secured during authentication. Firebase adds an extra layer of security for data synchronization.
- **Local Data Security:** Data stored on the local SQLite database is protected from unauthorized access.
- **Additional network security:** Data transmitted to Firebase is encrypted to prevent man-in-the-middle attacks. HTTPS protocol ensures secure data transmission.

3.1.3 Adaptability to Screen Sizes

- **Optimized for Mobile Devices:** The app is currently optimized for mobile screens with clear layouts and navigation. However, adjustments may be needed for larger devices like tablets to maintain an intuitive interface.
- **Responsive Layouts:** Using flexible layouts such as Constraint Layout ensures the app can adapt seamlessly to different screen sizes.
- **Future improvements:** Possibility to add text resizing and screen zooming features to assist visually impaired users

3.1.4 Real-World Deployment Considerations

- **Performance Optimization:** The app must be tested to handle large datasets and optimize loading times when multiple users interact simultaneously.
- **Data Synchronization Management:** Ensuring consistent data synchronization between local storage and Firebase, especially during offline and online transitions, is crucial.
- **Scalability:** Preparing the system to handle increased load as the number of classes and users grows.
- **User Training and Support:** Offering training materials, FAQs, and customer support channels for administrators.

3.1.5 Accessibility

- **Improved Accessibility for Disabled Users:** Adding text-to-speech support for visually impaired users and color contrast options for color-blind users can significantly enhance accessibility.
- **Keyboard Navigation:** Ensuring all app functionalities are accessible using a keyboard for users who cannot rely on touch input.

3.2 Customer App Evaluation

3.2.1 Human-Computer Interaction (HCI)

The Customer App is evaluated based on Nielsen's usability principles:

- **Visibility of System Status:** Users receive instant responses when adding classes to their cart or booking a class. Notifications such as “Added to Cart” or “Booking Successful” provide clear feedback.

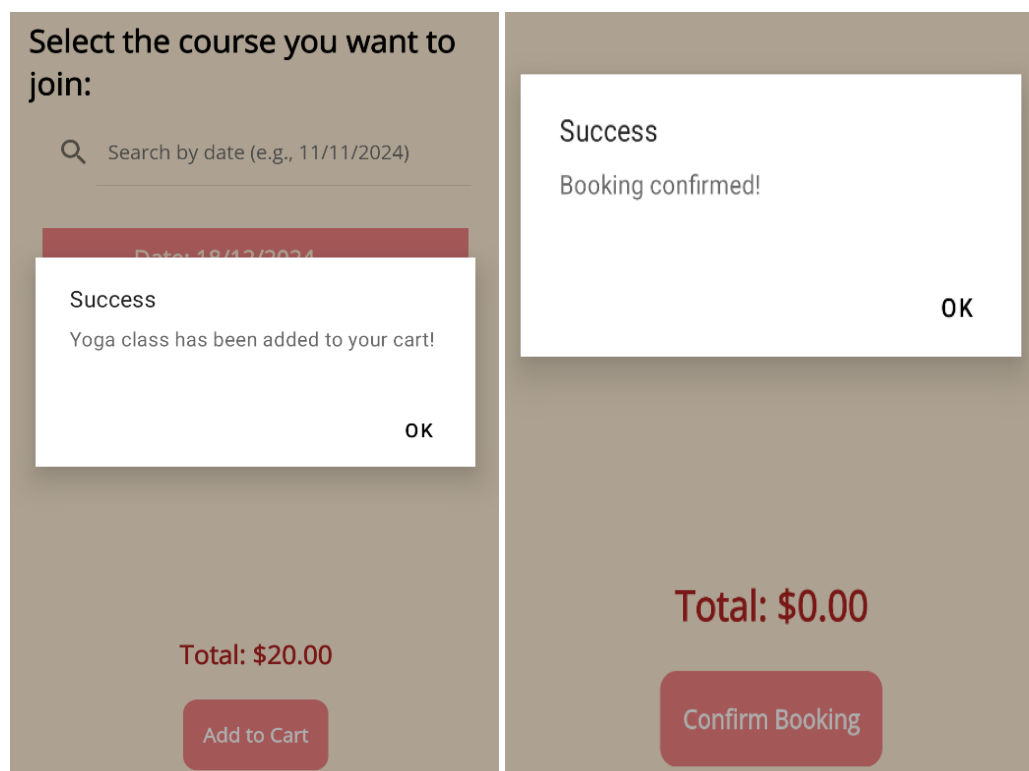


Figure 39. Responses after an event is completed

- **Match Between System and the Real World:** Yoga class details are displayed clearly with descriptions, times, and friendly visuals. The language used is intuitive and approachable.

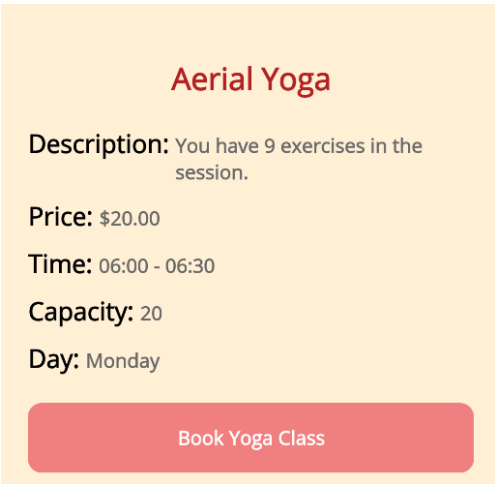


Figure 40. Yoga class details are displayed clearly

- **User Control and Freedom:** Users can easily remove classes from their cart or modify booking information before finalizing it.

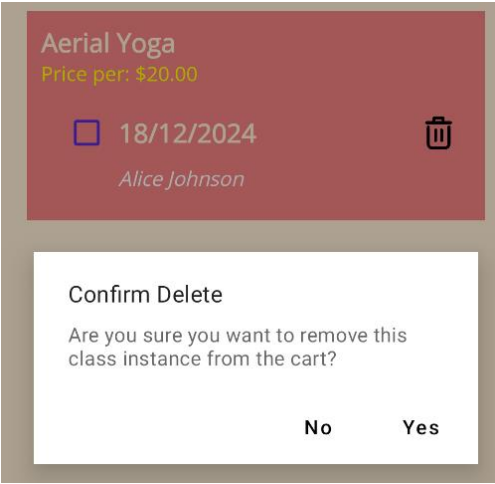


Figure 41. Delete class instances in the cart before creating a booking

- **Consistency and Standards:** The app maintains a consistent layout and navigation style across features like booking, searching, and cart management.



Figure 42. Consistent interface in terms of color and elements

- **Error Prevention:** Users are warned if they attempt to cancel a confirmed booking or reject a booking.

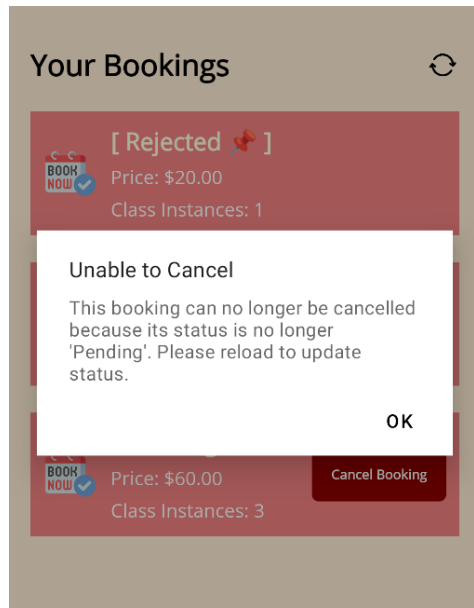


Figure 43. Notice of booking cannot be canceled

- **Recognition Rather Than Recall:** Recent bookings displayed on the homepage help users not to have to remember their last booking.



Figure 44. Latest Bookings in Homepage

- **Flexibility and Efficiency of Use:** Users can filter classes or class instances by date or time, making it easier to find classes and class instances that fit their schedules.

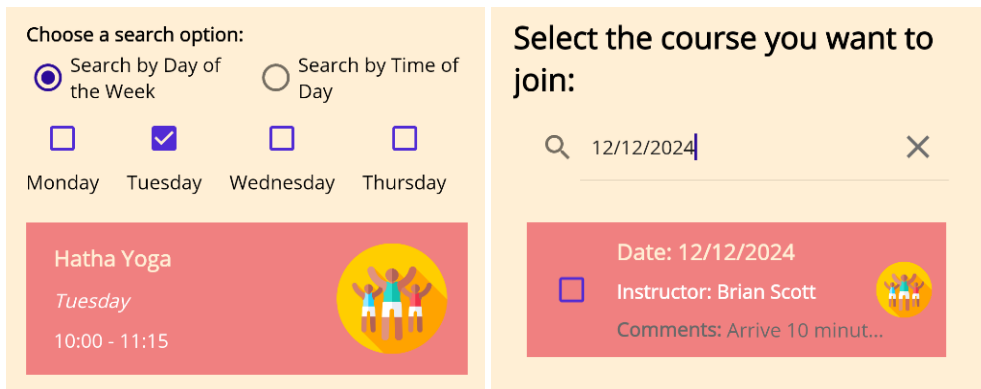


Figure 45. Search by day of week or date

- **Aesthetic and Minimalist Design:** The interface is clean and focuses on core tasks like viewing and booking classes without clutter or unnecessary elements.

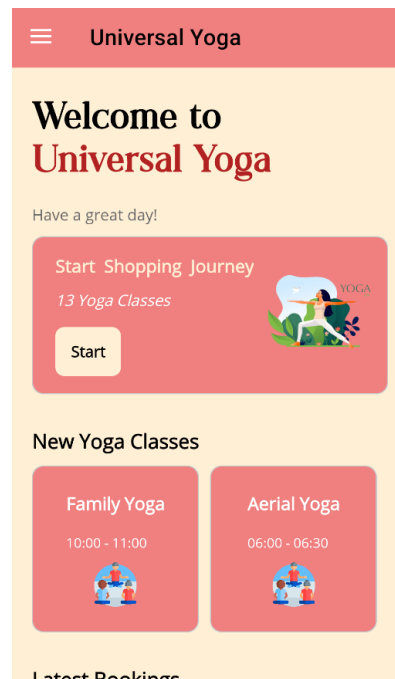


Figure 46. Simple interface, focusing on suggesting customers shopping yoga classes

- **Help Users Recognize, Diagnose, and Recover from Errors:** The app displays clear error messages to help users resolve issues during account-related processes such as registration, login, or password recovery.

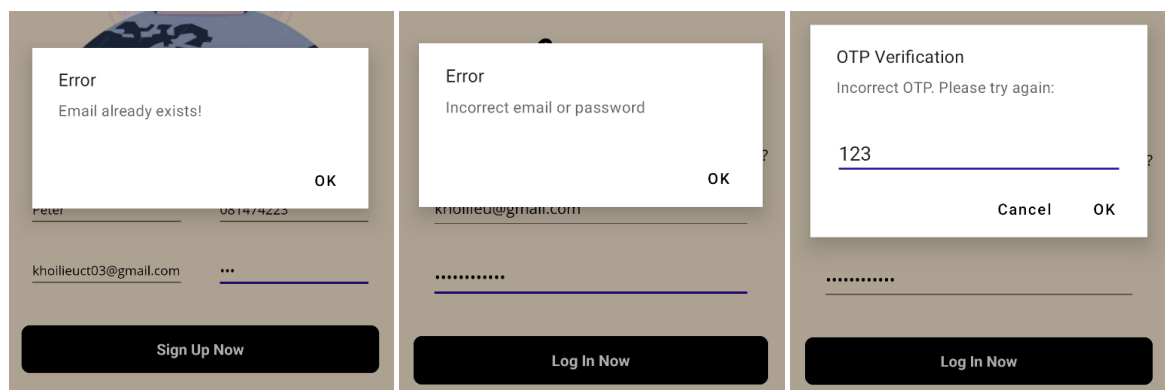


Figure 47. Display account-related errors clearly and in detail

3.2.2 Security

- **User Authentication:** Only logged-in users can book classes. Passwords are encrypted and protected. Implementing email or SMS verification for sensitive transactions can further enhance data protection.
- **Customer Data Security:** Booking data and personal details are securely stored, with robust authentication mechanisms in place.
- **Compliance with Data Protection Regulations:** Ensuring the app adheres to GDPR (General Data Protection Regulation) or similar local privacy laws for user data protection.

- **Additional feature:** users can manage their accounts by changing account information or deleting their accounts in case of need.

3.2.3 Adaptability to Screen Sizes

- **Mobile and Hybrid Device Compatibility:** The app, built using .NET MAUI, automatically adjusts to various screen sizes, including Android and iOS devices.
- **Large screen improvements:** For tablets and PCs, improvements to the display and navigation will be needed, this should be developed in additional functions.

3.2.4 Real-World Deployment Considerations

- **User Experience Optimization:** Extensive testing across different devices and network conditions is necessary to ensure smooth performance.
- **Multi-Language Support:** If deployed globally, the app should support multiple languages and ensure proper display for different character sets.
- **Integration with Payment Gateways:** Implementing secure, reliable payment gateway integrations for booking payments.

3.2.5 Accessibility

- **Enhanced Accessibility:** Integrating screen readers for visually impaired users, high-contrast color options, and larger fonts for elderly users will improve accessibility for a wider audience.
- **Voice Command Functionality:** Allowing voice-based commands for booking and searching classes to improve accessibility for users with mobility or vision impairments.

4. SECTION 4 - DESIGN

4.1 Admin Application:

4.1.1 Login Interface

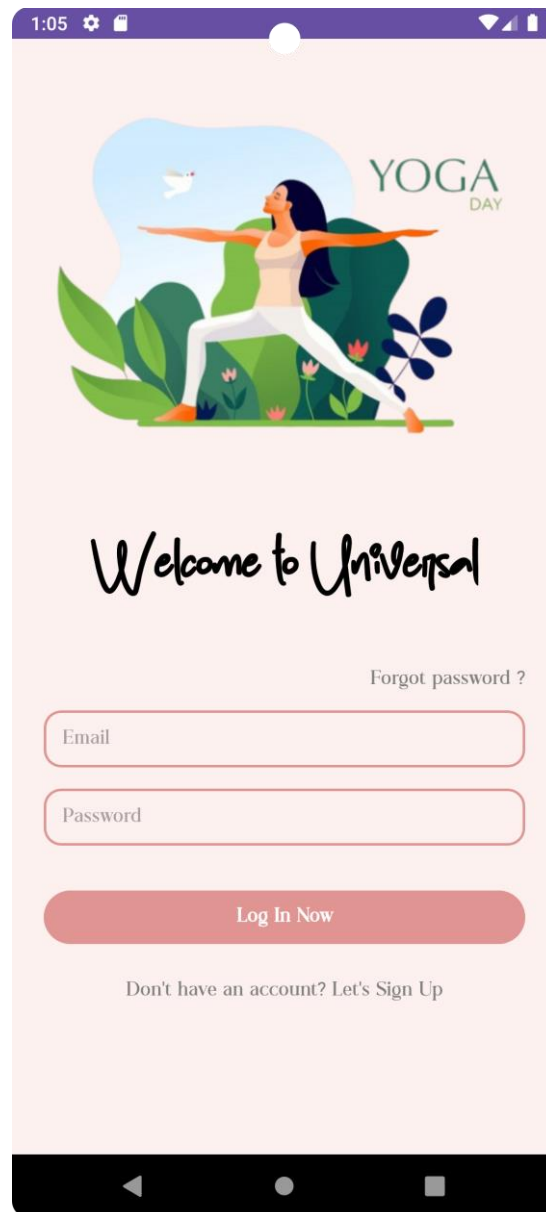


Figure 48. Login Interface

Figure 48 shows the Universal Yoga Admin application's login interface. This screen lets users input their email and password to access the admin features. There is also a link to sign up if they don't already have an account. This interface provides a secure access point to the app, ensuring only authorized personnel can manage the application's data.

4.1.2 Sign Up Interface

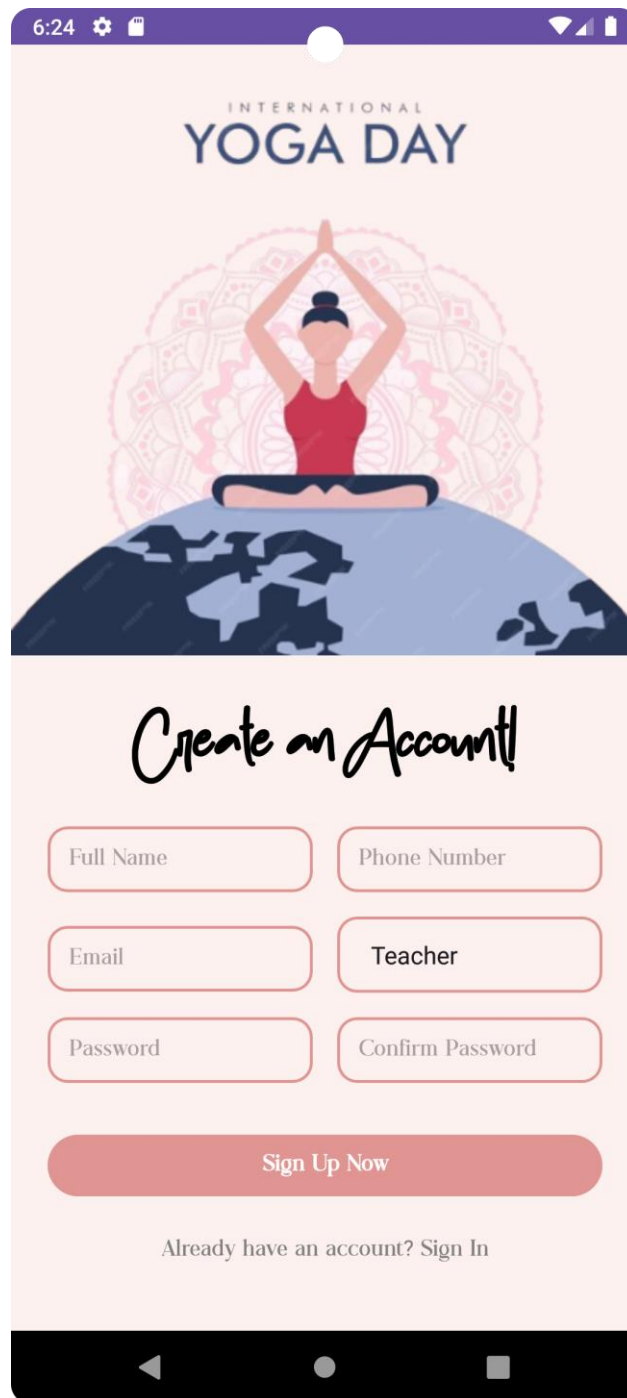


Figure 49. Sign Up Interface

Figure 49 shows the account creation interface for the Universal Yoga application. This screen allows new users to register by providing their full name, phone number, email, password, and password confirmation. The “Role” field, which is pre-set to “Teacher,” allows users to specify their role as “Customer,” or “Teacher.” This interface ensures that users register with the correct role, granting them the appropriate access and functionality according to their role in the application. It allows administrators to easily create “Customer” accounts, which are used for tasks that require interacting with the client-side application.

4.1.3 Authentication Handling Login

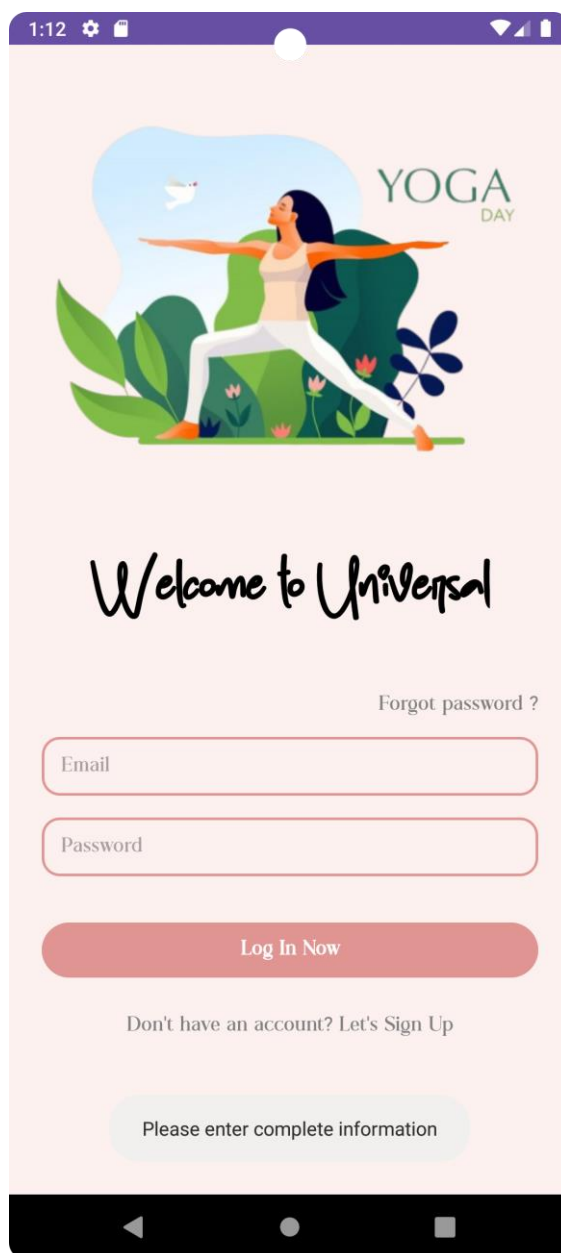


Figure 50. Incomplete fields when logging in

Figure 50 shows a notification message, "Please enter complete information," which appears when the user attempts to log in without filling in all required fields. This message serves as a prompt to ensure that both the email and password fields are completed before proceeding with the login process, enhancing user guidance and reducing login errors.

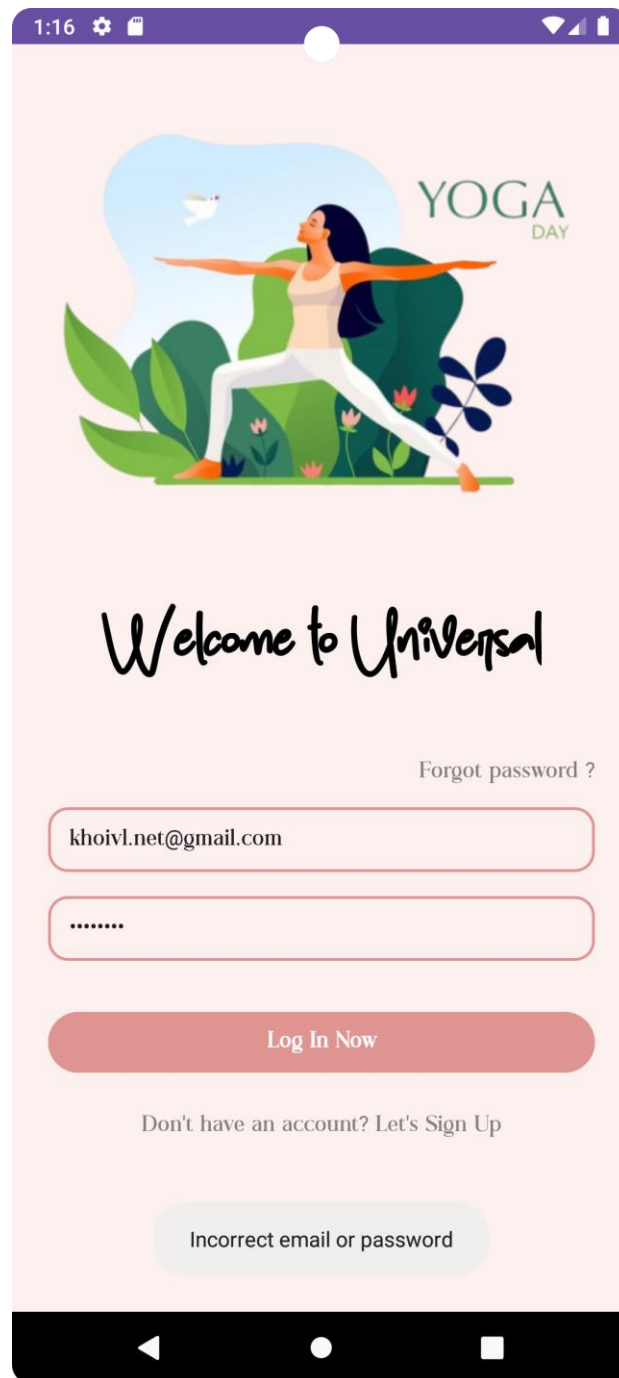


Figure 51. Incorrect authentication information

Figure 51 displays a notification message, "Incorrect email or password," which appears when the user attempts to log in with invalid credentials. This message alerts the user to check their email and password for accuracy before attempting to log in again, thereby providing feedback to help the user troubleshoot login issues.

4.1.4 Authentication Handling Sign Up

1:21

INTERNATIONAL
YOGA DAY

Create an Account!

Full Name Phone Number

Email Customer

Password Confirm Password

Sign Up Now

Please fill in all fields

Figure 52. Notification for incomplete registration fields.

Figure 52 shows a notification, "Please fill in all fields," which appears on the "Create an Account" screen when the user attempts to sign up without completing all required fields. This message prompts the user to ensure that all input fields are properly filled out to proceed with the registration, thereby improving the user experience by guiding them to complete the form.

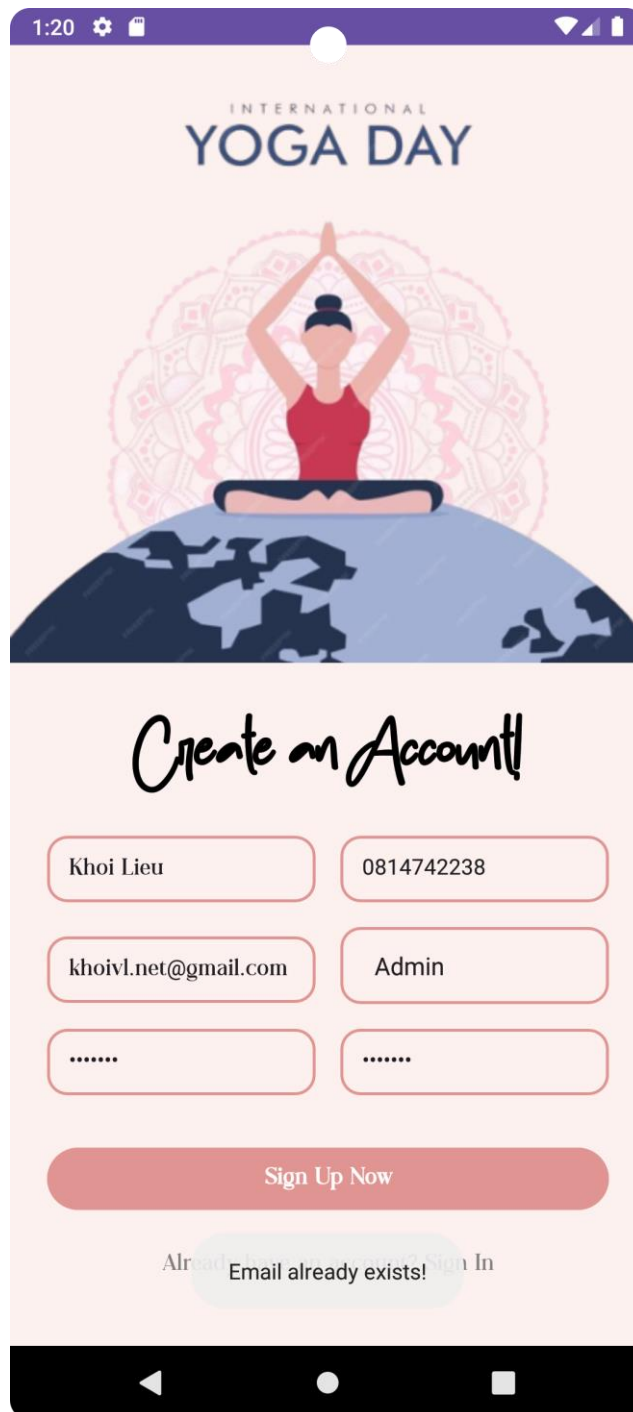


Figure 53. Notification for duplicate email in registration.

Figure 53 shows a notification, "Email already exists!" displayed on the "Create an Account" screen when a user attempts to register with an email that is already associated with an existing account. This alert guides the user to enter a unique email address, ensuring that each account has a distinct identifier and preventing conflicts in the registration process.

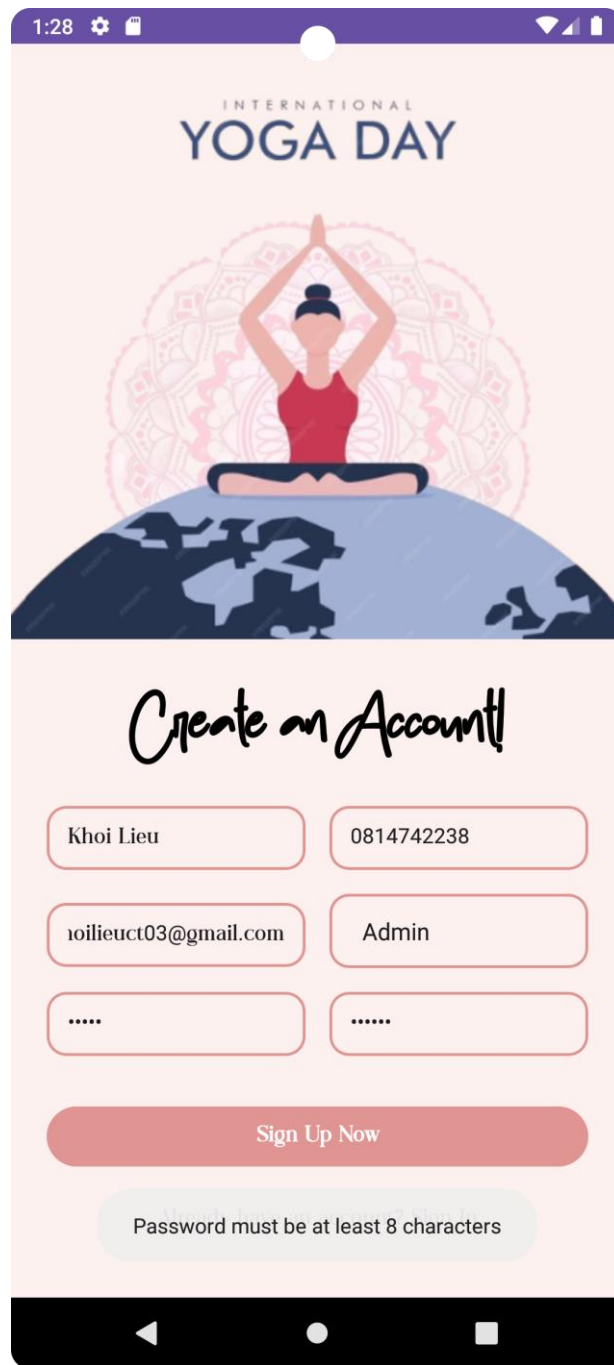


Figure 54. Notification for password length requirement in registration.

Figure 54 displays a notification, "Password must be at least 8 characters," on the "Create an Account" screen, indicating that the entered password does not meet the minimum length requirement. This message helps ensure that users create secure passwords, improving account security by enforcing a minimum character limit.

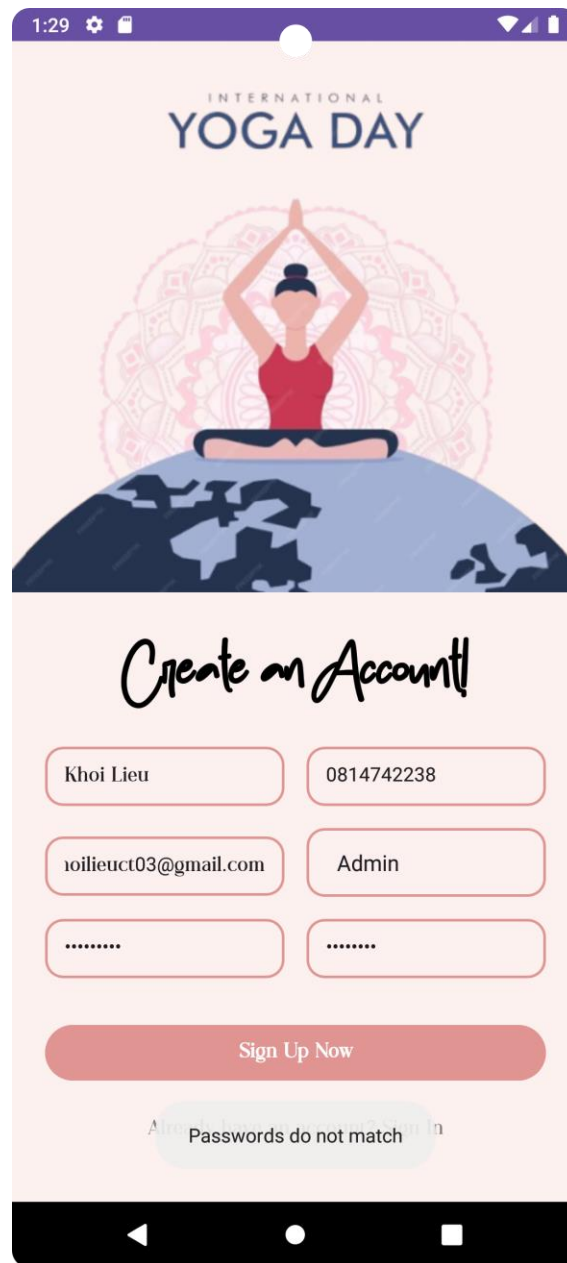


Figure 55. Notification for password mismatch in registration.

Figure 55 displays a notification, "Passwords do not match," on the "Create an Account" screen, indicating that the entries in the "Password" and "Confirm Password" fields are not identical. This message prompts the user to re-enter matching passwords, ensuring accuracy and consistency in account creation.

4.1.5 HomePage

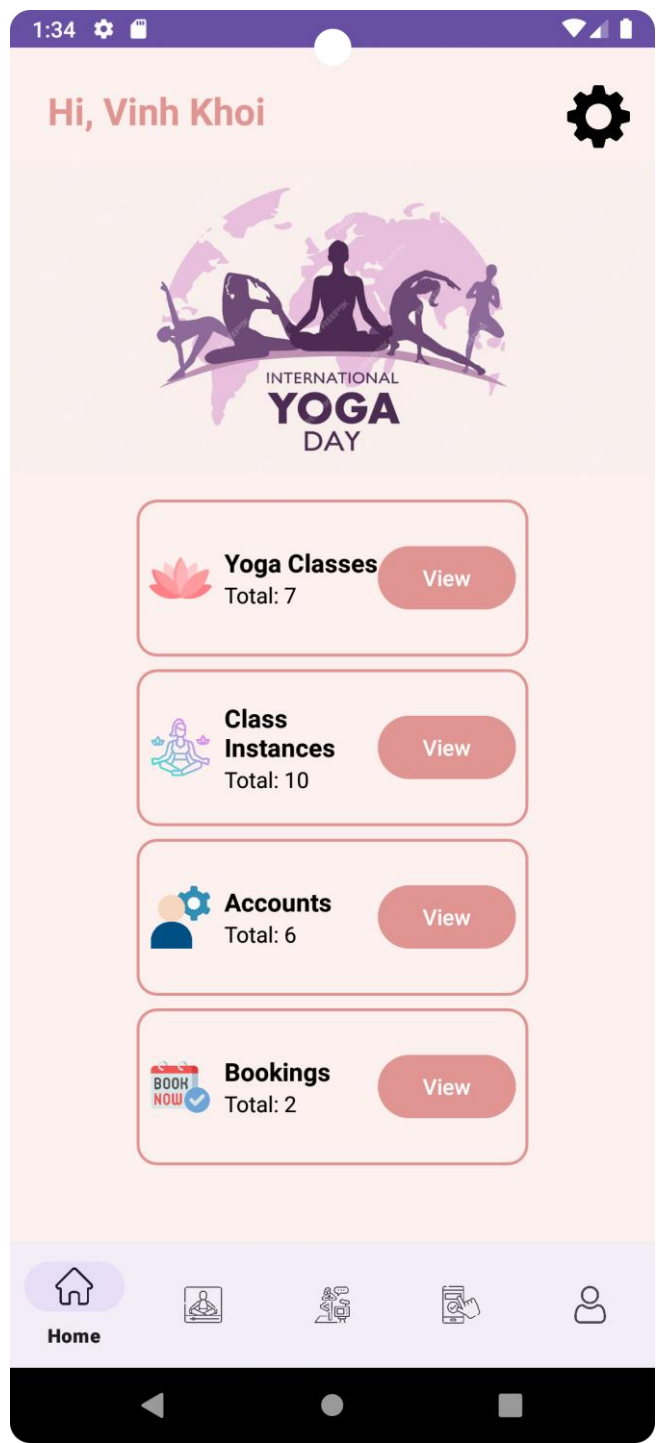


Figure 56. Home page of the admin app.

Figure 56 shows the home page of the admin app, designed to provide a quick overview of various management sections, including "Yoga Classes," "Class Instances," "Accounts," and "Bookings." Each section displays the total count and a "View" button, enabling administrators to access detailed lists or further management options. This interface streamlines navigation, allowing admins to efficiently monitor and manage different components of the app.

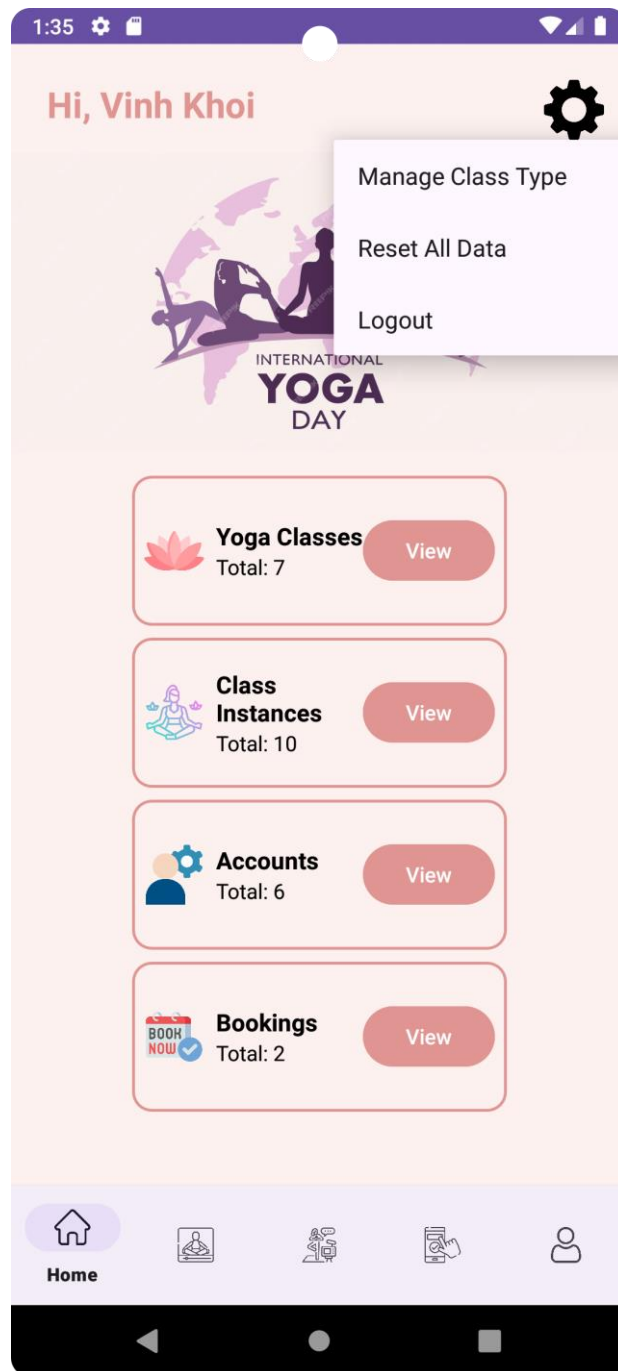


Figure 57. Settings menu on the admin home page.

Figure 57 illustrates the settings menu on the admin home page, accessible via the gear icon. This menu provides options such as "Manage Class Type" for updating or adding new yoga class categories, "Reset All Data" for clearing app data, and "Logout" for signing out of the application. This setup enhances admin control, enabling quick access to essential settings for application management and maintenance.

4.1.6 Yoga Classes Management

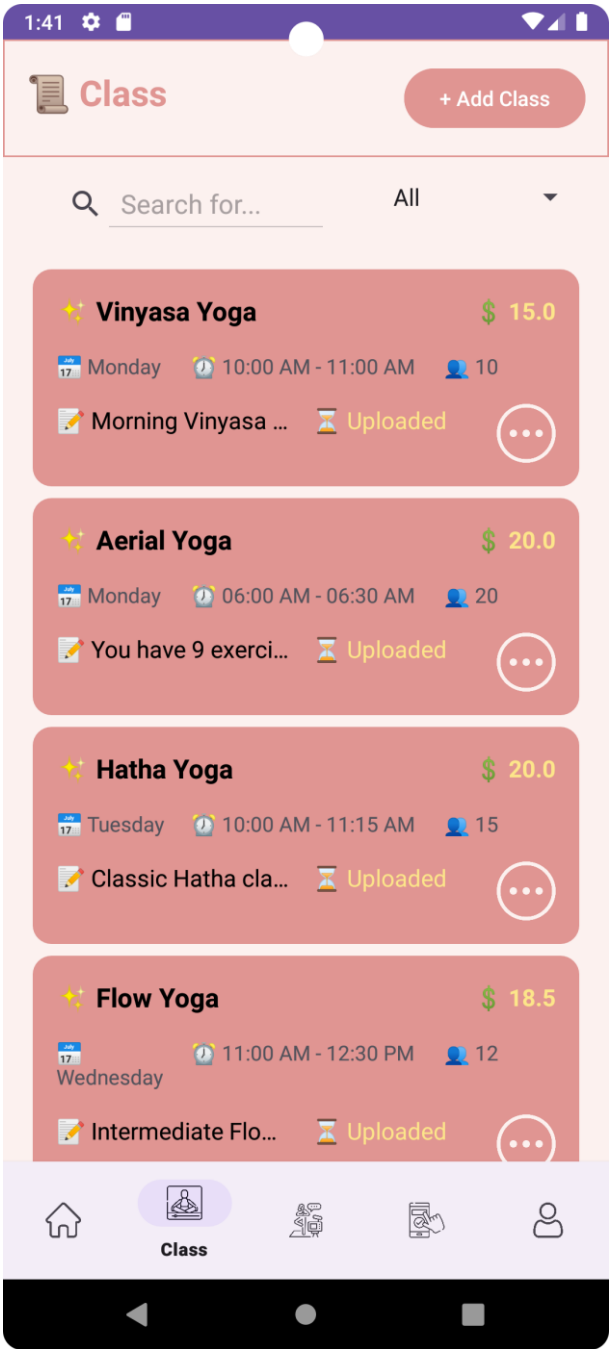


Figure 58. Yoga Classes Management Page.

Figure 58 shows the Yoga Classes Management page, where admins can view and manage different yoga classes. Each class entry displays essential information, including the class name, day, time, instructor, number of exercises, status (e.g., "Uploaded"), and price per class. The "Add Class" button at the top right allows admins to add new classes, while the three-dot icon on each entry provides further options for class-specific actions. This interface streamlines class management, enabling efficient monitoring and updating of yoga classes.

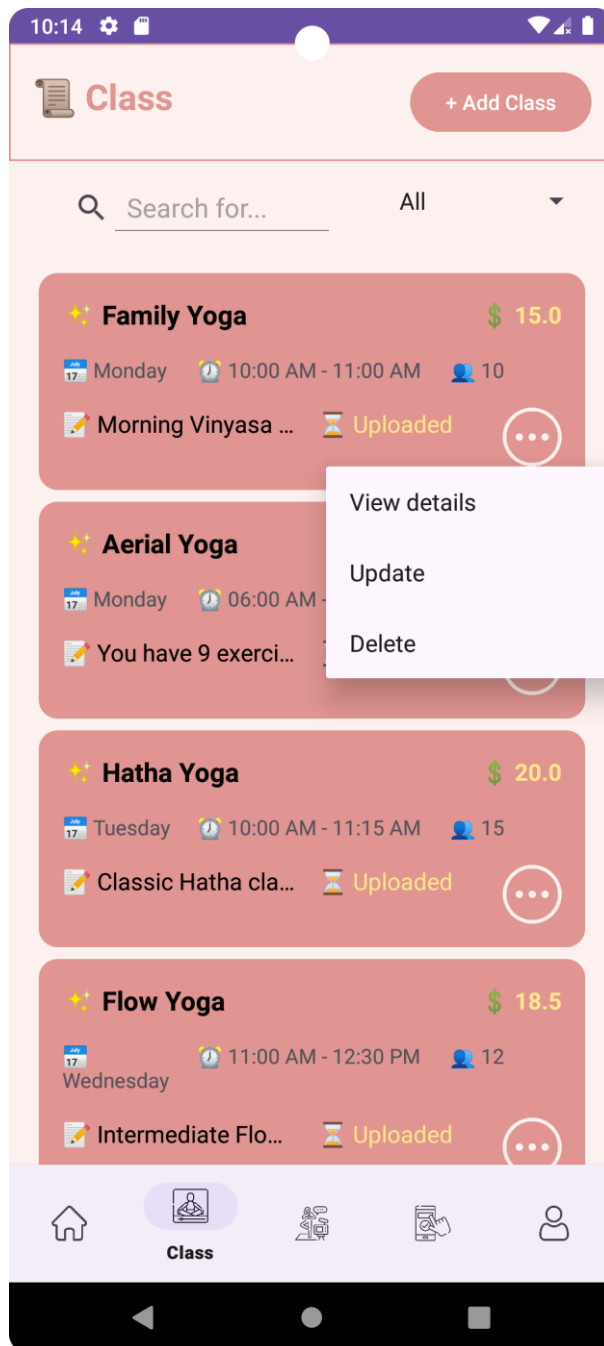


Figure 59. Options Menu in Yoga Classes Management.

Figure 59 displays the options menu accessed by tapping the three-dot icon on a yoga class entry within the Yoga Classes Management page. The menu includes three options: "View details," "Update," and "Delete." The "View details" option lets users see detailed information about the selected class. The "Update" option enables administrators to modify the class details, while the "Delete" option permits the class removal from the system. This feature offers quick access to essential management actions, facilitating efficient maintenance of accurate class information.

4.1.7 View Yoga Class Details

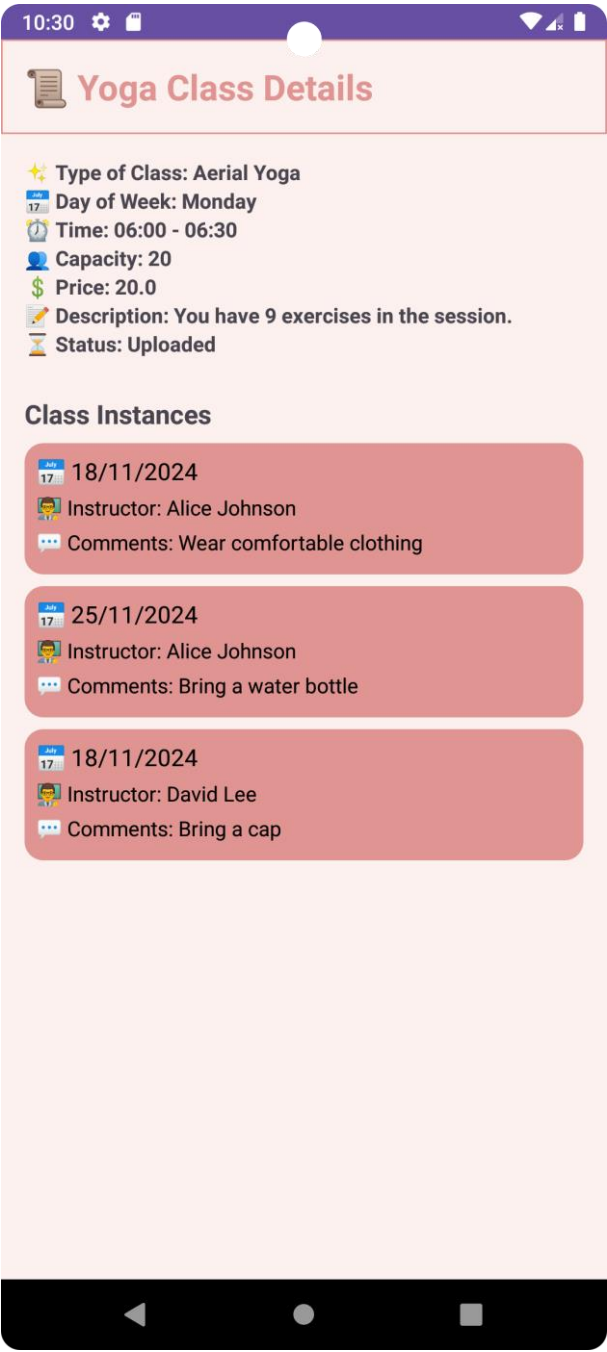


Figure 60. Yoga class details

Figure 60 displays a specific yoga class's type, schedule, capacity, price, description, and status. Below, associated class instances are listed with details like the date, instructor, and any comments, providing a comprehensive view of scheduled sessions.

4.1.8 Add Yoga Classes

1:43



WiFi: Connected

+

Add Class

Day of the Week

Monday

Time

23

00

01

59

00

01

Capacity

Capacity

Duration (minutes)

120

15

30

Price

Price (\$)

Type of Class

Kundalini Yoga

Description (Optional)

Description (Optional)

Save

Figure 61. Add Yoga Class Interface

Figure 61 displays the "Add Class" interface, designed for administrators to create new yoga classes. The form includes fields for selecting the day of the week, time, capacity, duration, price, class type, and an optional description. A "Save" button at the bottom allows users to confirm and save the new class details. This interface helps administrators efficiently input essential information for each yoga class, ensuring organized and accessible class scheduling.

The screenshot shows a mobile application interface for adding a class. At the top, there is a status bar with the time 1:44, a settings icon, a battery icon, and a WiFi icon labeled 'WiFi: Connected'. Below the status bar is a header with a red plus icon and the text 'Add Class'. The main form has several sections: 'Day of the Week' with a dropdown menu set to 'Monday'; 'Time' with two spinners showing '23:00' and '59:00'; 'Capacity' with an empty text input field; 'Duration (minutes)' with a spinner showing '15'; 'Price' with a text input field containing 'Price (\$)'; 'Type of Class' with a dropdown menu set to 'Kundalini Yoga'; and 'Description (Optional)' with a text input field containing 'Description (Optional)'. At the bottom of the form is a red 'Save' button. Below the 'Save' button is a light blue notification bubble that says 'Please enter the class capacity'. The bottom of the screen shows the Android navigation bar with back, home, and recent apps buttons.

Figure 62. Notification for Missing Class Capacity in Add Class Interface

Figure 62 displays a notification, "Please enter the class capacity," in the "Add Class" interface, indicating that the capacity field was left empty. This message prompts the user to input the maximum number of participants allowed in the class, ensuring that essential information is filled out before the class can be saved.

The screenshot shows a mobile application interface for adding a class. At the top, there is a status bar with the time 1:44, a settings icon, a battery icon, and a WiFi icon labeled 'WiFi: Connected'. Below the status bar is a header with a red plus icon and the text '+ Add Class'. The main form has a light pink background and contains the following fields:

- Day of the Week:** A dropdown menu with 'Monday' selected.
- Time:** Two time pickers. The first has options 23, 00, and 01. The second has options 59, 00, and 01.
- Capacity:** A text input field containing the number '20'.
- Duration (minutes):** A dropdown menu with options 120, 15, and 30.
- Price:** A text input field with the placeholder text 'Price (\$)'. This field is empty.
- Type of Class:** A dropdown menu with 'Kundalini Yoga' selected.
- Description (Optional):** A text input field with the placeholder text 'Description (Optional)'.

At the bottom of the form is a red 'Save' button. Below the 'Save' button is a light gray notification box with the text 'Please enter the class price'. The bottom of the screen shows a black navigation bar with three icons: a back arrow, a circle, and a square.

Figure 63. Notification for Missing Class Price in Add Class Interface

Figure 63 displays a notification, "Please enter the class price," in the "Add Class" interface, indicating that the price field was left empty. This message prompts the user to input the cost per session for the class, ensuring that all necessary details are provided before saving the class information.

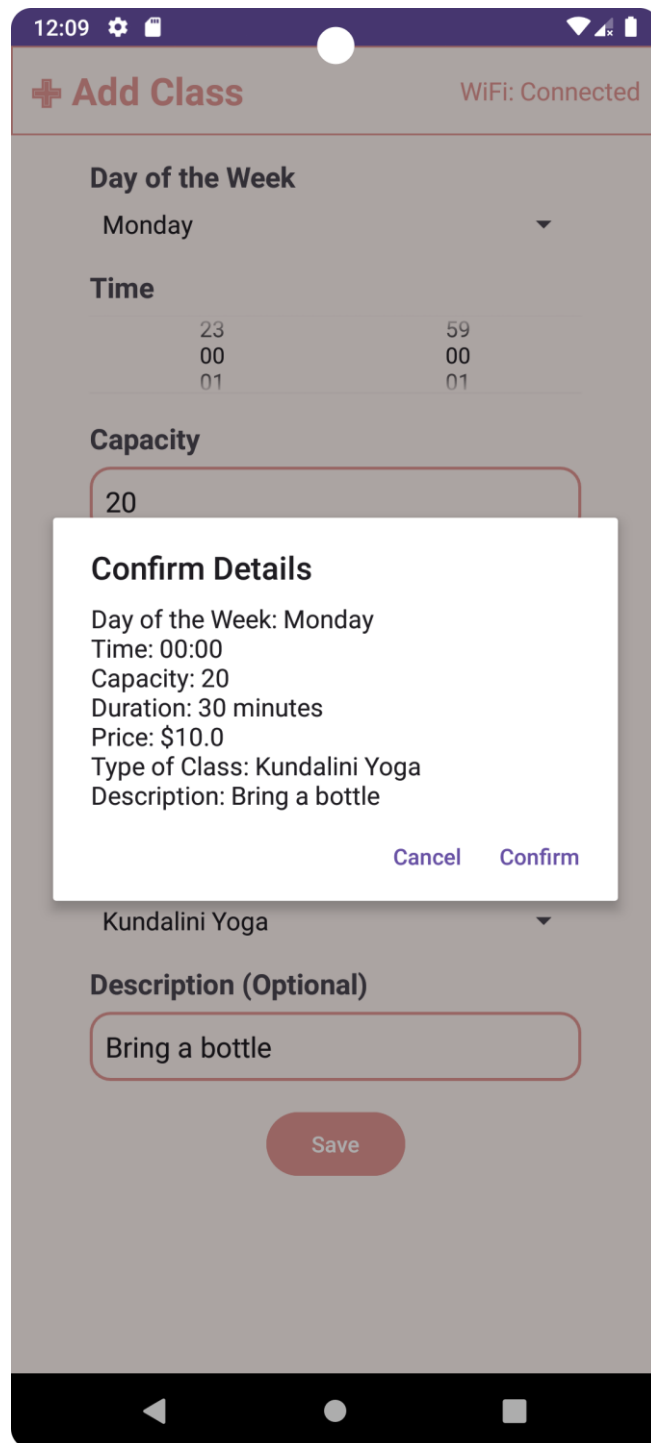


Figure 64. Confirm yoga class information before saving

Figure 64 shows the confirmation screen for yoga class information before saving. To ensure accurate information, the user is prompted to review all relevant details, including the day of the week, time, capacity, and..... The screen provides options to either save or cancel, allowing the user to make any necessary adjustments before finalizing the entry. This step serves as a final check to prevent errors and ensure that the correct details are stored in the system.

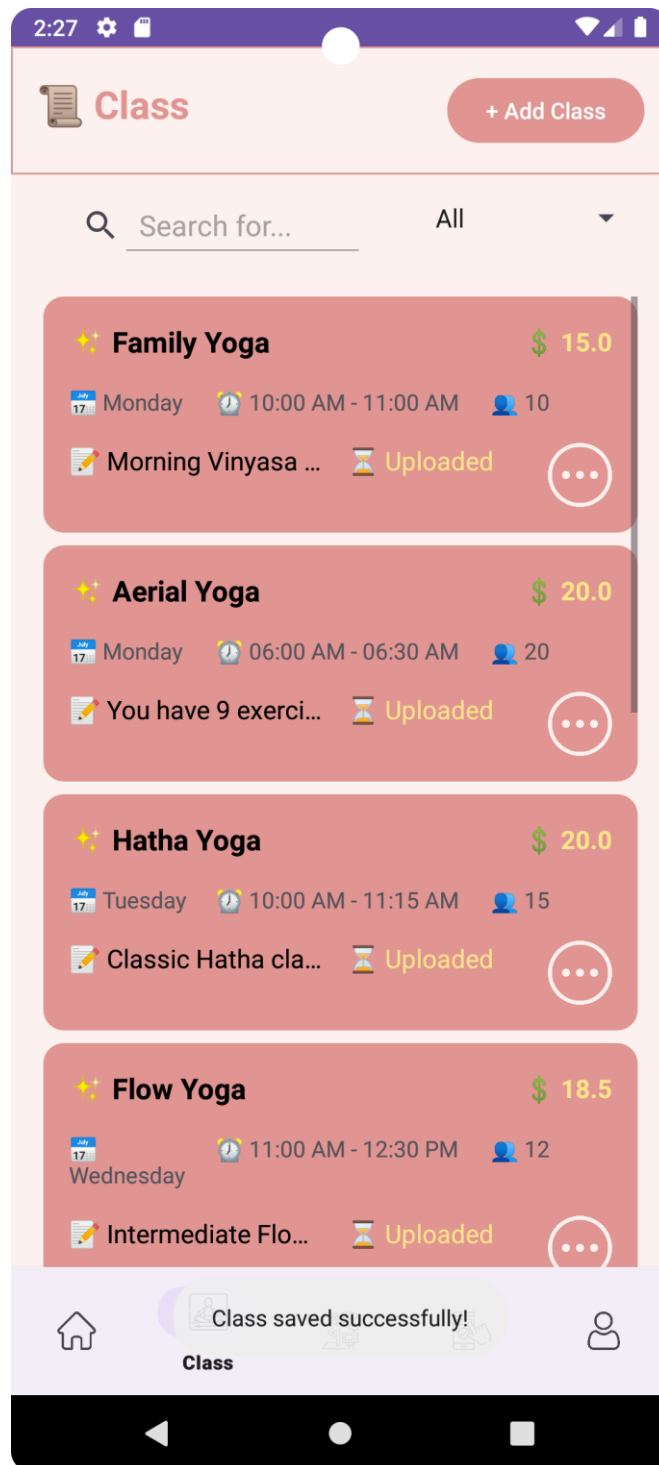


Figure 65. Confirmation message for successful class creation.

Figure 65 displays the confirmation message, "Class saved successfully!" at the bottom of the "Yoga Classes Management" screen. This notification confirms that a new yoga class has been successfully created and saved, providing users with immediate feedback after completing the save operation.

4.1.9 Update Yoga Classes

1:55





 Update class

WiFi: Connected

Day of the Week

Monday

Time

09

10

11

59

00

01

Capacity

10

Duration (minutes)

45

60

75

Price

15.0

Type of Class

Vinaya Yoga

Description (Optional)

Morning Vinaya for beginners

Save

Figure 66. Update class interface in the admin app.

Figure 66 shows the "Update Class" interface in the admin app, allowing administrators to modify details of an existing yoga class. The screen includes fields for the day of the week, time, capacity, duration, price, type of class, and an optional description. This interface enables administrators to make precise updates to a class's information, ensuring accurate scheduling and resource management.

The screenshot shows a mobile application interface for updating a class. At the top, there's a status bar with the time 1:58 and a purple header bar with the title '+ Update class' and 'WiFi: Connected'. The form fields are as follows:

- Day of the Week:** A dropdown menu currently showing 'Monday'.
- Time:** A time picker showing 10:00.
- Capacity:** A text input field with the placeholder text 'Capacity'. This field is highlighted with a red border, indicating it is required.
- Duration (minutes):** A dropdown menu with options 45, 60, and 75.
- Price:** A text input field containing the value '15.0'.
- Type of Class:** A dropdown menu currently showing 'Vinyasa Yoga'.
- Description (Optional):** A text input field containing the text 'Morning Vinyasa for beginners'.

At the bottom of the form is a red 'Save' button. Below the button, a light blue notification bubble contains the text 'Please enter the class capacity'.

Figure 67. Capacity requirement notification in the Update Class interface.

Figure 67 shows a notification in the "Update Class" interface indicating "Please enter the class capacity." This alert ensures that even during the update process, the system validates required fields to prevent missing essential information, similar to the checks in the add class interface.

The screenshot shows a mobile application interface for updating a class. At the top, there is a status bar with the time 1:59, a settings icon, a battery icon, and a WiFi icon labeled 'WiFi: Connected'. Below the status bar is a header with a red plus icon and the text 'Update class'. The main form has a light pink background and contains the following fields:

- Day of the Week:** A dropdown menu currently showing 'Monday'.
- Time:** A time picker showing 10:00.
- Capacity:** A text input field containing the number '10'.
- Duration (minutes):** A dropdown menu showing '60'.
- Price:** A text input field with the placeholder text 'Price (\$)'. This field is highlighted with a red border.
- Type of Class:** A dropdown menu currently showing 'Vinyasa Yoga'.
- Description (Optional):** A text input field containing the text 'Morning Vinyasa for beginners'.

At the bottom of the form is a red 'Save' button. Below the 'Save' button is a light blue notification bubble with the text 'Please enter the class price'.

Figure 68. Price requirement notification in the Update Class interface.

Figure 68 displays a notification, "Please enter the class price," on the "Update Class" screen. This message ensures that when updating a class, the system validates the entry of essential fields like price, maintaining data completeness and consistency.

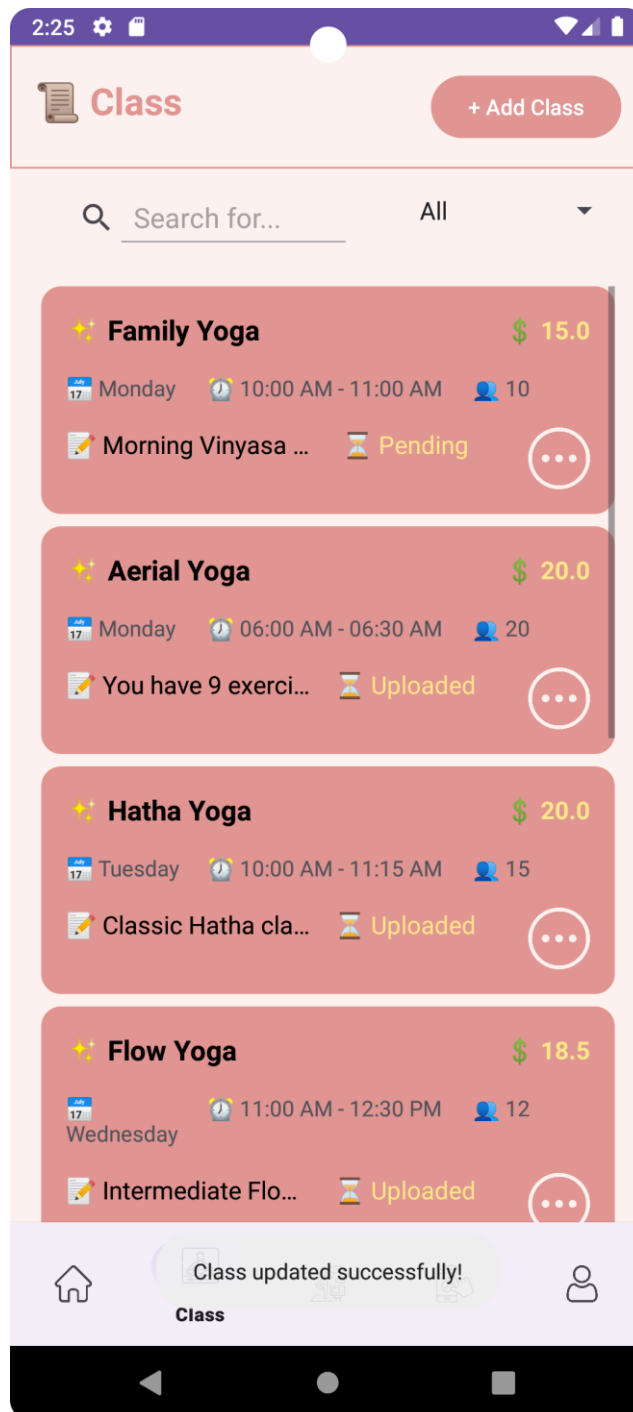


Figure 69. Successful class update notification on the Class Management screen.

Figure 69 shows a notification, "Class updated successfully!", displayed on the Class Management screen. This confirmation message appears after updating a class, reassuring users that their modifications have been successfully saved.

4.1.10 Delete Yoga Classes

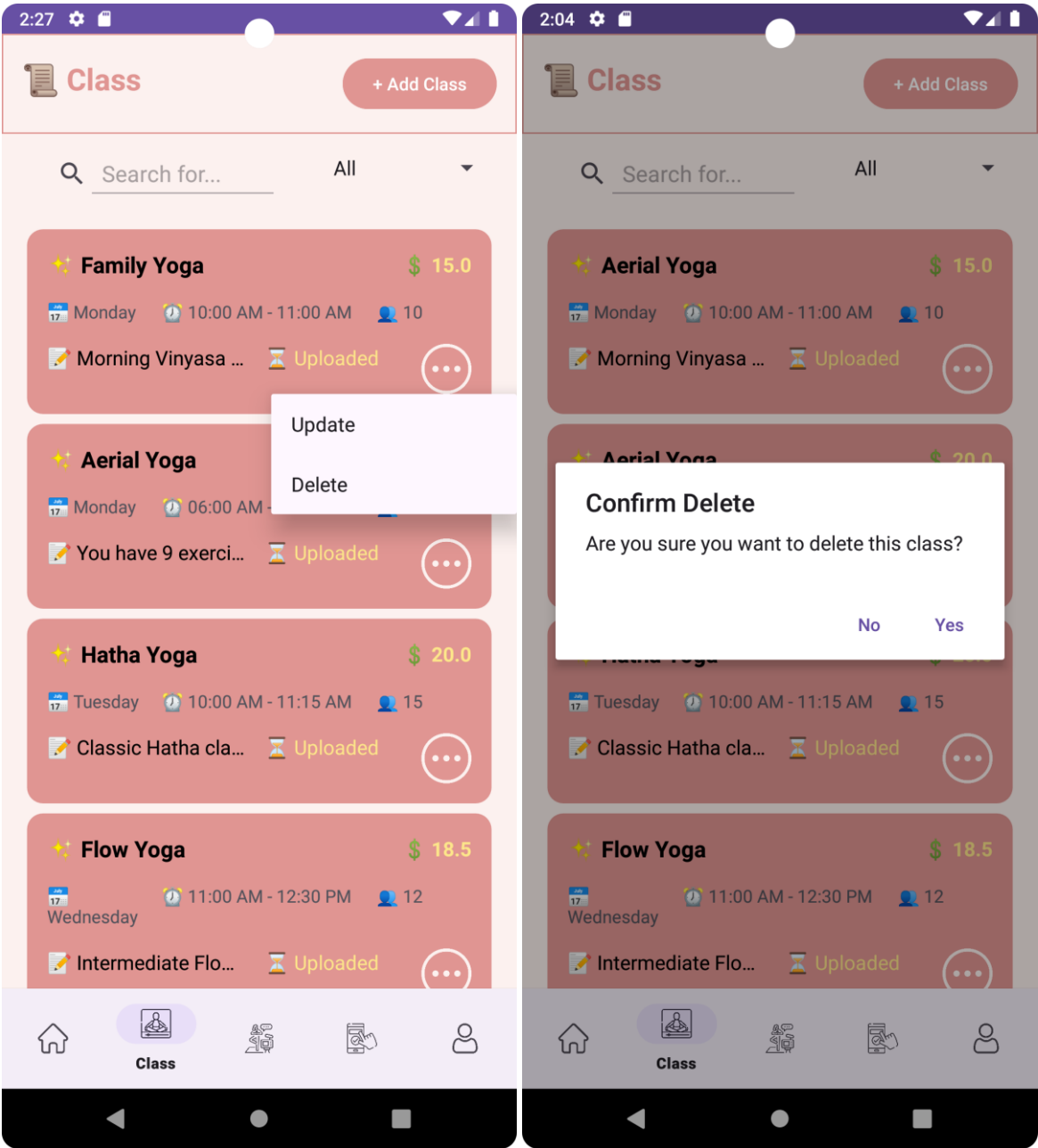


Figure 70. Confirmation dialog for deleting a class in the class management screen.

Figure 70 illustrates the confirmation dialog that appears when a user selects the "Delete" option from the menu on the class management screen. This dialog, labeled "Confirm Delete," prompts the user with a message asking if they are sure they want to delete the selected class. The options "Yes" and "No" ensure that accidental deletions are avoided, offering an additional layer of security for class data management.

4.1.11 Search Yoga Classes

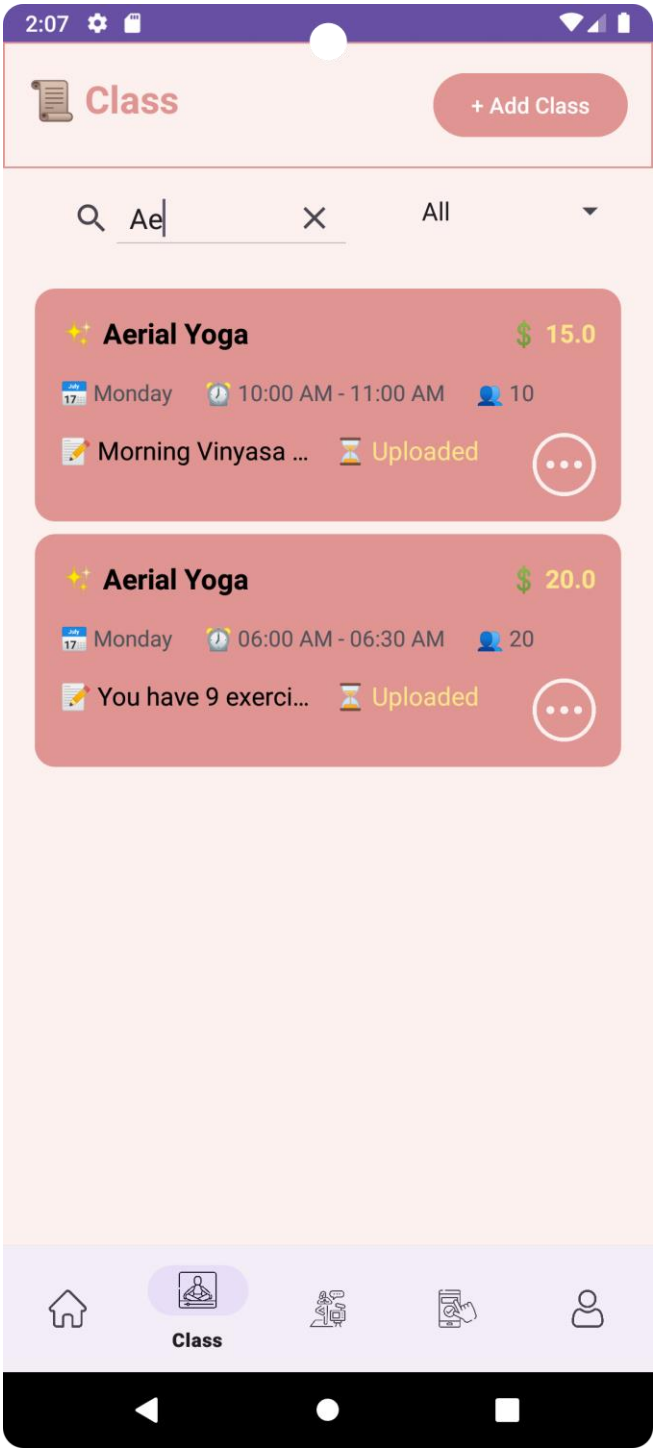


Figure 71. Search functionality for filtering classes by type.

Figure 71 shows the search feature on the class management screen, where the user has entered "Ae" in the search bar to filter classes by type. As a result, only classes matching the query, such as "Aerial Yoga," are displayed. This feature allows users to quickly find specific classes based on keywords, improving efficiency in managing and accessing class information.

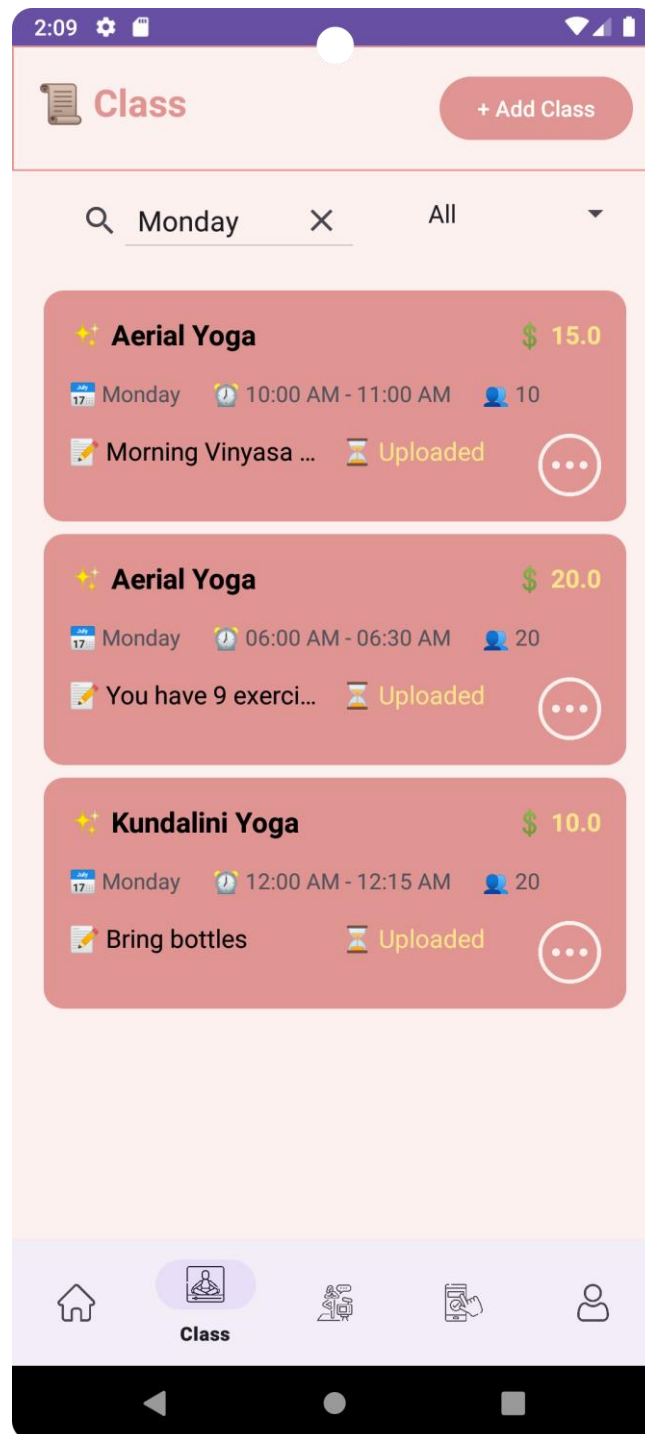


Figure 72. Search functionality for filtering classes by day of the week.

Figure 72 demonstrates the search function on the class management screen, where "Monday" is entered in the search bar to filter classes by day of the week. As a result, only classes scheduled on Monday, such as "Aerial Yoga" and "Kundalini Yoga," are displayed. This feature enables users to quickly locate classes for a specific day, aiding in efficient scheduling and class management.

4.1.12 Filter Yoga Classes

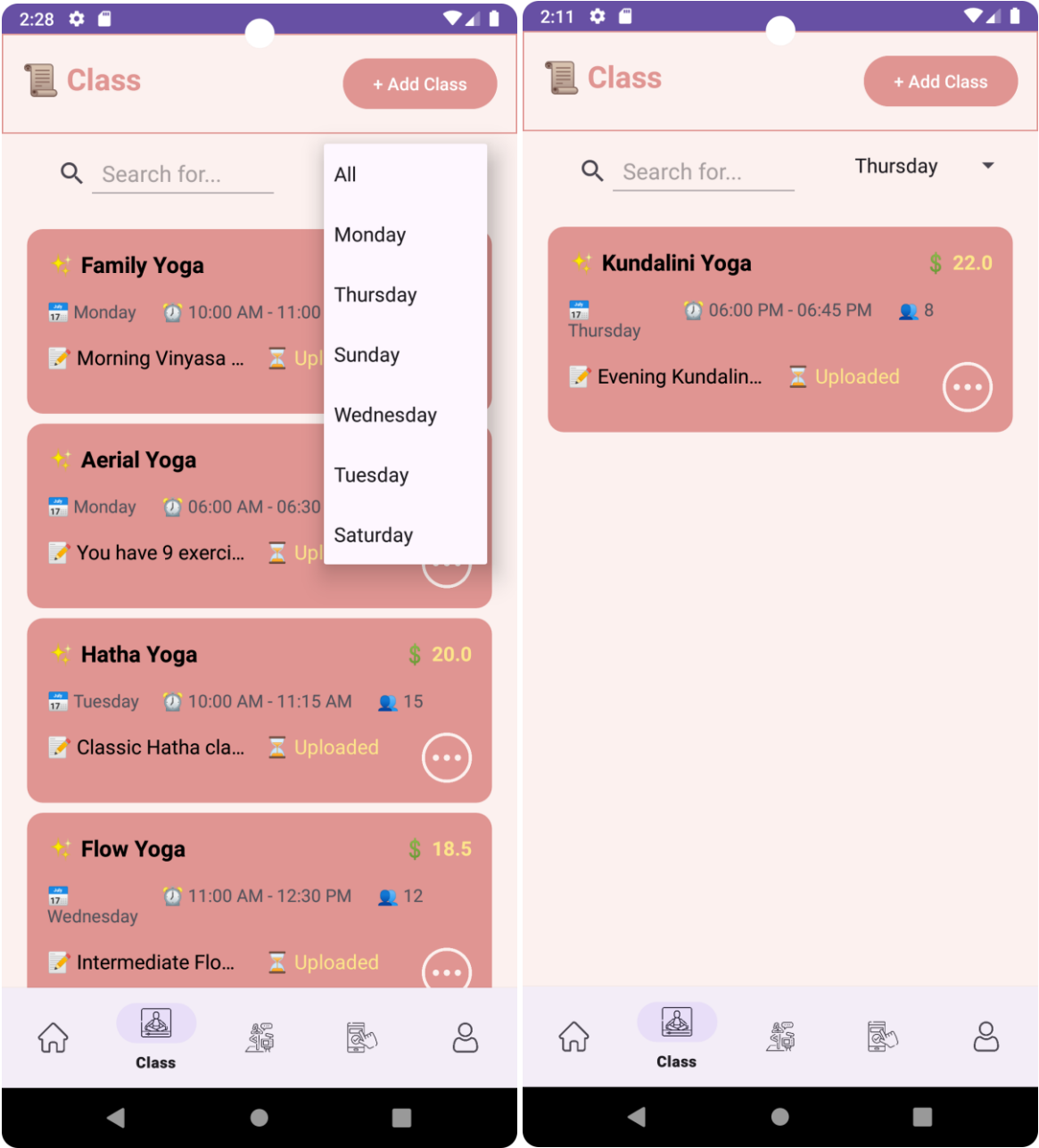


Figure 73. Filtering Yoga Classes by Day of the Week.

Figure 73 illustrates the filtering functionality on the "Class" management screen, allowing users to view yoga classes by specific days of the week. By selecting a day from the dropdown list, only classes scheduled for that day are displayed, providing a focused view for easier scheduling and class management.

4.1.13 Class Instance Management

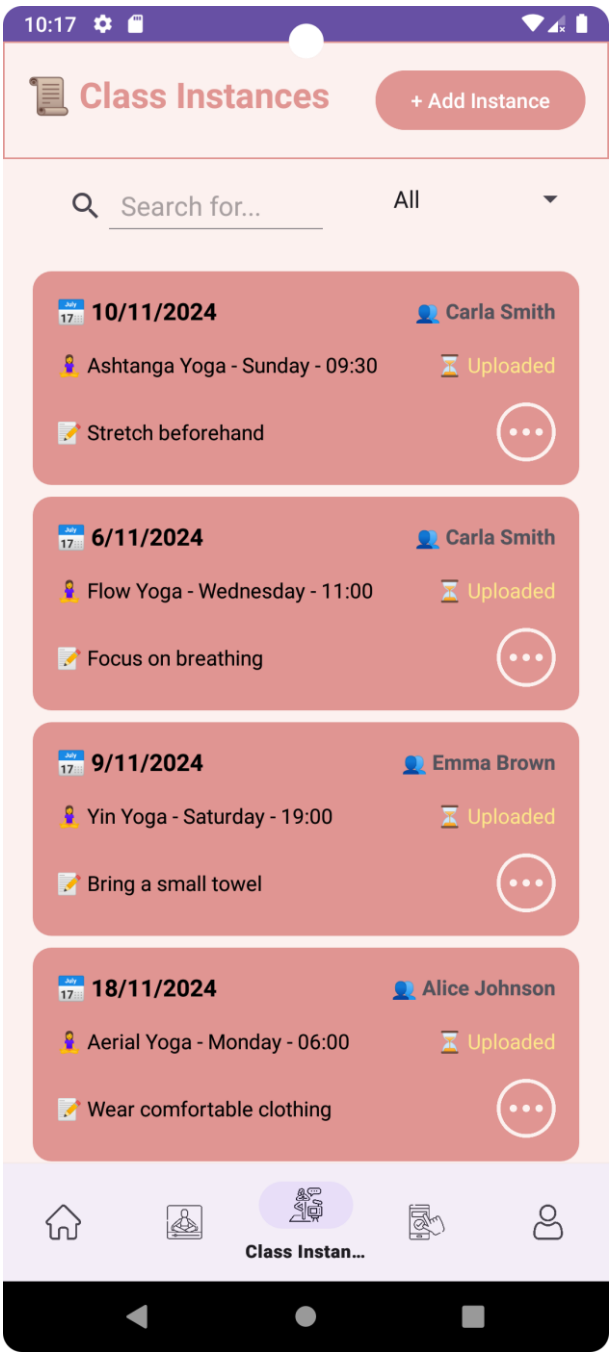


Figure 74. Class Instance Management Screen.

Figure 74 shows the "Class Instances" management screen, where each instance represents a scheduled yoga session. The view displays key details including the date, yoga type, instructor's name, and any additional comments or notes. It also includes an "Uploaded" status indicator and an options menu accessible through a three-dot icon, which provides management actions for each session. This setup allows administrators to effectively oversee and manage each scheduled class, with all relevant information available at a glance for streamlined organization.

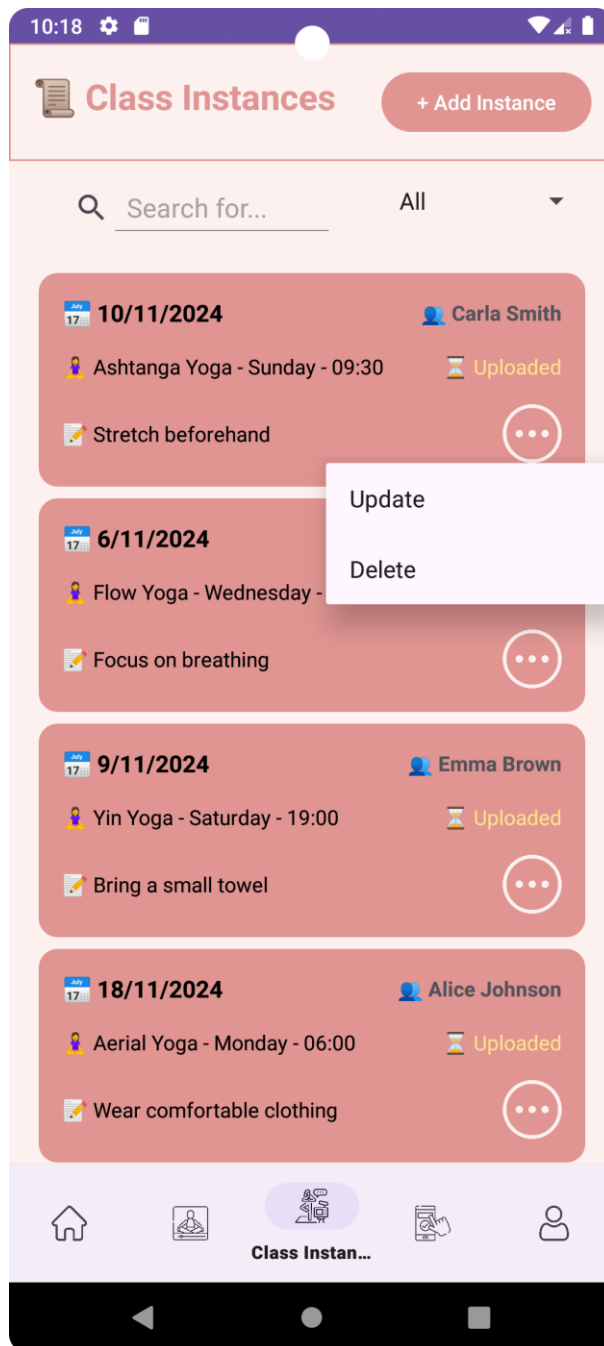
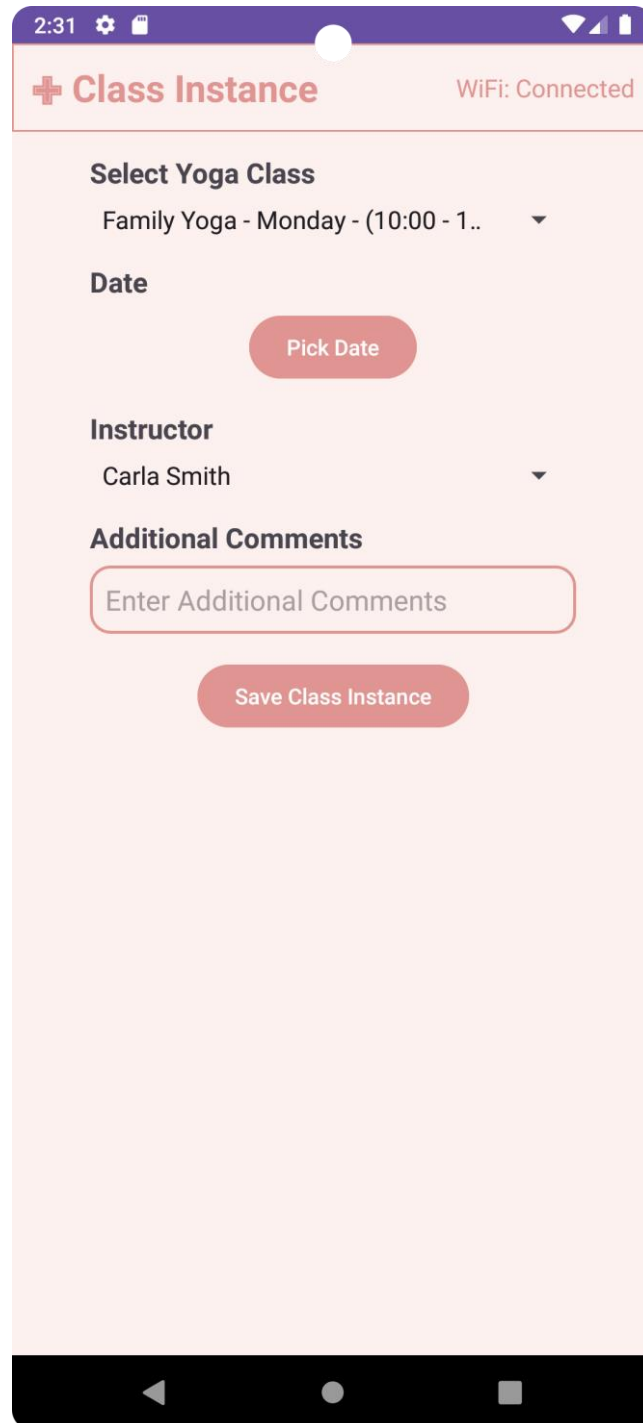


Figure 75. Options Menu for Class Instance Management.

Figure 75 illustrates the options menu for each class instance in the "Class Instances" management screen. Similar to the yoga class management interface, clicking the three-dot button provides "Update" and "Delete" options, allowing administrators to modify or remove specific class slots as needed.

4.1.14 Add Class Instance



The image shows a mobile application interface for adding a class instance. At the top, there is a purple status bar with the time 2:31, a settings icon, a folder icon, and a white circle. Below this is a red header bar with a white plus icon and the text "Class Instance" in white, and "WiFi: Connected" on the right. The main content area has a light pink background. It starts with the heading "Select Yoga Class" followed by a dropdown menu showing "Family Yoga - Monday - (10:00 - 1..". Below this is the heading "Date" followed by a red button labeled "Pick Date". Then, the heading "Instructor" is followed by a dropdown menu showing "Carla Smith". Below that is the heading "Additional Comments" followed by a text input field with the placeholder "Enter Additional Comments". At the bottom of the form is a red button labeled "Save Class Instance". The bottom of the screen shows a black navigation bar with three icons: a back arrow, a circle, and a square.

Figure 76. Add Class Instance Interface.

Figure 76 shows the "Add Class Instance" interface, where administrators can create a new slot for an existing yoga class. This screen allows the selection of a yoga class, picking a date, choosing an instructor, and entering additional comments. This setup streamlines the process of scheduling class instances for better management.

The screenshot shows a mobile application interface for adding a class instance. At the top, the status bar displays the time 2:31, a settings icon, a document icon, and a white circle. The app header is purple with a red plus icon and the text 'Class Instance' on the left, and 'WiFi: Connected' on the right. The main form area has a light pink background. It contains the following elements: a section header 'Select Yoga Class', a dropdown menu showing 'Family Yoga - Monday - (10:00 - 1..', a 'Date' label, a red 'Pick Date' button, an 'Instructor' label, a dropdown menu showing 'Carla Smith', an 'Additional Comments' label, a text input field with the placeholder 'Enter Additional Comments', and a red 'Save Class Instance' button. At the bottom, a light green notification bubble contains the text 'Date and Instructor are required!'. The bottom of the screen shows a black navigation bar with three icons: a back arrow, a circle, and a square.

Figure 77. Notification for Required Fields in Add Class Instance.

Figure 77 displays a notification, "Date and Instructor are required!" on the "Add Class Instance" screen, indicating that these fields must be completed before saving. This reminder helps ensure essential scheduling details are filled in, maintaining the accuracy of the class instance setup.

The screenshot shows a mobile application interface for adding a class instance. At the top, there is a purple header bar with the text '+ Class Instance' and 'WiFi: Connected'. Below the header, the screen is divided into sections. The first section is 'Select Yoga Class', which has a dropdown menu showing 'Family Yoga - Monday - (10:00 - 1..'. The second section is 'Date', which has a 'Pick Date' button. The third section is 'Instructor', which has a dropdown menu showing 'Carla Smith'. The fourth section is 'Additional Comments', which has a text input field with the placeholder 'Enter Additional Comments'. Below the input field is a 'Save Class Instance' button. At the bottom of the screen, there is a notification box with the text 'Please select a valid Monday that is not in the past.'

Figure 78. Notification for Incorrect Day Selection in Add Class Instance.

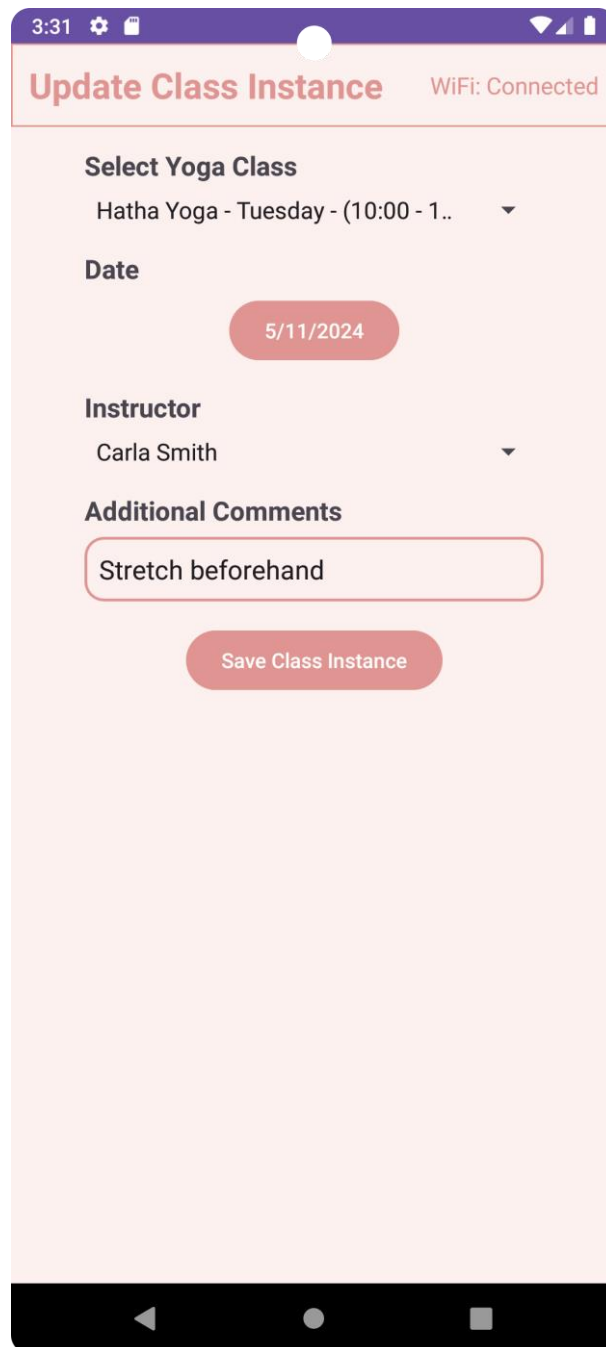
Figure 78 displays a notification, "Please select a valid Monday that is not in the past," on the "Add Class Instance" screen, indicating that the selected date does not match the required day of the week for the chosen yoga class. This message ensures that the scheduled date aligns with the class's set weekly schedule, maintaining consistency.

The screenshot shows a mobile application interface for adding a class instance. At the top, the status bar displays the time 3:06, a settings icon, a battery icon, and a WiFi icon labeled 'WiFi: Connected'. The app header is a purple bar with a white plus icon and the text 'Class Instance'. Below the header, the screen has a light pink background. The form includes a 'Select Yoga Class' dropdown menu with 'Ashtanga Yoga - Sunday - (09:30 -..' selected. Below this is a 'Date' field with a red pill-shaped button containing '3/11/2024'. The 'Instructor' dropdown menu shows 'Carla Smith'. An 'Additional Comments' text input field contains the text 'Bring a ball'. A red pill-shaped button labeled 'Save Class Instance' is positioned below the comments field. At the bottom of the form, a light blue rounded rectangle contains the message 'Instructor is not available for this time slot.' The bottom of the screen shows a black navigation bar with three white icons: a back arrow, a circle, and a square.

Figure 79. Notification for Instructor Unavailability in Add Class Instance.

Figure 79 displays a notification, "Instructor is not available for this time slot," on the "Add Class Instance" screen, indicating that the selected instructor is already assigned to another session during this time. This message helps prevent scheduling conflicts by ensuring the instructor's availability.

4.1.15 Update Class Instance



The image shows a mobile application interface for updating a class instance. At the top, a purple status bar displays the time 3:31, a settings icon, a document icon, and a white circle. Below this, a red header bar contains the title "Update Class Instance" and the status "WiFi: Connected". The main content area has a light pink background and includes the following fields: "Select Yoga Class" with a dropdown menu showing "Hatha Yoga - Tuesday - (10:00 - 1.."; "Date" with a red button showing "5/11/2024"; "Instructor" with a dropdown menu showing "Carla Smith"; and "Additional Comments" with a text input field containing "Stretch beforehand". A red button labeled "Save Class Instance" is positioned at the bottom of the form. The interface is framed by a black mobile navigation bar at the very bottom.

Figure 80. Update Class Instance Interface.

Figure 80 shows the "Update Class Instance" interface, which allows users to modify the details of an existing class instance, such as the selected yoga class, date, instructor, and additional comments. This screen is similar to the "Add Class Instance" interface, but it pre-fills existing data to facilitate updating specific fields without re-entering all information.

The screenshot shows a mobile application interface titled "Update Class Instance" with a status bar at the top indicating 4:01, settings, and WiFi connection. The interface includes a dropdown menu for "Select Yoga Class" currently set to "Flow Yoga - Wednesday - (11:00 -..)", a "Date" section with a "Select Date" button, and an "Instructor" dropdown menu currently set to "Carla Smith". Below these is an "Additional Comments" text input field containing the text "Stretch beforehand". A "Save Class Instance" button is positioned below the comments field. At the bottom of the screen, a light gray notification bubble displays the message "Date and Instructor are required!".

Figure 81. Error Handling in Update Class Instance Interface.

Figure 81 demonstrates the "Update Class Instance" interface, showing that error handling is enforced similarly to the "Add Class Instance" screen. Here, a notification "Date and Instructor are required!" is displayed, ensuring that essential fields are completed before updating. This consistent validation improves data integrity across both addition and update processes.

The screenshot displays a mobile application interface titled "Update Class Instance" with a status bar at the top showing the time 3:32 and "WiFi: Connected". The interface includes several form fields: "Select Yoga Class" with a dropdown menu showing "Hatha Yoga - Tuesday - (10:00 - 1..", "Date" with a date picker showing "5/11/2024", "Instructor" with a dropdown menu showing "Carla Smith", and "Additional Comments" with a text input field containing "Stretch beforehand". A "Save Class Instance" button is located below the comments field. At the bottom, a light gray message box contains the text: "Please select a valid Tuesday that is not in the past."

Figure 82. Date Validation Error in Update Class Instance Interface.

Figure 82 shows the "Update Class Instance" interface with a validation message, "Please select a valid Tuesday that is not in the past." This message appears when the selected date does not match the required day of the week for the class or is set in the past. This validation helps users select appropriate future dates that align with the class schedule.

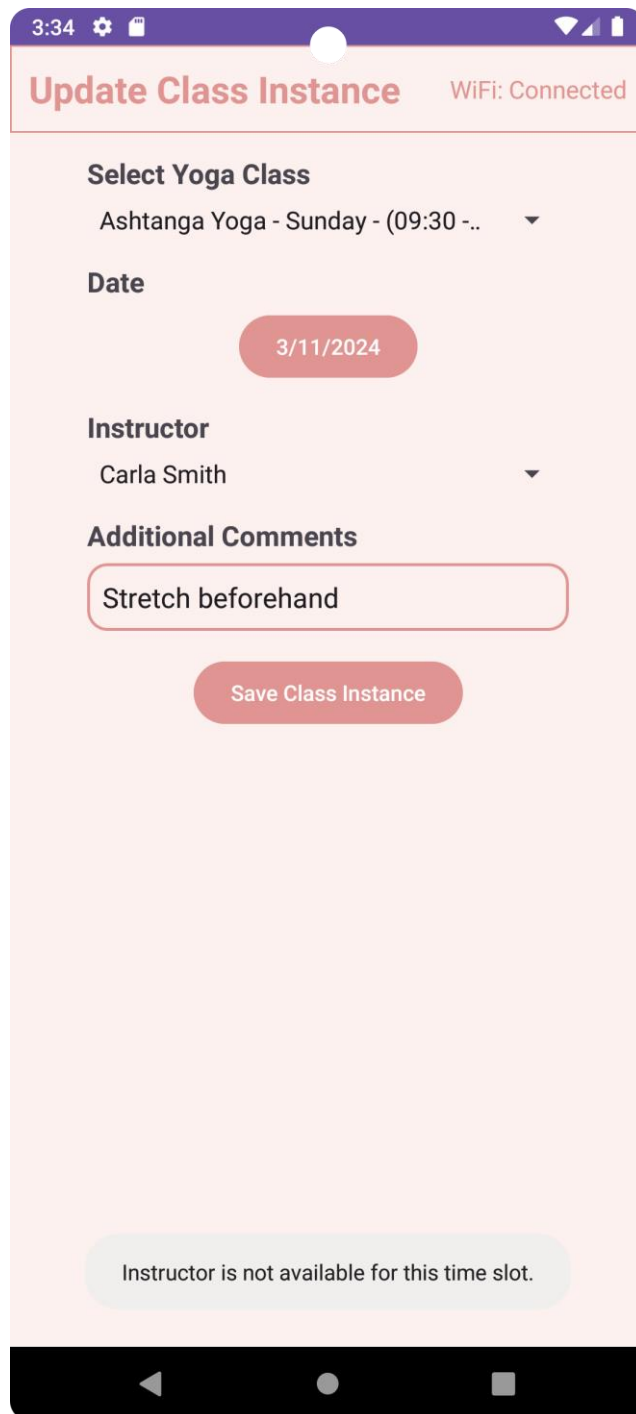


Figure 83. Instructor Availability Conflict in Update Class Instance Interface.

Figure 83 shows the "Update Class Instance" interface with a validation message, "Instructor is not available for this time slot." This message is displayed when the selected instructor already has another class scheduled at the same time, helping to avoid scheduling conflicts and ensuring instructor availability.

4.1.16 Delete Class Instance

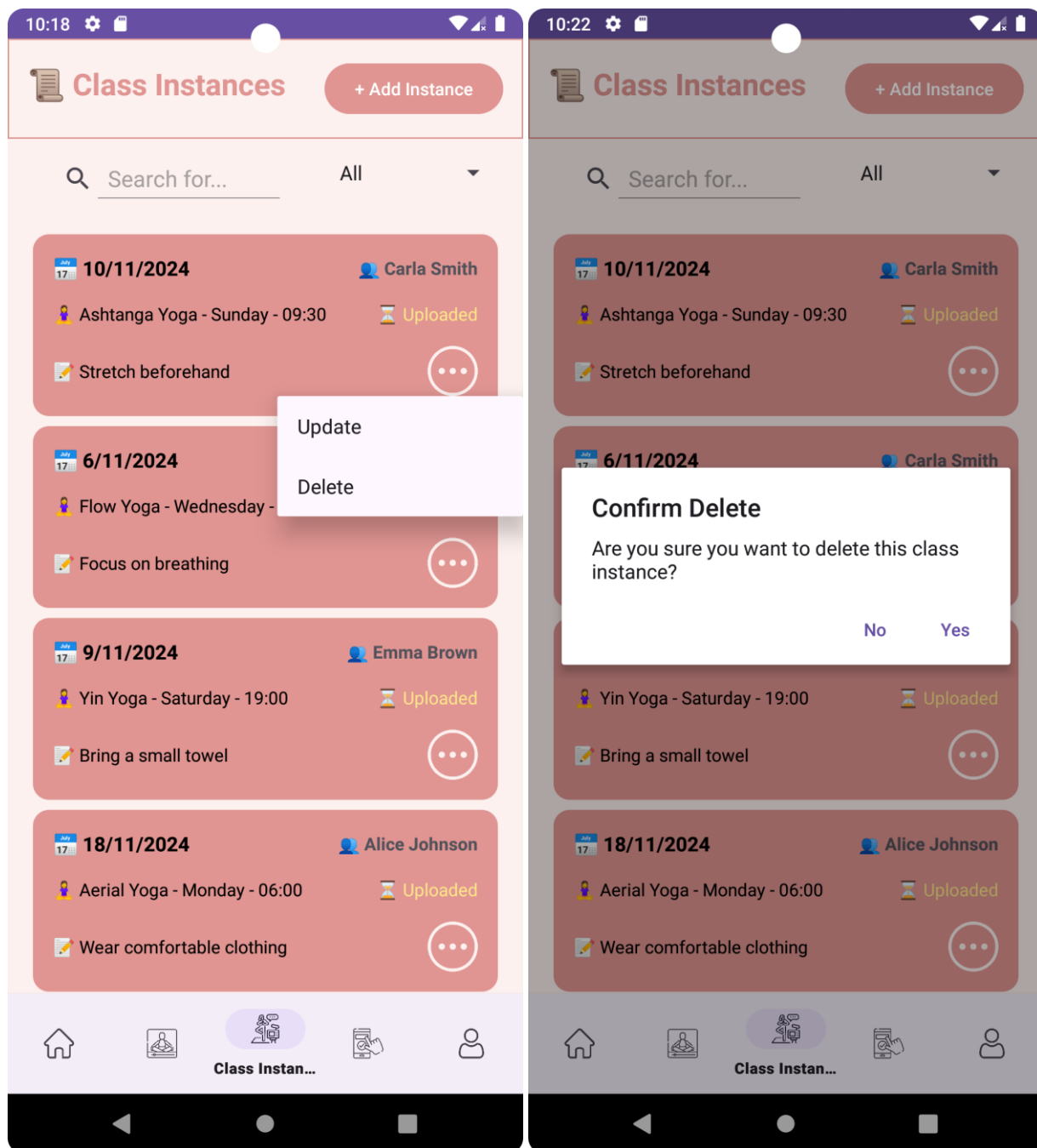


Figure 84. Options and confirmation dialog for deleting a class instance.

Figure 84 illustrates the delete functionality in the Class Instances management interface. Tapping the three-dot icon beside a class instance reveals options for updating or deleting the instance. Selecting "Delete" triggers a confirmation dialog, prompting the user to confirm their decision, helping to prevent accidental deletions.

4.1.17 Search and Advanced Search for Class Instance

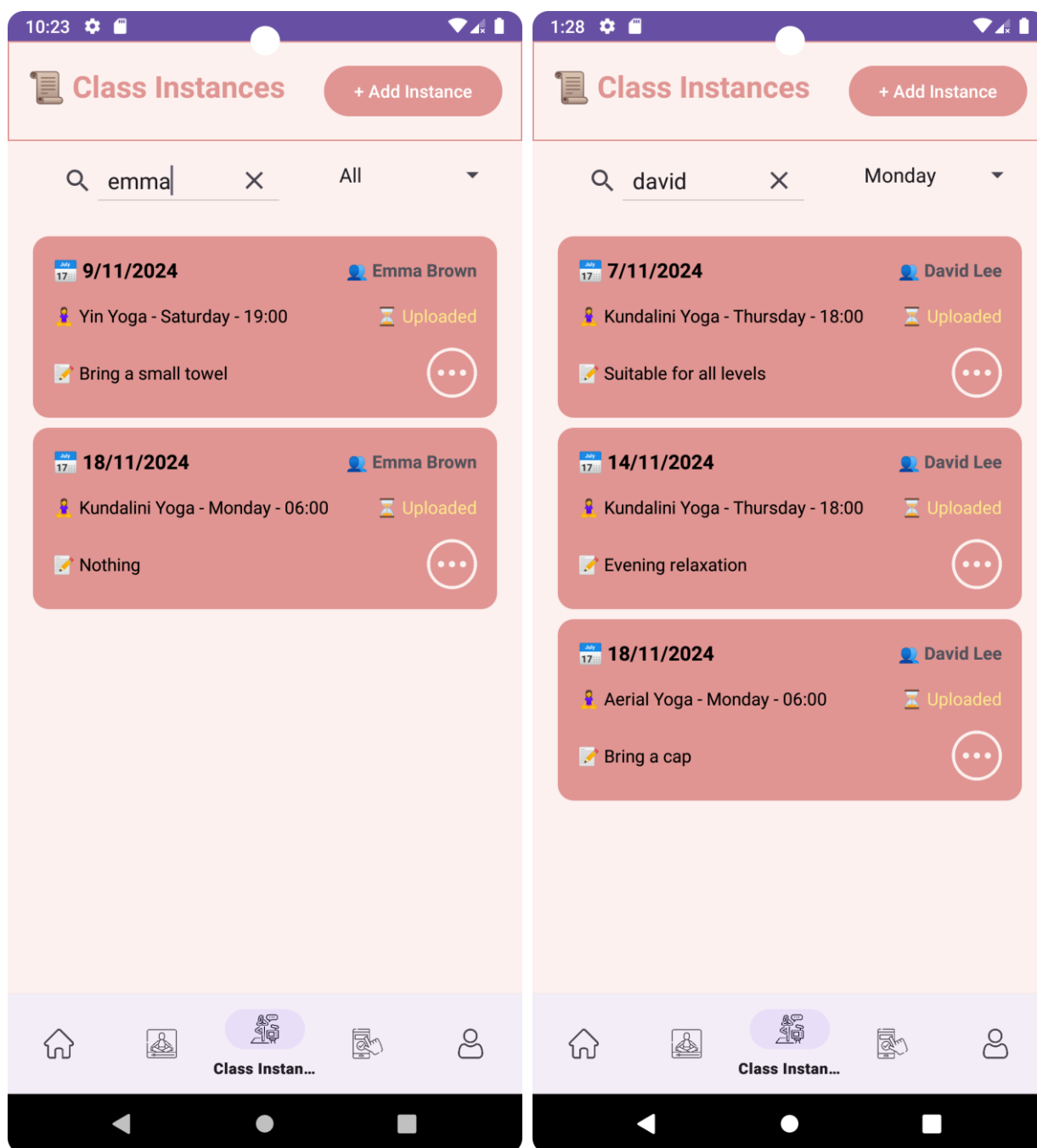


Figure 85. Comprehensive search function in Class Instances management.

Figure 85 illustrates the comprehensive search functionality in the Class Instances management interface. By typing "Emma" or "David," the system filters results based on fields like instructor name, class name, date, and comments. Advanced search also allows for combining the search bar with days of the week, such as "Monday," enabling precise results and enhancing usability.

4.1.18 Filter Class Instances

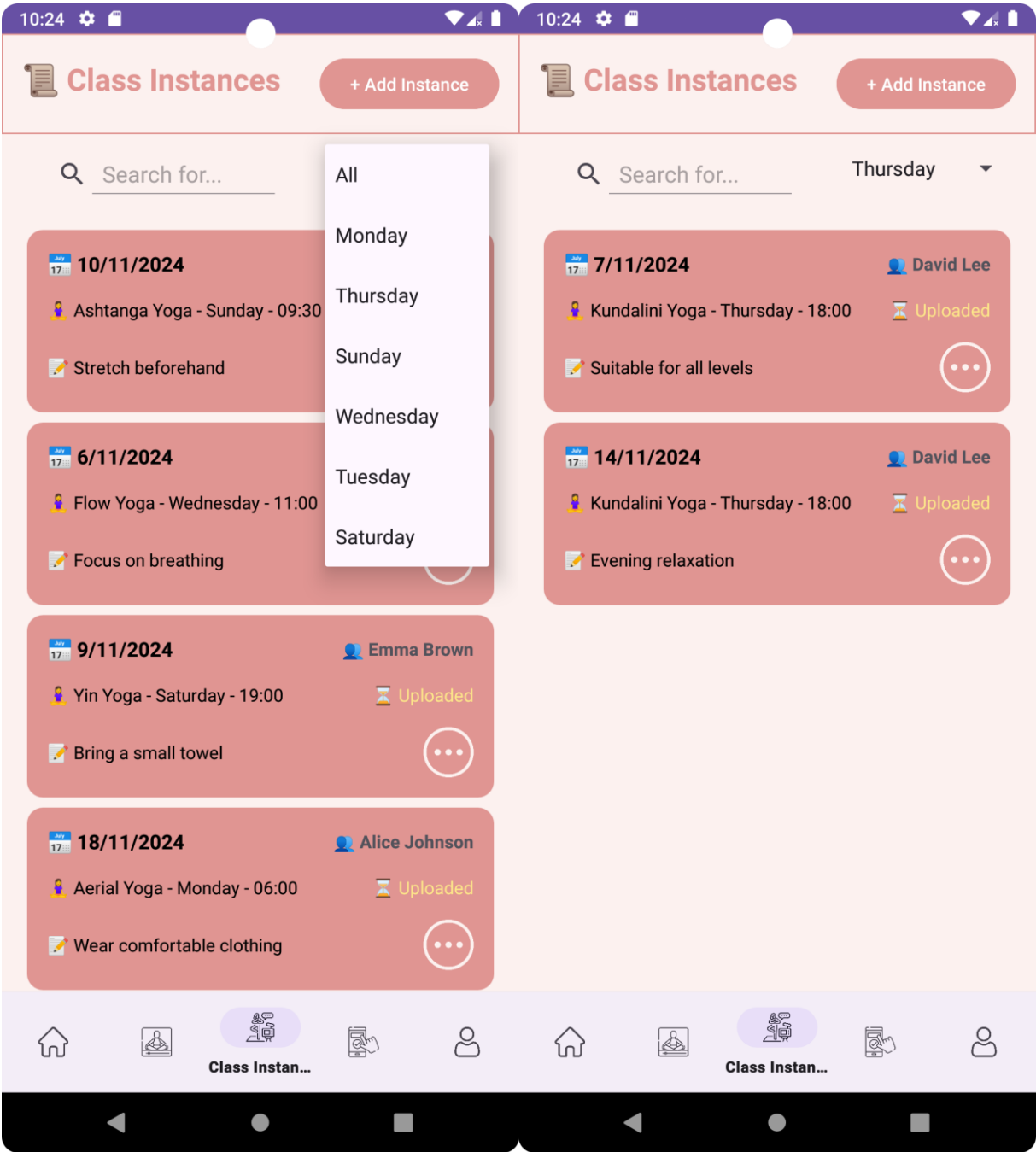


Figure 86. Class instances filtered by selected day of the week for quick access.

Figure 86 shows the filtering functionality for class instances. The dropdown allows users to select a day of the week, displaying only matching classes. The second image illustrates filtered results for "Thursday." This feature simplifies navigation and focuses on relevant schedules.

4.1.19 Account Management

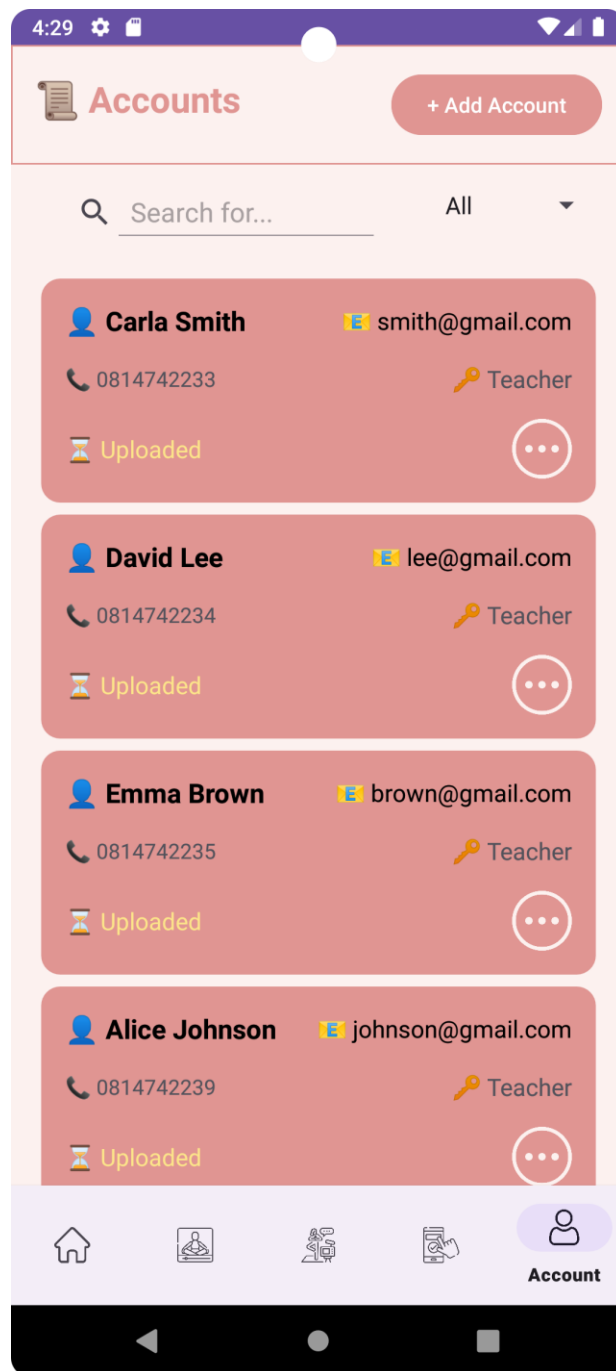


Figure 87. Account Management interface for viewing and managing user accounts.

Figure 87 shows the Account Management interface, which provides an organized view of all registered user accounts, including their name, contact information, email, role, and current status. Each account entry features an icon indicating the role (e.g., Teacher), and an "Uploaded" status shows that the account data has been synchronized. This interface enables administrators to oversee user accounts efficiently, with options for further actions via the three-dot menu on each entry, such as updating or deleting accounts.

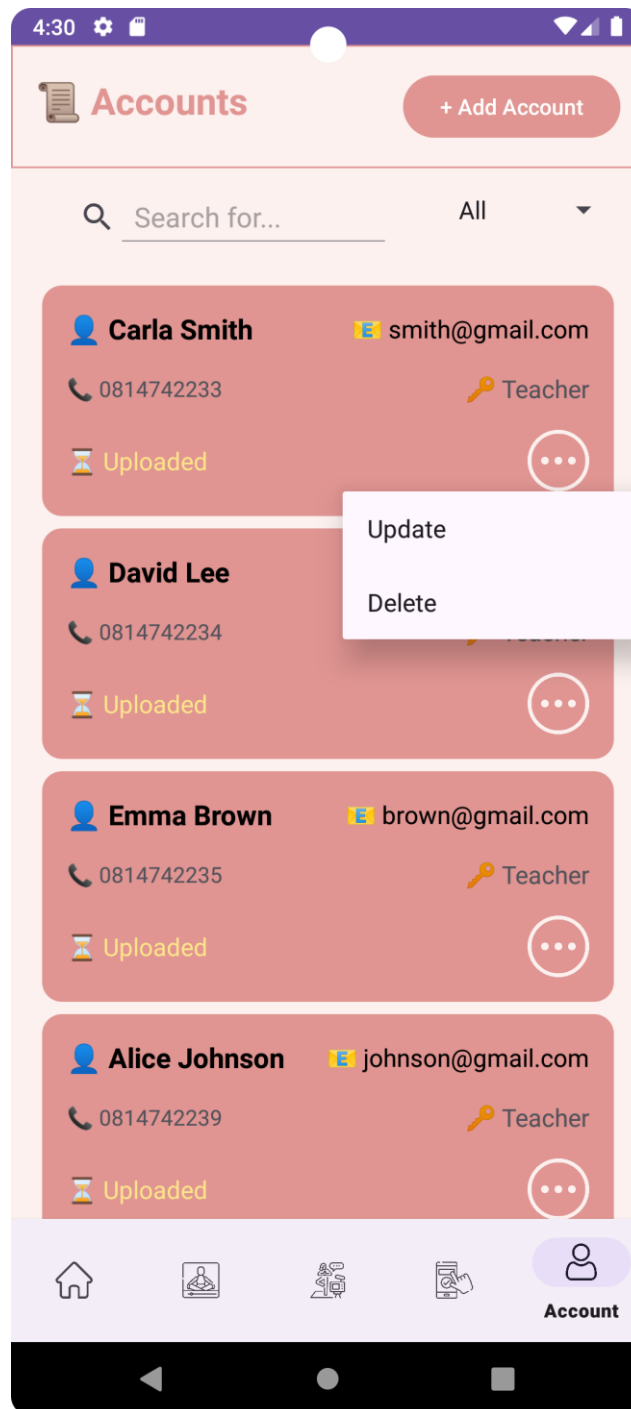
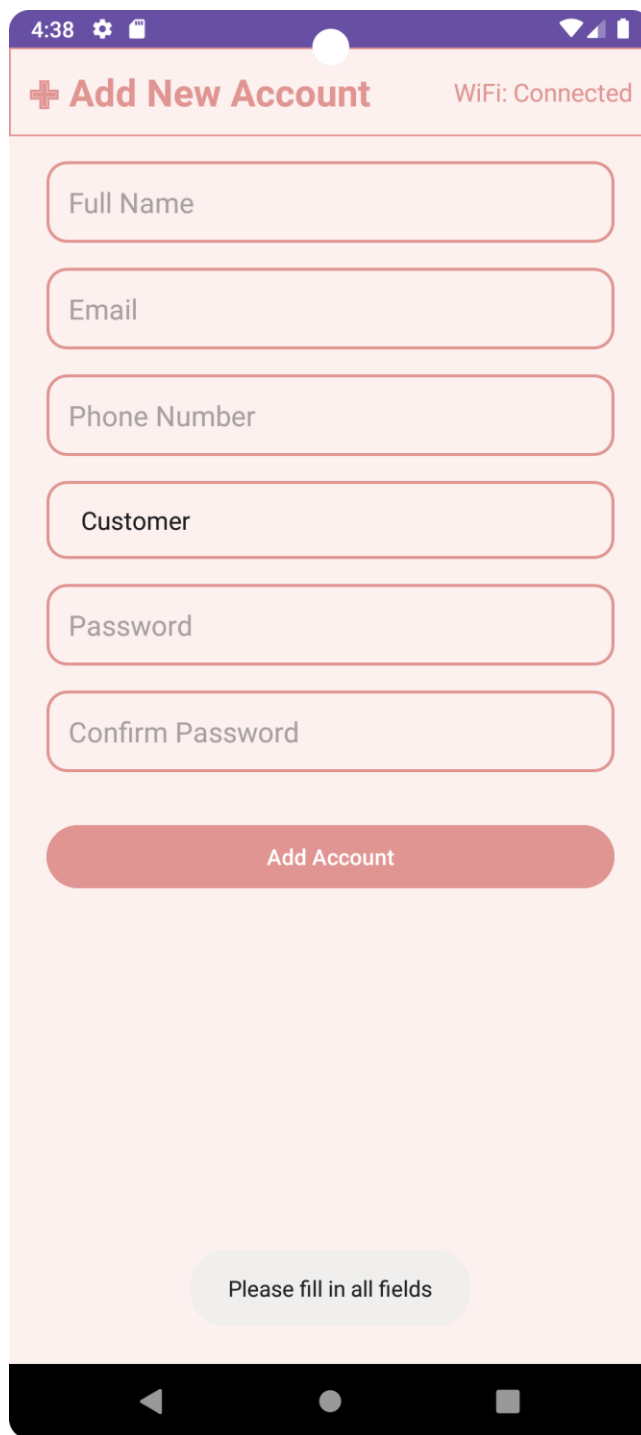


Figure 88. Options for updating or deleting accounts in Account Management.

Figure 88 displays the options menu in the Account Management interface, which appears when selecting the three-dot icon next to a user account. Similar to the Yoga Class and Class Instance management screens, this menu provides options to update or delete the selected account, giving administrators the flexibility to manage user information efficiently.

4.1.20 Add Account



The image shows a mobile application interface for adding a new account. At the top, there is a status bar with the time 4:38, a settings icon, a battery icon, and a WiFi icon. Below the status bar, the title bar contains a red plus icon, the text "Add New Account", and the text "WiFi: Connected". The main form area has a light pink background and contains six input fields: "Full Name", "Email", "Phone Number", "Customer", "Password", and "Confirm Password". Each field is a rounded rectangle with a red border. Below the input fields is a red button with the text "Add Account". At the bottom of the form area, there is a light gray rounded rectangle with the text "Please fill in all fields". The bottom of the screen shows a black navigation bar with three icons: a back arrow, a home circle, and a recent apps square.

Figure 89. Add New Account interface with required fields notification.

Figure 89 shows the "Add New Account" interface, where administrators can input essential information for creating a new user account, including full name, email, phone number, role selection, and password fields. The notification "Please fill in all fields" appears if any required fields are left blank, ensuring that all necessary details are provided before creating the account.

4:44

+ Add New Account WiFi: Connected

Khoi Lieu

khoivl.net@gmail.com

0814742238

Teacher

.....

.....

Add Account

Email already exists!

Figure 90. Add New Account interface with duplicate email notification.

Figure 90 shows the "Add New Account" interface where, upon attempting to create an account, a notification "Email already exists!" appears. This message prevents the duplication of email addresses, ensuring each user account has a unique email for identification and access.

4:44

+ Add New Account WiFi: Connected

Khoi Lieu

khoilieuct03@gmail.com

0814742238

Teacher

.....

.....

Add Account

Password must be at least 8 characters

Figure 91. Add New Account interface with password length requirement notification.

Figure 91 displays the "Add New Account" interface where a notification, "Password must be at least 8 characters," appears. This message informs the user that the password entered does not meet the minimum length requirement, enforcing a standard for account security.

The image shows a mobile application interface for adding a new account. At the top, there is a status bar with the time 4:45, a settings icon, a battery icon, and a signal strength indicator. Below the status bar, the app header has a red background with a white plus icon and the text "Add New Account" in white. To the right of the header, it says "WiFi: Connected". The main form area has a light pink background and contains several input fields with rounded corners and red borders. The fields are: a name field containing "Khoi Lieu", an email field containing "khoilieu03@gmail.com", a phone number field containing "0814742238", a role field containing "Teacher", a password field containing ".....", and a confirm password field containing ".....|". Below these fields is a red button with the text "Add Account". At the bottom of the form, there is a light gray notification bubble with the text "Passwords do not match". The bottom of the screen shows a black navigation bar with three white icons: a back arrow, a home circle, and a recent apps square.

4:45

+ Add New Account WiFi: Connected

Khoi Lieu

khoilieu03@gmail.com

0814742238

Teacher

.....

.....|

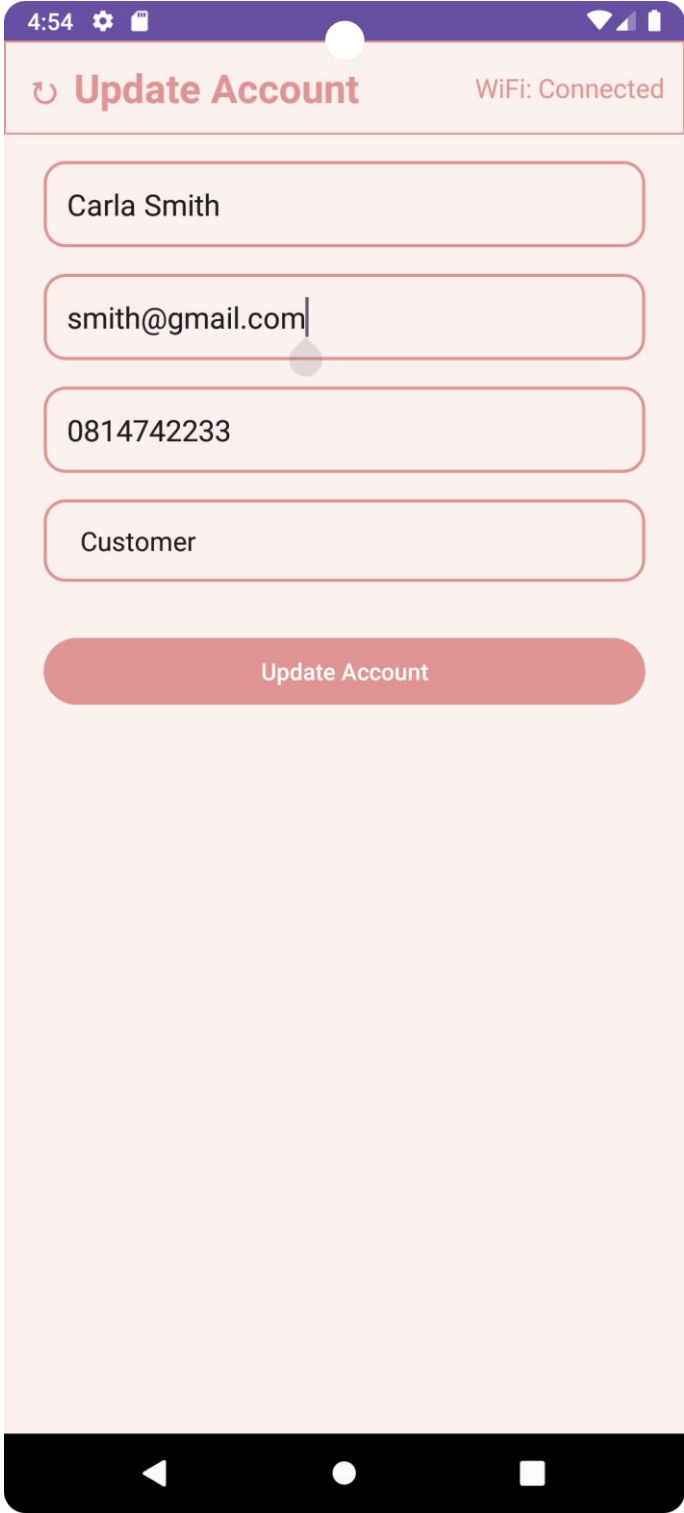
Add Account

Passwords do not match

Figure 92. Add New Account interface with password mismatch notification.

Figure 92 shows the "Add New Account" interface, displaying a notification, "Passwords do not match," at the bottom. This message informs the user that the entered passwords in the "Password" and "Confirm Password" fields do not match, prompting them to correct the input before proceeding.

4.1.21 Update Account



The image shows a mobile application interface for updating an account. At the top, there is a status bar with the time 4:54, a settings icon, a battery icon, and a WiFi icon. Below the status bar, the title "Update Account" is displayed in a red font, accompanied by a refresh icon. To the right of the title, it says "WiFi: Connected". The main form consists of four input fields stacked vertically, each with a red border and rounded corners. The first field contains the name "Carla Smith". The second field contains the email address "smith@gmail.com" with a cursor at the end. The third field contains the phone number "0814742233". The fourth field contains the role "Customer". Below these fields is a large, rounded red button with the text "Update Account" in white. The bottom of the screen shows a black navigation bar with three white icons: a back arrow, a home circle, and a recent apps square.

Figure 93. Update Account interface without password field.

Figure 93 displays the "Update Account" interface, where users can modify details like name, email, phone number, and role. Notably, there is no field for updating the password, ensuring additional security for customer accounts by limiting sensitive changes.

The image shows a mobile application interface for updating an account. At the top, there is a status bar with the time 4:34, a settings icon, a battery icon, and a WiFi icon. Below the status bar, the title bar contains a refresh icon, the text "Update Account", and the status "WiFi: Connected". The main content area has four text input fields: "Carla Smith", "smith@gmail.com", "0814742233", and "Customer". The "Customer" field is disabled, indicated by a light gray background. Below the fields is a red button labeled "Update Account". At the bottom, there is a black navigation bar with three icons: a back arrow, a home circle, and a recent apps square.

Figure 94. Update Account interface with disabled role field.

Figure 94 shows the "Update Account" interface where the role field is disabled. This restriction occurs because the account currently has the "Teacher" role and is assigned to manage a class instance. Changing the role to "Customer" would be impractical, as a customer account cannot serve as an instructor for a class instance. This design prevents role changes that could disrupt the application's structure and responsibilities.

The image shows a mobile application interface for updating an account. At the top, a purple status bar displays the time 4:57, a settings icon, a battery icon, and a white circle. Below this, a red header bar contains a refresh icon, the text "Update Account", and "WiFi: Connected". The main content area is a light pink rectangle containing four rounded rectangular input fields. The first field is labeled "Full Name" and is empty. The second field contains the email address "khoivl.net@gmail.com". The third field contains the phone number "0814742233". The fourth field is labeled "Customer" and is empty. Below these fields is a red rounded rectangular button labeled "Update Account". At the bottom of the pink area is a light blue rounded rectangular button labeled "Please fill in all fields". The entire interface is set against a white background with a black Android-style navigation bar at the very bottom.

Figure 95. Update Account interface with empty required fields warning.

Figure 95 displays the "Update Account" interface, where a notification, "Please fill in all fields," appears because required fields are left empty. This prompt ensures that users complete all necessary information before updating the account, helping maintain accurate and complete account details.

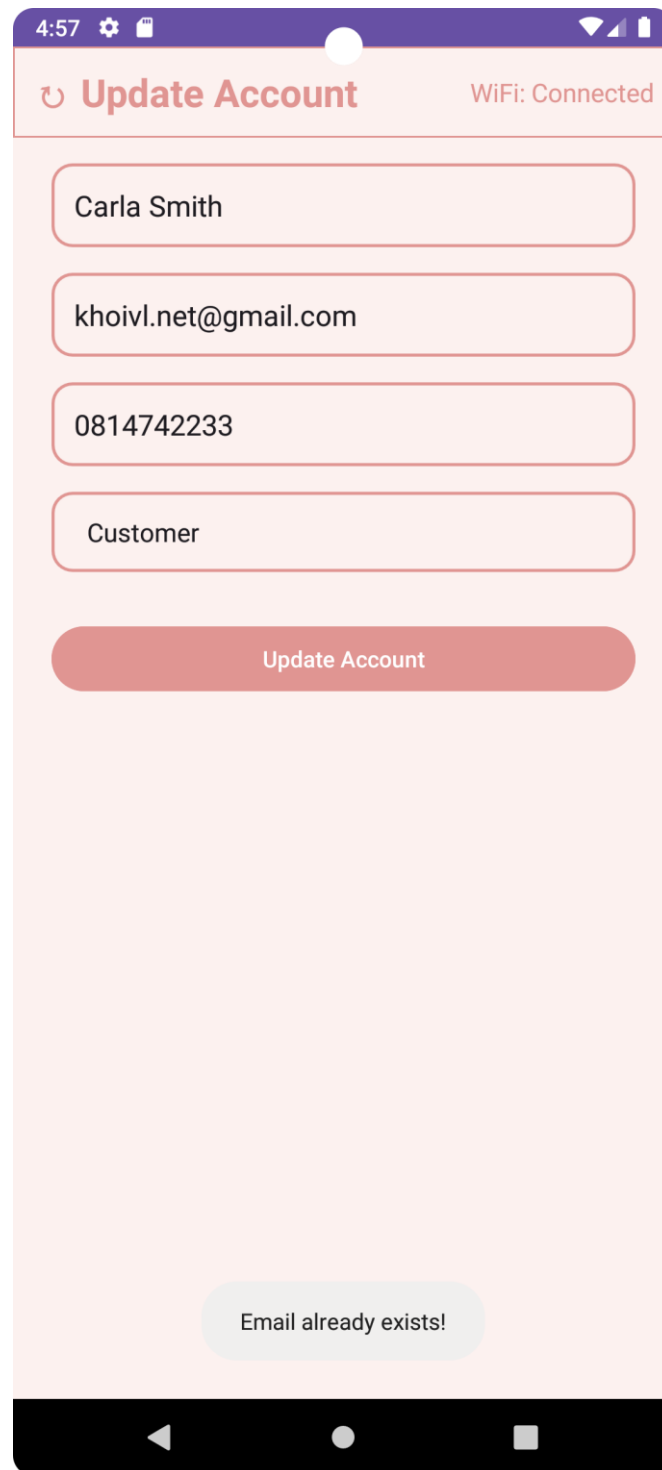


Figure 96. Duplicate email warning in Update Account.

Figure 96 shows the "Update Account" interface displaying the warning "Email already exists!" This indicates that the entered email is already associated with another account. The system maintains consistency in handling exceptions, as this validation also exists in the "Add New Account" interface, ensuring that emails remain unique across accounts.

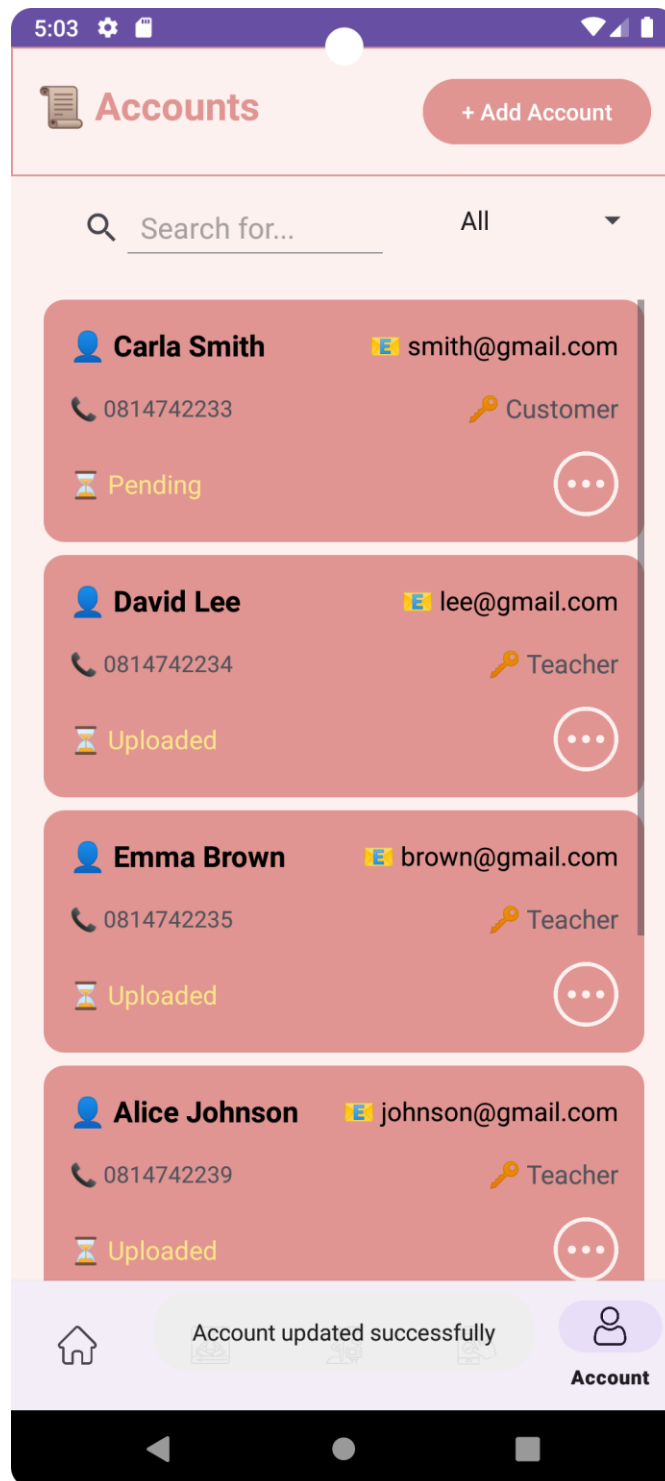


Figure 97. Successful account update notification.

Figure 97 shows the "Accounts" management interface displaying a notification, "Account updated successfully." This confirms that the account details have been modified without issues. Consistent with the application's design, this interface ensures users are informed of successful updates, maintaining clear feedback throughout the user experience.

4.1.22 Delete Account

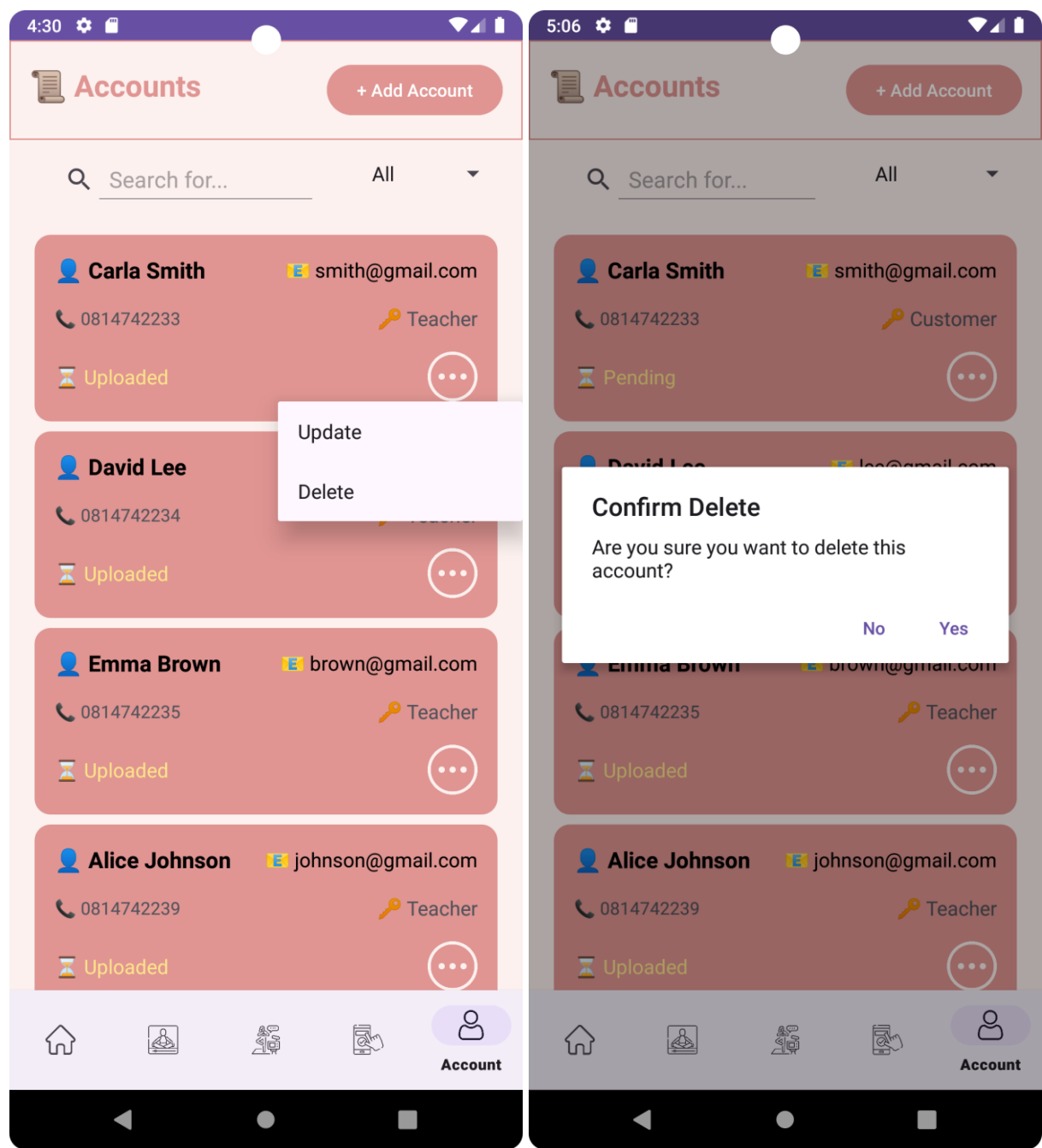


Figure 98. Delete Account Confirmation Dialog

Figure 98 illustrates the account management interface, specifically highlighting the delete function. When the three-dot menu beside an account is selected, options like "Update" and "Delete" appear. Upon choosing "Delete," a confirmation dialog box is displayed, asking, "Are you sure you want to delete this account?" with "No" and "Yes" options. This feature ensures users must confirm their decision before permanently removing an account, providing an extra layer of data protection.

4.1.23 Search Account

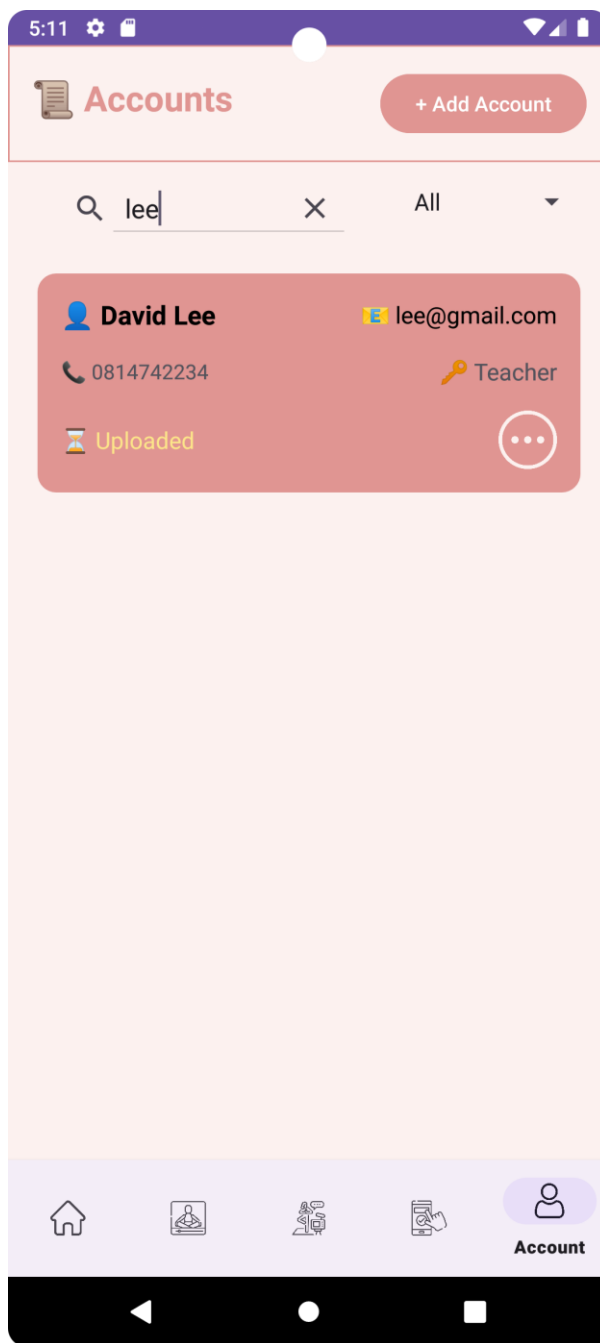


Figure 99. Search Account Functionality

Figure 99 demonstrates the account management screen with the search functionality in use. The user has entered "lee" into the search bar, filtering the list to display only the account matching the query, "David Lee." This feature enhances usability by allowing users to quickly locate specific accounts based on name or other identifying information.

4.1.24 Filter Account

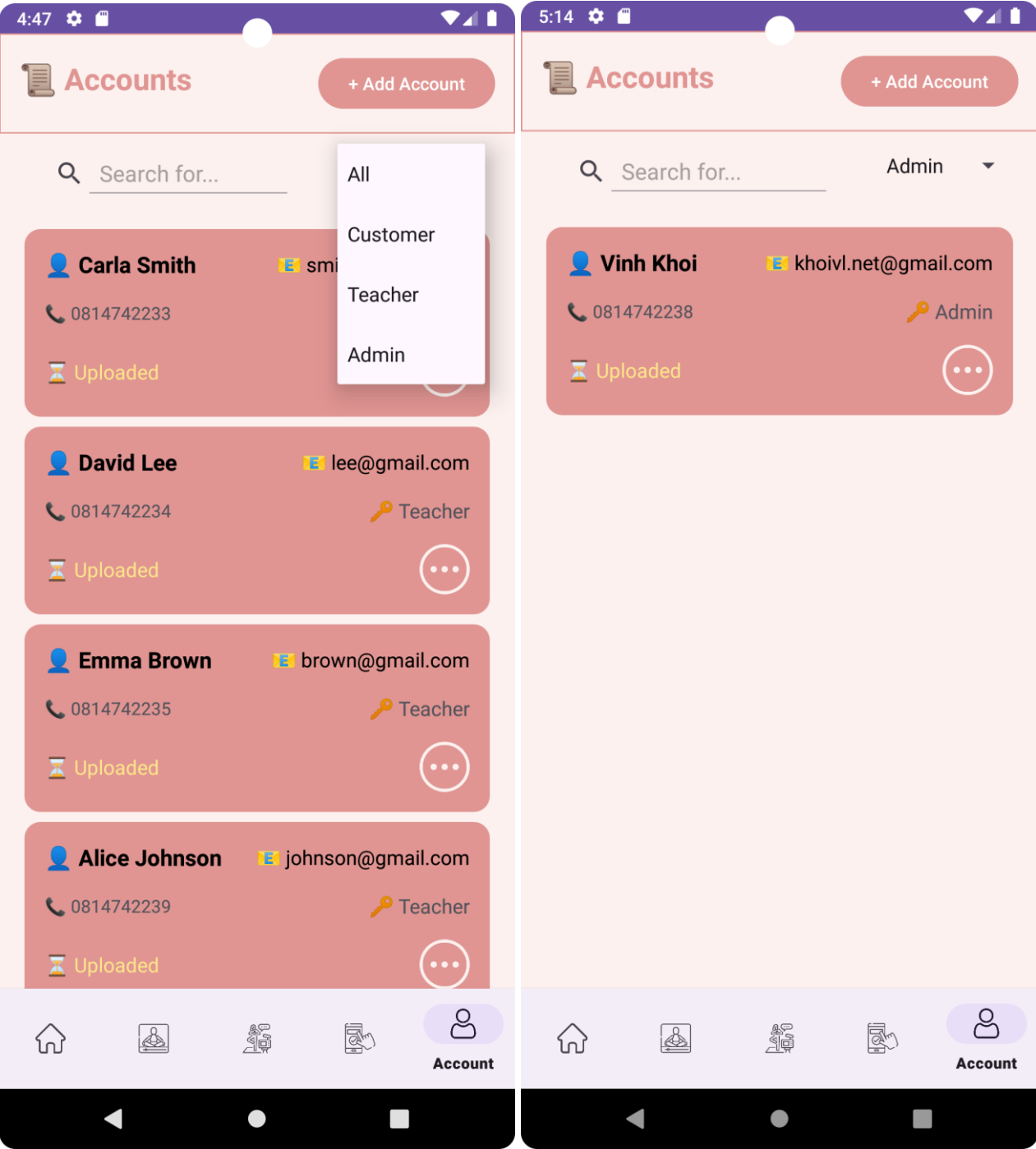


Figure 100. Role-based Filtering in Account Management

Figure 100 shows the account management screen with the role-based filtering functionality. The user can filter accounts by roles such as "Customer," "Teacher," and "Admin." In the second image, selecting "Admin" in the filter dropdown displays only the account with the "Admin" role, "Vinh Khoi." This feature enhances efficiency by allowing users to quickly find accounts associated with specific roles.

4.1.25 Booking Management

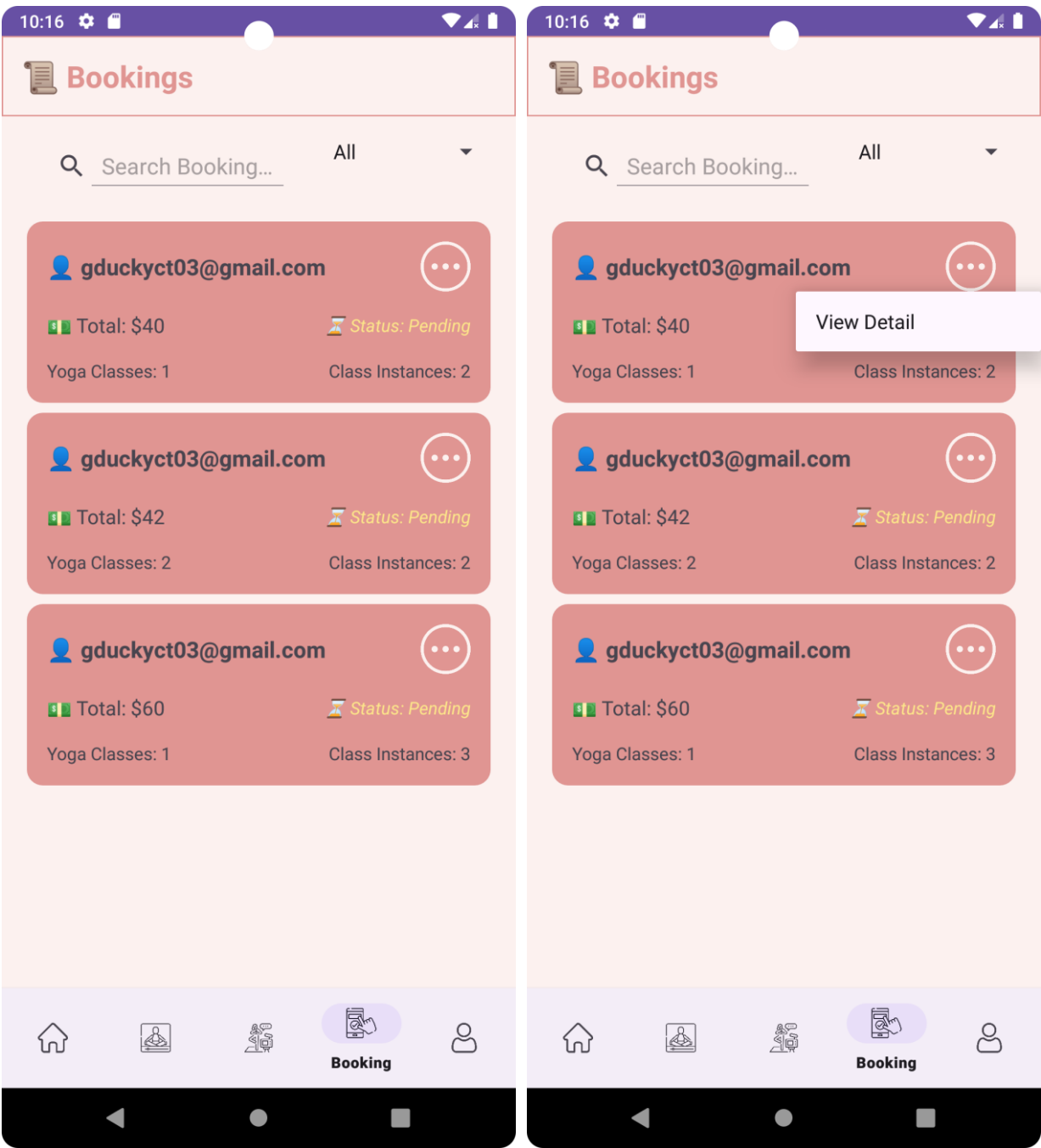


Figure 101. Booking Management Interface

Figure 101 illustrates the booking management screen. It displays a list of bookings, including details such as the customer's name, total amount, and the count of yoga classes and class instances in each booking. The booking status is indicated next to the total, showing if it is "Pending," "Confirmed," or "Rejected." Users can click on the three-dot menu for each booking entry to view further details, as shown in the second image, where the "View Detail" option is available.

4.1.26 View Booking Detail

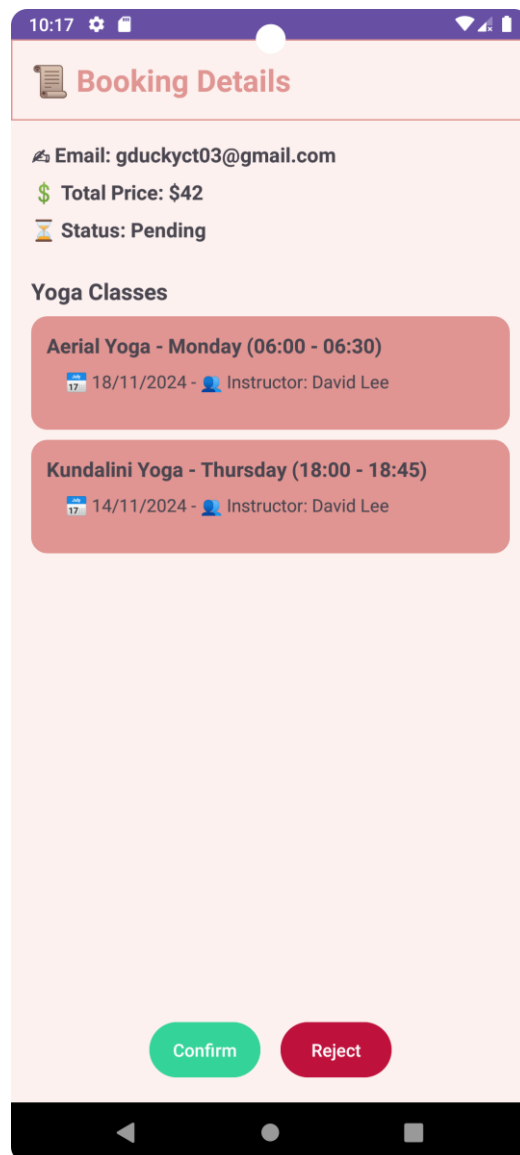


Figure 102. View Booking Detail Screen

Figure 102 illustrates the detailed view of a booking with a "Pending" status. This screen provides comprehensive booking information, including the customer's full name, total price of the booking, and its current status. Since the booking is in the "Pending" state, two action buttons **Confirm** and **Reject** are displayed at the bottom of the screen. These buttons allow the admin to either approve the booking, moving it to a "Confirmed" status, or reject it, changing the status to "Rejected." This dynamic display of options based on the booking's status enhances the admin's ability to manage bookings efficiently by showing only relevant actions.

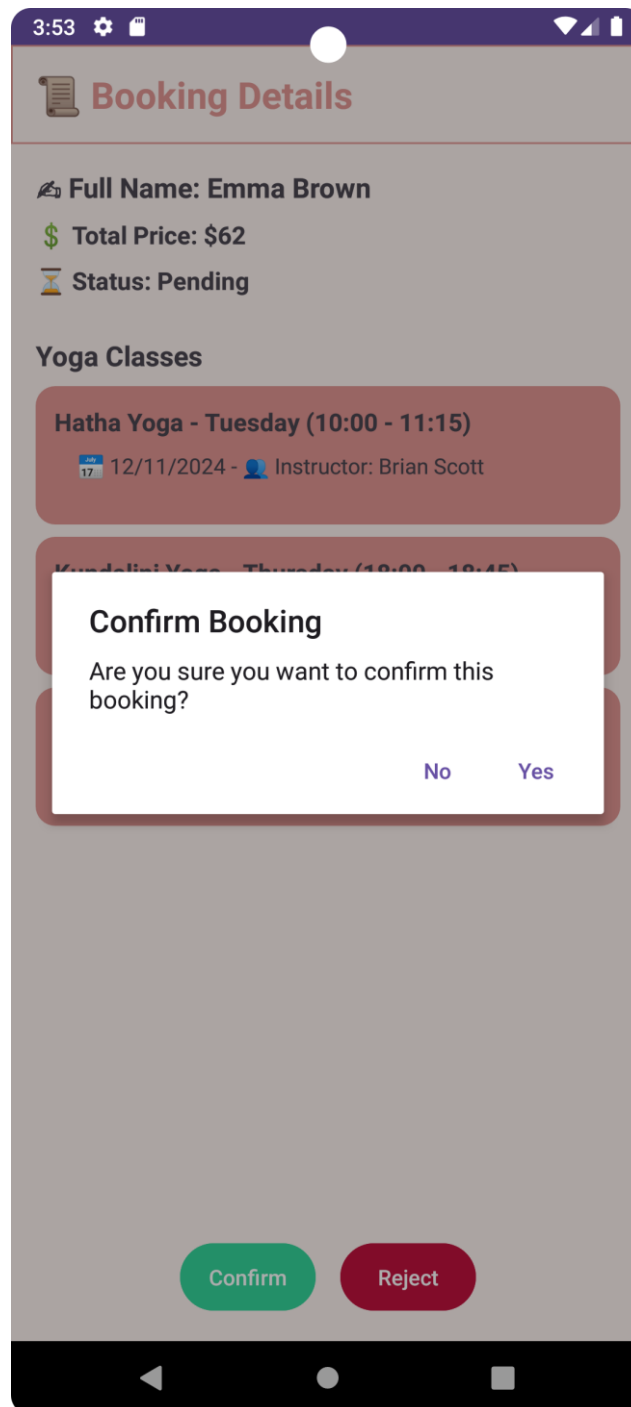


Figure 103. Booking Confirmation Prompt

Figure 103 shows the confirmation prompt that appears when the user presses the **Confirm** button on the Booking Details screen. This dialog asks, "Are you sure you want to confirm this booking?" giving the admin an option to either proceed with the confirmation by selecting **Yes** or cancel the action by selecting **No**. This prompt serves as a safeguard, ensuring that the admin reviews the decision before changing the booking status to "Confirmed."

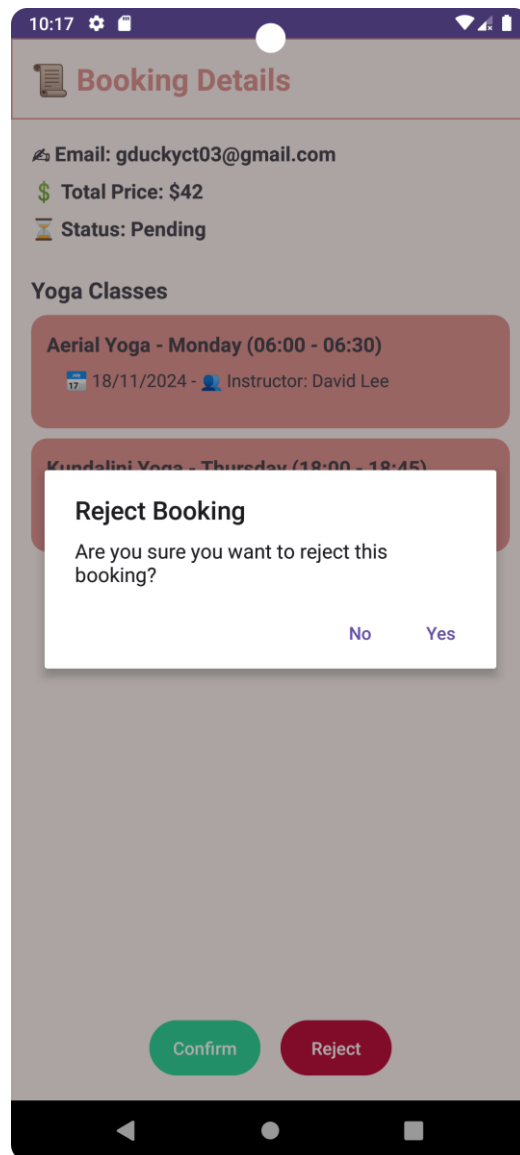


Figure 104. Booking Rejection Prompt

Figure 104 illustrates the rejection prompt that appears when the user presses the **Reject** button on the Booking Details screen. This dialog asks, "Are you sure you want to reject this booking?" allowing the admin to either proceed with the rejection by selecting **Yes** or cancel the action by selecting **No**. This confirmation step ensures that the admin reviews the action carefully before changing the booking status to "Rejected."

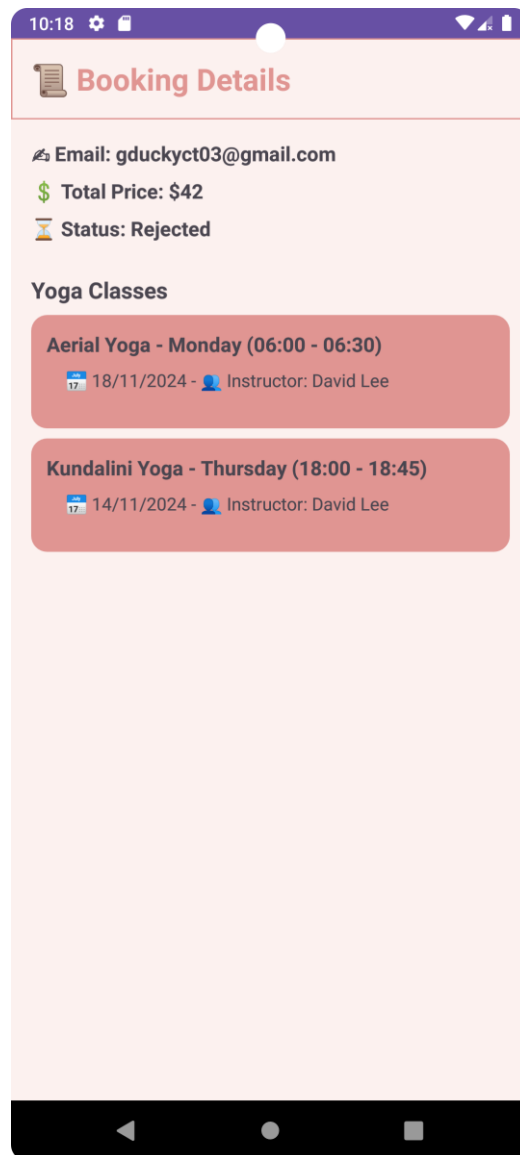


Figure 105. Booking Details After Status Change

Figure 105 shows the Booking Details screen after the booking status has been updated to "Confirmed." The confirmation and rejection buttons are no longer visible, reflecting the finalized status of the booking. This streamlined view provides clear feedback that no further actions are required, indicating a completed transaction for both the user and the admin.

4.1.27 Search Booking

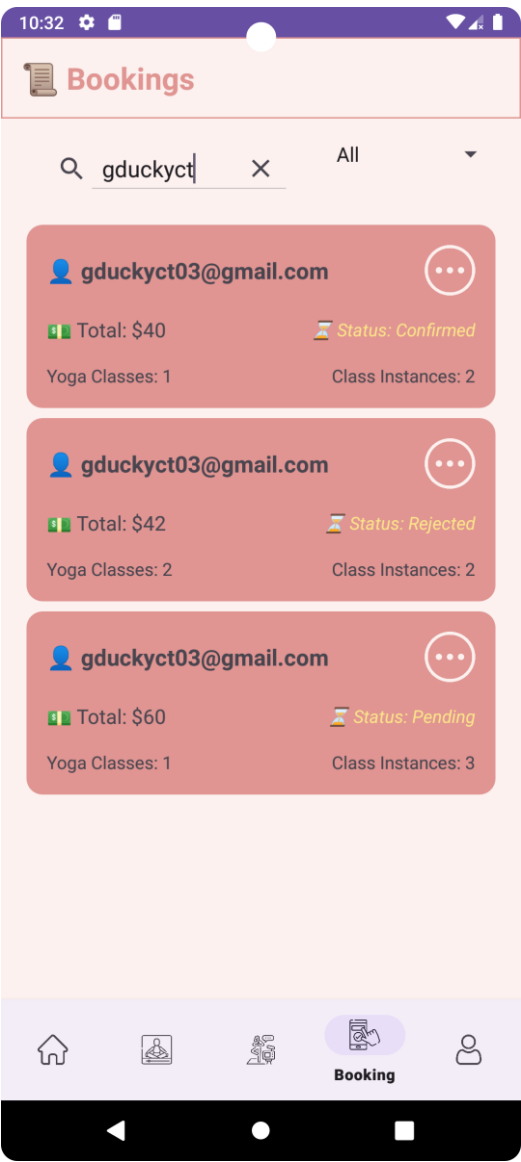


Figure 106. Search Booking Functionality

Figure 106 demonstrates the search functionality within the booking management screen. Here, the user has entered "gduckyct" in the search bar, resulting in a filtered display that shows only gduckyct03@gmail.com's booking details. This search feature allows users to efficiently locate specific bookings based on client names or other identifying details.

4.1.28 Filter Booking

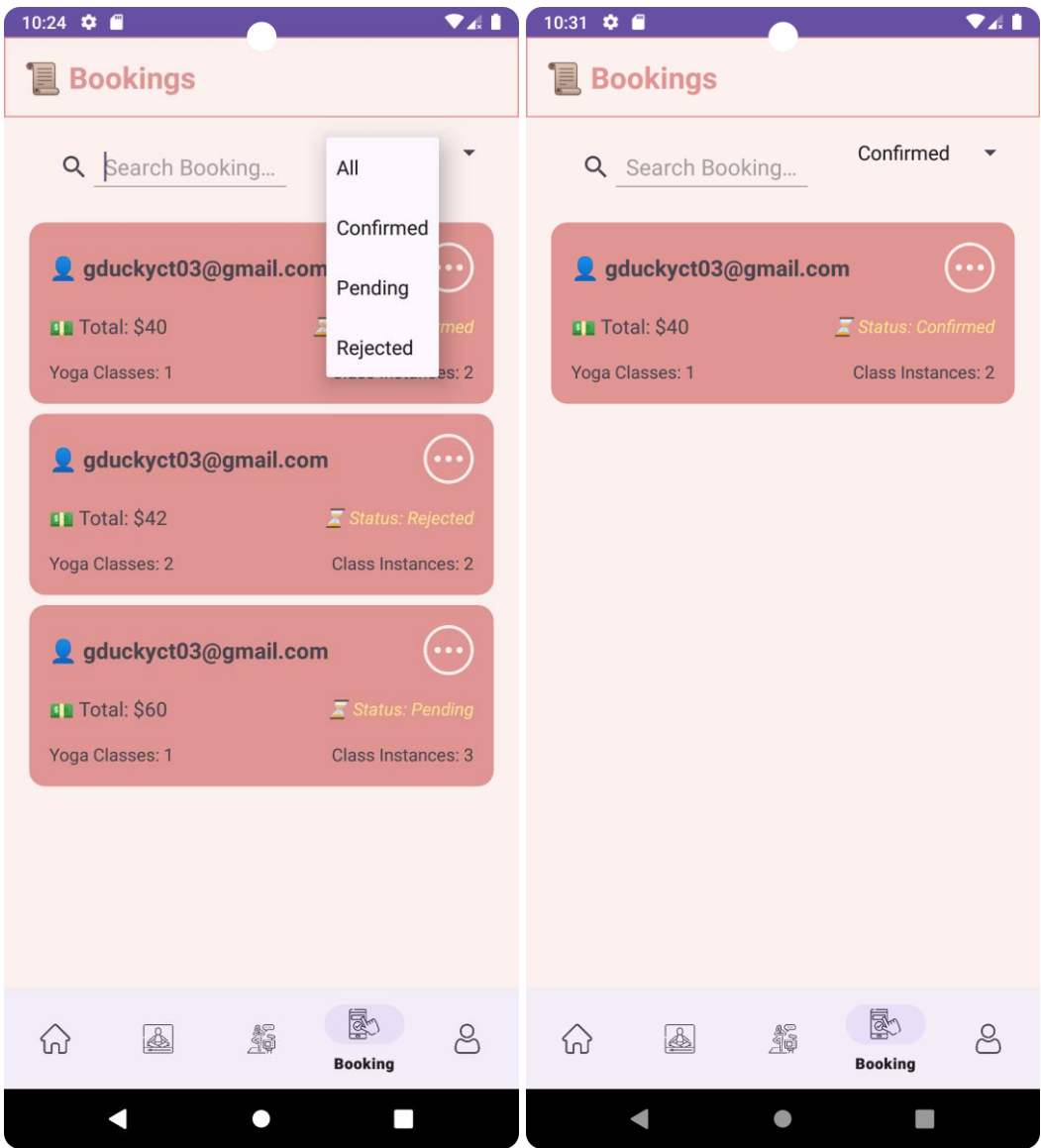


Figure 107. Filter Booking by Status

Figure 107 displays the booking management screen with the filter functionality activated. In the dropdown menu, the user has selected "Confirmed" limiting the displayed bookings to only those with a "Confirmed" status. This functionality allows users to efficiently sort bookings by their confirmation status (e.g., Confirmed, Pending, Rejected), enhancing the user experience by providing a more organized view.

4.1.29 Class Type Management

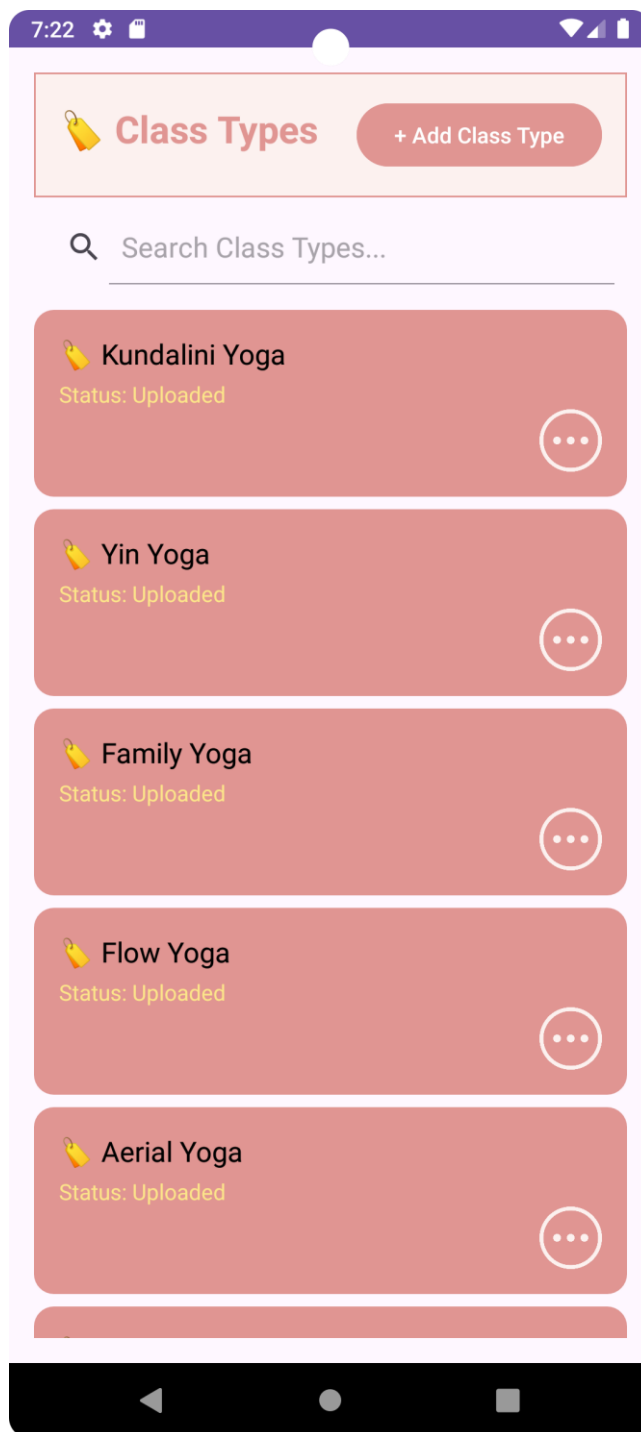


Figure 108. Class Type Management

Figure 108 illustrates the class type management screen, which displays a list of different yoga class types such as "Kundalini Yoga", "Yin Yoga", "Family Yoga", "Flow Yoga", and "Aerial Yoga". Each class type displays a status of "Uploaded", indicating that the class has been synced to the database. The search bar at the top allows users to quickly find specific class types, while the "+ Add Class Type" button allows new class types to be added. The ellipsis icon next to each class type provides additional options for management actions such as updating or deleting.

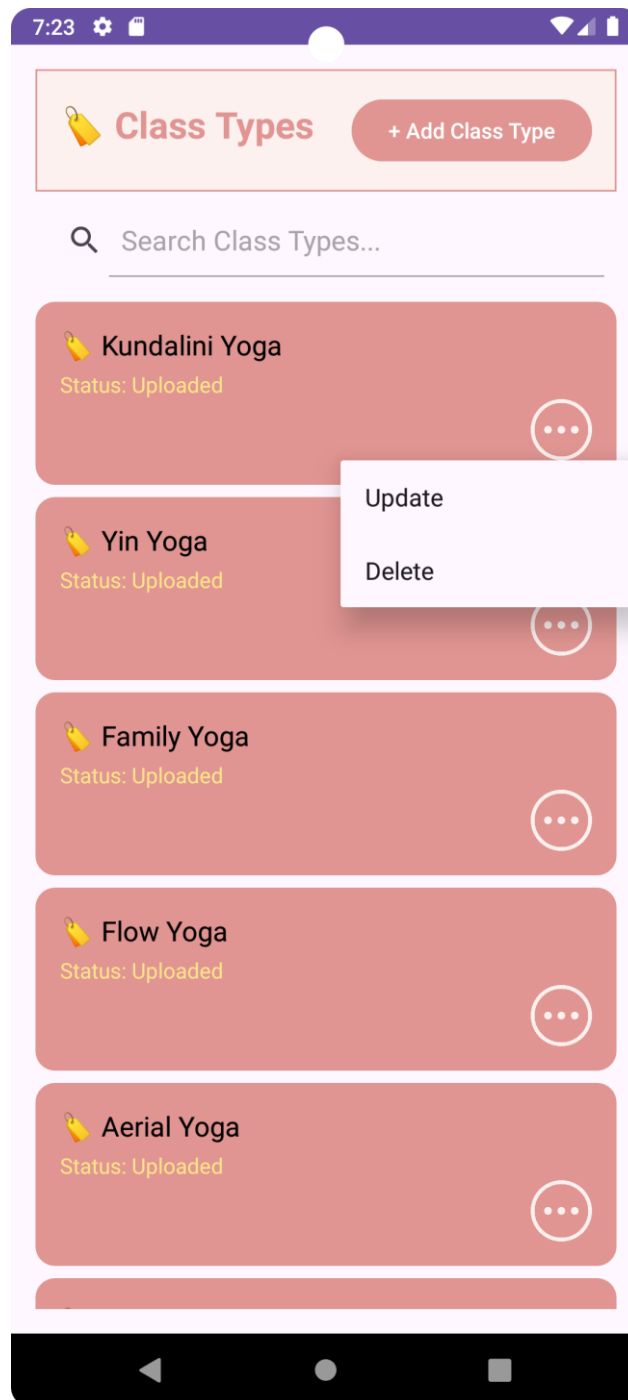


Figure 109. Class Type Management Options

Figure 109 showcases the options available within the class type management screen. By selecting the ellipsis icon next to a class type, users can access a menu with options to "Update" or "Delete" the selected class type. This functionality allows administrators to make adjustments or remove class types as needed, providing flexibility in managing the different types of yoga classes offered within the system.

4.1.30 Add Class Type

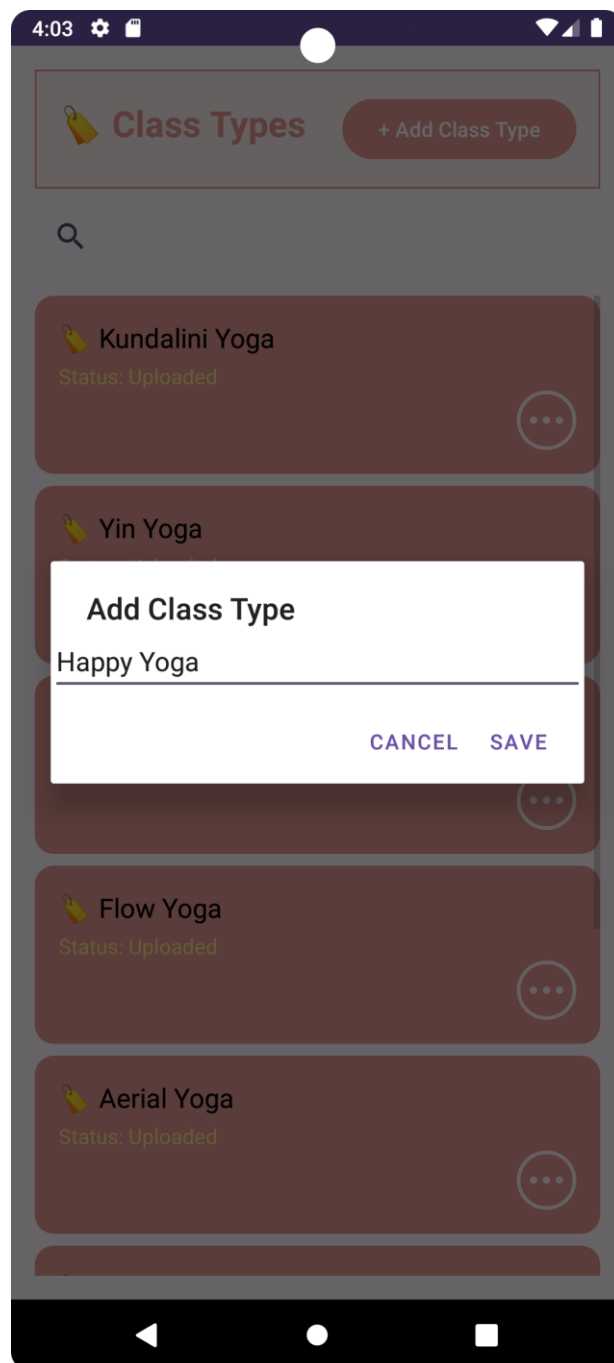


Figure 110. Add Class Type Dialog

Figure 110 illustrates the dialog box for adding a new class type. In this example, the user is entering "Happy Yoga" as the name of the new class type. The dialog provides two options: "Cancel" to exit without saving, and "Save" to add the new class type to the list. This feature allows administrators to expand the variety of classes available in the system by easily adding new class types.

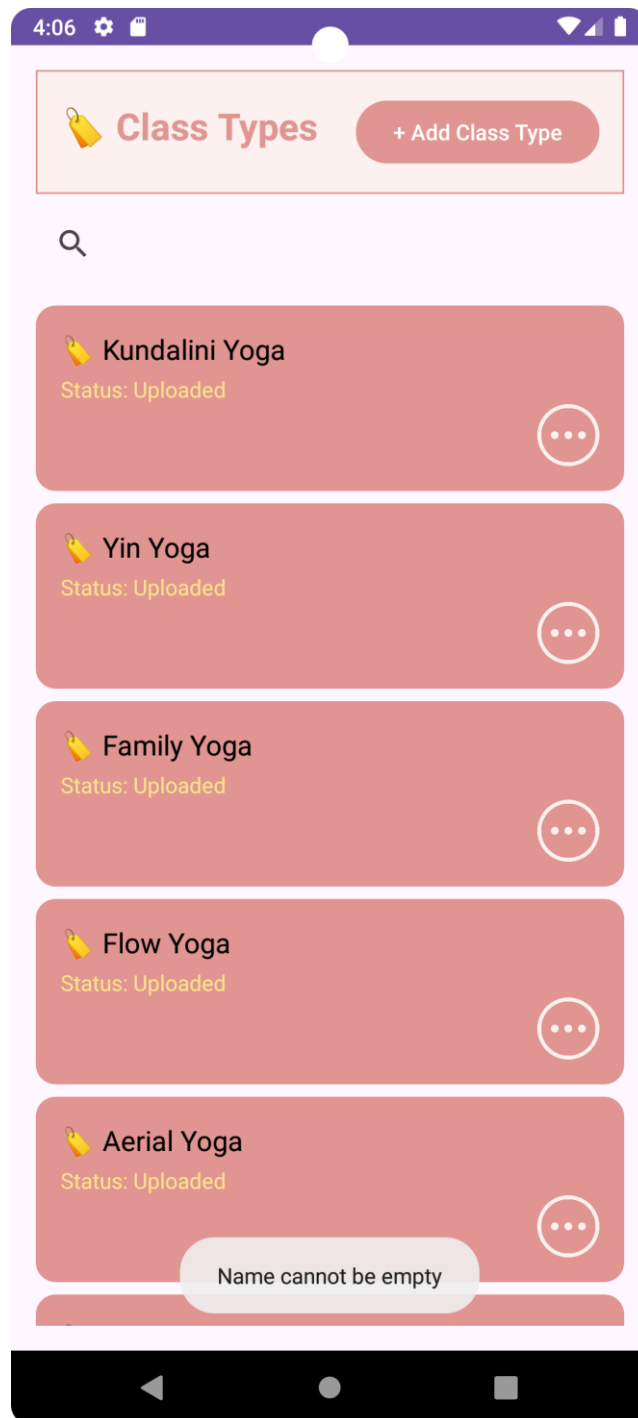


Figure 111. Add Class Type Validation

Figure 111 shows the validation message that appears when attempting to add a new layer type without entering a name. The "Name cannot be empty" message ensures that the user provides a valid layer type name before saving. This validation helps maintain data integrity by preventing layer types from being added or updated without a name.

4.1.31 Update Class Type

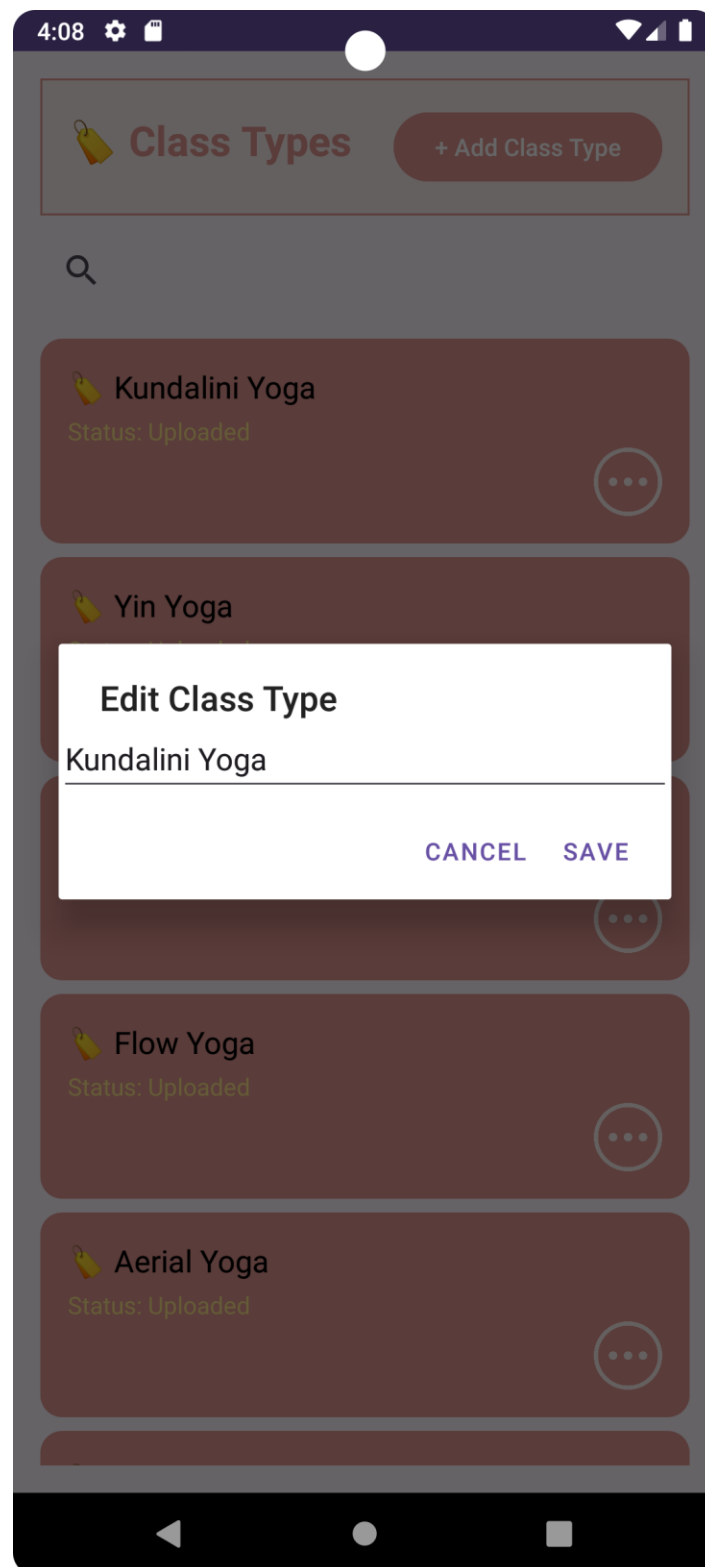


Figure 112. Edit Class Type

Figure 112 illustrates the dialog for editing a class type name. In this example, the name "Kundalini Yoga" is being modified. Users can update the class type name and save the changes, or cancel to discard any edits. This functionality enables users to keep class types accurately labeled and up-to-date.

4.1.32 Search Class Type

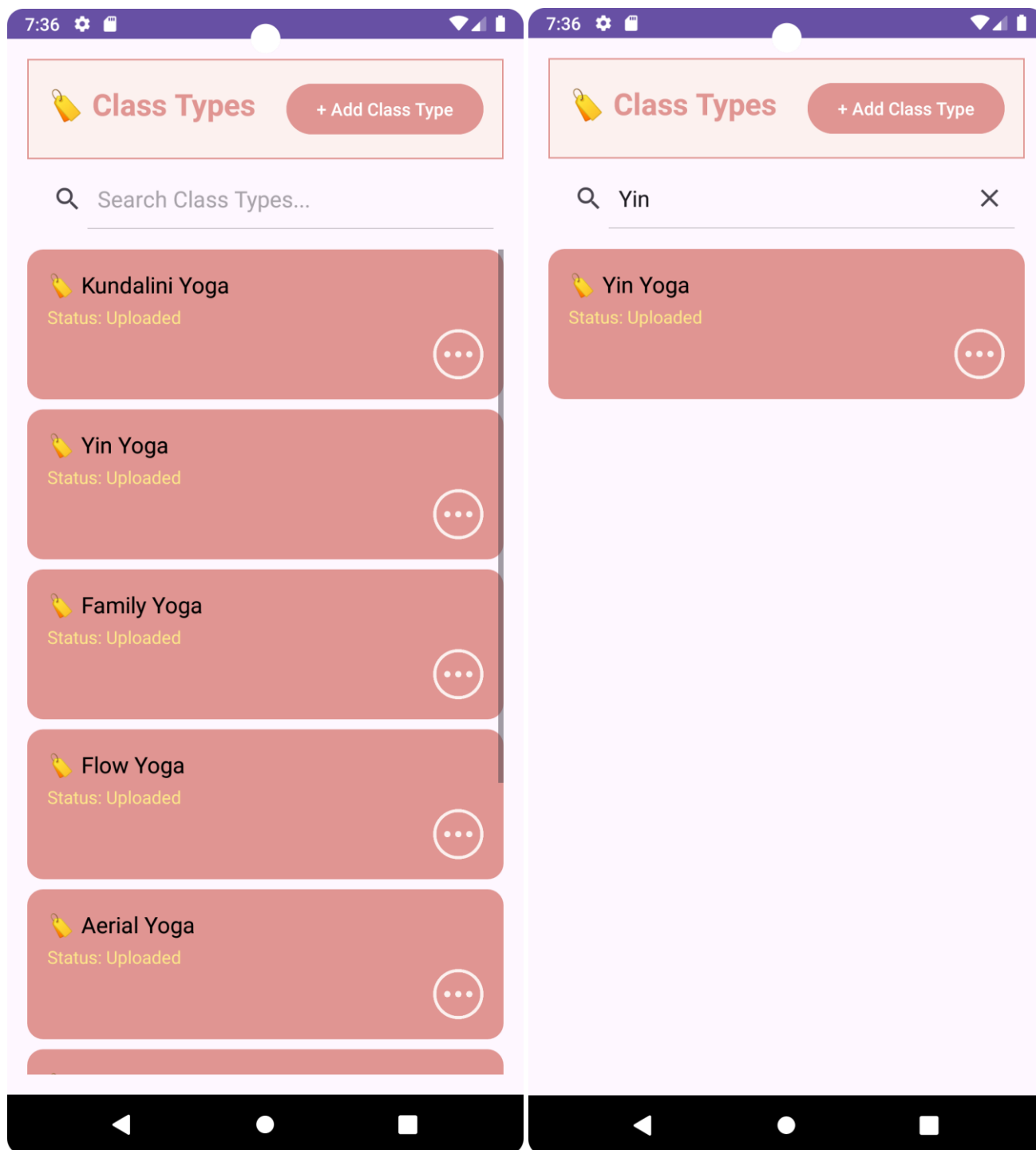


Figure 113. Search Class Type Functionality

Figure 113 illustrates the search functionality within the class type management screen. The first image shows the full list of class types, while in the second image, the user has entered "Yin" in the search bar, filtering the list to display only the "Yin Yoga" type. This feature allows users to quickly find specific class types, enhancing navigation and usability within the class type management interface.

4.1.33 Reset All Data

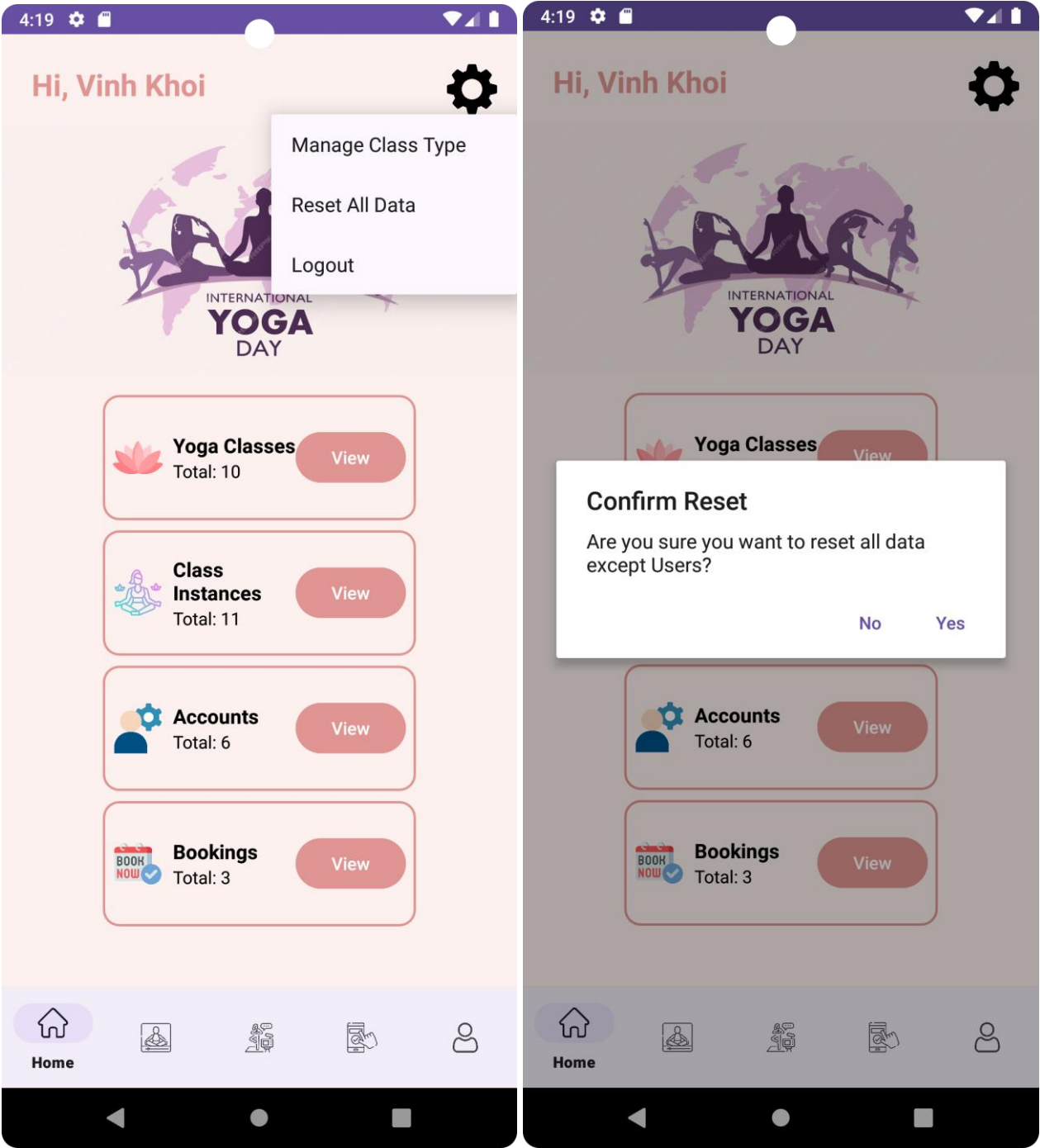


Figure 114. Home Screen Options and Data Reset Confirmation

Figure 114 shows the dialog box when "Reset All Data" is selected, asking the user if they want to erase all data except user information, helping to prevent accidental data loss.

4.1.34 Sync all data from Firebase to Sqlite

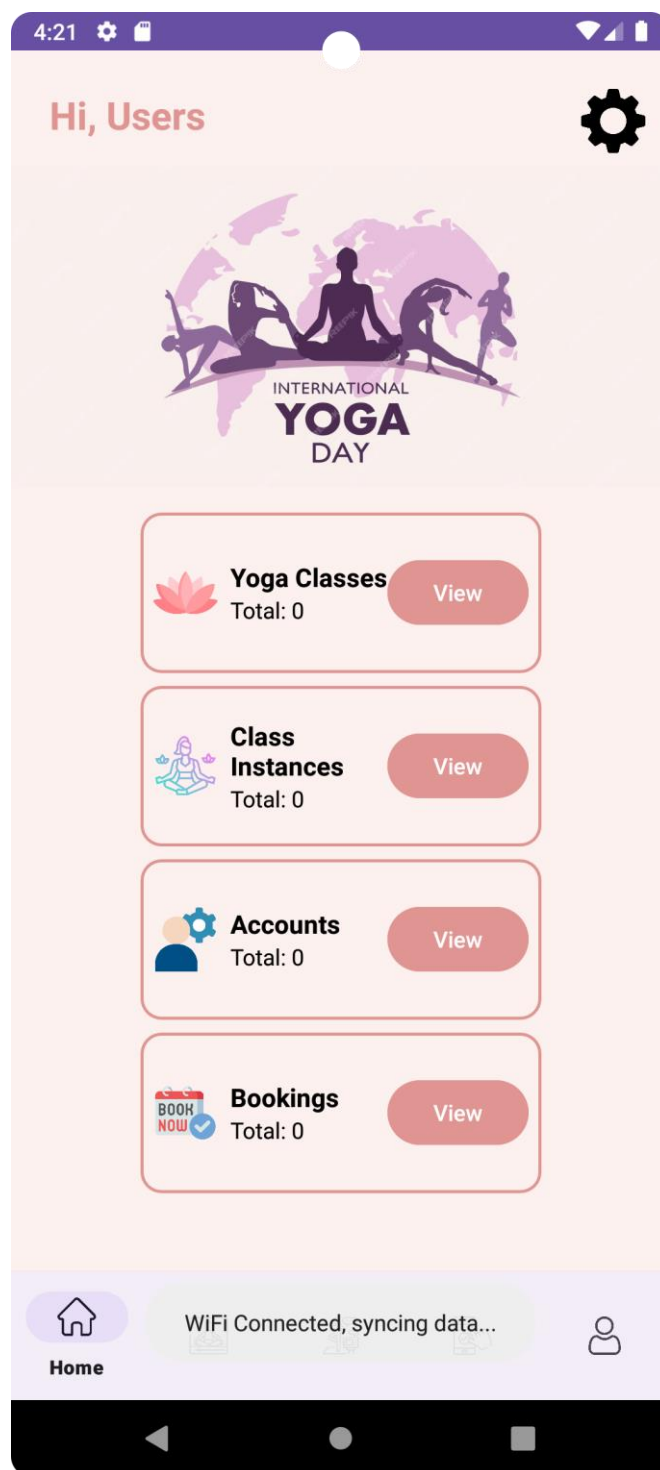


Figure 115. Synchronizing Data from Firebase to SQLite

Figure 115 shows the application's home screen with a synchronization message at the bottom, indicating that data is being synced from Firebase to the local SQLite database. The initial counts for Yoga Classes, Class Instances, Accounts, and Bookings are all set to zero, as the data has yet to populate. This sync process allows the app to work offline by ensuring all necessary information is available locally.

4.1.35 Exception when updating a yoga class

7:52 WiFi: Connected

Update class

Day of the Week
Monday

Time
05 59
06 00
07 01

Capacity
20

Duration (minutes)
15
30
45

Price
20.0

Type of Class
Aerial Yoga

Description (Optional)
You have 9 exercises in the session.

Save

WiFi Connected, syncing data...

Figure 116. Update Yoga Class - Limited Field Update

Figure 116 illustrates the "Update Yoga Class" screen, where only the "Description" field is editable. For this specific yoga class, restrictions are in place preventing changes to fields such as "Day of the Week," "Time," "Price," and "Type of Class." This limitation is enforced because this yoga class has existing class instances linked by its ID, with scheduled dates depending on the "Day of the Week" field. Altering key fields like time or price could create inconsistencies for customers who have already booked sessions, as their expectations would be based on the original class details.

4.1.36 Catch logic errors in adding and updating class instances

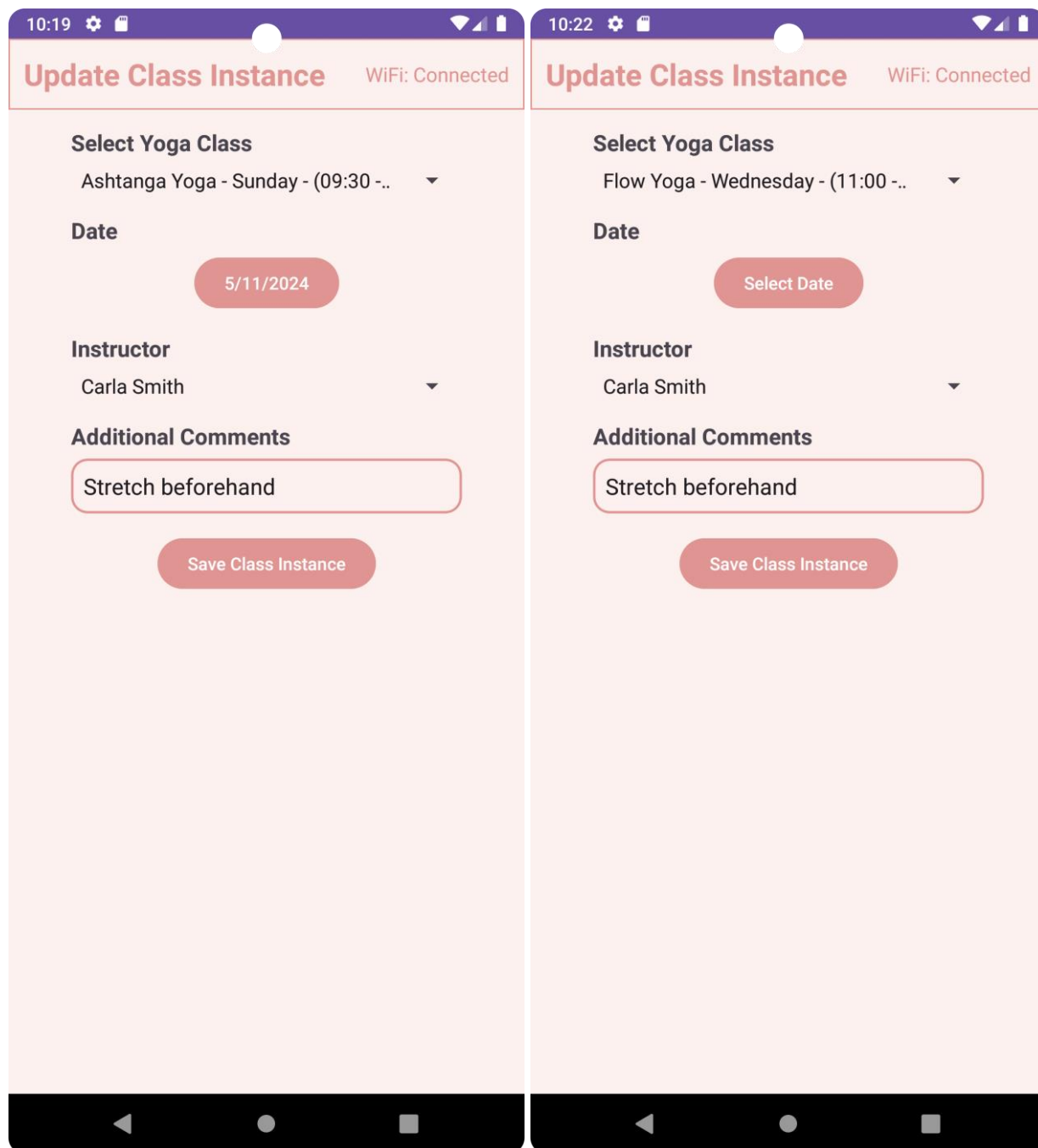


Figure 117. Update Class Instance - Date Reset on Yoga Class Change

Figure 117 demonstrates the "Update Class Instance" screen, highlighting a logic fix to ensure consistency between the selected date and the "Day of the Week" of the chosen yoga class. Initially, users could select a date and then change the yoga class without resetting the date, which could lead to mismatches if the new yoga class had a different "Day of the Week." After the logic fix, when a user selects a different yoga class, the date field is reset, prompting the user to select a new date that aligns with the "Day of the Week" of the newly chosen yoga class.

4.1.37 Teacher Page

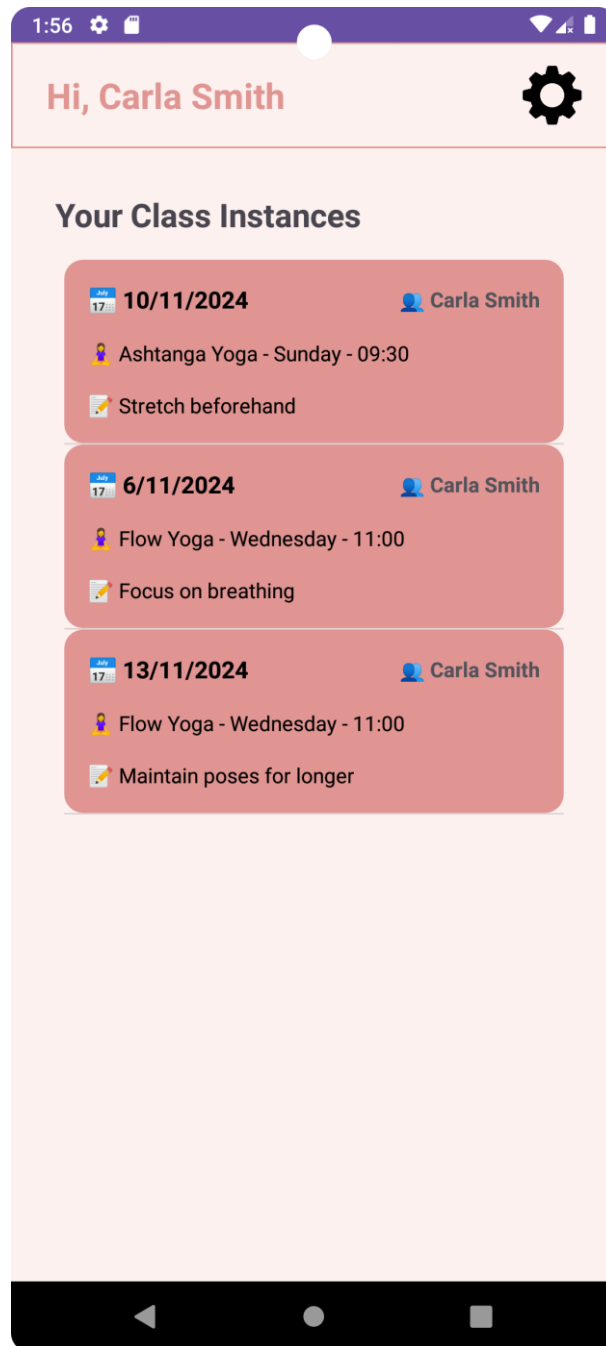
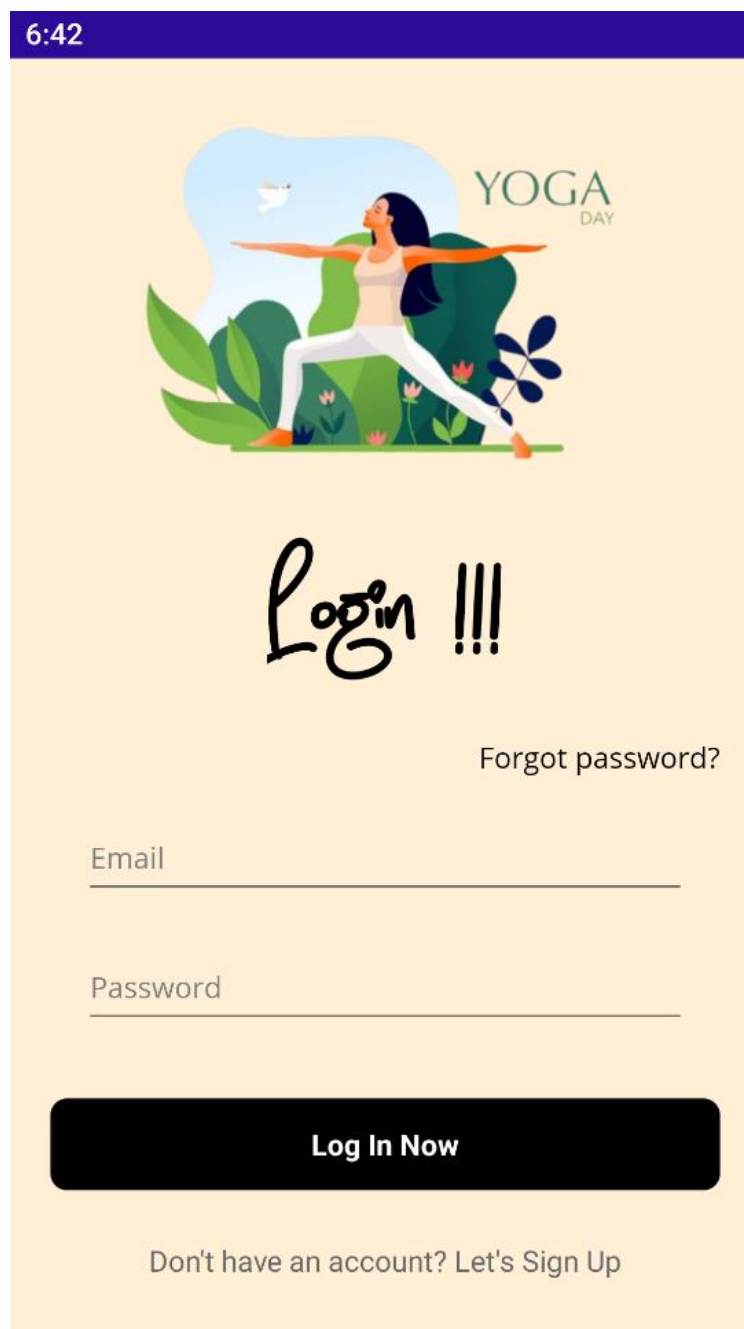


Figure 118. Teacher Page

Figure 118 shows the teacher's management page, allowing instructors to oversee and manage the class instances they are assigned to teach. It provides an organized interface to track teachers' class instances activities and schedules.

4.2 Customer Application:

3.2.1 Login:



The image shows a mobile application login screen for 'YOGA DAY'. At the top, a dark blue status bar displays the time '6:42'. The main background is a light beige color. At the top center, there is an illustration of a woman in a white yoga pose (Warrior II) with her arms extended, set against a backdrop of green foliage and a blue sky with a white dove. To the right of the illustration, the text 'YOGA DAY' is written in a green, sans-serif font. Below the illustration, the word 'Login' is written in a large, black, cursive font, followed by three exclamation marks '!!!'. To the right of this, the text 'Forgot password?' is displayed in a small, black, sans-serif font. Below this, there are two input fields: one labeled 'Email' and another labeled 'Password', both in a small, grey, sans-serif font. Below the input fields is a large, black, rounded rectangular button with the text 'Log In Now' in white, bold, sans-serif font. At the bottom of the screen, the text 'Don't have an account? Let's Sign Up' is displayed in a small, grey, sans-serif font.

Figure 119. Customer Login Interface

Figure 119 displays the "Login" screen designed for customers accessing the yoga application. The interface prominently features fields for entering the user's email and password, along with a visually appealing yoga-themed background that enhances the user experience. The design includes a "Log In Now" button, enabling users to submit their login information, and a "Forgot password?" link for password recovery. Additionally, users are prompted to sign up if they do not have an existing account, creating a seamless onboarding experience for new users.

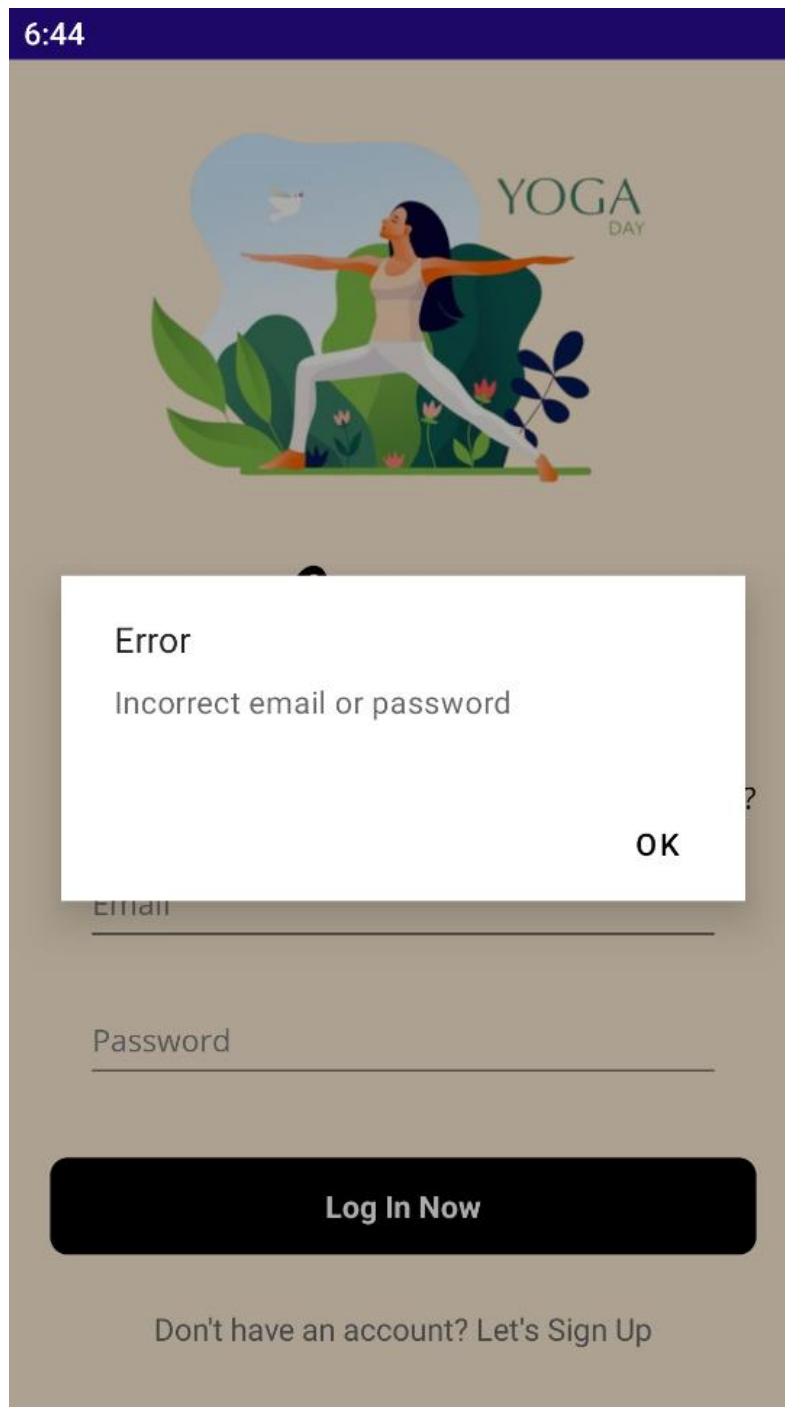


Figure 120. Login Error Message

Figure 120 presents an error message displayed on the "Login" screen when users attempt to log in with incorrect credentials. The message clearly states, "Error: Incorrect email or password," providing immediate feedback to the user that the login attempt has failed due to invalid information. This prompt ensures users are aware of the issue and encourages them to recheck their entries before attempting to log in again. The design maintains the aesthetic of the login interface while emphasizing the importance of correct input for successful authentication.

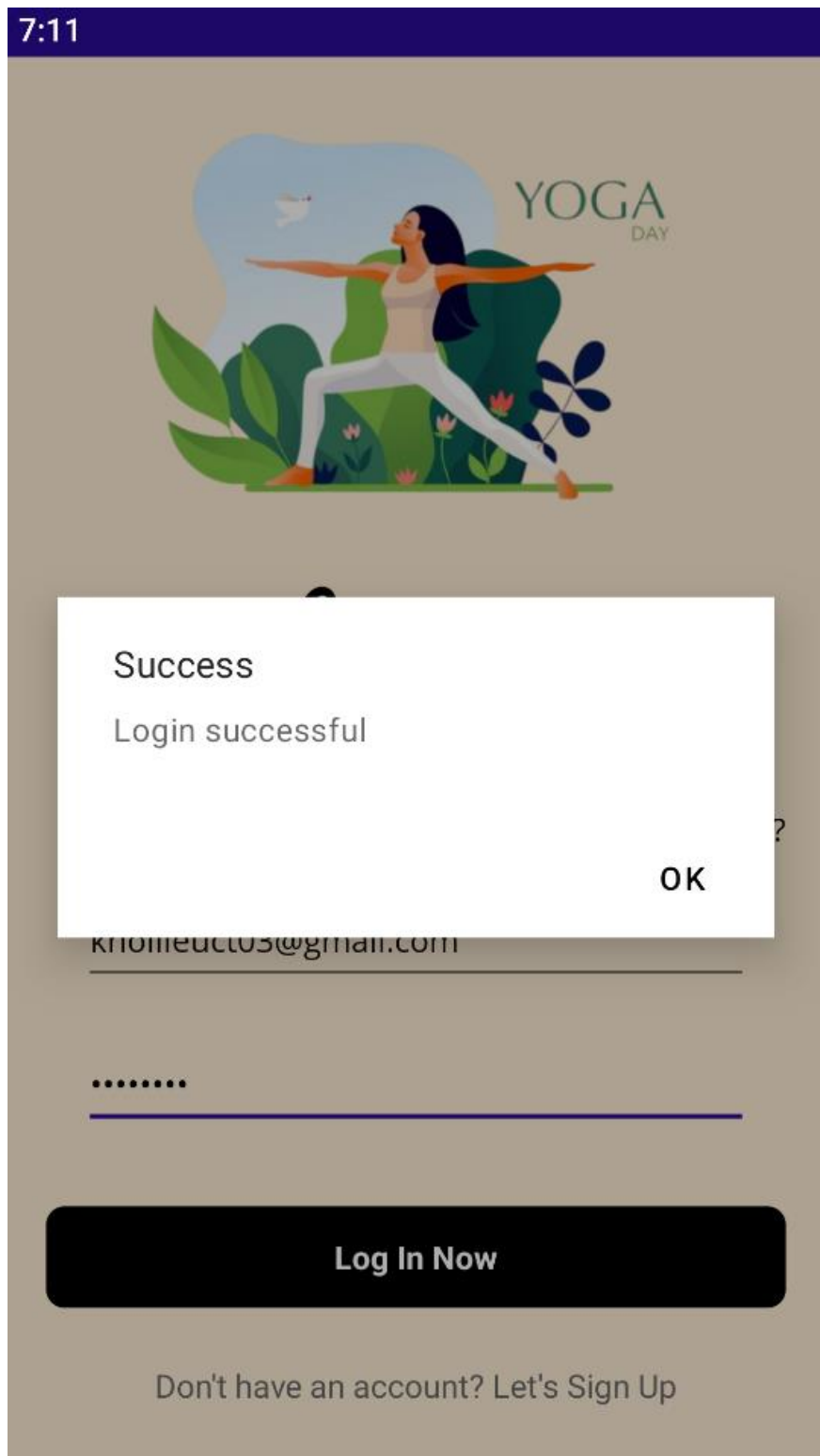


Figure 121. Successful Login Notification

Figure 121 displays a success notification after a user successfully logs into the application. The message states, "Success: Login successful," confirming that the user's credentials have been accepted, and the login process is complete. This feedback reassures users that they can now access their account and navigate the application seamlessly.

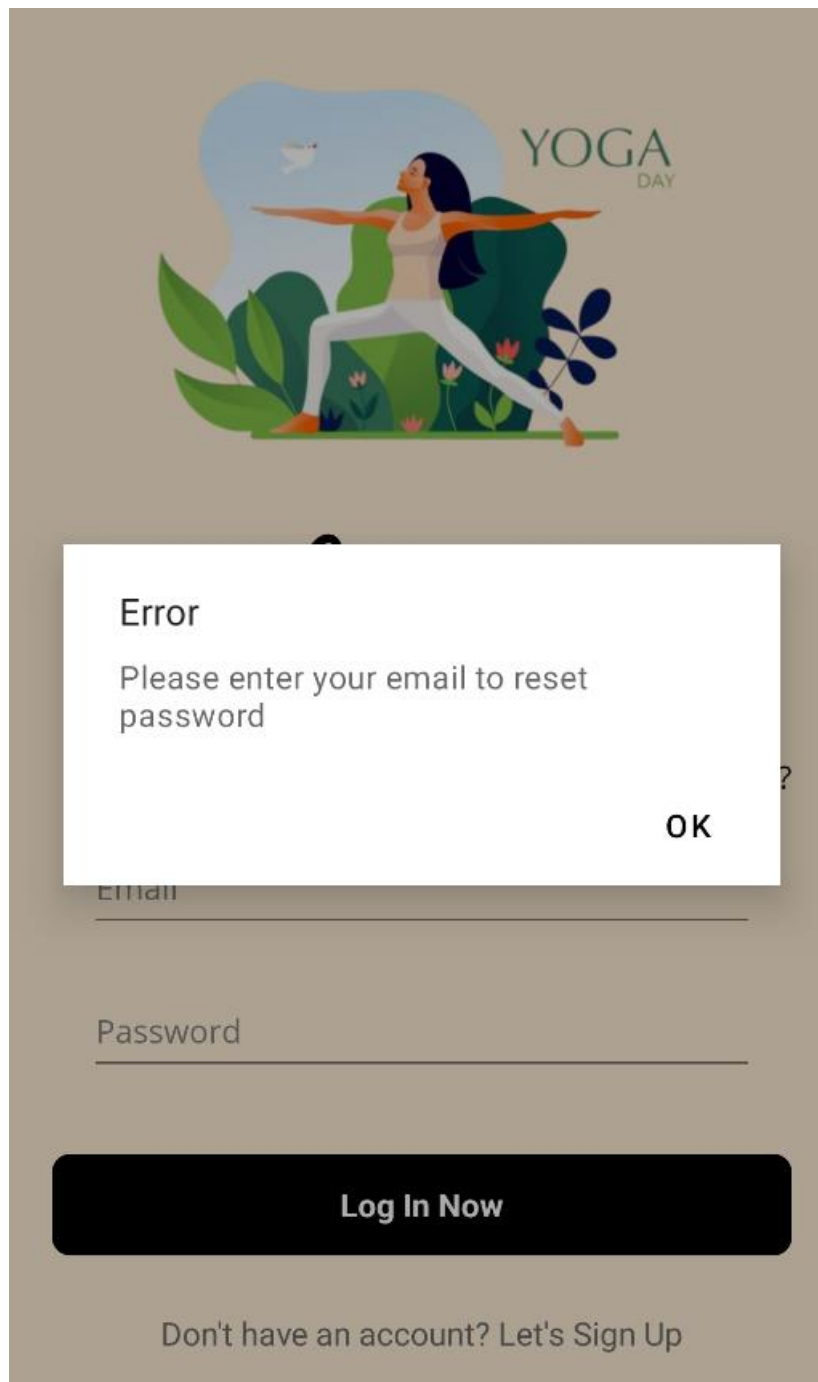


Figure 122. Password Reset Error Notification

Figure 122 displays an error notification on the login screen of a yoga app, instructing the user to "Please enter your email to reset password." This prompt appears when the user attempts to reset their password without providing an email. The message helps guide users to input the necessary information to proceed with the password reset process.

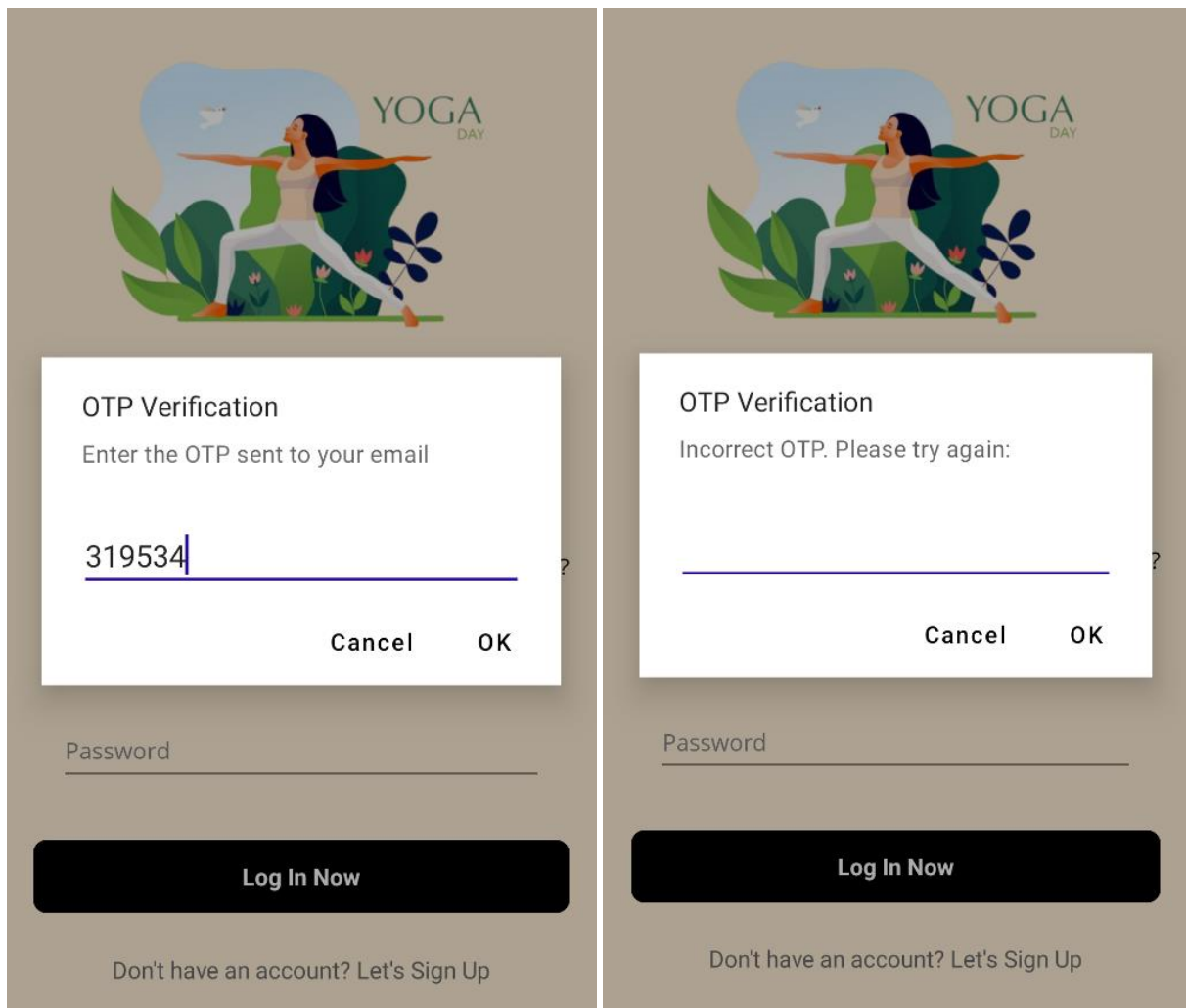


Figure 123. OTP Verification Interface

Figure 123 shows an OTP verification interface. In the first image, the user is prompted to enter the OTP sent to their email. This dialog provides fields to input the OTP and options to "Cancel" or "OK." In the second image, an error message "Incorrect OTP. Please try again:" appears, indicating that the OTP entered was incorrect. This interface ensures that the user can securely verify their identity through an OTP system before proceeding.

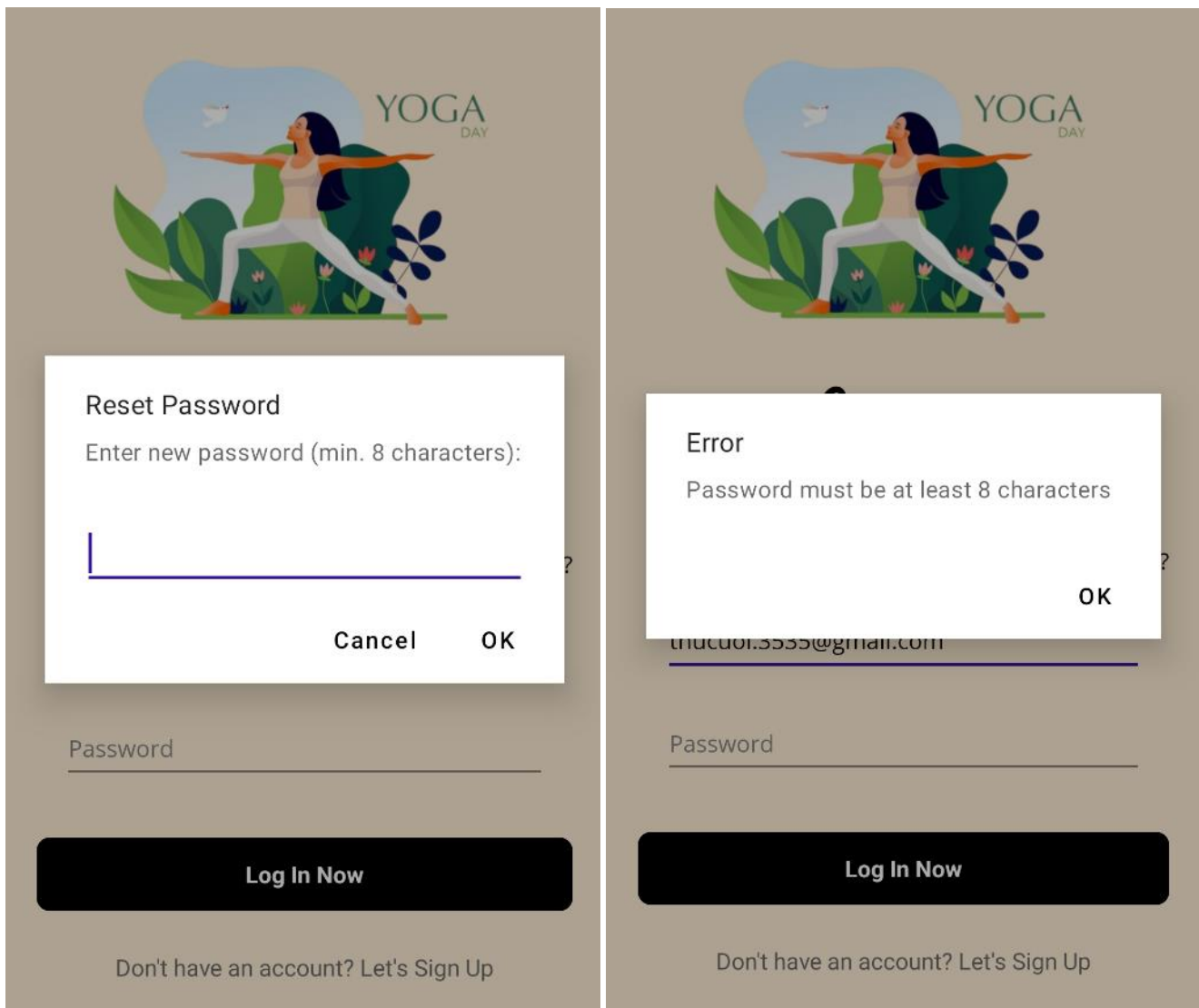


Figure 124. Reset Password Dialog and Error Message

Figure 124 shows the "Reset Password" interface, where the user is prompted to enter a new password with a minimum length requirement of 8 characters. In the first image, the dialog box requests the user to "Enter new password (min. 8 characters)." If the user enters a password that does not meet the length requirement, the second image displays an error message: "Password must be at least 8 characters," guiding the user to provide a valid input. This helps ensure that the password meets security standards.

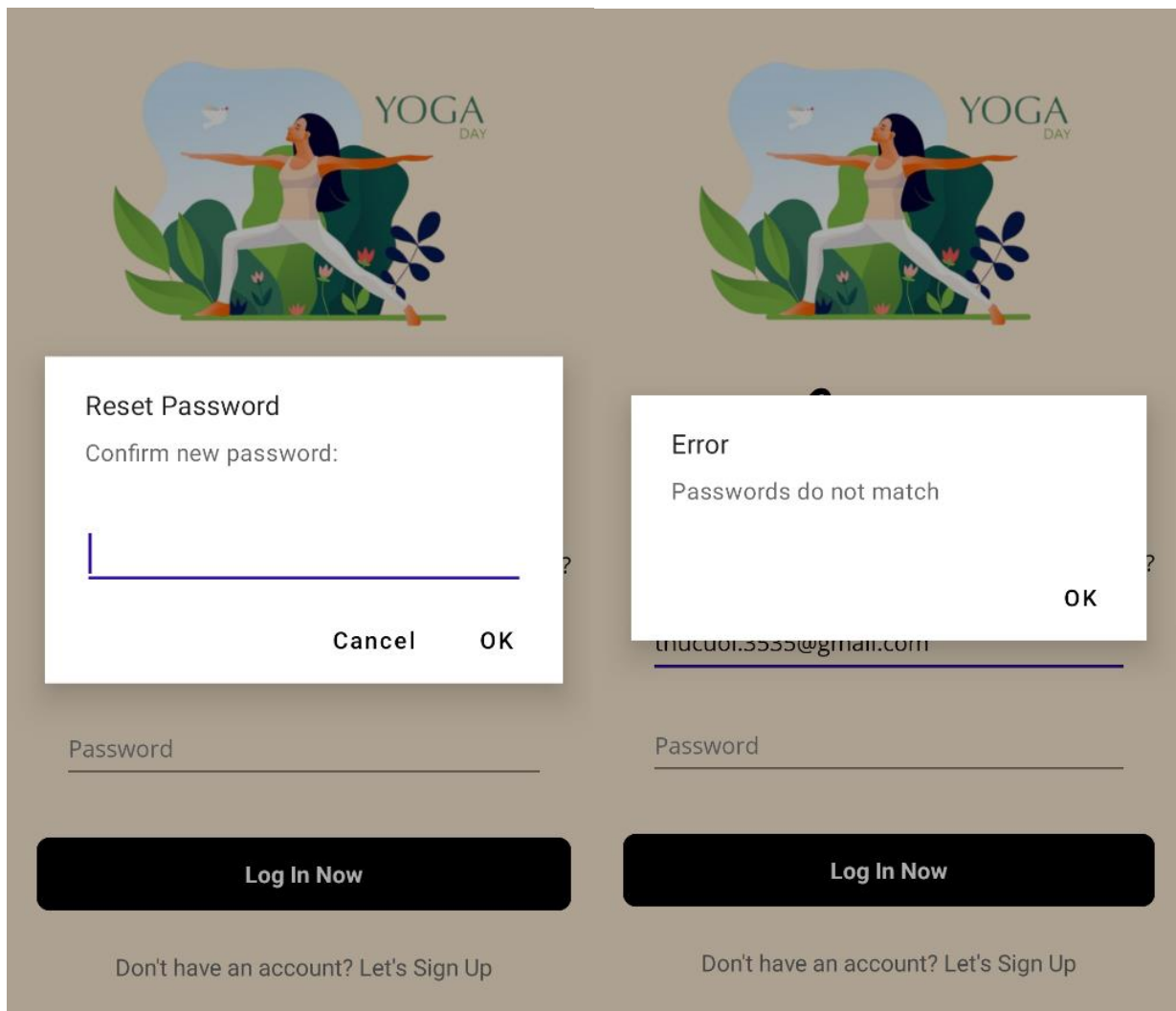


Figure 125. Reset Password Confirmation and Error Notification

Figure 125 shows a "Reset Password" prompt, where the user is asked to confirm their new password. The second image displays an error message stating, "Password must be at least 8 characters," which appears if the entered password does not meet the minimum length requirement. This notification helps ensure that users create secure passwords, enhancing account safety.

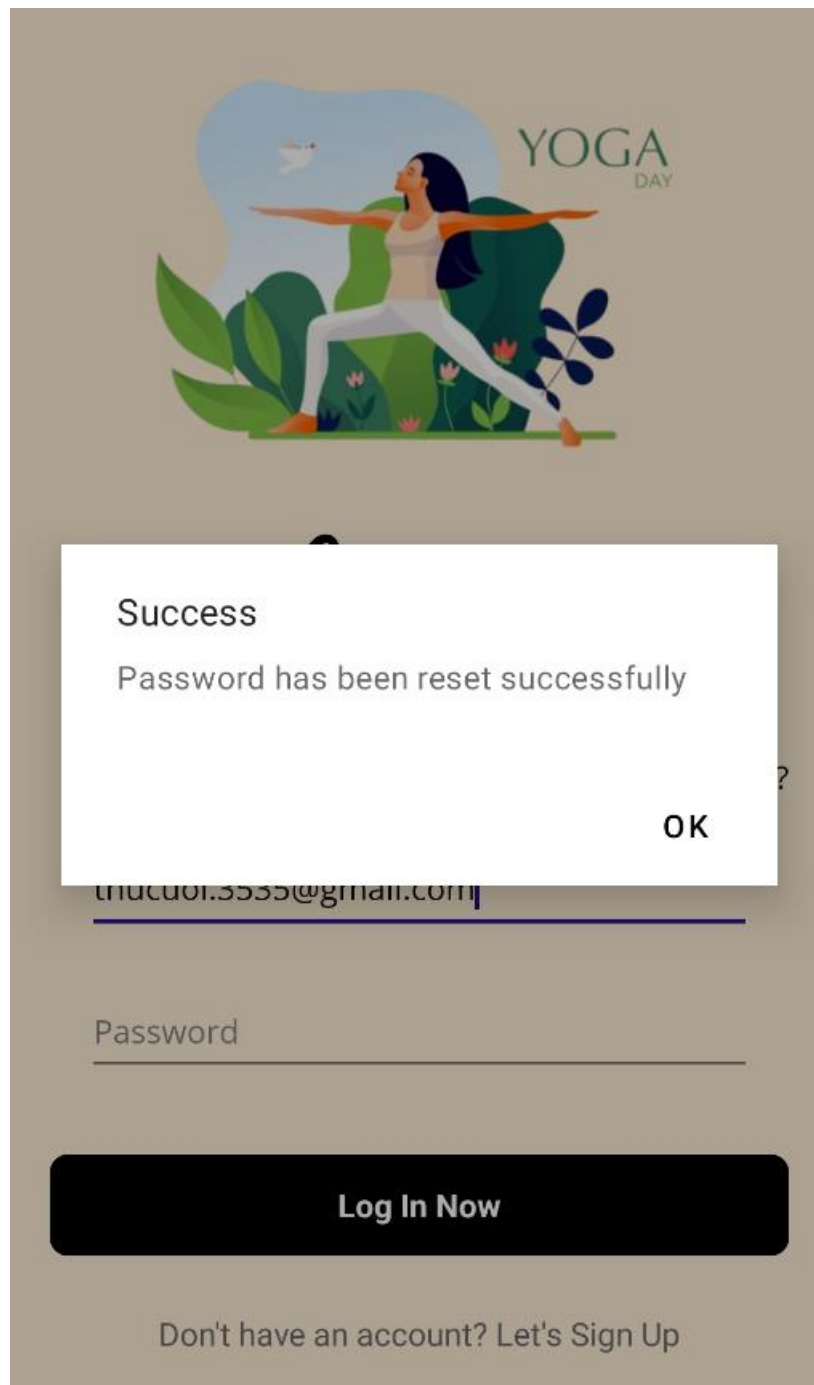


Figure 126. Password Reset Success Notification

Figure 126 shows a success message that reads, "Password has been reset successfully." This notification appears after the user successfully resets their password, confirming that the change has been made. It provides immediate feedback to the user, ensuring they are informed that their new password is now active and ready for use.

3.2.2 Sign up:

6:43

INTERNATIONAL
YOGA DAY



Create an Account!

Full Name

Phone Number

Email

Password

Sign Up Now

Already have an account? Sign In

Figure 127. Account Registration Interface

Figure 127 showcases the "Create an Account" screen where users can register for the application. The form prompts users to enter their full name, phone number, email, and password. This intuitive design facilitates user registration by providing clear input fields, encouraging new users to create their accounts. The call-to-action button "Sign Up Now" is prominently displayed, making it easy for users to complete the registration process. Additionally, there is a prompt for existing users to sign in, enhancing user navigation.

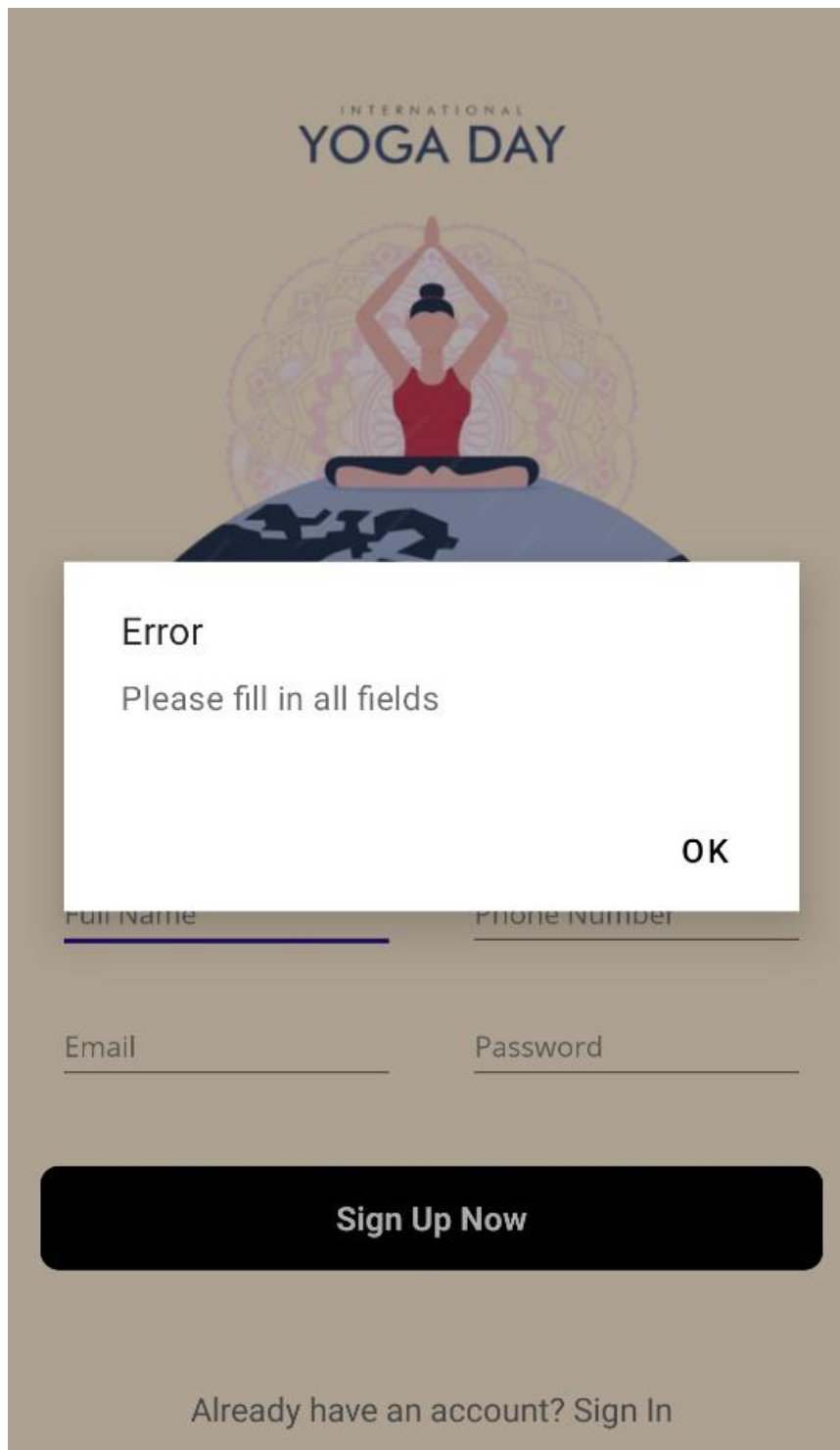


Figure 128. Account Registration Error Notification

Figure 128 displays an error message that appears when users attempt to register without completing all required fields. The notification clearly states, "Please fill in all fields," prompting users to ensure that they provide the necessary information for their registration. This feature enhances the user experience by preventing incomplete submissions, thereby ensuring that all required data is collected for account creation.

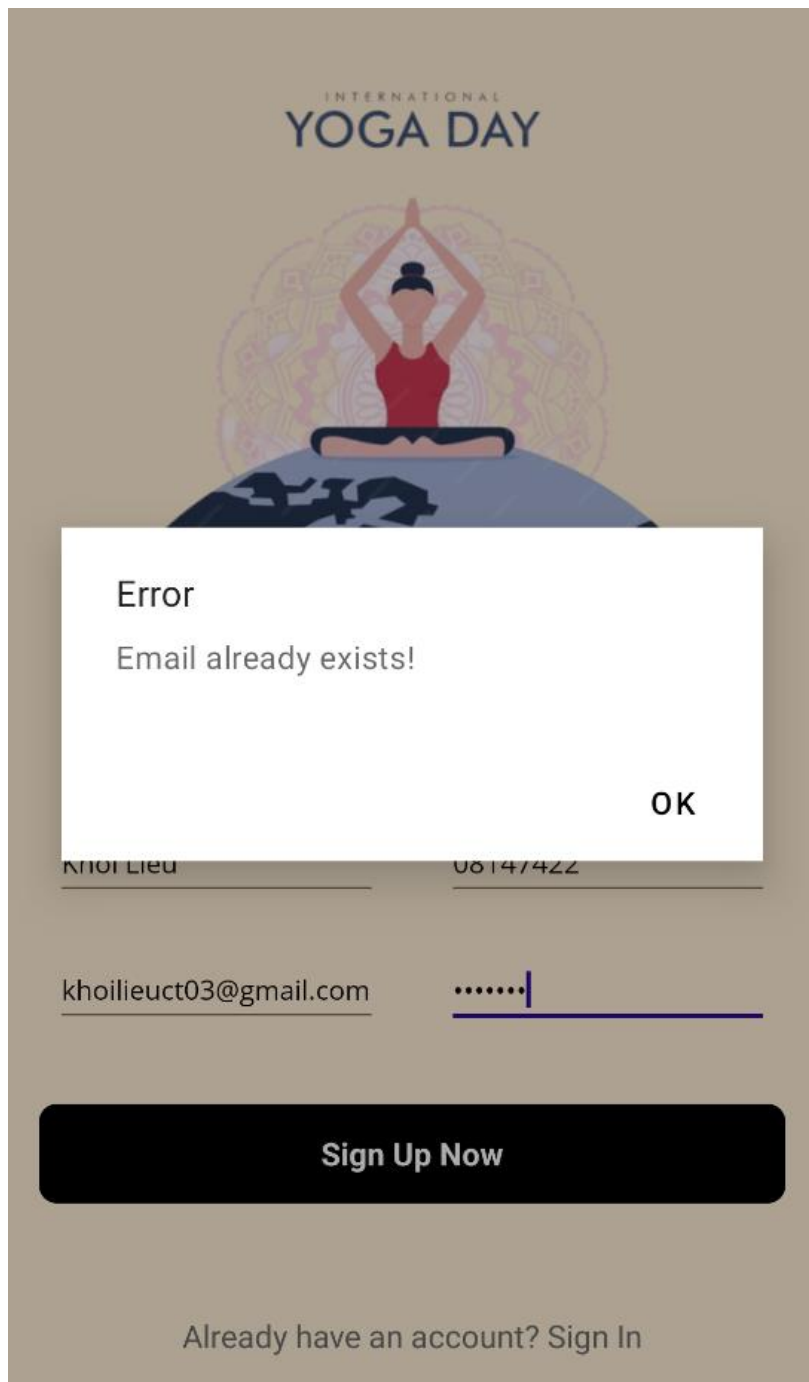


Figure 129. Account Registration Duplicate Email Error

Figure 129 shows an error notification that alerts users when they attempt to register with an email address that already exists in the system. The message reads, "Email already exists!" This prompt informs users that they cannot use the same email for multiple accounts, encouraging them to either log in or use a different email address for registration. This feature helps maintain data integrity by preventing duplicate accounts from being created.

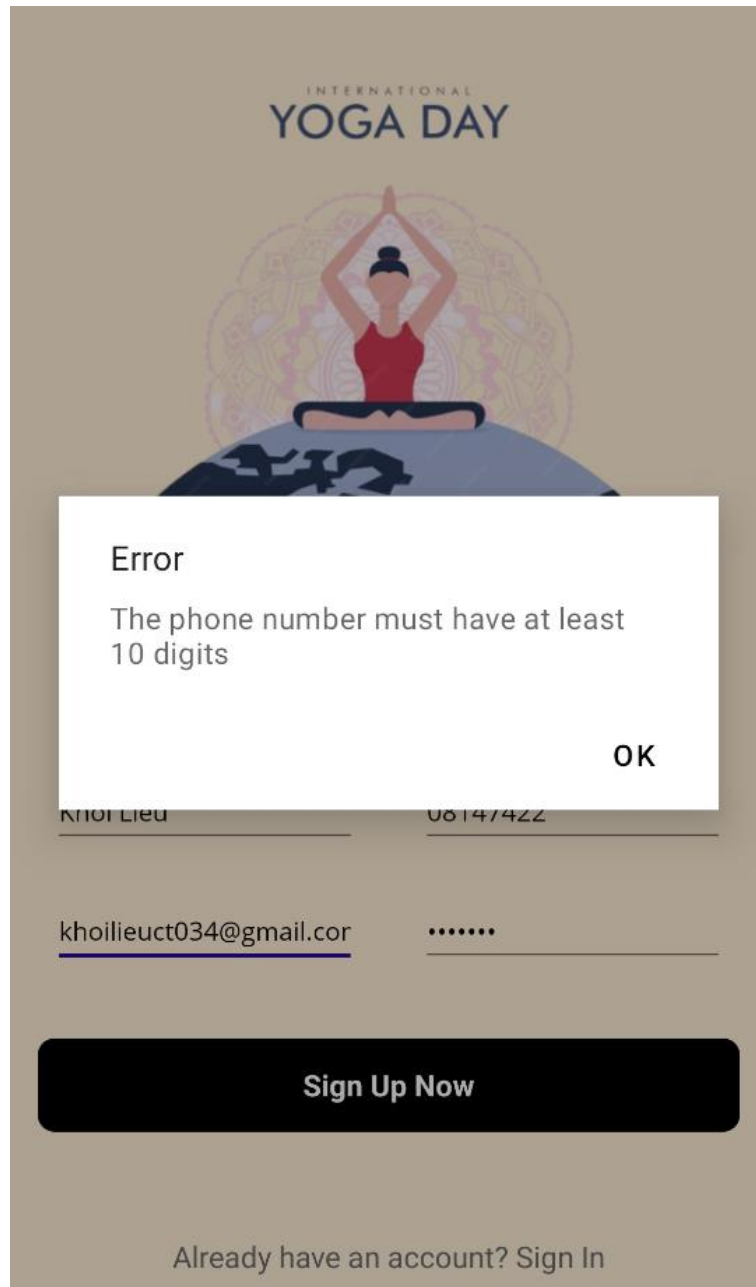


Figure 130. Account Registration Phone Number Length Error

Figure 130 displays an error message indicating that the phone number entered by the user does not meet the required length criteria. The notification states, "The phone number must have at least 10 digits." This message guides users to provide a valid phone number, ensuring that it adheres to standard formatting for contact information. The validation process is crucial in maintaining accurate records and effective communication with users.

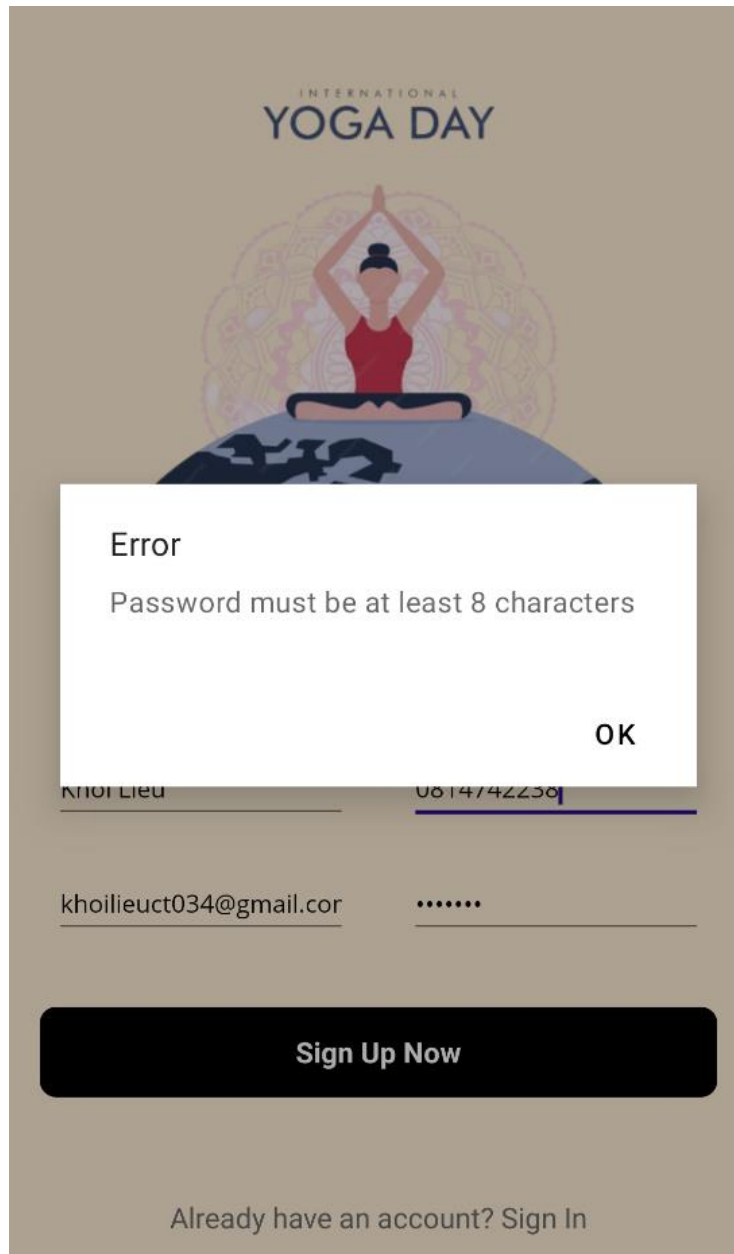


Figure 131. Registration Error Notification - Password Validation

Figure 131 illustrates an error notification that says "Password must be at least 8 characters" when a user sets a password that doesn't meet the minimum length requirement. This alert encourages users to create strong passwords, enhancing security for their accounts.

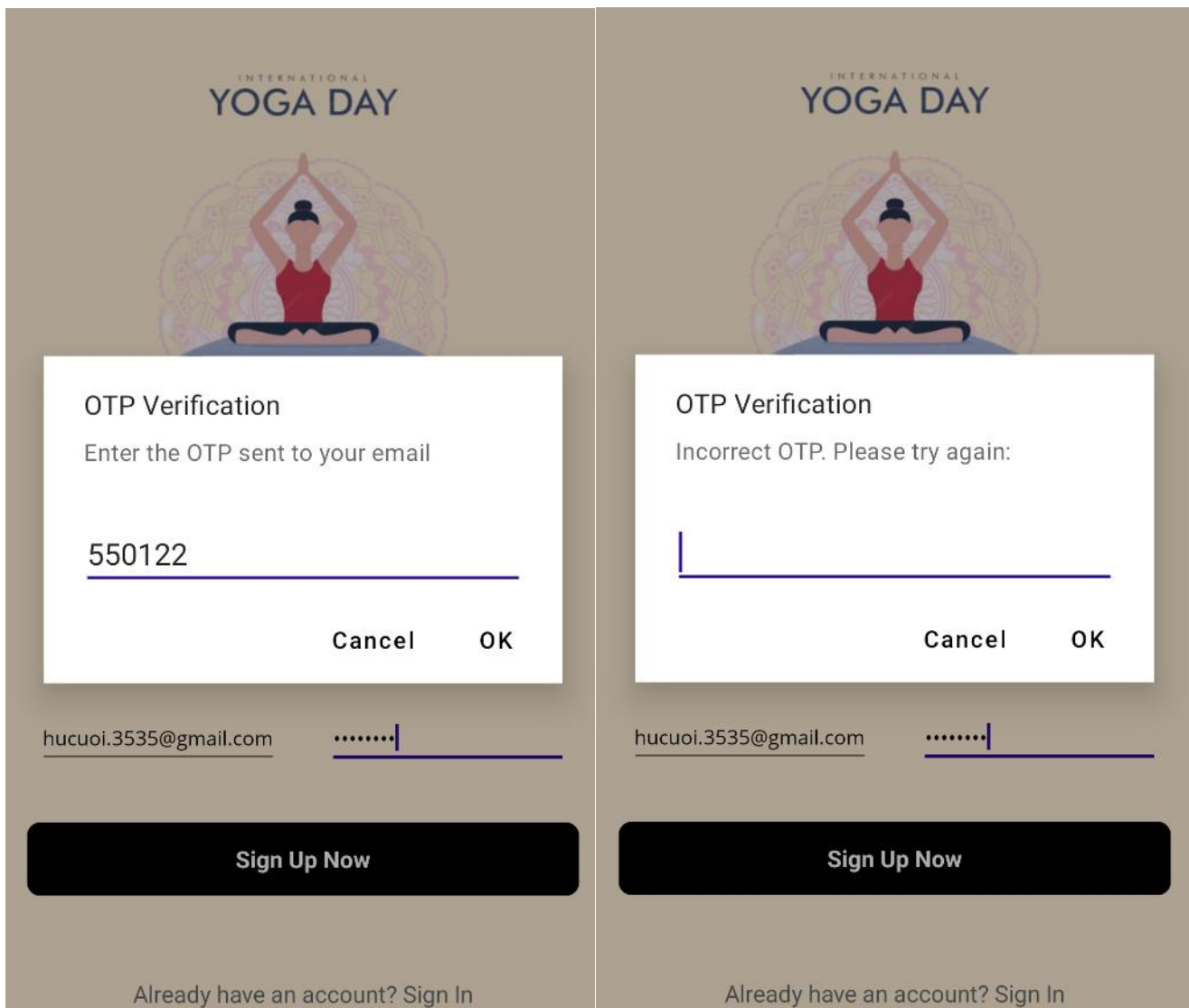


Figure 132. OTP Verification During Sign-Up

Figure 132 shows the OTP verification interface during the sign-up process. In the first image, the user is prompted to enter the OTP sent to their email to verify their account. The second image displays an error message, "Incorrect OTP. Please try again," indicating that the entered OTP was invalid. This interface helps ensure account security by verifying the user's email before proceeding with registration.

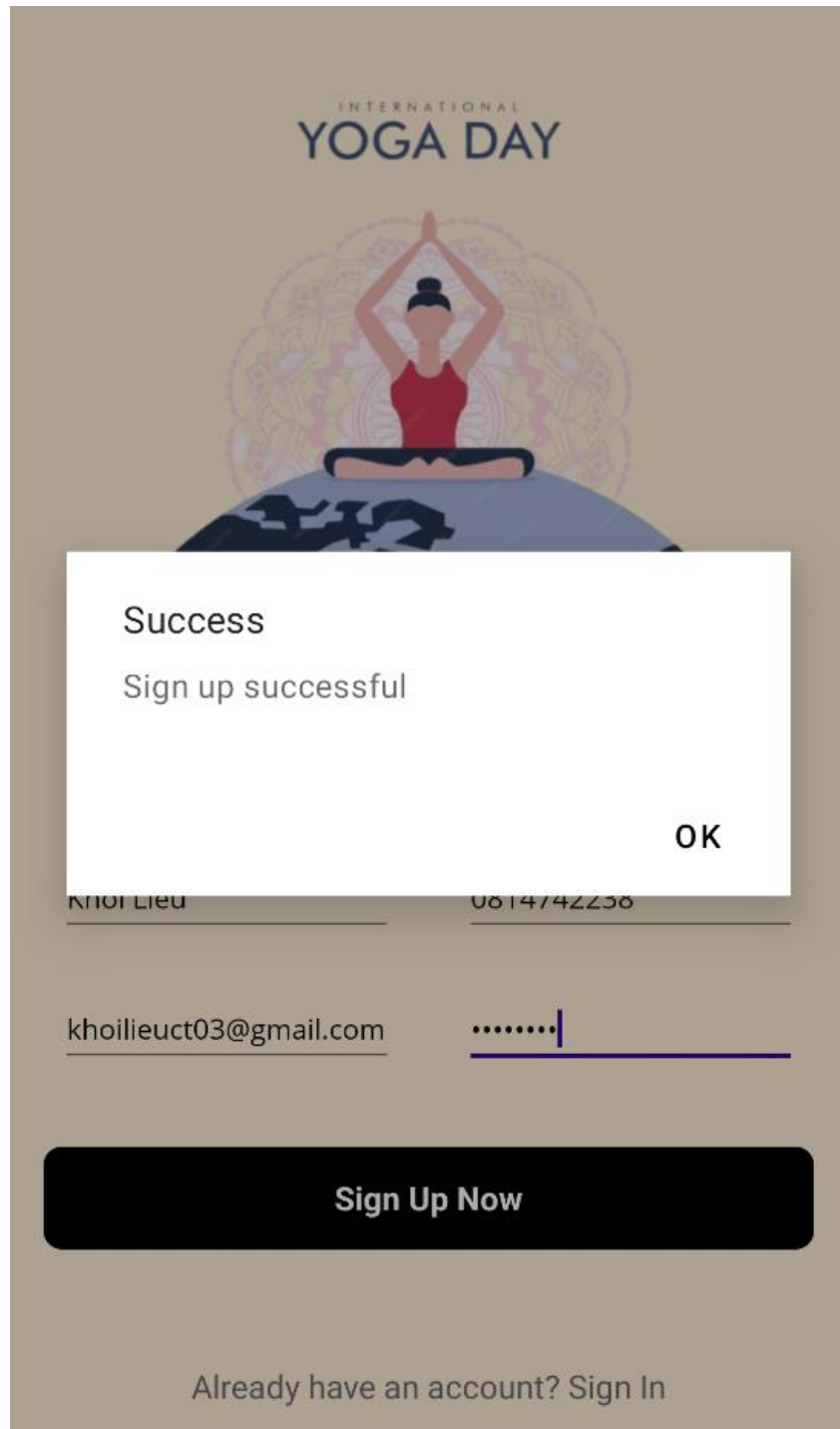


Figure 133. Successful Registration Notification

Figure 133 illustrates a successful registration message that reads “Registration Successful”. This message confirms that the user has successfully created an account by entering the correct OTP, allowing them to proceed to the next steps in the app, such as logging in and accessing yoga classes. Positive feedback enhances the user experience by providing peace of mind after creating an account.

3.2.3 Home page:

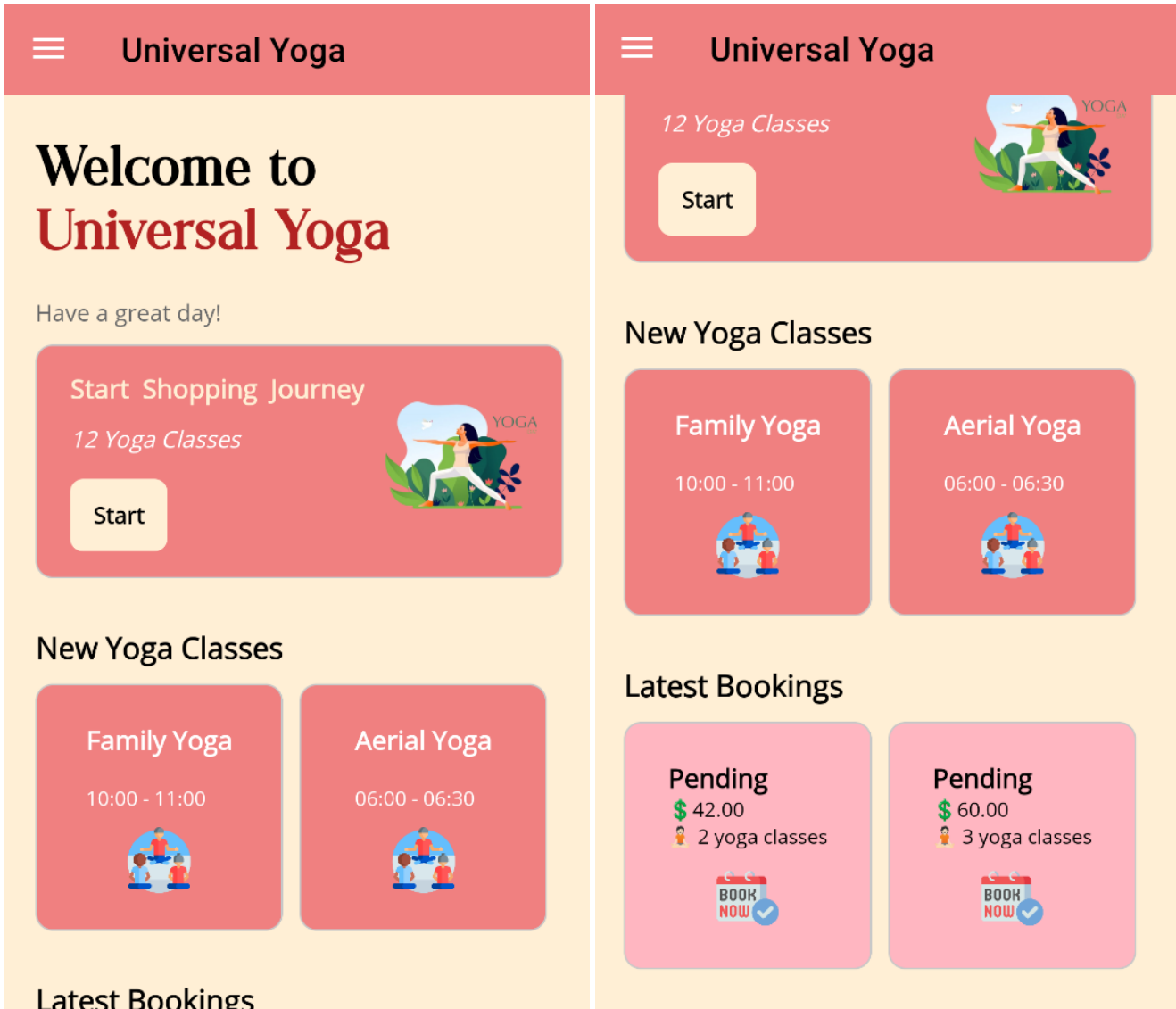


Figure 134. Home Page Interface of Universal Yoga

Figure 134 illustrates the home page of the Universal Yoga application. This interface welcomes users to the platform and provides them with a summary of available yoga classes and their latest bookings. The "Start Shopping Journey" button encourages users to begin exploring yoga classes, while the "New Yoga Classes" section showcases the latest offerings with their respective schedules. This layout enhances user engagement by offering quick access to important features, helping users easily navigate through the app.

3.2.4 Sidebar of the application:

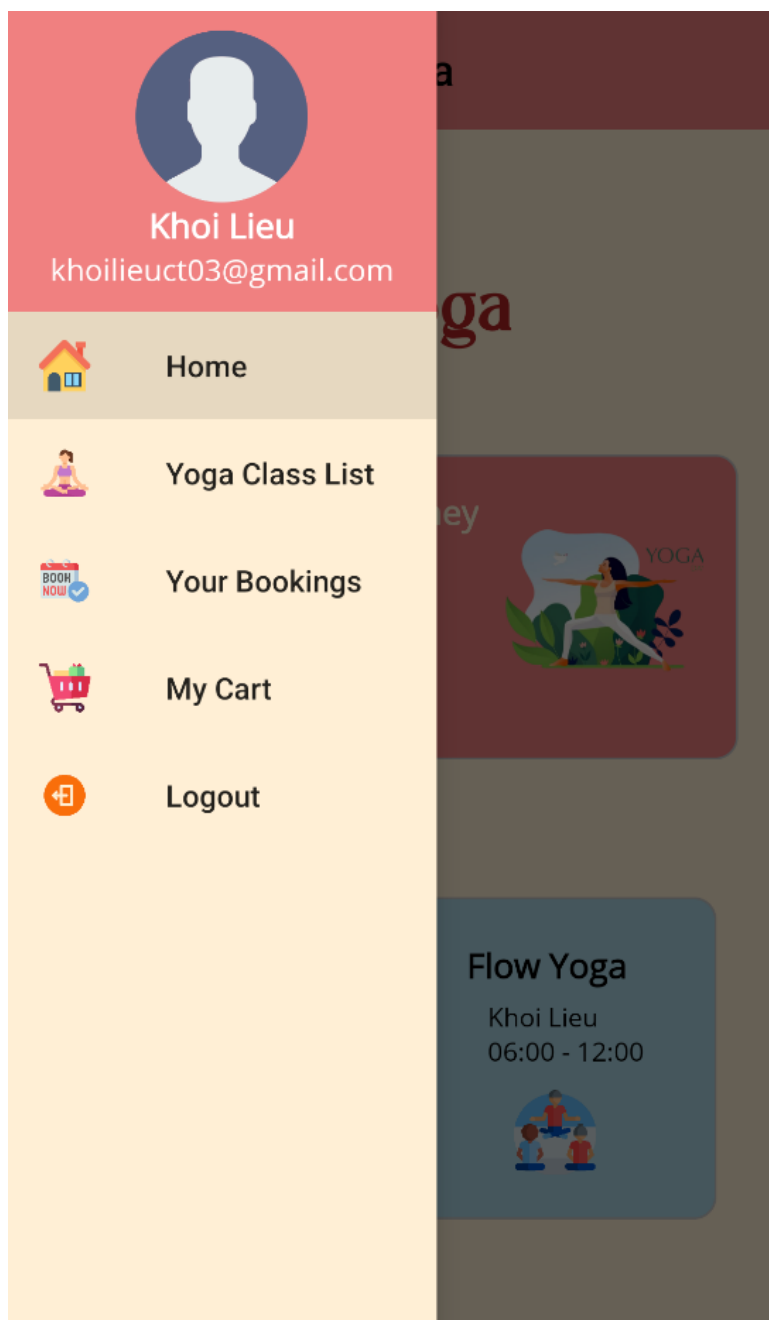


Figure 135. Sidebar Navigation Menu of Universal Yoga

Figure 135 showcases the sidebar navigation menu of the Universal Yoga application. This interface provides users with quick access to various sections, including "Home," "Yoga Class List," "Your Bookings," "My Cart," and "Logout." The sidebar enhances user experience by allowing easy navigation through the app's features, enabling users to efficiently manage their yoga classes and bookings while maintaining a user-friendly design.

3.2.5 Yoga class list:

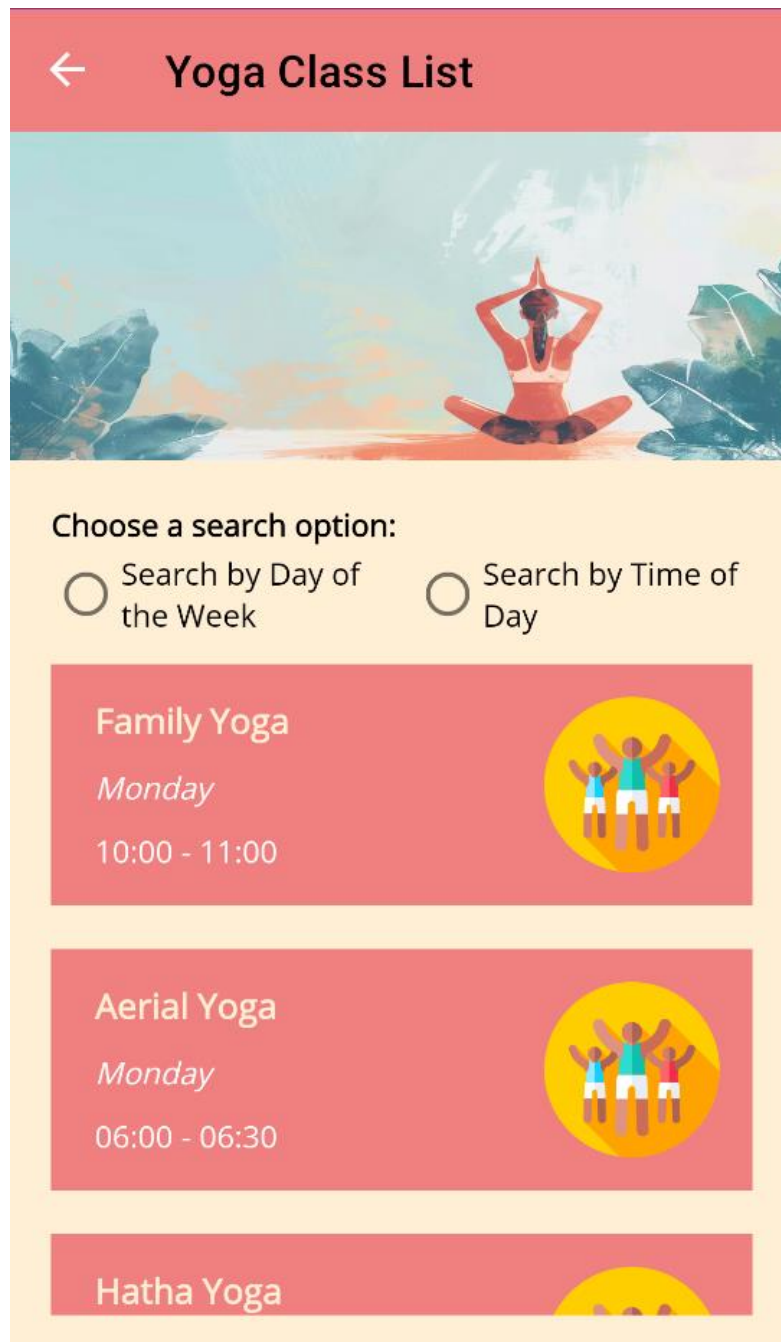


Figure 136. Yoga Class List Interface

Figure 136 presents the Yoga Class List interface, where users can select their preferred search method. Users can choose to search for classes either by "Day of the Week" or "Time of Day." This design enhances usability by providing options for filtering yoga classes, making it easier for users to find suitable classes based on their schedules. The list includes various yoga classes, displaying relevant information such as the class name, day, and time.

3.2.6 Search yoga class by day of the week:

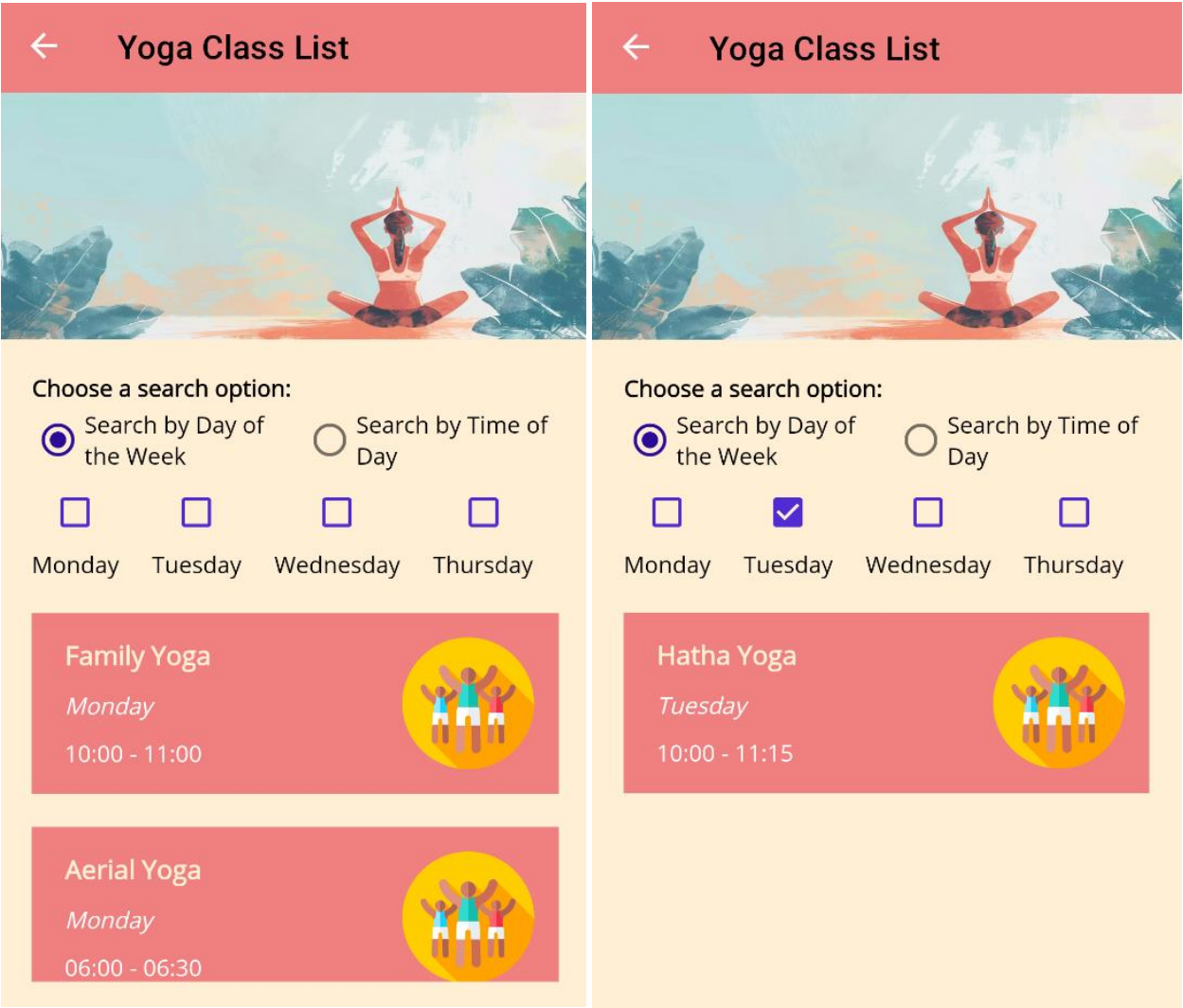


Figure 137. Search Yoga Classes by Day of the Week

Figure 137 displays the "Yoga Class List" interface, where users can search for yoga classes based on the day of the week. The user has selected "Search by Day of the Week," allowing them to check available classes on specific days. The second image shows the results for "Tuesday," highlighting the "Hatha Yoga" class with its scheduled time. This functionality enhances user experience by enabling quick access to relevant classes based on their availability.

3.2.7 Search yoga class by time of day:

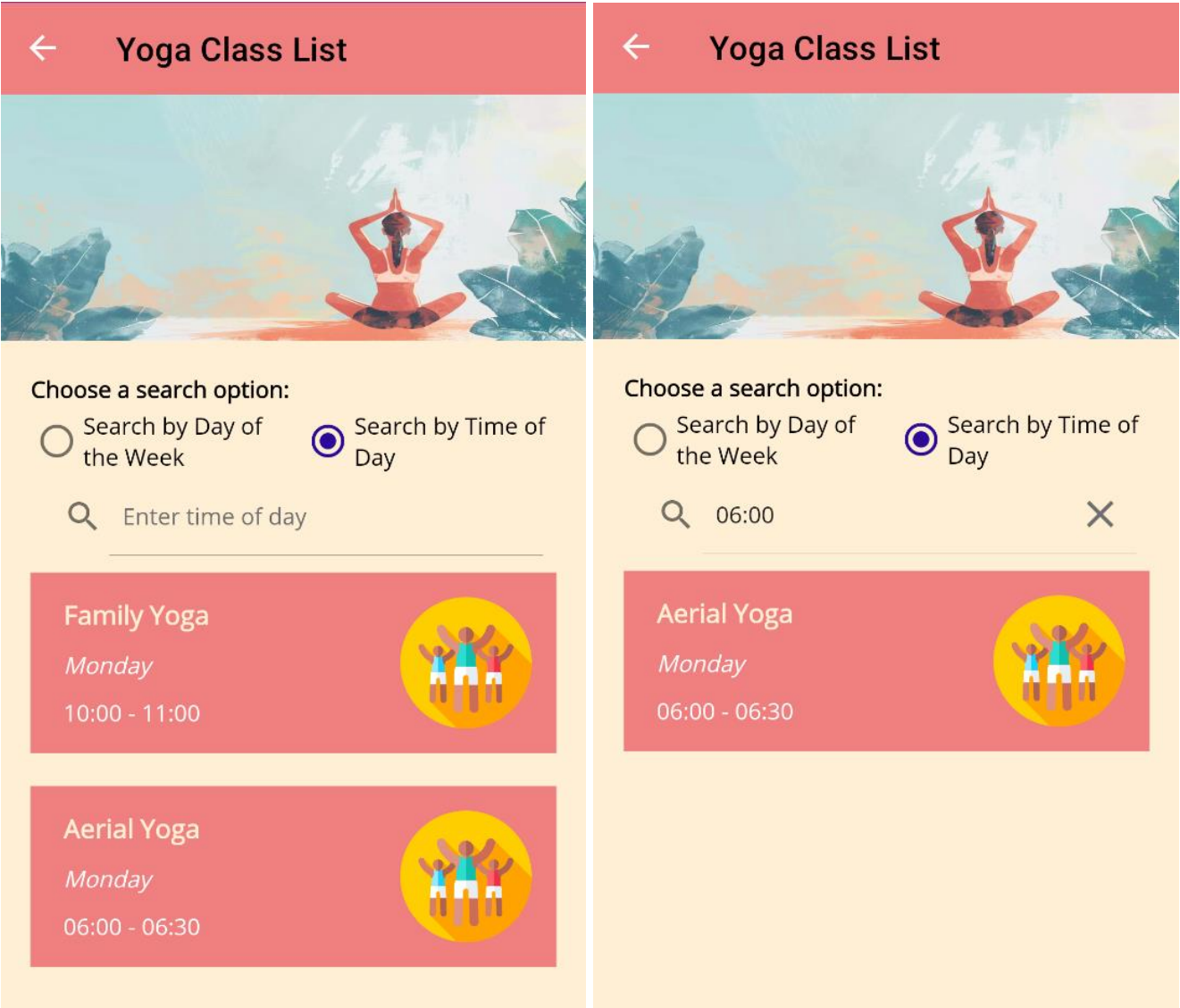


Figure 138. Search Yoga Class by Time of Day

Figure 138 illustrates the "Yoga Class List" screen, where users can search for yoga classes based on the time of day. Initially, the user selects the "Search by Time of Day" option, allowing them to enter a specific time to find classes that match. After entering "06:00," the system displays the relevant yoga class, "Aerial Yoga," scheduled from 06:00 to 06:30. This feature enhances user experience by facilitating quick access to classes at desired times.

3.2.8 Yoga class details:

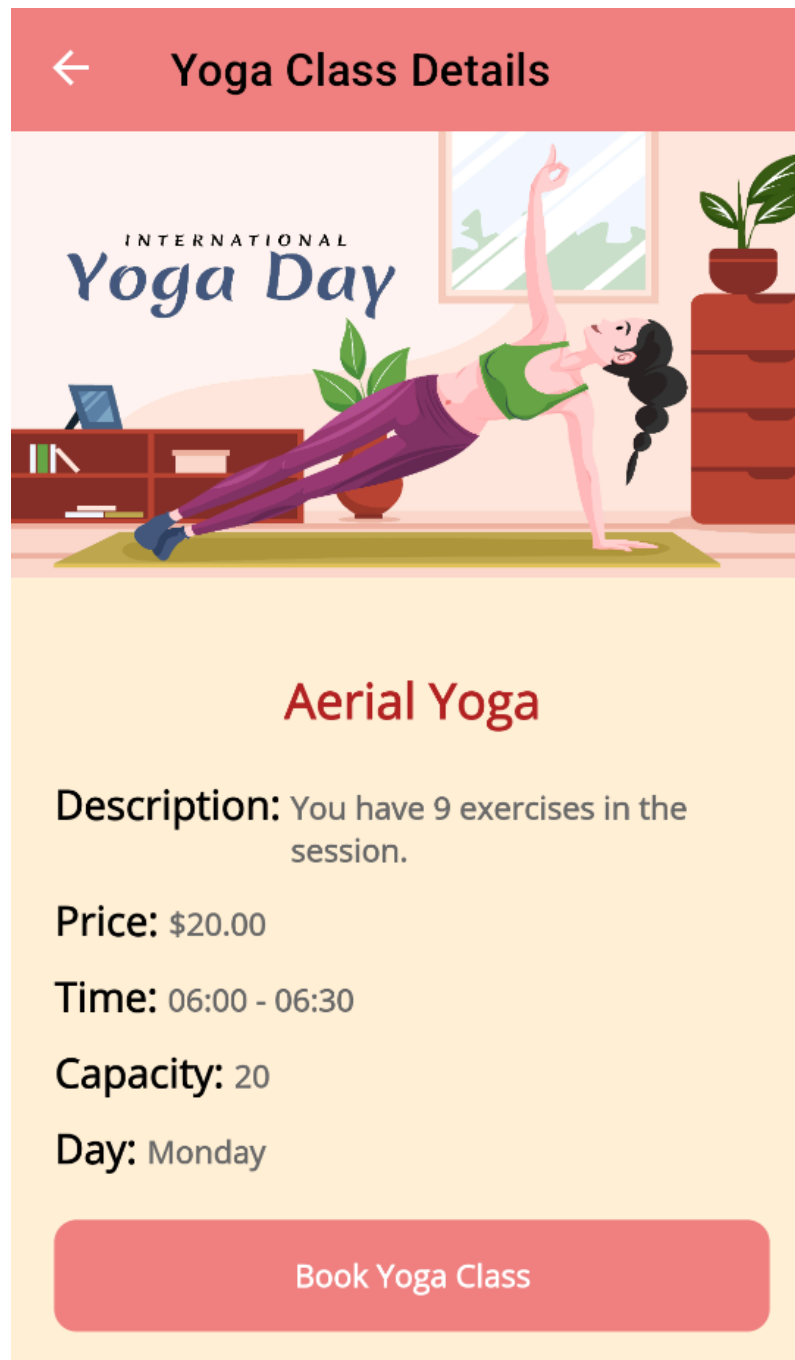


Figure 139. Yoga Class Details

Figure 139 displays the "Yoga Class Details" screen for the "Aerial Yoga" class. This interface provides essential information about the class, including its description, price, scheduled time, capacity, and day of the week. The user is presented with a clear overview of what to expect during the session, such as the number of exercises involved. This detailed information aids users in making informed decisions before booking the class.

3.2.9 Book yoga class:

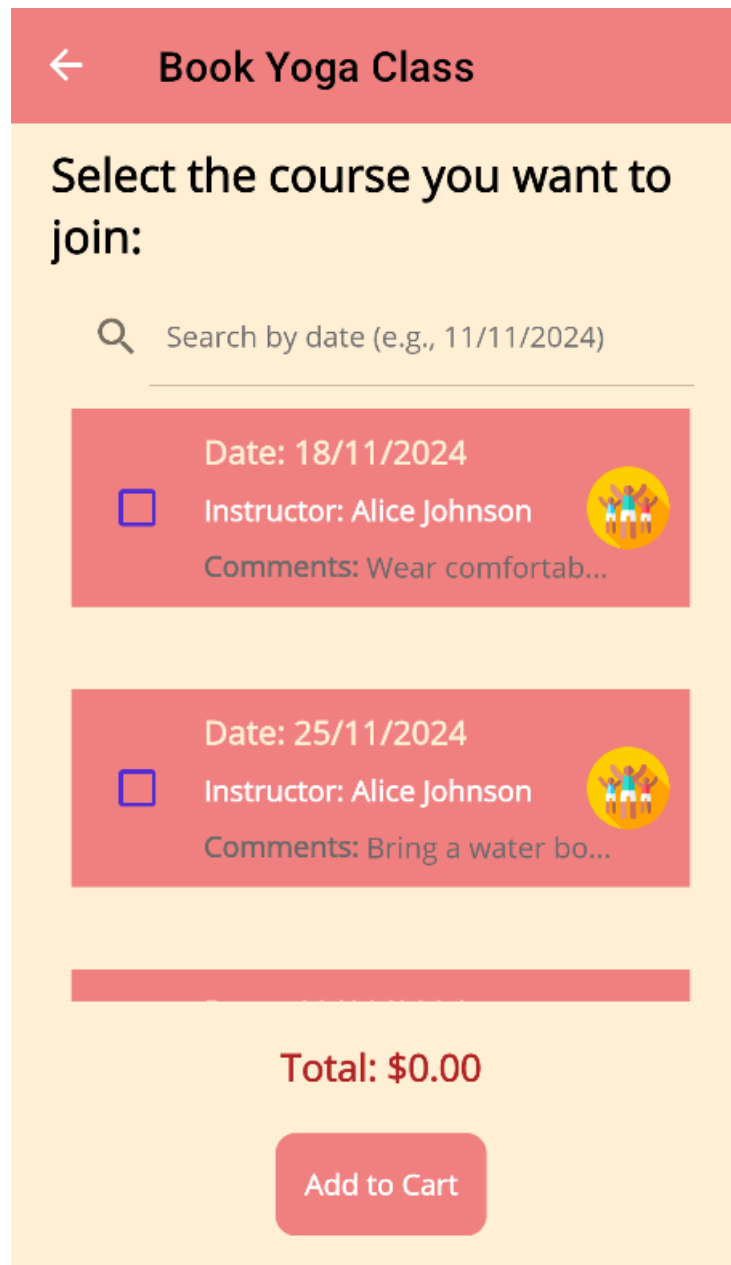


Figure 140. Book Yoga Class

Figure 140 presents the "Book Yoga Class" screen, where users can select the class instances they wish to join. The interface lists available class instances, including the date, instructor's name, and any additional comments. This information helps users decide which class to book, making the booking process straightforward and user-friendly. The total cost is displayed at the bottom, reinforcing transparency in pricing, and the "Add to Cart" button allows users to proceed with their selections conveniently.

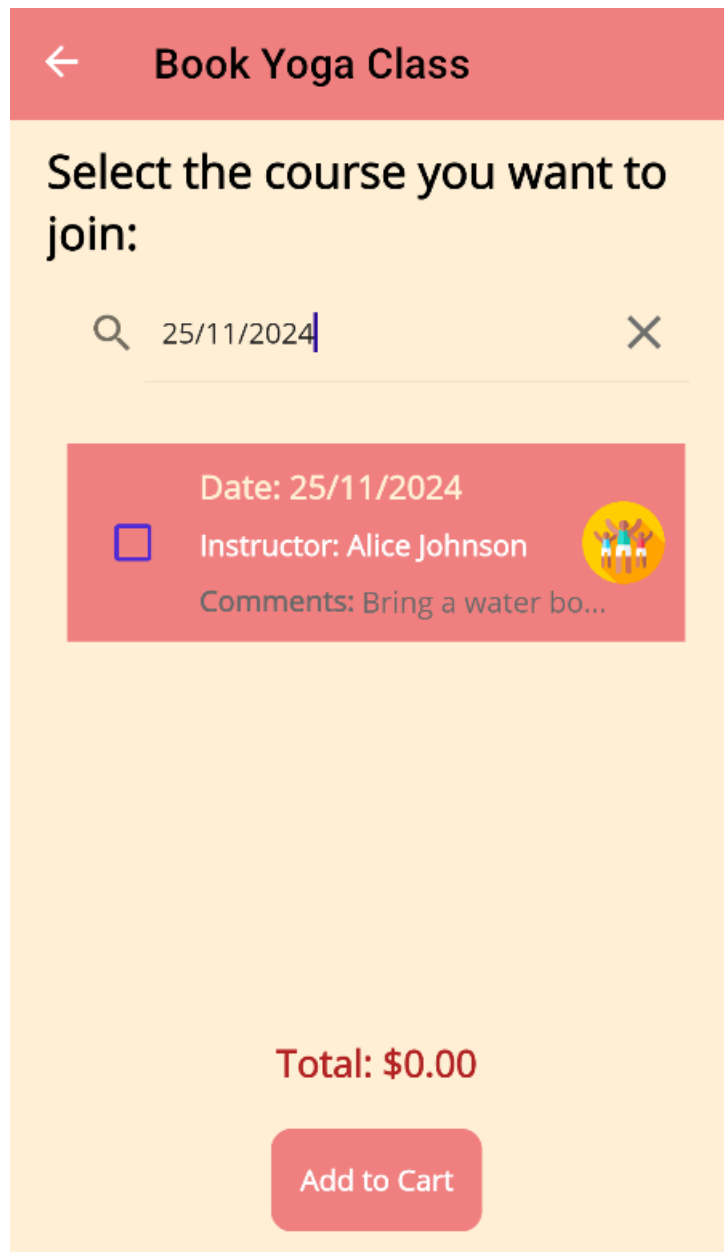


Figure 141. Date-based class instance search feature.

Figure 141 shows the search functionality for filtering yoga class instances based on date. Users can input a specific date into the search bar, and matching class instances will be displayed in real-time below. The search feature makes it easy to locate available sessions by date, allowing users to quickly find and select the desired yoga class instance, enhancing user convenience and navigation efficiency within the booking interface.

Book Yoga Class

Select the course you want to join:

 Search by date (e.g., 11/11/2024)

☒

Date: 18/11/2024

Instructor: Alice Johnson

Comments: Wear comfortab...



☐

Date: 25/11/2024

Instructor: Alice Johnson

Comments: Bring a water bo...



Total: \$20.00

Add to Cart

Figure 142. Book Yoga Class - Course Selection

Figure 142 displays the "Book Yoga Class" screen, where the user has selected a class instance to join. The selected session shows a checkbox, indicating the user's choice, along with details such as the date, instructor, and any additional comments. The total cost is updated to \$20.00, reflecting the selected class, and the "Add to Cart" button remains available for proceeding with the booking.

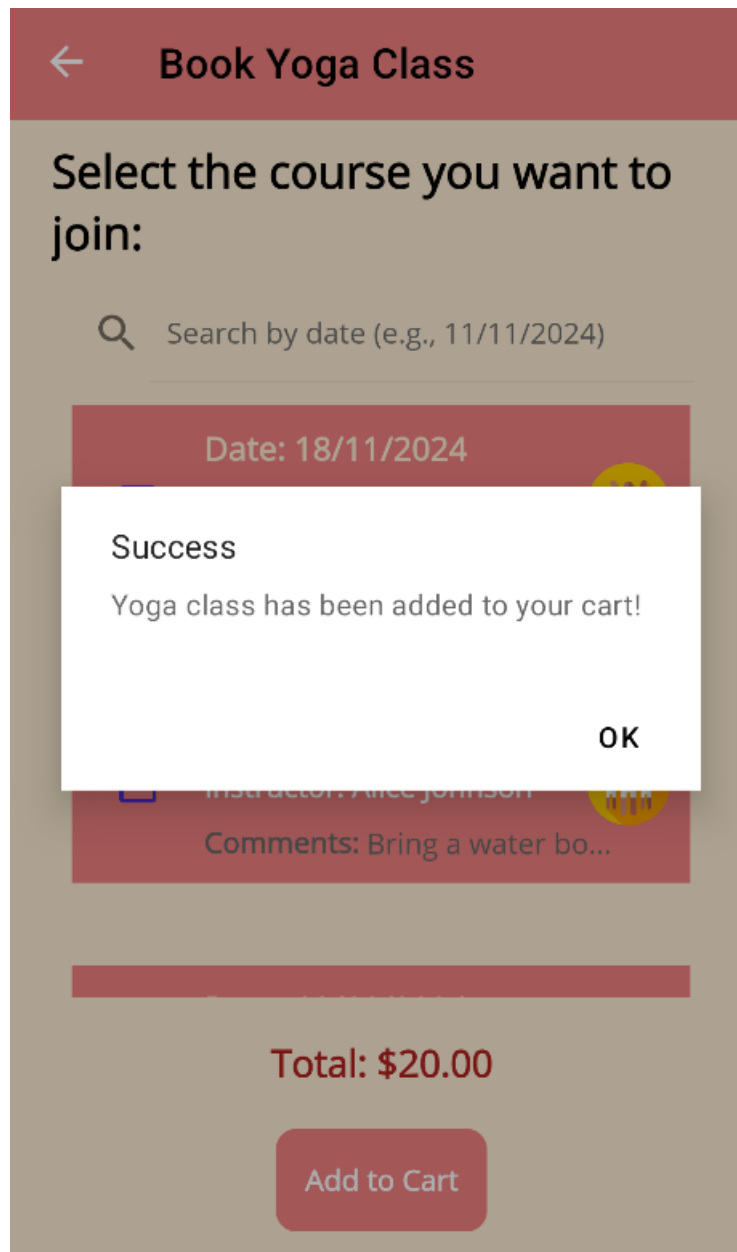


Figure 143. Book Yoga Class - Successful Addition to Cart

Figure 143 shows the "Book Yoga Class" screen, where the user has successfully added a yoga class to their cart. A success notification appears, stating that the yoga class has been added to the cart. The user can still see the details of the selected class, including the date, instructor, and additional comments, while the total amount is updated to \$40.00, indicating the cost of the selected classes.

Book Yoga Class

Select the course you want to join:

Search by date (e.g., 11/11/2024)

Date: 18/11/2024

Instructor: Alice Johnson

Comments: Wear comfortab...

Date: 25/11/2024

Instructor: Alice Johnson

Comments: Bring a water bo...

Total: \$0.00

Add to Cart

Figure 144. Book Yoga Class - Disabled Selection for Existing Class Instances

Figure 144 illustrates the "Book Yoga Class" screen, where previously selected class instances have their checkboxes disabled, preventing users from selecting them again. The details for each class, including the date, instructor, and comments, are still displayed, but the user can no longer modify the selection for the class on 11/11/2024. The total remains at \$0.00, indicating no new selections can be made for that date.

COMP1786

Page 137 | 284

3.2.10 Shopping cart:

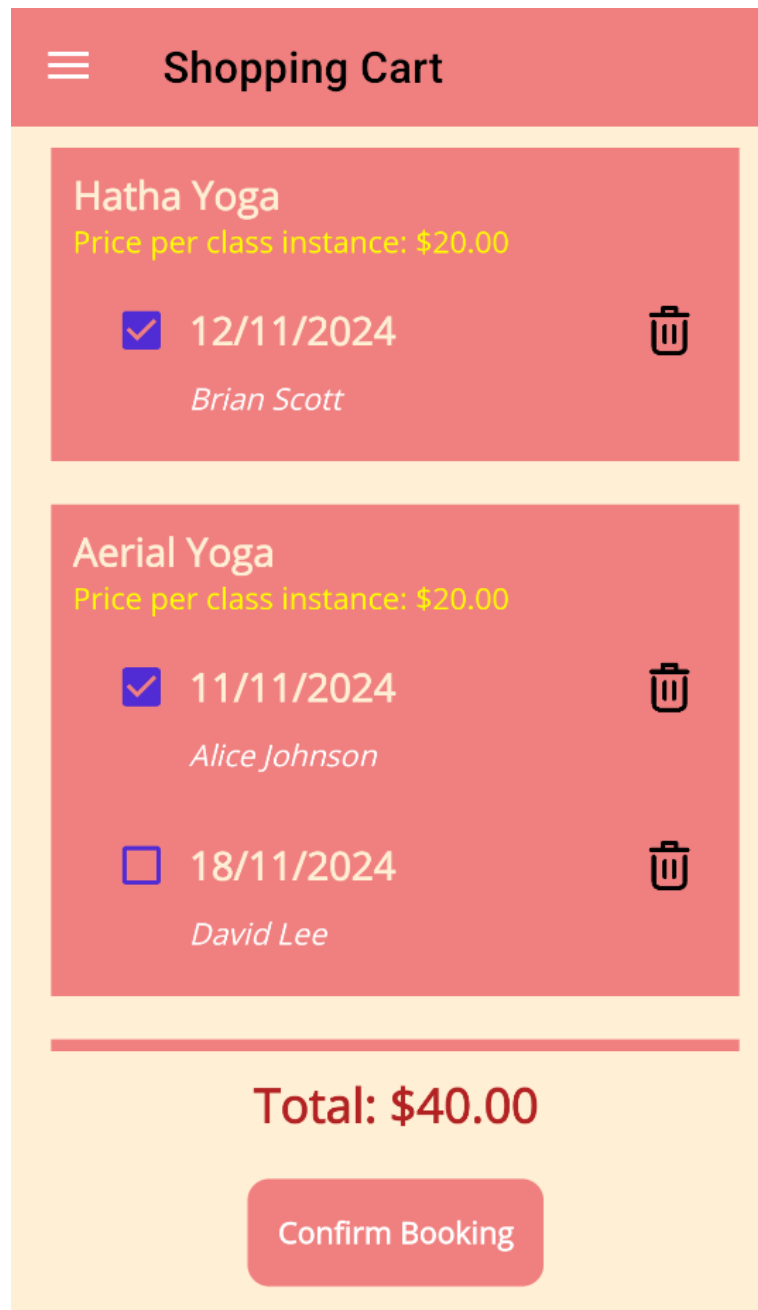


Figure 145. Shopping Cart - Selected Class Instances

Figure 145 illustrates the "Shopping Cart" interface, where users can select the yoga class instances they wish to book. The cart displays each class instance according to the yoga classes that have been previously added to the cart, including "Hatha Yoga" and "Aerial Yoga," along with the corresponding date, instructor, and price per class instance. Users can check the boxes next to the class instances they want to book. The total amount due is \$40.00, and there is an option to remove any selected class instance using the trash icon.

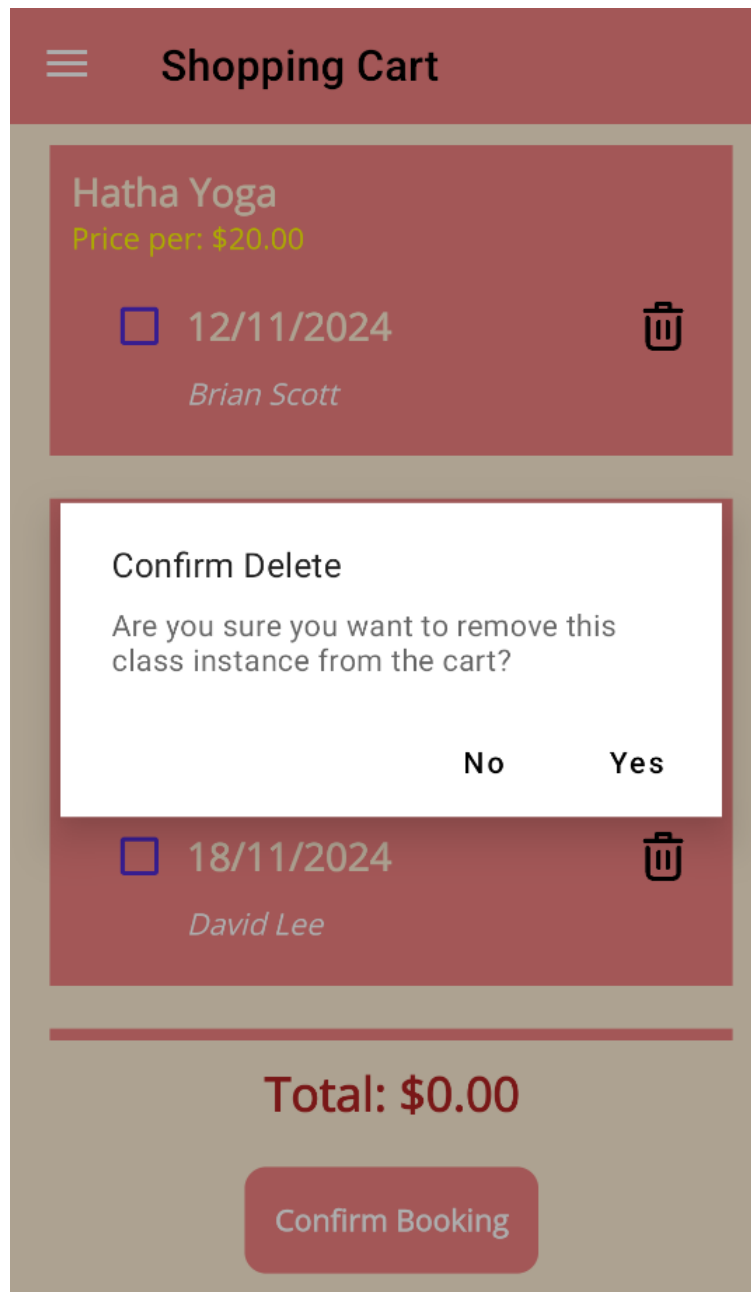


Figure 146. Shopping Cart - Confirm Delete

Figure 146 shows the "Shopping Cart" interface with a confirmation dialog prompting the user to confirm the deletion of a selected class instance. The dialog helps ensure users are certain about removing the class instance from their cart before proceeding.

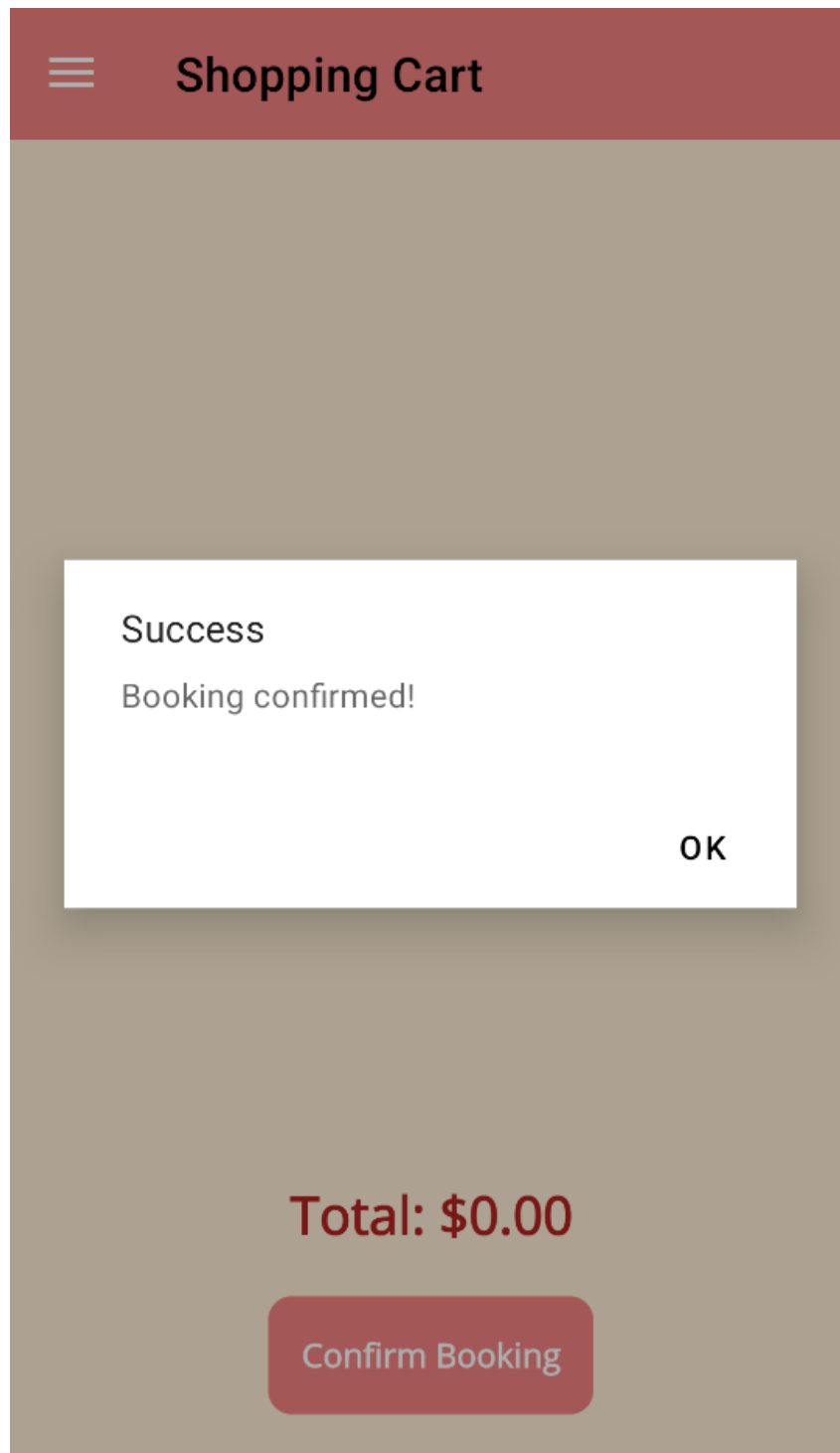


Figure 147. Booking Confirmation Success

Figure 147 shows the "Booking confirmed!" success message displayed after the user completes the booking process in the shopping cart. This confirmation assures the user that their selected yoga classes have been successfully booked, providing a sense of completion and clarity about the transaction.

3.2.11 My bookings:

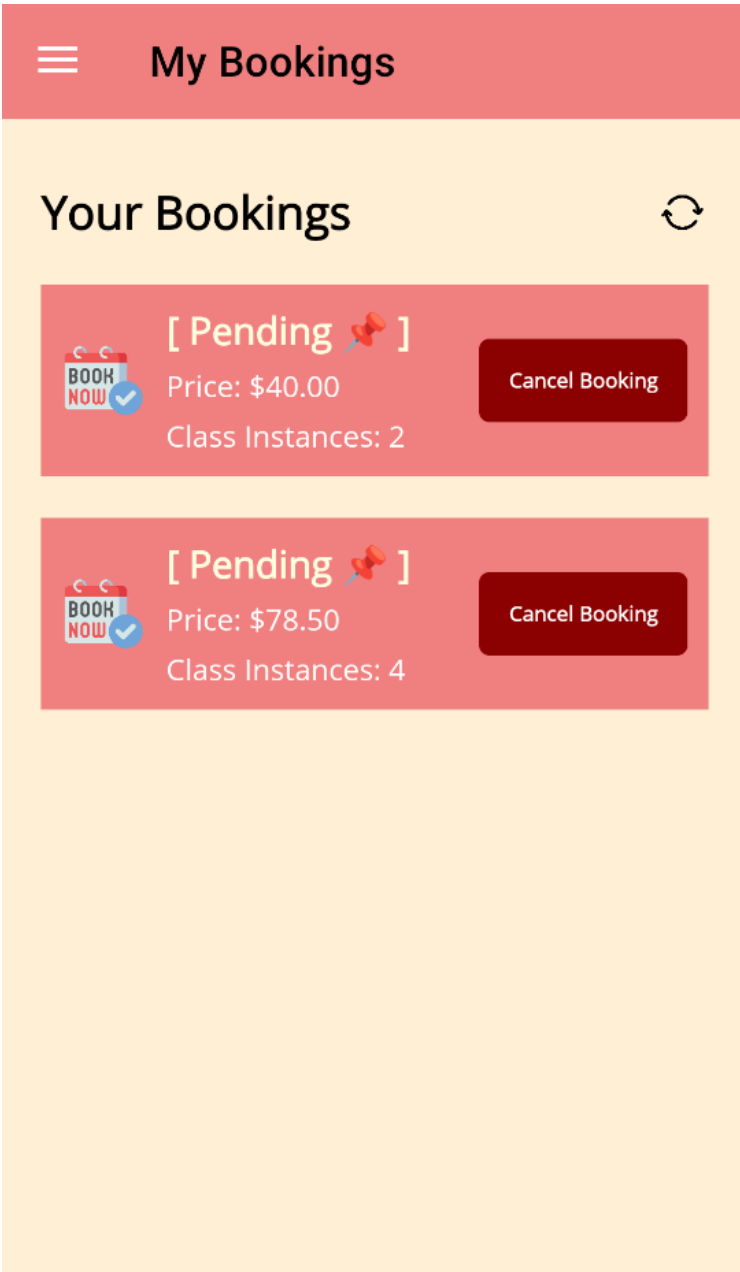


Figure 148. My Bookings - Pending Status

Figure 148 illustrates the "My Bookings" screen, where users can view their current pending yoga class bookings. Each booking entry lists the price and the number of class instances. A "Cancel Booking" button is provided next to each booking, enabling users to cancel their reservations if necessary. A refresh icon is also available at the top right, allowing users to reload the page to check for the most up-to-date information on their bookings.

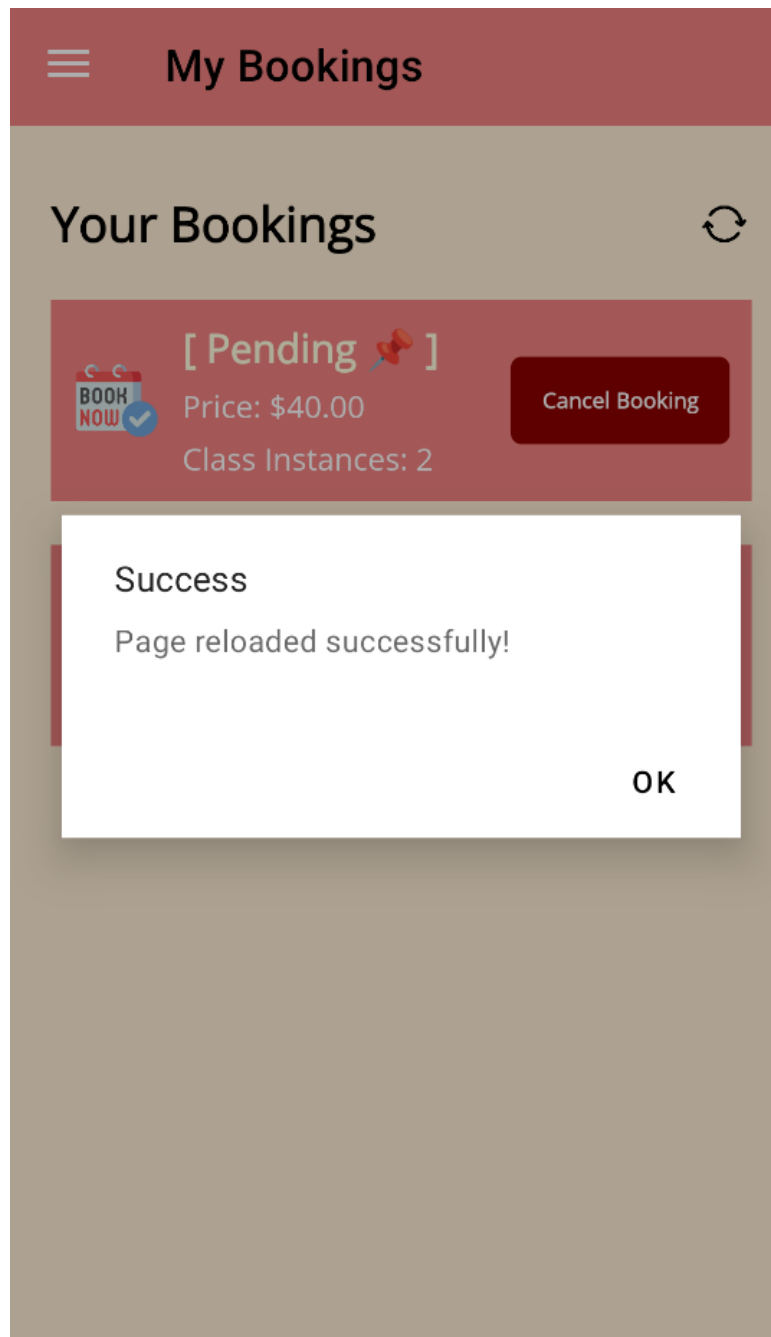


Figure 149. My Bookings - Page Reload Confirmation

Figure 149 shows the "My Bookings" screen with a confirmation message displayed after using the refresh icon. This message, "Page reloaded successfully!" informs the user that the booking page has been successfully updated, reflecting the latest booking information. The screen still displays the pending booking with options to view details or cancel if necessary.

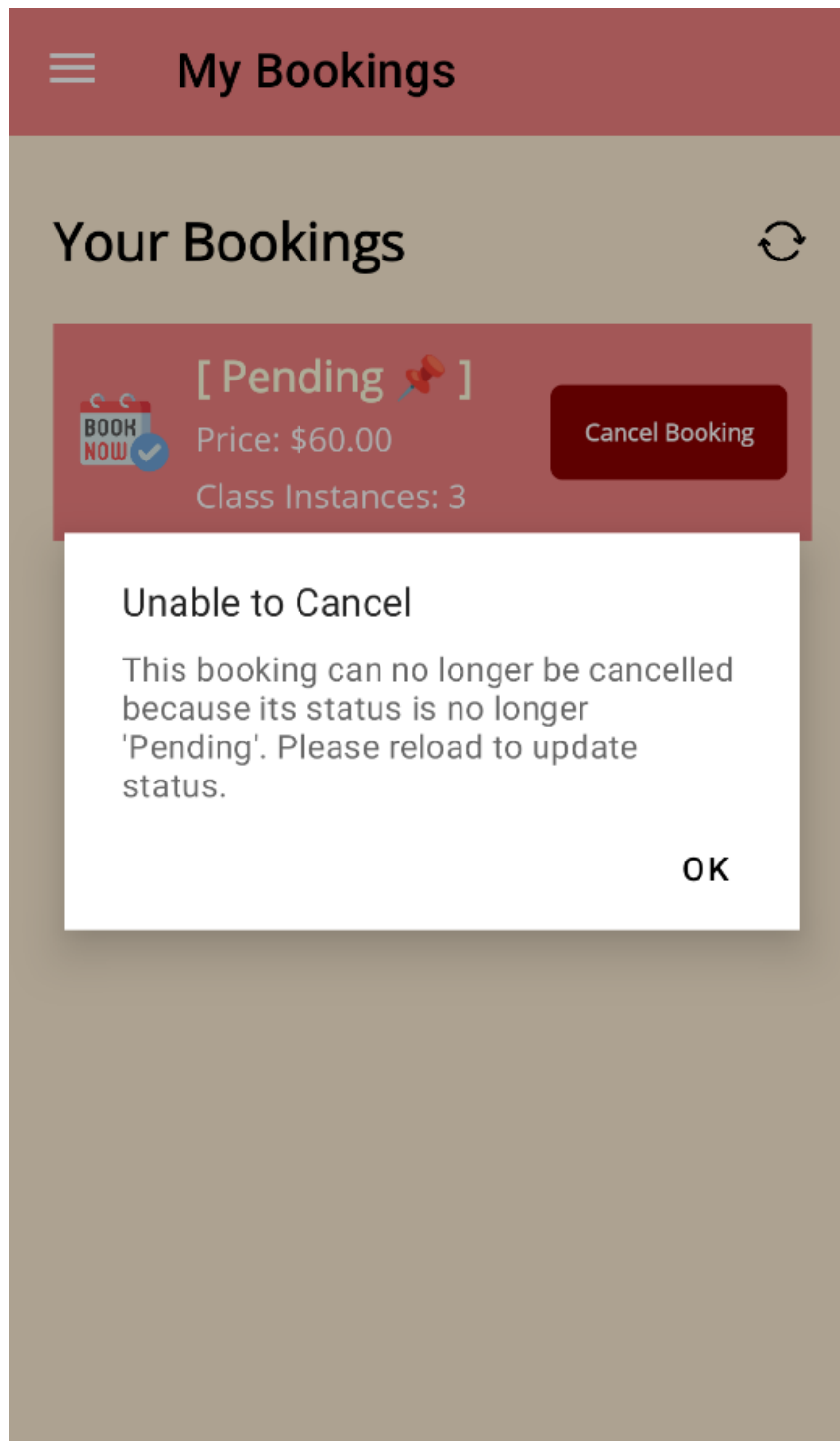


Figure 150. Unable to Cancel Booking - Status Update Required

Figure 150 displays a notification in the "My Bookings" screen indicating that the selected booking can no longer be canceled due to a change in its status. The message, "This booking can no longer be cancelled because its status is no longer 'Pending'. Please reload to update status," prompts the user to refresh the page to see the updated status, as the booking status has changed since the last load.

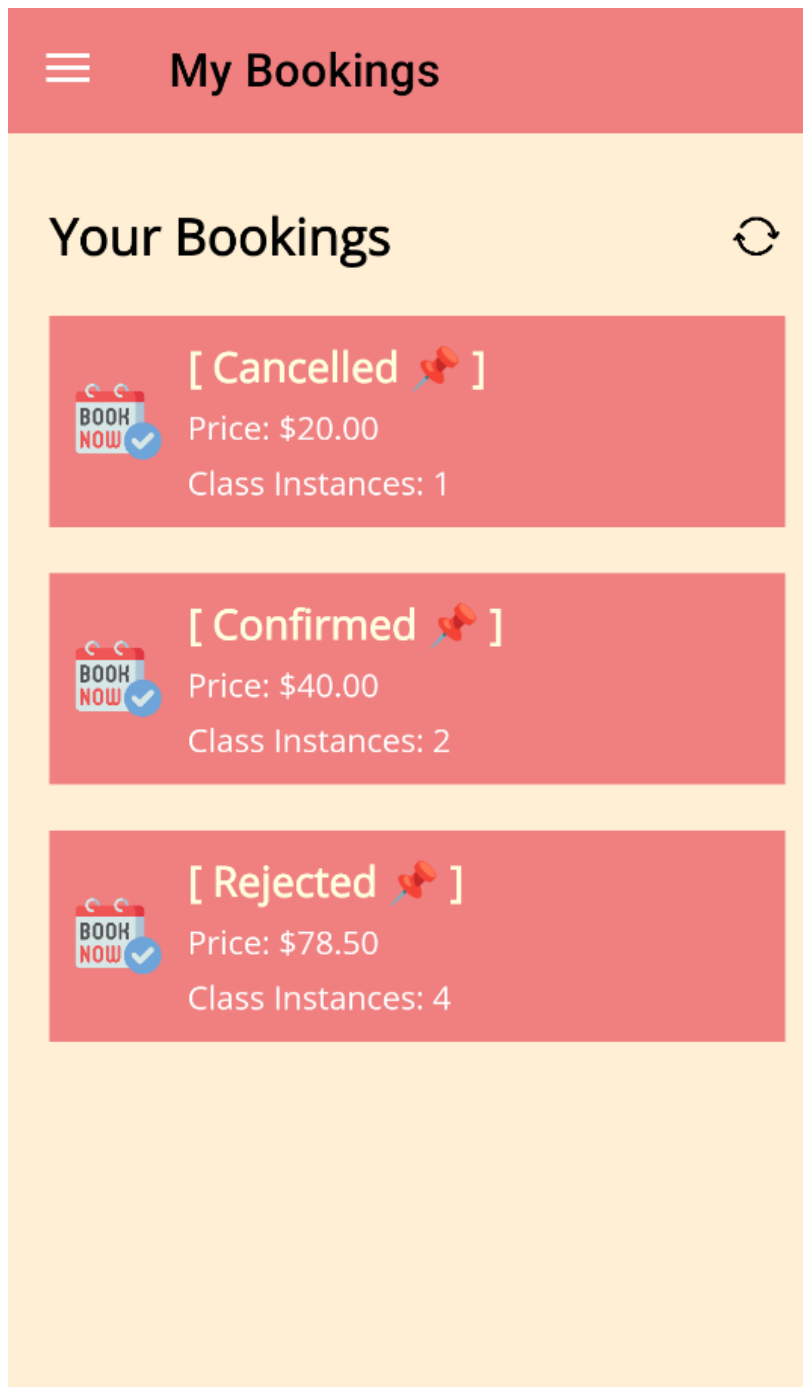


Figure 151. Display of Various Booking Statuses with Auto-Refresh Option

Figure 151 illustrates the "My Bookings" page, showcasing bookings with different statuses: "Cancelled," "Confirmed," and "Rejected." The page allows users to manually refresh by tapping the loading icon, and it also checks for status updates every 10 seconds, automatically reloading if there are changes from the database, ensuring the user views the latest booking information.

3.2.12 My booking details:

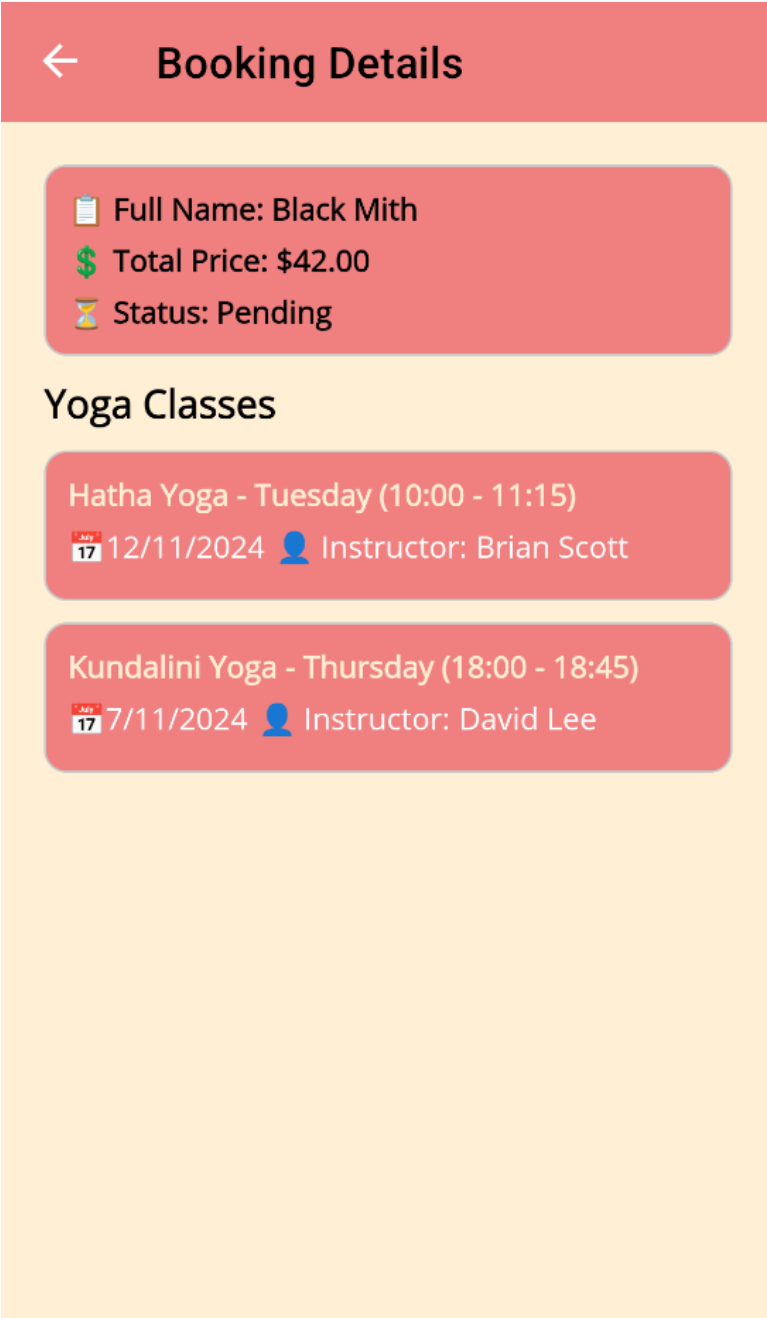


Figure 152. Pending Booking Details with Grouped Yoga Class Instances

Figure 152 shows the "Booking Details" page for a pending booking. It displays the user’s name, the total price of the booking, and the status as "Pending." Below this summary, yoga classes are grouped, each showing specific class instances that the user has booked. For each yoga class, relevant instances are listed with details such as type, day, time, date, and instructor. This layout helps users easily review all the individual sessions they have booked, organized by yoga class.

5. SECTION 4 - CODE

5.1 Admin Application

5.1.1 DatabaseHelper.java

```
@Override
public void onCreate(SQLiteDatabase db) {

    String createClassTypeTable = "CREATE TABLE " + TABLE_CLASS_TYPE + "(" +
        COLUMN_CLASS_TYPE_ID + " TEXT PRIMARY KEY,"
        + COLUMN_CLASS_TYPE_NAME + " TEXT,"
        + COLUMN_CLASS_TYPE_STATUS + " TEXT)";
    db.execSQL(createClassTypeTable);

    String createBookingTable = "CREATE TABLE " + TABLE_BOOKINGS + "(" +
        COLUMN_BOOKING_ID + " TEXT PRIMARY KEY," +
        COLUMN_BOOKING_STATUS + " TEXT," +
        COLUMN_CAN_CANCEL + " INTEGER," +
        COLUMN_TOTAL_PRICE + " INTEGER," +
        COLUMN_BOOKING_USER_ID + " TEXT," +
        COLUMN_CLASS_INSTANCES + " TEXT" +
        ")";
    db.execSQL(createBookingTable);

    String createUserTable = "CREATE TABLE " + TABLE_USERS + "(" +
        COLUMN_USER_ID + " TEXT PRIMARY KEY," +
        COLUMN_FULL_NAME + " TEXT," +
        COLUMN_EMAIL + " TEXT," +
        COLUMN_PASSWORD + " TEXT," +
        COLUMN_PHONE_NUMBER + " TEXT," +
        COLUMN_STATUS + " TEXT," +
        COLUMN_ROLE + " TEXT)";
    db.execSQL(createUserTable);

    String createYogaClassesTable = "CREATE TABLE " + TABLE_YOGA_CLASSES + "(" +
        COLUMN_ID + " TEXT PRIMARY KEY," +
        COLUMN_DAY_OF_WEEK + " TEXT," +
        COLUMN_TIME + " TEXT," +
        COLUMN_CAPACITY + " INTEGER," +
        COLUMN_DURATION + " INTEGER," +
        COLUMN_PRICE + " REAL," +
        COLUMN_TYPE_OF_CLASS + " TEXT," +
        COLUMN_DESCRIPTION + " TEXT," +
        COLUMN_STATUS + " TEXT)";
    db.execSQL(createYogaClassesTable);

    String createTableClassInstance = "CREATE TABLE " + TABLE_CLASS_INSTANCES + "(" +
        COLUMN_ID + " TEXT PRIMARY KEY," +
        COLUMN_CLASS_ID + " TEXT," +
        COLUMN_DATE + " TEXT," +
        COLUMN_INSTRUCTOR + " TEXT," +
        COLUMN_ADDITIONAL_COMMENTS + " TEXT," +
        COLUMN_STATUS + " TEXT," +
        "FOREIGN KEY(" + COLUMN_CLASS_ID + ") REFERENCES " + TABLE_YOGA_CLASSES + "(" + COLUMN_ID + ")";
    db.execSQL(createTableClassInstance);
}
```

This code shows the `onCreate()` method in the `DatabaseHelper` class, responsible for initializing the database by creating essential tables. Figure defines SQL statements for creating tables like `ClassType`,

Bookings, Users, YogaClasses, and ClassInstances. Each table is structured with relevant columns, setting primary keys and foreign key references where necessary. This setup is essential for organizing data related to users, bookings, yoga classes, and their instances, ensuring efficient data storage and retrieval in the application.

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_USERS);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_YOGA_CLASSES);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_CLASS_INSTANCES);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_BOOKINGS);

    onCreate(db);
}
```

This code shows the `onUpgrade()` method in the `DatabaseHelper` class, which handles database upgrades by dropping existing tables (Users, YogaClasses, ClassInstances, Bookings) and recreating them. This method ensures that any structural changes to the database schema in a new version are applied by clearing outdated tables and reinitializing them, thus maintaining data structure consistency in the application.

```
@Override
public void onDowngrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_USERS);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_YOGA_CLASSES);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_CLASS_INSTANCES);
    db.execSQL("DROP TABLE IF EXISTS " + TABLE_BOOKINGS);

    onCreate(db);
}
```

This code displays the `onDowngrade()` method in the `DatabaseHelper` class, which manages database downgrades by removing existing tables (Users, YogaClasses, ClassInstances, Bookings) and recreating them. This approach ensures compatibility with older database versions by clearing current tables and reinitializing them, thus maintaining the application's structural integrity during a version rollback.

4.1.1.1 Yoga Classes

```
public boolean updateYogaClassDescription(String classId, String description, String status) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_DESCRIPTION, description);
    values.put(COLUMN_STATUS, status);

    int rowsUpdated = db.update(TABLE_YOGA_CLASSES, values, "id = ?", new String[]{classId});
    db.close();
    return rowsUpdated > 0;
}
```

This code shows the `updateYogaClassDescription` method, which updates the description and status of a specific yoga class in `TABLE_YOGA_CLASSES`. This method uses `classId` as the identifier to ensure that only the targeted yoga class is updated with new description and status values. This functionality allows for flexible management of class information without affecting other class data.

```
public boolean addYogaClass(YogaClass classModel) {
    SQLiteDatabase db = this.getWritableDatabase();
```

```

ContentValues contentValues = new ContentValues();
contentValues.put(COLUMN_ID, classModel.getId());
contentValues.put(COLUMN_DAY_OF_WEEK, classModel.getDayOfWeek());
contentValues.put(COLUMN_TIME, classModel.getTime());
contentValues.put(COLUMN_CAPACITY, classModel.getCapacity());
contentValues.put(COLUMN_DURATION, classModel.getDuration());
contentValues.put(COLUMN_PRICE, classModel.getPrice());
contentValues.put(COLUMN_TYPE_OF_CLASS, classModel.getTypeOfClass());
contentValues.put(COLUMN_DESCRIPTION, classModel.getDescription());
contentValues.put("status", classModel.getStatus());

long result = db.insert(TABLE_YOGA_CLASSES, null, contentValues);
db.close();

return result != -1;
}

```

This code illustrates the `addYogaClass` method, which is used to insert a new yoga class into `TABLE_YOGA_CLASSES`. It takes in a `YogaClass` model object and populates various fields like day, time, capacity, and type. This method enables adding new classes to the database, making them available for scheduling and booking.

```

public boolean updateYogaClass(YogaClass classModel) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();

    contentValues.put("dayOfWeek", classModel.getDayOfWeek());
    contentValues.put("time", classModel.getTime());
    contentValues.put("capacity", classModel.getCapacity());
    contentValues.put("duration", classModel.getDuration());
    contentValues.put("price", classModel.getPrice());
    contentValues.put("typeOfClass", classModel.getTypeOfClass());
    contentValues.put("description", classModel.getDescription());
    contentValues.put("status", classModel.getStatus());

    int result = db.update("yoga_classes", contentValues, "id = ?", new String[]{classModel.getId()});
    db.close();

    return result > 0;
}

```

This code presents the `updateYogaClass` method, designed to modify details of an existing yoga class in `TABLE_YOGA_CLASSES`. This method updates fields such as day, time, capacity, and type based on the provided `YogaClass` model. It allows for seamless updates to class information, aiding in effective schedule management.

```

public void deleteYogaClass(String id) {
    SQLiteDatabase db = this.getWritableDatabase();
    db.delete(TABLE_YOGA_CLASSES, "id = ?", new String[]{String.valueOf(id)});
    db.close();
}

```

This code demonstrates the `deleteYogaClass` method, which removes a specific yoga class from `TABLE_YOGA_CLASSES` using its id. This function allows the app to delete classes that are no longer needed, keeping the database organized and up-to-date.

```

public YogaClass getYogaClassById(String id) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.query(TABLE_YOGA_CLASSES, null, COLUMN_ID + " = ?",
        new String[]{id}, null, null, null);

    if (cursor != null && cursor.moveToFirst()) {
        YogaClass yogaClass = new YogaClass(
            cursor.getString(cursor.getColumnIndex(COLUMN_ID)),
            cursor.getString(cursor.getColumnIndex(COLUMN_DAY_OF_WEEK)),
            cursor.getString(cursor.getColumnIndex(COLUMN_TIME)),
            cursor.getInt(cursor.getColumnIndex(COLUMN_CAPACITY)),
            cursor.getInt(cursor.getColumnIndex(COLUMN_DURATION)),
            cursor.getDouble(cursor.getColumnIndex(COLUMN_PRICE)),
            cursor.getString(cursor.getColumnIndex(COLUMN_TYPE_OF_CLASS)),
            cursor.getString(cursor.getColumnIndex(COLUMN_DESCRIPTION)),
            cursor.getString(cursor.getColumnIndex(COLUMN_STATUS))
        );
        cursor.close();
        return yogaClass;
    }

    if (cursor != null) {
        cursor.close();
    }

    return null;
}

```

This code depicts the `getYogaClassById` method, which retrieves a specific yoga class by id from `TABLE_YOGA_CLASSES`. This method is useful for displaying detailed information for a single class, allowing users to view or edit the class details as needed.

```

public Cursor getAllYogaClasses() {
    SQLiteDatabase db = this.getReadableDatabase();
    return db.rawQuery("SELECT * FROM " + TABLE_YOGA_CLASSES, null);
}

```

This code shows the `getAllYogaClasses` method, which retrieves all yoga class records from `TABLE_YOGA_CLASSES`. This method is primarily used to display a complete list of available classes for viewing or booking purposes.

```

public Cursor getYogaClassesWithStatus(String status) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM yoga_classes WHERE status = ?";
    return db.rawQuery(query, new String[]{status});
}

```

This code illustrates the `getYogaClassesWithStatus` method, which retrieves all yoga classes in `TABLE_YOGA_CLASSES` with a specific status. This allows filtering classes based on their status, such as showing only active or archived classes.

```

public Cursor getYogaClass(String dayOfWeek, String time, int capacity,
    int duration, double price, String typeOfClass, String description) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM yoga_classes WHERE dayOfWeek = ? AND " +
        "time = ? AND capacity = ? AND duration = ? AND price = ? AND typeOfClass = ? AND description = ?";
    return db.rawQuery(query, new String[]{dayOfWeek, time, String.valueOf(capacity),

```

```
String.valueOf(duration), String.valueOf(price), typeOfClass, description));
}
```

This code shows the `getYogaClass` method, which allows fetching yoga classes from `TABLE_YOGA_CLASSES` that match specific criteria such as day, time, capacity, duration, price, type, and description. This advanced filtering helps users search for classes that meet their requirements.

```
public boolean updateYogaClassStatus(String id, String status) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("status", status);

    int result = db.update(TABLE_YOGA_CLASSES, contentValues, "id = ?", new String[]{id});
    db.close();

    return result > 0;
}
```

This code presents the `updateYogaClassStatus` method, which updates the status of a specific yoga class in `TABLE_YOGA_CLASSES` using its id. This function is useful for changing the class's status, such as marking it as active, inactive, or archived.

4.1.1.2 Class Instance

```
public ClassInstance getClassInstanceById(String classInstanceId) {
    SQLiteDatabase db = this.getReadableDatabase();

    Cursor cursor = db.query(TABLE_CLASS_INSTANCES,
        null,
        COLUMN_ID + " = ?",
        new String[]{classInstanceId},
        null, null, null);

    if (cursor != null && cursor.moveToFirst()) {
        ClassInstance classInstance = new ClassInstance(
            cursor.getString(cursor.getColumnIndex(COLUMN_ID)),
            cursor.getString(cursor.getColumnIndex(COLUMN_CLASS_ID)),
            cursor.getString(cursor.getColumnIndex(COLUMN_DATE)),
            cursor.getString(cursor.getColumnIndex(COLUMN_INSTRUCTOR)),
            cursor.getString(cursor.getColumnIndex(COLUMN_ADDITIONAL_COMMENTS)),
            cursor.getString(cursor.getColumnIndex(COLUMN_STATUS))
        );

        cursor.close();
        return classInstance;
    }

    if (cursor != null) {
        cursor.close();
    }
    return null;
}
```

This code displays the `getClassInstanceById` method, which retrieves a specific class instance from the `TABLE_CLASS_INSTANCES` table based on its ID. By executing a query, it checks for the class instance matching the provided `classInstanceId`. If found, the method constructs a `ClassInstance` object containing

details such as the instance's ID, class ID, date, instructor, additional comments, and status. This function facilitates fetching detailed information about a particular class instance for further processing or display.

```
public boolean deleteClassInstancesByClassId(String classId) {
    SQLiteDatabase db = this.getWritableDatabase();
    int rowsDeleted = db.delete(TABLE_CLASS_INSTANCES, "classId = ?", new String[]{classId});
    db.close();
    return rowsDeleted > 0;
}
```

This code shows the `deleteClassInstancesByClassId` method, which deletes all class instances associated with a specific class ID in the `TABLE_CLASS_INSTANCES` table. This method removes records where the `classId` matches the provided ID, ensuring that all instances of a class are deleted when necessary. This is useful for data cleanup or when an entire class is removed from the system.

```
public List<ClassInstance> getClassInstancesByInstructorId(String instructorId) {
    List<ClassInstance> classInstanceList = new ArrayList<>();
    SQLiteDatabase db = this.getReadableDatabase();

    Cursor cursor = db.query(TABLE_CLASS_INSTANCES, null, "instructor = ?",
        new String[]{instructorId}, null, null, null);

    if (cursor != null && cursor.moveToFirst()) {
        do {
            ClassInstance classInstance = new ClassInstance(
                cursor.getString(cursor.getColumnIndex("id")),
                cursor.getString(cursor.getColumnIndex("classId")),
                cursor.getString(cursor.getColumnIndex("date")),
                cursor.getString(cursor.getColumnIndex("instructor")),
                cursor.getString(cursor.getColumnIndex("additionalComments")),
                cursor.getString(cursor.getColumnIndex("status"))
            );
            classInstanceList.add(classInstance);
        } while (cursor.moveToNext());
        cursor.close();
    }

    return classInstanceList;
}
```

This code presents the `getClassInstancesByInstructorId` method, which retrieves a list of class instances associated with a specific instructor in the `TABLE_CLASS_INSTANCES` table. The method uses the `instructorId` to find matching entries and compiles them into a list of `ClassInstance` objects. This function is essential for instructors to view all their scheduled class sessions.

```
public Cursor getClassInstancesByUserId(String userId) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM class_instances WHERE instructor = ?";
    return db.rawQuery(query, new String[]{userId});
}
```

This code shows the `getClassInstancesByUserId` method, which retrieves class instances associated with a user based on their `userId`. This method creates a query to select records from the `class_instances` table where the instructor is the specified user. This allows viewing all class instances taught by a particular instructor.

```

public boolean addClassInstance(ClassInstance instance) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("id", instance.getId());
    contentValues.put("classId", instance.getClassId());
    contentValues.put("date", instance.getDate());
    contentValues.put("instructor", instance.getInstructor());
    contentValues.put("additionalComments", instance.getAdditionalComments());
    contentValues.put("status", instance.getStatus());

    long result = db.insert(TABLE_CLASS_INSTANCES, null, contentValues);
    db.close();
    return result != -1;
}

```

This code displays the `addClassInstance` method, which inserts a new class instance into the `TABLE_CLASS_INSTANCES` table. It collects various attributes, including ID, class ID, date, instructor, additional comments, and status, and saves them to the database. This method enables the creation of new class instances within the app.

```

public boolean updateClassInstance(ClassInstance instance) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("classId", instance.getClassId());
    contentValues.put("date", instance.getDate());
    contentValues.put("instructor", instance.getInstructor());
    contentValues.put("additionalComments", instance.getAdditionalComments());
    contentValues.put("status", instance.getStatus());

    int result = db.update(TABLE_CLASS_INSTANCES, contentValues, "id = ?", new String[]{instance.getId()});
    db.close();
    return result > 0;
}

```

This code shows the `updateClassInstance` method, which updates an existing class instance in the `TABLE_CLASS_INSTANCES` table. It modifies details such as class ID, date, instructor, additional comments, and status based on the `instanceId`. This method is used when existing class instance details need modification.

```

public void deleteClassInstance(String id) {
    SQLiteDatabase db = this.getWritableDatabase();
    db.delete(TABLE_CLASS_INSTANCES, "id = ?", new String[]{id});
    db.close();
}

```

This code presents the `deleteClassInstance` method, which deletes a single class instance from the `TABLE_CLASS_INSTANCES` table using its ID. This function removes a specific instance when it is no longer needed.

```

public Cursor getClassInstancesByClassId(String classId) {
    SQLiteDatabase db = this.getReadableDatabase();
    return db.query("class_instances", null, "classId = ?", new String[]{classId},
        null, null, null);
}

```

This code illustrates the `getClassInstancesByClassId` method, which retrieves class instances based on a specific class ID. This function executes a query to collect instances of a certain class, helping to group all related sessions for easy access.

```
public Cursor getAllClassInstances() {
    SQLiteDatabase db = this.getReadableDatabase();
    return db.rawQuery("SELECT * FROM class_instances", null);
}
```

This code shows the `getAllClassInstances` method, which retrieves all class instances from the `TABLE_CLASS_INSTANCES` table. It performs a query to fetch every instance available, providing a complete list of all class sessions stored in the database.

```
public Cursor getClassInstancesWithStatus(String status) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM class_instances WHERE status = ?";
    return db.rawQuery(query, new String[]{status});
}
```

This code displays the `getClassInstancesWithStatus` method, which retrieves class instances with a specified status from the `TABLE_CLASS_INSTANCES` table. This query-based function allows filtering sessions based on their current status, such as active or completed.

```
public boolean updateClassInstanceStatus(String id, String status) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("status", status);
    int result = db.update("class_instances", contentValues, "id = ?", new String[]{id});
    db.close();
    return result > 0;
}
```

This code shows the `updateClassInstanceStatus` method, which updates the status of a specific class instance in the database. This method takes the instance ID and new status as parameters, then uses a `ContentValues` object to set the new status value. The database is then updated using the provided ID to locate the correct record. This function helps ensure that the status of a class instance is accurately recorded and modified as needed.

4.1.1.3 Booking

```
public Cursor getAllBookings() {
    SQLiteDatabase db = this.getReadableDatabase();

    String query = "SELECT * FROM " + TABLE_BOOKINGS;

    return db.rawQuery(query, null);
}
```

This code shows the `getAllBookings()` method, which retrieves all records from the `TABLE_BOOKINGS` in the database. This method executes a SQL query to select all columns from the booking table and returns a `Cursor` object that can be used to navigate through the booking data.

```
public void updateBooking(Booking booking) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put("status", booking.getStatus());
    values.put("canCancel", booking.isCanCancel() ? 1 : 0);

    db.update(TABLE_BOOKINGS, values, "id = ?", new String[]{booking.getId()});
}
```

This code illustrates the `updateBooking()` method, which updates specific details of a booking in the database. It sets the status and `canCancel` fields in the `TABLE_BOOKINGS` based on the provided `Booking` object, enabling modifications to the booking's status and cancellation eligibility.

```
public void addBooking(Booking booking) {
    SQLiteDatabase db = this.getWritableDatabase();

    ContentValues values = new ContentValues();
    values.put(COLUMN_BOOKING_ID, booking.getId());
    values.put(COLUMN_STATUS, booking.getStatus());
    values.put(COLUMN_CAN_CANCEL, booking.isCanCancel() ? 1 : 0);
    values.put(COLUMN_TOTAL_PRICE, booking.getTotalPrice());
    values.put(COLUMN_BOOKING_USER_ID, booking.getUserId());

    String classInstancesJson = new Gson().toJson(booking.getClassInstances());

    values.put(COLUMN_CLASS_INSTANCES, classInstancesJson);

    db.insert(TABLE_BOOKINGS, null, values);
    db.close();
}
```

This code shows the `addBooking()` method, which inserts a new booking record into the `TABLE_BOOKINGS`. This method saves all necessary booking details, including status, `canCancel`, `totalPrice`, and user information, as well as the associated class instances converted to JSON format.

```
public Booking getBookingById(String bookingId) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.query(TABLE_BOOKINGS, null, "id = ?", new String[]{bookingId},
        null, null, null);

    if (cursor != null && cursor.moveToFirst()) {
        String id = cursor.getString(cursor.getColumnIndex("id"));
        String status = cursor.getString(cursor.getColumnIndex("status"));
        boolean canCancel = cursor.getInt(cursor.getColumnIndex("canCancel")) == 1;
        int totalPrice = cursor.getInt(cursor.getColumnIndex("totalPrice"));
        String userId = cursor.getString(cursor.getColumnIndex("userId"));

        String classInstancesJson = cursor.getString(cursor.getColumnIndex("classInstances"));
        List<ClassInstanceInfo> classInstances = new Gson().fromJson(classInstancesJson,
            new TypeToken<List<ClassInstanceInfo>>() {}.getType());

        Booking booking = new Booking(id, status, canCancel, totalPrice, userId, classInstances);

        cursor.close();
        return booking;
    }

    if (cursor != null) {
        cursor.close();
    }
    return null;
}
```

This code presents the `getBookingById()` method, which fetches a specific booking record from the `TABLE_BOOKINGS` using the booking ID. It extracts details such as status, cancellation eligibility, total price, and associated class instances, allowing for a comprehensive view of an individual booking.

```
public boolean isBookingExists(String bookingId) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery("SELECT 1 FROM Bookings WHERE id = ?", new String[]{bookingId});
    boolean exists = (cursor.getCount() > 0);
    cursor.close();
    return exists;
}
```

This code shows the `isBookingExists()` method, which verifies whether a specific booking ID exists in the `TABLE_BOOKINGS`. This method runs a SQL query to check for the presence of the booking ID, returning `true` if found, enabling validation of booking records.

4.1.1.4 User

```
public boolean addUser(User user) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
}
```

```

contentValues.put(COLUMN_USER_ID, user.getId());
contentValues.put(COLUMN_FULL_NAME, user.getFullName());
contentValues.put(COLUMN_EMAIL, user.getEmail());
contentValues.put(COLUMN_PASSWORD, user.getPassword());
contentValues.put(COLUMN_PHONE_NUMBER, user.getPhoneNumber());
contentValues.put(COLUMN_STATUS, "Waiting");
contentValues.put(COLUMN_ROLE, user.getRole());

long result = db.insert(TABLE_USERS, null, contentValues);
db.close();
return result != -1;
}

```

This code shows the `addUser` method, which is responsible for inserting a new user into the `TABLE_USERS` database. Using `ContentValues`, it maps various user attributes such as ID, full name, email, password, phone number, status, and role to respective database columns. After preparing these values, it inserts them into `TABLE_USERS`. If the insertion is successful, it returns `true`, confirming the user's addition to the database.

```

public Cursor getUsersByRole(String role) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM Users WHERE role = ?";
    return db.rawQuery(query, new String[]{role});
}

```

This code illustrates the `getUsersByRole` method, which retrieves users from the `TABLE_USERS` based on a specific role. It constructs an SQL query to select all user records where the role column matches the provided role parameter. This method returns a `Cursor` object, allowing further iteration through the query results.

```

public boolean updateUser(User user) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put(COLUMN_FULL_NAME, user.getFullName());
    contentValues.put(COLUMN_EMAIL, user.getEmail());
    contentValues.put(COLUMN_PHONE_NUMBER, user.getPhoneNumber());
    contentValues.put(COLUMN_PASSWORD, user.getPassword());
    contentValues.put(COLUMN_STATUS, user.getStatus());
    contentValues.put(COLUMN_ROLE, user.getRole());

    int result = db.update(TABLE_USERS, contentValues, COLUMN_USER_ID + " = ?", new String[]{user.getId()});
    db.close();
    return result > 0;
}

```

This code shows the `updateUser` method, which updates an existing user's details in the `TABLE_USERS` database. The method uses `ContentValues` to map updated user attributes such as full name, email, phone number, password, status, and role. The update is applied to the record where `COLUMN_USER_ID` matches the user's ID. A `true` return value indicates a successful update.

```

public User getUserByEmailAndPassword(String email, String password) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM users WHERE email = ?";

    Cursor cursor = db.rawQuery(query, new String[]{email});

    if (cursor != null && cursor.moveToFirst()) {
        String storedSaltedHashedPassword = cursor.getString(cursor.getColumnIndex(COLUMN_PASSWORD));
        String[] parts = storedSaltedHashedPassword.split(":");
        if (parts.length == 2) {
            String storedSalt = parts[0];
            String storedHash = parts[1];

            String userInputHash = hashPasswordWithSalt(password, storedSalt);

            if (userInputHash.split(":")[1].equals(storedHash)) {
                User user = new User(
                    cursor.getString(cursor.getColumnIndex(COLUMN_USER_ID)),
                    cursor.getString(cursor.getColumnIndex(COLUMN_EMAIL)),
                    cursor.getString(cursor.getColumnIndex(COLUMN_FULL_NAME)),
                    cursor.getString(cursor.getColumnIndex(COLUMN_PHONE_NUMBER)),
                    storedSaltedHashedPassword,
                    cursor.getString(cursor.getColumnIndex(COLUMN_STATUS)),
                    cursor.getString(cursor.getColumnIndex(COLUMN_ROLE))
                );
            }
        }
    }
    return user;
}

```

```

    );
    cursor.close();
    return user;
}
}
}

if (cursor != null) {
    cursor.close();
}

return null;
}

```

This code presents the `getUserByEmailAndPassword` method, which is used for user authentication during login. It queries `TABLE_USERS` by email, retrieves the stored hashed password with salt, and compares it with the input password hashed in the same way. If they match, a `User` object with user details is returned, confirming successful authentication.

```

public boolean isUserExists(String userId) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.rawQuery("SELECT 1 FROM Users WHERE id = ?", new String[]{userId});
    boolean exists = (cursor.getCount() > 0);
    cursor.close();
    return exists;
}

```

This code displays the `isUserExists` method, which checks the existence of a user by their ID. It executes an SQL query on `TABLE_USERS` to see if any records match the given user ID. The method returns true if a matching user is found, confirming the user's existence in the database.

```

public String hashPasswordWithSalt(String password, String salt) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        String saltedPassword = salt + password;
        byte[] hash = digest.digest(saltedPassword.getBytes());
        StringBuilder hexString = new StringBuilder();
        for (byte b : hash) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return salt + ":" + hexString.toString();
    } catch (NoSuchAlgorithmException e) {
        e.printStackTrace();
        return null;
    }
}

```

This code shows the `hashPasswordWithSalt` method, which generates a secure hash of a password combined with a salt. Using SHA-256, this method creates a hashed string with hexadecimal encoding. The result includes the salt and hashed password, enhancing security by making password storage more robust.

```

public void deleteUser(String userId) {
    SQLiteDatabase db = this.getWritableDatabase();
    db.delete(TABLE_USERS, COLUMN_USER_ID + " = ?", new String[]{userId});
    db.close();
}

```

This code illustrates the `deleteUser` method, which deletes a user record from `TABLE_USERS` based on the specified user ID. By matching the `COLUMN_USER_ID`, this method removes the relevant record and then closes the database connection.

```

public boolean isEmailExists(String email) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM users WHERE email = ?";
    Cursor cursor = db.rawQuery(query, new String[]{email});

    boolean exists = (cursor.getCount() > 0);
    cursor.close();
    return exists;
}

```

This code shows the `isEmailExists` method, which checks whether a given email is already registered in the `TABLE_USERS`. It performs an SQL query to count records with the specified email. A return value of `true` indicates that the email exists in the database.

```
public Cursor getAllUsers() {
    SQLiteDatabase db = this.getReadableDatabase();
    return db.rawQuery("SELECT * FROM users", null);
}
```

This code presents the `getAllUsers` method, which retrieves all user records from `TABLE_USERS`. This method executes a SQL query to select all columns, returning a `Cursor` object that allows iteration over the retrieved user data.

```
public Cursor getUsersWithStatus(String status) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM users WHERE status = ?";
    return db.rawQuery(query, new String[]{status});
}
```

This code displays the `getUsersWithStatus` method, which retrieves users based on a specific status. It queries `TABLE_USERS` to select records where the status column matches the provided status value, returning a `Cursor` object for further processing.

```
public boolean updateUserStatus(String userId, String status) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("status", status);
    int result = db.update("users", contentValues, "id = ?", new String[]{userId});
    db.close();
    return result > 0;
}
```

This code shows the `updateUserStatus` method, which is designed to update the status of a specific user in the database. This method takes `userId` and `status` as parameters and updates the status column in the users table for the user with the matching id. The method returns `true` if the update was successful, making it useful for modifying a user's status based on specific conditions.

```
public User getUserById(String userId) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.query(TABLE_USERS, null, COLUMN_USER_ID + " = ?",
        new String[]{userId}, null, null, null);

    if (cursor != null && cursor.moveToFirst()) {
        User user = new User(
            cursor.getString(cursor.getColumnIndex(COLUMN_USER_ID)),
            cursor.getString(cursor.getColumnIndex(COLUMN_EMAIL)),
            cursor.getString(cursor.getColumnIndex(COLUMN_FULL_NAME)),
            cursor.getString(cursor.getColumnIndex(COLUMN_PHONE_NUMBER)),
            cursor.getString(cursor.getColumnIndex(COLUMN_PASSWORD)),
            cursor.getString(cursor.getColumnIndex(COLUMN_STATUS)),
            cursor.getString(cursor.getColumnIndex(COLUMN_ROLE))
        );
        cursor.close();
        return user;
    }

    if (cursor != null) {
        cursor.close();
    }
    return null;
}
```

This code presents the `getUserById` method, which retrieves a user's details from the database based on their `userId`. The method queries the users table for a row with a matching `userId` and returns a `User` object populated with the user's information, including `userId`, email, full name, phone number, password, status, and role. This method is useful for fetching and using user details within the application.

4.1.1.5 ClassType

```

public boolean addClassType(ClassType classType) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_CLASS_TYPE_ID, classType.getId());
    values.put(COLUMN_CLASS_TYPE_NAME, classType.getName());
    values.put(COLUMN_CLASS_TYPE_STATUS, classType.getStatus());
    long result = db.insert(TABLE_CLASS_TYPE, null, values);
    db.close();
    return result != -1;
}

```

This method inserts a new `ClassType` record into the `TABLE_CLASS_TYPE`. It creates a `ContentValues` object to store column-value pairs for insertion, then inserts these values using the `db.insert()` method. If the insertion is successful, the method returns `true`; otherwise, it returns `false`. The database connection is closed after the operation.

```

public Cursor getAllClassTypes() {
    SQLiteDatabase db = this.getReadableDatabase();
    return db.rawQuery("SELECT * FROM " + TABLE_CLASS_TYPE, null);
}

```

This method retrieves all records from the `TABLE_CLASS_TYPE` table using a `rawQuery()` call. It returns a `Cursor` object containing the query results, which can be used to iterate over the data.

```

public boolean updateClassType(ClassType classType) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();

    values.put(COLUMN_CLASS_TYPE_NAME, classType.getName());
    values.put("status", "Pending");

    int result = db.update(TABLE_CLASS_TYPE, values, "id = ?", new String[]{classType.getId()});
    db.close();

    return result > 0;
}

```

This function updates the name and sets the status of a `ClassType` record with a specific id in the `TABLE_CLASS_TYPE` table. The `ContentValues` object stores the new values, and the `db.update()` method performs the update operation. The method returns `true` if the update was successful and `false` otherwise.

```

public Cursor getClassTypesWithStatus(String status) {
    SQLiteDatabase db = this.getReadableDatabase();
    String query = "SELECT * FROM class_type WHERE status = ?";
    return db.rawQuery(query, new String[]{status});
}

```

This method retrieves `ClassType` records from the `TABLE_CLASS_TYPE` table where the status matches the input parameter. It uses `rawQuery()` with a selection argument and returns a `Cursor` containing the matching records.

```

public boolean updateClassTypeStatus(String id, String status) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("status", status);

    int result = db.update("class_type", contentValues, "id = ?", new String[]{id});
    db.close();

    return result > 0;
}

```

This method updates the status of a `ClassType` record identified by the id. A `ContentValues` object stores the new status, and the `db.update()` method performs the update. It returns `true` if one or more rows were affected by the update.

```

public boolean deleteClassType(String id) {
    SQLiteDatabase db = this.getWritableDatabase();
    int result = db.delete(TABLE_CLASS_TYPE, "id = ?", new String[]{id});
    db.close();
    return result > 0;
}

```

This function deletes a `ClassType` record from the `TABLE_CLASS_TYPE` table using the `id` as a condition. It uses the `db.delete()` method and returns `true` if the deletion was successful, or `false` otherwise.

5.1.2 DataSyncHelper.java

```
public void syncDataFromFirebaseIfNeeded() {
    syncClassInstancesFromFirebase();

    syncYogaClassesFromFirebase();

    syncUsersFromFirebase();

    syncBookingsFromFirebase();

    syncClassTypesFromFirebase();
}
```

This code shows the `syncDataFromFirebaseIfNeeded()` method in the `DataSyncHelper` class. This method is responsible for initiating the data synchronization process from Firebase for multiple data types: class instances, yoga classes, users, bookings, and class types. By calling specific sync methods for each data type, it ensures that the local database stays updated with the latest data from Firebase.

```
private void syncClassInstancesFromFirebase() {
    Cursor cursor = db.getAllClassInstances();

    if (cursor.getCount() == 0) {
        if (isWifiConnected()) {
            firebaseHelper.getAllClassInstances(new FirebaseHelper.DataStatusClassInstance() {
                @Override
                public void onDataLoaded(List<ClassInstance> classInstances) {
                    for (ClassInstance classInstance : classInstances) {
                        db.addClassInstance(classInstance);
                    }
                    Log.d("DataSync", "ClassInstance data synced from Firebase to SQLite.");
                }
            });

            @Override
            public void onError(String error) {
                Log.e("DataSync", "Error syncing ClassInstance data from Firebase: " + error);
            }
        });
    } else {
        Log.d("DataSync", "No Wi-Fi connection. ClassInstance data sync skipped.");
    }
} else {
    Log.d("DataSync", "ClassInstance data already exists in SQLite. No sync needed.");
}
}
cursor.close();
}
```

This code displays the `syncClassInstancesFromFirebase()` method in the `DataSyncHelper` class. This method checks if class instance data needs to be synced by verifying if there are no entries in the local database. If connected to Wi-Fi, it retrieves all class instances from Firebase and adds them to the local SQLite database, logging the status of the synchronization process.

```
private void syncYogaClassesFromFirebase() {
    Cursor cursor = db.getAllYogaClasses();

    if (cursor.getCount() == 0) {
        if (isWifiConnected()) {
            firebaseHelper.getAllYogaClasses(new FirebaseHelper.DataStatusYogaClass() {
                @Override
                public void onDataLoaded(List<YogaClass> yogaClasses) {
                    for (YogaClass yogaClass : yogaClasses) {
                        db.addYogaClass(yogaClass);
                    }
                    Log.d("DataSync", "YogaClass data synced from Firebase to SQLite.");
                }
            });

            @Override
            public void onError(String error) {
                Log.e("DataSync", "Error syncing YogaClass data from Firebase: " + error);
            }
        });
    } else {
        Log.d("DataSync", "No Wi-Fi connection. YogaClass data sync skipped.");
    }
}
```

```

    } else {
        Log.d("DataSync", "YogaClass data already exists in SQLite. No sync needed.");
    }
    cursor.close();
}

```

This code shows the `syncYogaClassesFromFirebase()` method in the `DataSyncHelper` class. Similar to the previous method, it checks if the local database contains yoga class data. If empty and connected to Wi-Fi, it retrieves yoga classes from Firebase and inserts them into SQLite, with logs indicating successful sync or errors if they occur.

```

public void syncUsersFromFirebase() {
    if (isWifiConnected()) {
        firebaseHelper.getAllUsers(new FirebaseHelper.DataStatusUsers() {
            @Override
            public void onDataLoaded(List<User> userList) {
                for (User user : userList) {
                    if (db.isUserExists(user.getId())) {
                        db.updateUser(user);
                    } else {
                        db.addUser(user);
                    }
                }
                Log.d("DataSync", "User data synced from Firebase to SQLite.");
            }

            @Override
            public void onError(String error) {
                Log.e("DataSync", "Error syncing User data from Firebase: " + error);
            }
        });
    } else {
        Log.d("DataSync", "No Wi-Fi connection. User data sync skipped.");
    }
}

```

This code illustrates the `syncUsersFromFirebase()` method in the `DataSyncHelper` class. This method synchronizes user data from Firebase if Wi-Fi is available. It updates existing user records or inserts new ones as needed, ensuring that the local database reflects Firebase's current user data, with logging for status tracking.

```

public void syncBookingsFromFirebase() {
    if (isWifiConnected()) {
        firebaseHelper.getAllBookings(new FirebaseHelper.DataStatusBookings() {
            @Override
            public void onDataLoaded(List<Booking> bookings) {
                for (Booking booking : bookings) {
                    if (db.isBookingExists(booking.getId())) {
                        db.updateBooking(booking);
                    } else {
                        db.addBooking(booking);
                    }
                }
                Log.d("DataSync", "Booking data synced from Firebase to SQLite.");
            }

            @Override
            public void onError(String error) {
                Log.e("DataSync", "Error syncing Booking data from Firebase: " + error);
            }
        });
    } else {
        Log.d("DataSync", "No Wi-Fi connection. Booking data sync skipped.");
    }
}

```

This code shows the `syncBookingsFromFirebase()` method in the `DataSyncHelper` class. This method synchronizes booking data by retrieving it from Firebase if Wi-Fi is connected. It either updates existing booking entries or adds new ones, keeping the local database in sync with Firebase and logging the sync status.

```

public void syncClassTypesFromFirebase() {
    Cursor cursor = db.getAllClassTypes();

    if (cursor.getCount() == 0 && isWifiConnected()) {

```

```

firebaseHelper.getAllClassTypes(new FirebaseHelper.DataStatusClassType() {
    @Override
    public void onDataLoaded(List<ClassType> classTypes) {
        for (ClassType classType : classTypes) {
            db.addClassType(classType);
        }
        Log.d("DataSync", "ClassType data synced from Firebase to SQLite.");
    }

    @Override
    public void onError(String error) {
        Log.e("DataSync", "Error syncing ClassType data from Firebase: " + error);
    }
});
}
cursor.close();
}

```

This code illustrates the `syncClassTypesFromFirebase()` method in the `DataSyncHelper` class. This method checks if class type data exists locally and syncs from Firebase if Wi-Fi is available. It inserts any missing class types, ensuring the local database includes the latest class type definitions from Firebase.

```

private boolean isWifiConnected() {
    ConnectivityManager cm = (ConnectivityManager) context.getSystemService(Context.CONNECTIVITY_SERVICE);
    NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
    return activeNetwork != null && activeNetwork.getType() == ConnectivityManager.TYPE_WIFI;
}

```

This code displays the `isWifiConnected()` method in the `DataSyncHelper` class. This method checks if the device is connected to a Wi-Fi network by utilizing Android's `ConnectivityManager`. It returns a boolean value indicating the Wi-Fi connection status, used to determine if data sync should proceed.

5.1.3 FirebaseHelper.java

```

public FirebaseHelper() {
    firebaseDatabase = FirebaseDatabase.getInstance("https://yogaapp-42561-default-rtdb.asia-southeast1.firebaseio.com/");
    databaseReferenceClass = firebaseDatabase.getReference("YogaClasses");
    databaseReferenceClassInstance = firebaseDatabase.getReference("ClassInstances");
    databaseReferenceUsers = firebaseDatabase.getReference("Users");
    databaseReferenceBookings = firebaseDatabase.getReference("Bookings");
    databaseReferenceClassType = firebaseDatabase.getReference("ClassType");
}

```

This code shows the initialization of `FirebaseHelper`, where the `Firebase Realtime Database` instance and references for various data paths are set up. Figure establishes the connection to the `Firebase database` and initializes references for paths like `YogaClasses`, `ClassInstances`, `Users`, `Bookings`, and `ClassType`. These references allow the application to interact with specific parts of the `Firebase database`, facilitating data retrieval and manipulation for each defined category.

```

public void deleteAllClassTypes() {
    databaseReferenceClassType.removeValue();
}
public void deleteAllBookings() {
    databaseReferenceBookings.removeValue();
}
public void deleteAllYogaClasses() {
    databaseReferenceClass.removeValue();
}
public void deleteAllClassInstances() {
    databaseReferenceClassInstance.removeValue();
}
public void addUpdateYogaClassToFirebase(YogaClass yogaClass) {
    databaseReferenceClass.child(yogaClass.getId()).setValue(yogaClass);
}
public void deleteYogaClassFromFirebase(String classId) {
    databaseReferenceClass.child(classId).removeValue();
}
public void addUpdateYogaClassInstancesToFirebase(ClassInstance classInstance) {
    databaseReferenceClassInstance.child(classInstance.getId()).setValue(classInstance);
}
public void deleteYogaClassInstancesFromFirebase(String classInstanceId) {
    databaseReferenceClassInstance.child(classInstanceId).removeValue();
}
public void addUserToFirebase(User user) {
    databaseReferenceUsers.child(user.getId()).setValue(user);
}
}

```

```
public void deleteUserFromFirebase(String userId) {
    databaseReferenceUsers.child(userId).removeValue();
}
```

This code shows several methods for managing data entries in Firebase, including methods to delete all entries for class types, bookings, yoga classes, and class instances, as well as to add or update individual entries. These methods enable synchronization between local and cloud storage by ensuring Firebase remains consistent with the local database.

```
public void updateBookingStatusAndCancel(String bookingId, String status, boolean canCancel) {
    Log.d("Update Booking", bookingId);
    DatabaseReference bookingRef = databaseReferenceBookings.child(bookingId);

    Map<String, Object> updates = new HashMap<>();
    updates.put("status", status);
    updates.put("canCancel", canCancel);

    bookingRef.updateChildren(updates);
}
```

This code illustrates the `updateBookingStatusAndCancel` method, which updates the status and cancellation ability of a booking in Firebase. This method allows for selective updates to specific booking fields, enhancing control over data modification without fully overwriting entries.

```
public void getAllUsers(final DataStatusUsers dataStatus) {
    databaseReferenceUsers.addListenerForSingleValueEvent(new ValueEventListener() {
        @Override
        public void onDataChange(DataSnapshot dataSnapshot) {
            List<User> userList = new ArrayList<>();
            for (DataSnapshot snapshot : dataSnapshot.getChildren()) {
                User user = snapshot.getValue(User.class);
                userList.add(user);
            }
            dataStatus.onDataLoaded(userList);
        }

        @Override
        public void onCancelled(DatabaseError databaseError) {
            dataStatus.onError(databaseError.getMessage());
        }
    });
}
```

This code depicts the `getAllUsers` method, which retrieves all user data from Firebase. It listens for a single event to collect user entries, then triggers callbacks upon successful data retrieval or in the case of errors, making it integral to user data synchronization.

```
public void getAllClassInstances(final DataStatusClassInstance dataStatus) {
    databaseReferenceClassInstance.addListenerForSingleValueEvent(new ValueEventListener() {
        @Override
        public void onDataChange(DataSnapshot dataSnapshot) {
            List<ClassInstance> classInstances = new ArrayList<>();
            for (DataSnapshot snapshot : dataSnapshot.getChildren()) {
                ClassInstance classInstance = snapshot.getValue(ClassInstance.class);
                classInstances.add(classInstance);
            }
            dataStatus.onDataLoaded(classInstances);
        }

        @Override
        public void onCancelled(DatabaseError databaseError) {
            dataStatus.onError(databaseError.getMessage());
        }
    });
}
```

This code illustrates the `getAllClassInstances` method, which retrieves all class instances from Firebase. This method listens for a single value event and processes each `DataSnapshot` to create `ClassInstance` objects. Upon successful data loading, it calls `onDataLoaded` with the list of instances, while any error triggers `onError`. This method ensures synchronization of class instances between Firebase and the local app.

```

public void getAllYogaClasses(final DataStatusYogaClass dataStatus) {
    databaseReferenceClass.addListenerForSingleValueEvent(new ValueEventListener() {
        @Override
        public void onDataChange(DataSnapshot snapshot) {
            List<YogaClass> yogaClasses = new ArrayList<>();
            for (DataSnapshot dataSnapshot : snapshot.getChildren()) {
                YogaClass yogaClass = dataSnapshot.getValue(YogaClass.class);
                yogaClasses.add(yogaClass);
            }
            dataStatus.onDataLoaded(yogaClasses);
        }

        @Override
        public void onCancelled(DatabaseError error) {
            dataStatus.onError(error.getMessage());
        }
    });
}

```

This code shows the `getAllYogaClasses` method, responsible for fetching all yoga class data from Firebase. It processes each entry into a `YogaClass` object and calls relevant callbacks, facilitating data synchronization for yoga classes.

```

public void getAllBookings(final DataStatusBookings dataStatus) {
    databaseReferenceBookings.addListenerForSingleValueEvent(new ValueEventListener() {
        @Override
        public void onDataChange(DataSnapshot snapshot) {
            List<Booking> bookingsList = new ArrayList<>();

            for (DataSnapshot bookingSnapshot : snapshot.getChildren()) {
                String id = bookingSnapshot.child("id").getValue(String.class);
                String status = bookingSnapshot.child("status").getValue(String.class);
                boolean canCancel = bookingSnapshot.child("canCancel").getValue(Boolean.class);
                double totalPriceDouble = bookingSnapshot.child("totalPrice").getValue(Double.class);
                int totalPrice = (int) totalPriceDouble;
                String userId = bookingSnapshot.child("userId").getValue(String.class);

                List<ClassInstanceInfo> classInstanceInfos = new ArrayList<>();

                for (DataSnapshot classInstanceSnapshot : bookingSnapshot.child("classInstances").getChildren()) {
                    String classId = classInstanceSnapshot.child("classId").getValue(String.class);
                    String classInstanceId = classInstanceSnapshot.child("classInstanceId").getValue(String.class);

                    ClassInstanceInfo classInstanceInfo = new ClassInstanceInfo(classId, classInstanceId);
                    classInstanceInfos.add(classInstanceInfo);
                }

                Booking booking = new Booking(id, status, canCancel, totalPrice, userId, classInstanceInfos);
                bookingsList.add(booking);
            }
            dataStatus.onDataLoaded(bookingsList);
        }

        @Override
        public void onCancelled(DatabaseError error) {
            dataStatus.onError(error.getMessage());
        }
    });
}

```

This code details the `getAllBookings` method, which retrieves all bookings from Firebase, including details like status, total price, and associated class instances. This method supports comprehensive booking data synchronization.

```

public void addUpdateClassTypeToFirebase(ClassType classType) {
    databaseReferenceClassType.child(classType.getId()).setValue(classType);
}

public void deleteClassTypeFromFirebase(String classTypeId) {
    databaseReferenceClassType.child(classTypeId).removeValue();
}

```

This code presents the methods `addUpdateClassTypeToFirebase` and `deleteClassTypeFromFirebase`, which handle adding/updating or deleting class types in Firebase. These methods are essential for managing class type data in the cloud.

```

public void getAllClassTypes(final DataStatusClassType dataStatus) {
    databaseReferenceClassType.addListenerForSingleValueEvent(new ValueEventListener() {
        @Override

```

```

public void onDataChange(DataSnapshot dataSnapshot) {
    List<ClassType> classTypes = new ArrayList<>();
    for (DataSnapshot snapshot : dataSnapshot.getChildren()) {
        ClassType classType = snapshot.getValue(ClassType.class);
        classTypes.add(classType);
    }
    dataStatus.onDataLoaded(classTypes);
}

@Override
public void onCancelled(DatabaseError databaseError) {
    dataStatus.onError(databaseError.getMessage());
}
});
}

```

This code displays the `getAllClassTypes` method, which fetches all class types from Firebase. It listens for a single value event and iterates through each `DataSnapshot` to construct `ClassType` objects. After successful data loading, the method invokes `onDataLoaded` with the list of class types. If an error occurs, the `onError` method is triggered with the error message, ensuring reliable synchronization of class types from Firebase.

```

public interface DataStatusBookings {
    void onDataLoaded(List<Booking> bookings);
    void onError(String error);
}

public interface DataStatusUsers {
    void onDataLoaded(List<User> userList);
    void onError(String error);
}

public interface DataStatusClassInstance {
    void onDataLoaded(List<ClassInstance> classInstances);
    void onError(String error);
}

public interface DataStatusYogaClass {
    void onDataLoaded(List<YogaClass> yogaClasses);
    void onError(String error);
}

public interface DataStatusClassType {
    void onDataLoaded(List<ClassType> classTypes);
    void onError(String error);
}

```

This code shows the `DataStatus` interfaces, which define `onDataLoaded` and `onError` callbacks for handling Firebase data synchronization events across various entities, ensuring structured data retrieval and error management.

5.1.4 NetworkChangeReceiver.java

```

public NetworkChangeReceiver(Context context) {
    this.context = context;
    this.databaseHelper = new DatabaseHelper(context);
    this.firebaseHelper = new FirebaseHelper();
    this.dataSyncHelper = new DataSyncHelper(context, databaseHelper);
    startSyncTask();
}

```

This code shows the constructor of `NetworkChangeReceiver`, where it initializes `DatabaseHelper`, `FirebaseHelper`, and `DataSyncHelper`. This setup prepares the receiver to monitor network changes and synchronize data when connectivity changes occur.

```

private void startSyncTask() {
    syncTask = new Runnable() {
        @Override
        public void run() {
            if (isConnected) {
                syncClassInstances();
                syncYogaClass();
                syncUsers();
                syncClassTypes();
                dataSyncHelper.syncUsersFromFirebase();
            }
        }
    };
    syncTask.run();
}

```

```

        dataSyncHelper.syncBookingsFromFirebase();
    }
    handler.postDelayed(this, 30000);
}
};
handler.post(syncTask);
}

```

This code illustrates the `startSyncTask()` method, which sets up a runnable task to synchronize data every 30 seconds if connected. It ensures that all data types, such as class instances and users, are kept up-to-date with Firebase.

```

@Override
public void onReceive(Context context, Intent intent) {
    ConnectivityManager cm = (ConnectivityManager) context.getSystemService(Context.CONNECTIVITY_SERVICE);
    NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
    isConnected = activeNetwork != null && activeNetwork.getType() == ConnectivityManager.TYPE_WIFI;

    if (isConnected) {
        syncClassInstances();
        syncYogaClass();
        syncUsers();
        syncClassTypes();
        dataSyncHelper.syncUsersFromFirebase();
        dataSyncHelper.syncBookingsFromFirebase();
        Toast.makeText(context, "WiFi Connected, syncing data...", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(context, "WiFi Disconnected, sync paused", Toast.LENGTH_SHORT).show();
    }
}

```

This code displays the `onReceive()` method in `NetworkChangeReceiver`, which detects network changes. If connected to Wi-Fi, it initiates data synchronization; otherwise, it displays a notification that syncing is paused.

```

private void syncYogaClass() {
    if (isConnected) {
        Cursor cursor = databaseHelper.getYogaClassesWithStatus("Pending");

        if (cursor != null && cursor.moveToFirst()) {
            do {
                String classId = cursor.getString(cursor.getColumnIndex("id"));
                String className = cursor.getString(cursor.getColumnIndex("typeOfClass"));
                String dayOfWeek = cursor.getString(cursor.getColumnIndex("dayOfWeek"));
                String time = cursor.getString(cursor.getColumnIndex("time"));
                int capacity = cursor.getInt(cursor.getColumnIndex("capacity"));
                int duration = cursor.getInt(cursor.getColumnIndex("duration"));
                double price = cursor.getDouble(cursor.getColumnIndex("price"));
                String description = cursor.getString(cursor.getColumnIndex("description"));

                YogaClass yogaClass = new YogaClass(classId, dayOfWeek, time, capacity, duration, price, className, description, "Uploaded");

                firebaseHelper.addUpdateYogaClassToFirebase(yogaClass);

                databaseHelper.updateYogaClassStatus(classId, "Uploaded");

            } while (cursor.moveToNext());
            cursor.close();
        }
    }
}

```

This code shows the `syncYogaClass()` method, which retrieves Yoga classes with a "Pending" status from the local database and uploads them to Firebase. After uploading, it updates their status to "Uploaded".

```

private void syncClassInstances() {
    if (isConnected) {
        Cursor cursor = databaseHelper.getClassInstancesWithStatus("Pending");

        if (cursor != null && cursor.moveToFirst()) {
            do {
                String id = cursor.getString(cursor.getColumnIndex("id"));
                String classId = cursor.getString(cursor.getColumnIndex("classId"));
                String date = cursor.getString(cursor.getColumnIndex("date"));
                String instructor = cursor.getString(cursor.getColumnIndex("instructor"));
                String additionalComments = cursor.getString(cursor.getColumnIndex("additionalComments"));

                ClassInstance classInstance = new ClassInstance(id, classId, date, instructor, additionalComments, "Uploaded");

                firebaseHelper.addUpdateYogaClassInstancesToFirebase(classInstance);
            }
        }
    }
}

```

```

        databaseHelper.updateClassInstanceStatus(id, "Uploaded");

    } while (cursor.moveToNext());
    cursor.close();
}
}
}

```

This code illustrates the `syncClassInstances()` method, which uploads all "Pending" class instances to Firebase. Each instance is marked as "Uploaded" in the local database after being synced.

```

private void syncUsers() {
    if (isConnected) {
        Cursor cursor = databaseHelper.getUsersWithStatus("Pending");

        if (cursor != null && cursor.moveToFirst()) {
            do {
                String userId = cursor.getString(cursor.getColumnIndex("id"));
                String email = cursor.getString(cursor.getColumnIndex("email"));
                String fullName = cursor.getString(cursor.getColumnIndex("fullName"));
                String phoneNumber = cursor.getString(cursor.getColumnIndex("phoneNumber"));
                String password = cursor.getString(cursor.getColumnIndex("password"));
                String role = cursor.getString(cursor.getColumnIndex("role"));

                User user = new User(userId, email, fullName, phoneNumber, password, "Uploaded", role);

                firebaseHelper.addUserToFirebase(user);

                databaseHelper.updateUserStatus(userId, "Uploaded");

            } while (cursor.moveToNext());
            cursor.close();
        }
    }
}

```

This code displays the `syncUsers()` method, which retrieves users with a "Pending" status from the local database, uploads them to Firebase, and updates their status to "Uploaded".

```

private void syncClassTypes() {
    if (isConnected) {
        Cursor cursor = databaseHelper.getClassTypesWithStatus("Pending");
        if (cursor != null && cursor.moveToFirst()) {
            do {
                String typeId = cursor.getString(cursor.getColumnIndex("id"));
                String typeName = cursor.getString(cursor.getColumnIndex("name"));
                String typeStatus = cursor.getString(cursor.getColumnIndex("status"));

                ClassType classType = new ClassType(typeId, typeName, "Uploaded");

                firebaseHelper.addUpdateClassTypeToFirebase(classType);

                databaseHelper.updateClassTypeStatus(typeId, "Uploaded");

            } while (cursor.moveToNext());
            cursor.close();
        }
    }
}

```

This code shows the `syncClassTypes()` method, which finds class types marked as "Pending" in the local database, uploads them to Firebase, and changes their status to "Uploaded" after successful synchronization.

5.1.5 Booking.java

```

package com.example.yogaapp.Model;
import java.util.List;
public class Booking {
    private String id;
    private String status;
    private boolean canCancel;
    private int totalPrice;
    private String userId;

    private List<ClassInstanceInfo> classInstances;

    public Booking() {}

    public Booking(String id, String status, boolean canCancel, int totalPrice, String userId, List<ClassInstanceInfo> classInstances) {
        this.id = id;
        this.status = status;
    }
}

```

```

        this.canCancel = canCancel;
        this.totalPrice = totalPrice;
        this.userId = userId;
        this.classInstances = classInstances;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public boolean isCanCancel() { return canCancel; }
    public void setCanCancel(boolean canCancel) { this.canCancel = canCancel; }

    public int getTotalPrice() { return totalPrice; }
    public void setTotalPrice(int totalPrice) { this.totalPrice = totalPrice; }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }

    public List<ClassInstanceInfo> getClassInstances() { return classInstances; }
    public void setClassInstances(List<ClassInstanceInfo> classInstances) { this.classInstances = classInstances; }
}

```

This code shows the Booking class, which represents a booking entity in the application. This class includes fields like id, status, canCancel, totalPrice, userId, and a list of classInstances to hold associated class sessions. The constructor initializes these fields, and getter and setter methods allow accessing and modifying each field. This structure enables the application to manage booking details for users, including the ability to track the status and associated classes for each booking.

5.1.6 ClassInstance.java

```

package com.example.yogaapp.Model;
public class ClassInstance {
    private String id;
    private String classId;
    private String date;
    private String instructor;
    private String additionalComments;
    private String status;

    public ClassInstance() {
    }

    public ClassInstance(String id, String classId, String date, String instructor, String additionalComments, String status) {
        this.id = id;
        this.classId = classId;
        this.date = date;
        this.instructor = instructor;
        this.additionalComments = additionalComments;
        this.status = status;
    }

    public String getId() {
        return id;
    }
    public void setId(String id) {
        this.id = id;
    }

    public String getClassId() {
        return classId;
    }
    public void setClassId(String classId) {
        this.classId = classId;
    }

    public String getDate() {
        return date;
    }
    public void setDate(String date) {
        this.date = date;
    }

    public String getInstructor() {
        return instructor;
    }
    public void setInstructor(String instructor) {
        this.instructor = instructor;
    }
}

```

```

}

public String getAdditionalComments() {
    return additionalComments;
}
public void setAdditionalComments(String additionalComments) { this.additionalComments = additionalComments;}

public String getStatus() {
    return status;
}
public void setStatus(String status) {
    this.status = status;
}
}
}

```

This code shows the `ClassInstance` class, which defines a yoga class session with properties such as `id`, `classId`, `date`, `instructor`, `additionalComments`, and `status`. This class is utilized to represent individual sessions, linking them to specific classes and providing details like the instructor and any additional comments. It enables the app to manage and display specific instances of yoga classes.

5.1.7 ClassInstanceInfo.java

```

package com.example.yogaapp.Model;

public class ClassInstanceInfo {
    private String classId;
    private String classInstanceId;

    public ClassInstanceInfo() {}

    public ClassInstanceInfo(String classId, String classInstanceId) {
        this.classId = classId;
        this.classInstanceId = classInstanceId;
    }

    public String getClassId() {
        return classId;
    }

    public void setClassId(String classId) {
        this.classId = classId;
    }

    public String getClassInstanceId() {
        return classInstanceId;
    }

    public void setClassInstanceId(String classInstanceId) { this.classInstanceId = classInstanceId; }
}

```

This code shows the `ClassInstanceInfo` class, which contains minimal details with two fields: `classId` and `classInstanceId`. This class is used to store basic reference information for specific instances of a yoga class. It allows the app to link particular sessions with class identifiers in a lightweight way, useful for referencing without carrying the full class details.

5.1.8 ClassSpinnerItem.java

```

package com.example.yogaapp.Model;

public class ClassSpinnerItem {
    private String classId;
    private String classInfo;

    public ClassSpinnerItem(String classId, String classInfo) {
        this.classId = classId;
        this.classInfo = classInfo;
    }

    public String getClassId() {
        return classId;
    }

    public String getClassInfo() {
        return classInfo;
    }

    @Override
    public String toString() {

```

```
    return classInfo;
  }
}
```

This code shows the `ClassSpinnerItem` class, which is used to represent items in a dropdown spinner with two fields: `classId` and `classInfo`. The `classId` holds the unique identifier for the class, while `classInfo` provides a readable name or description. The `toString()` method is overridden to display `classInfo` in the spinner dropdown, allowing users to select a class based on recognizable information.

5.1.9 ClassType.java

```
package com.example.yogaapp.Model;

public class ClassType {
    private String id;
    private String name;
    private String status;

    public ClassType() {
    }

    public ClassType(String id, String name, String status) {
        this.id = id;
        this.name = name;
        this.status = status;
    }

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

This code shows the `ClassType` class, designed to manage details of different yoga class types. It contains three private fields: `id` (a unique identifier for the class type), `name` (the name of the class type), and `status` (indicating the class type's current status). The class includes a parameterized constructor to initialize these fields and a default constructor for flexibility. Additionally, it provides public getter and setter methods for each field, allowing external access and modification, ensuring controlled access to the class type's details.

5.1.10 User.java

```
package com.example.yogaapp.Model;

public class User {
    private String id;
    private String email;
    private String fullName;
    private String phoneNumber;
    private String password;
    private String status;
    private String role;

    public User() {}

    public User(String id, String email, String fullName, String phoneNumber, String password, String status, String role) {
        this.id = id;
        this.email = email;
        this.fullName = fullName;
        this.phoneNumber = phoneNumber;
        this.password = password;
        this.status = status;
        this.role = role;
    }

    public String getId() {
        return id;
    }
}
```

```

}
public void setId(String id) { this.id = id; }

public String getEmail() {
    return email;
}
public void setEmail(String email) {
    this.email = email;
}

public String getFullName() {
    return fullName;
}
public void setFullName(String fullName) {
    this.fullName = fullName;
}

public String getPhoneNumber() {
    return phoneNumber;
}
public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
}

public String getPassword() {
    return password;
}
public void setPassword(String password) {
    this.password = password;
}

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

public String getRole() { return role; }
public void setRole(String role) {
    this.role = role;
}
}

```

This code illustrates the User class, which encapsulates the attributes and methods related to a user in the application. The class includes private fields such as id, email, fullName, phoneNumber, password, status, and role. Each field has its own getter and setter methods, allowing external classes to access and modify these properties. This structure helps manage user-specific information and facilitates user authentication and role management within the app.

5.1.11 YogaClass.java

```

package com.example.yogaapp.Model;
public class YogaClass {
    private String id, dayOfWeek, time, typeOfClass, description, status;
    private int capacity, duration;
    private double price;

    public YogaClass() {}

    public YogaClass(String id,String dayOfWeek, String time, int capacity, int duration, double price, String typeOfClass, String description, String status) {
        this.id=id;
        this.dayOfWeek = dayOfWeek;
        this.time = time;
        this.capacity = capacity;
        this.duration = duration;
        this.price = price;
        this.typeOfClass = typeOfClass;
        this.description = description;
        this.status = status;
    }

    public String getId() {
        return id;
    }
    public void setId(String id) {
        this.id = id;
    }
    public String getDayOfWeek() { return dayOfWeek; }
    public void setDayOfWeek(String dayOfWeek) { this.dayOfWeek = dayOfWeek; }
    public String getTime() { return time; }
    public void setTime(String time) { this.time = time; }
    public int getCapacity() { return capacity; }
    public void setCapacity(int capacity) { this.capacity = capacity; }
    public int getDuration() { return duration; }
    public void setDuration(int duration) { this.duration = duration; }
}

```

```

public double getPrice() { return price; }
public void setPrice(double price) { this.price = price; }
public String getClassName() { return typeOfClass; }
public void setClassName(String typeOfClass) { this.typeOfClass = typeOfClass; }
public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }
public String getStatus() {
    return status;
}
public void setStatus(String status) {
    this.status = status;
}
}

```

This code shows the YogaClass class, which encapsulates various properties related to a yoga class. It contains fields such as id, dayOfWeek, time, capacity, duration, price, typeOfClass, description, and status. These fields represent key details about each class, including its scheduling, pricing, and description. The class provides a constructor for initializing all fields, along with getter and setter methods to access and modify each property. This structure supports the effective management and retrieval of yoga class information within the application.

5.1.12 WelcomeActivity.java

```

public class WelcomeActivity extends AppCompatActivity {

    private Button loginButton, signUpButton;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_welcome);

        loginButton = findViewById(R.id.loginButton);
        signUpButton = findViewById(R.id.signUpButton);

        loginButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent loginIntent = new Intent(WelcomeActivity.this, LoginActivity.class);
                startActivity(loginIntent);
            }
        });

        signUpButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent signUpIntent = new Intent(WelcomeActivity.this, SignUpActivity.class);
                startActivity(signUpIntent);
            }
        });
    }
}

```

This code illustrates the WelcomeActivity class, which serves as the entry point for users when they open the application. This activity defines two buttons: loginButton and signUpButton. Within the onCreate method, each button is assigned an OnClickListener. When clicked, loginButton directs the user to LoginActivity, while signUpButton leads to SignUpActivity. This setup provides a simple user interface, allowing users to either log in or create a new account as they start using the app.

5.1.13 LoginActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);

    etEmail = findViewById(R.id.etEmail);
    etPassword = findViewById(R.id.etPassword);
    btnLogin = findViewById(R.id.btnLogin);
    tvSignUp = findViewById(R.id.tvSignUp);

    databaseHelper = new DatabaseHelper(this);

    networkReceiver = new NetworkChangeReceiver(this);
}

```

```

IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
registerReceiver(networkReceiver, filter);

btnLogin.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String email = etEmail.getText().toString();
        String password = etPassword.getText().toString();

        if (TextUtils.isEmpty(email) || TextUtils.isEmpty(password)) {
            Toast.makeText(LoginActivity.this, "Please enter complete information", Toast.LENGTH_SHORT).show();
            return;
        }

        User user = databaseHelper.getUserByEmailAndPassword(email, password);

        if (user != null) {
            if (user.getRole().equals("Admin")) {
                Intent adminIntent = new Intent(LoginActivity.this, AdminPageActivity.class);
                adminIntent.putExtra("USER_FULLNAME", user.getFullName());
                startActivity(adminIntent);
            } else if (user.getRole().equals("Teacher")) {
                Intent teacherIntent = new Intent(LoginActivity.this, TeacherPageActivity.class);
                teacherIntent.putExtra("USER_ID", user.getId());
                startActivity(teacherIntent);
            } else {
                Toast.makeText(LoginActivity.this, "Incorrect email or password", Toast.LENGTH_SHORT).show();
                Intent loginIntent = new Intent(LoginActivity.this, LoginActivity.class);
                startActivity(loginIntent);
            }
            finish();
        } else {
            Toast.makeText(LoginActivity.this, "Incorrect email or password", Toast.LENGTH_SHORT).show();
        }
    }
});

tvSignUp.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent signUpIntent = new Intent(LoginActivity.this, SignUpActivity.class);
        startActivity(signUpIntent);
    }
});
}

```

This code shows the `onCreate()` method in the `LoginActivity` class, which initializes the login page for the application. Figure sets up the activity layout, connects UI components such as the email and password fields (`etEmail`, `etPassword`), and the login and sign-up buttons (`btnLogin`, `tvSignUp`). It also initializes `databaseHelper` for database access and `networkReceiver` for monitoring network changes. The purpose of this setup is to allow users to log in, where, upon clicking `btnLogin`, the system verifies the credentials and displays appropriate messages. For instance, if the login fails, it shows a "Incorrect email or password" message.

5.1.14 SignUpActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_signup);

    etFullName = findViewById(R.id.etFullName);
    etPhoneNumber = findViewById(R.id.etPhoneNumber);
    etEmail = findViewById(R.id.etEmail);
    spRole = findViewById(R.id.spRole);
    etPassword = findViewById(R.id.etPassword);
    etConfirmPassword = findViewById(R.id.etConfirmPassword);
    btnSignUp = findViewById(R.id.btnSignUp);
    tvSignIn = findViewById(R.id.tvSignIn);

    ArrayAdapter<CharSequence> roleAdapter = ArrayAdapter.createFromResource(this,
        R.array.roles_array, android.R.layout.simple_spinner_item);
    roleAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spRole.setAdapter(roleAdapter);

    networkReceiver = new NetworkChangeReceiver(this);
    IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
    registerReceiver(networkReceiver, filter);
}

```

```

databaseHelper = new DatabaseHelper(this);
firebaseHelper = new FirebaseHelper();

btnSignUp.setOnClickListener(v -> {
    String email = etEmail.getText().toString();
    String password = etPassword.getText().toString().trim();
    String fullName = etFullName.getText().toString();
    String confirmPassword = etConfirmPassword.getText().toString();
    String phoneNumber = etPhoneNumber.getText().toString();
    String role = spRole.getSelectedItem().toString();

    if (TextUtils.isEmpty(email) || TextUtils.isEmpty(password) || TextUtils.isEmpty(fullName) || TextUtils.isEmpty(phoneNumber)) {
        Toast.makeText(SignUpActivity.this, "Please fill in all fields", Toast.LENGTH_SHORT).show();
        return;
    }
    if (databaseHelper.isEmailExists(email)) {
        Toast.makeText(SignUpActivity.this, "Email already exists!", Toast.LENGTH_SHORT).show();
        return;
    }
    if (password.length() < 8) {
        Toast.makeText(SignUpActivity.this, "Password must be at least 8 characters", Toast.LENGTH_SHORT).show();
        return;
    }
    if (!password.equals(confirmPassword)) {
        Toast.makeText(SignUpActivity.this, "Passwords do not match", Toast.LENGTH_SHORT).show();
        return;
    }
    String salt = generateSalt();
    String saltedHashedPassword = databaseHelper.hashPasswordWithSalt(password, salt);

    User user = new User(UUID.randomUUID().toString(), email, fullName, phoneNumber, saltedHashedPassword, "Pending", role);

    databaseHelper.addUser(user);
    firebaseHelper.addUserToFirebase(user);

    Toast.makeText(SignUpActivity.this, "Sign up successful", Toast.LENGTH_SHORT).show();
    Intent loginIntent = new Intent(SignUpActivity.this, LoginActivity.class);
    startActivity(loginIntent);
    finish();
});

tvSignIn.setOnClickListener(v -> {
    Intent loginIntent = new Intent(SignUpActivity.this, LoginActivity.class);
    startActivity(loginIntent);
    finish();
});
});
}

```

This code shows the `onCreate()` function in `SignUpActivity`, where essential UI components and listeners are set up. The function initializes input fields such as email, phone number, and password, connects these fields to a `DatabaseHelper` and `FirebaseHelper` for user data handling, and registers a `NetworkChangeReceiver` to monitor connectivity. Figure also implements validation for user registration, providing feedback messages to the user when fields are empty, passwords don't match, or certain criteria (like password length) aren't met. A success message confirms successful signup and redirects the user to the login page.

```

private String generateSalt() {
    SecureRandom sr = new SecureRandom();
    byte[] salt = new byte[16];
    sr.nextBytes(salt);
    return Base64.getEncoder().encodeToString(salt);
}

```

This code shows the `generateSalt()` function in `SignUpActivity`, which creates a random 16-byte salt used for password hashing. The method utilizes `SecureRandom` to generate secure random bytes and encodes the byte array to a Base64 string, ensuring added security for stored passwords. This salt is combined with the user's password before hashing, protecting against dictionary attacks.

5.1.15 TeacherPageActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_teacher_page);

    tvWelcomeTeacher = findViewById(R.id.tvWelcomeTeacher);
}

```

```

ivSettingsTeacher = findViewById(R.id.ivSettingsTeacher);
listViewClassInstances = findViewById(R.id.listViewClassInstances);
databaseHelper = new DatabaseHelper(this);
loggedInTeacherId = getIntent().getStringExtra("USER_ID");
User instructor = databaseHelper.getUserById(loggedInTeacherId);

if (loggedInTeacherId != null) {
    tvWelcomeTeacher.setText("Hi, " + instructor.getFullName());
} else {
    tvWelcomeTeacher.setText("Hi, Users");
}
ivSettingsTeacher.setOnClickListener(v -> {
    ivSettingsTeacher.setRotation(ivSettingsTeacher.getRotation() + 90);

    PopupMenu popup = new PopupMenu(TeacherPageActivity.this, ivSettingsTeacher);
    popup.getMenuInflater().inflate(R.menu.settings_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        if (item.getItemId() == R.id.menu_logout) {
            Intent intent = new Intent(TeacherPageActivity.this, LoginActivity.class);
            intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
            startActivity(intent);
            return true;
        }
        return false;
    });
    popup.show();
});
if (loggedInTeacherId != null) {
    loadClassInstancesForTeacher(loggedInTeacherId);
} else {
    Toast.makeText(this, "Error: Could not find teacher ID.", Toast.LENGTH_SHORT).show();
}
}

```

This code illustrates the `onCreate()` method within `TeacherPageActivity`, which initializes the view for a logged-in teacher. This method retrieves the teacher's ID from the intent, fetches their data from the database, and displays a welcome message. It also configures a settings menu with a logout option, allowing teachers to sign out. If the teacher ID is valid, it loads the associated class instances; otherwise, it shows an error message, aiding in user experience by confirming the correct setup or notifying of issues.

```

private void loadClassInstancesForTeacher(String teacherId) {
    List<ClassInstance> classInstances = databaseHelper.getClassInstancesByInstructorId(teacherId);
    if (classInstances != null && !classInstances.isEmpty()) {
        classInstanceAdapter = new ClassInstanceAdapter(this, classInstances, databaseHelper, true);
        listViewClassInstances.setAdapter(classInstanceAdapter);
    } else {
        Toast.makeText(this, "No class instances found for this teacher.", Toast.LENGTH_SHORT).show();
    }
}

```

This code shows the `loadClassInstancesForTeacher()` method, which fetches and displays class instances assigned to a teacher based on their ID. If the teacher has class instances, it initializes a `ClassInstanceAdapter` to display them in a list view. If no class instances are found, a toast notification alerts the teacher with the message "No class instances found for this teacher." This enhances user experience by either presenting relevant data or notifying the teacher if there is none.

5.1.16 AdminPageActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_admin_page);

    tvWelcomeAdmin = findViewById(R.id.tvWelcomeAdmin);
    ivSettings = findViewById(R.id.ivSettings);
    btnViewYogaClass = findViewById(R.id.btnViewYogaClass);
    btnViewClassInstance = findViewById(R.id.btnViewClassInstance);
    btnViewAccounts = findViewById(R.id.btnViewAccounts);
    btnViewBookings = findViewById(R.id.btnViewBookings);

    networkReceiver = new NetworkChangeReceiver(this);
    IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
    registerReceiver(networkReceiver, filter);

    btnViewYogaClass.setOnClickListener(new View.OnClickListener() {

```

```

@Override
public void onClick(View v) {
    Intent intent = new Intent(AdminPageActivity.this, ClassListActivity.class);
    startActivity(intent);
}
});

btnViewClassInstance.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(AdminPageActivity.this, ClassInstanceListActivity.class);
        startActivity(intent);
    }
});

btnViewAccounts.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(AdminPageActivity.this, AccountListActivity.class);
        startActivity(intent);
    }
});

btnViewBookings.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(AdminPageActivity.this, BookingListActivity.class);
        startActivity(intent);
    }
});

dbHelper = new DatabaseHelper(this);

String userFullname = getIntent().getStringExtra("USER_FULLNAME");
if (userFullname != null) {
    tvWelcomeAdmin.setText("Hi, " + userFullname);
} else {
    tvWelcomeAdmin.setText("Hi, Users");
}

ivSettings.setOnClickListener(v -> {
    PopupMenu popup = new PopupMenu(AdminPageActivity.this, ivSettings);
    popup.getMenuInflater().inflate(R.menu.settings_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        int itemId = item.getItemId();
        if (itemId == R.id.menu_logout) {
            Intent intent = new Intent(AdminPageActivity.this, LoginActivity.class);
            intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
            startActivity(intent);
            return true;
        } else if (itemId == R.id.menu_manage_class_type) {
            Intent intent = new Intent(AdminPageActivity.this, ManageClassTypesActivity.class);
            startActivity(intent);
            return true;
        } else if (itemId == R.id.menu_reset_data) {
            confirmResetData();
            return true;
        }
        return false;
    });
    popup.show();
});

loadStatistics();

BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
bottomNavigationView.setSelectedItemId(R.id.navigation_home);

bottomNavigationView.setOnNavigationItemSelectedListener(new BottomNavigationView.OnNavigationItemSelectedListener() {
    @Override
    public boolean onNavigationItemSelected(@NonNull MenuItem item) {
        int id = item.getItemId();

        if (id == R.id.navigation_home) {
            return true;
        } else if (id == R.id.navigation_class) {
            startActivity(new Intent(AdminPageActivity.this, ClassListActivity.class));
            return true;
        } else if (id == R.id.navigation_classinstance) {
            startActivity(new Intent(AdminPageActivity.this, ClassInstanceListActivity.class));
            return true;
        } else if (id == R.id.navigation_booking) {
            startActivity(new Intent(AdminPageActivity.this, BookingListActivity.class));
            return true;
        } else if (id == R.id.navigation_account) {
            startActivity(new Intent(AdminPageActivity.this, AccountListActivity.class));
            return true;
        }
    }
});

```

```

    }
    return false;
  }
  });
}

```

This code illustrates the `onCreate()` method in `AdminPageActivity`. This method sets up the admin dashboard, initializing user interface elements for viewing different sections such as Yoga Classes, Class Instances, Accounts, and Bookings. Buttons are set with `OnClickListeners` to navigate to corresponding pages. The method also includes a `NetworkChangeReceiver` to monitor connectivity status and loads user-specific greetings. Additionally, a bottom navigation bar is configured with listeners, enabling seamless navigation across core sections of the admin app. This setup ensures admins can efficiently manage the app's main entities.

```

private void confirmResetData() {
    new AlertDialog.Builder(this)
        .setTitle("Confirm Reset")
        .setMessage("Are you sure you want to reset all data except Users?")
        .setPositiveButton("Yes", (dialog, which) -> resetAllData())
        .setNegativeButton("No", null)
        .show();
}

```

This code shows the `confirmResetData()` method in `AdminPageActivity`, which displays a confirmation dialog before resetting data. The dialog asks the user to confirm if they want to reset all data except user accounts. If "Yes" is selected, the `resetAllData()` method is triggered, proceeding with the reset. If "No" is selected, the action is canceled. This method ensures that administrators have a chance to reconsider before performing a major data reset.

```

private void resetAllData() {
    DatabaseHelper db = new DatabaseHelper(this);

    SQLiteDatabase database = db.getWritableDatabase();
    database.delete("class_type", null, null);
    database.delete("bookings", null, null);
    database.delete("yoga_classes", null, null);
    database.delete("class_instances", null, null);
    database.close();

    FirebaseHelper firebaseHelper = new FirebaseHelper();
    firebaseHelper.deleteAllClassTypes();
    firebaseHelper.deleteAllBookings();
    firebaseHelper.deleteAllYogaClasses();
    firebaseHelper.deleteAllClassInstances();

    Toast.makeText(this, "All data reset except Users", Toast.LENGTH_SHORT).show();
}

```

This code illustrates the `resetAllData()` method in `AdminPageActivity`, which deletes all data except user accounts. This method first clears the specified tables (`class_type`, `bookings`, `yoga_classes`, `class_instances`) in the local `SQLite` database. Then, it calls methods from `FirebaseHelper` to remove corresponding data (`ClassTypes`, `Bookings`, `YogaClasses`, `ClassInstances`) from `Firebase`. A toast message, "All data reset except Users," confirms the action's completion. This function allows administrators to perform a selective data reset, retaining only user information.

```

private void loadStatistics() {
    int totalYogaClasses = getCountFromTable("yoga_classes");
    int totalClassInstances = getCountFromTable("class_instances");
    int totalAccounts = getCountFromTable("users");
    int totalBookings = getCountFromTable("bookings");

    TextView tvYogaClassCount = findViewById(R.id.tvYogaClassCount);
    TextView tvClassInstanceCount = findViewById(R.id.tvClassInstanceCount);
    TextView tvAccountCount = findViewById(R.id.tvAccountCount);
    TextView tvBookingCount = findViewById(R.id.tvBookingCount);

    tvYogaClassCount.setText("Total: " + totalYogaClasses);
}

```

```

tvClassInstanceCount.setText("Total: " + totalClassInstances);
tvAccountCount.setText("Total: " + totalAccounts);
tvBookingCount.setText("Total: " + totalBookings);
}

```

This code shows the `loadStatistics()` method in `AdminPageActivity`, designed to display statistics on various entities within the app. It retrieves counts for `YogaClasses`, `ClassInstances`, `Accounts`, and `Bookings` by calling `getCountFromTable()` on each respective table. These counts are then displayed in `TextView` elements for each category, providing an overview of the total items in each database table. This method enables administrators to quickly assess the current state of key data points in the application.

```

private int getCountFromTable(String tableName) {
    Cursor cursor = dbHelper.getReadableDatabase().rawQuery("SELECT COUNT(*) FROM " + tableName, null);
    if (cursor != null && cursor.moveToFirst()) {
        int count = cursor.getInt(0);
        cursor.close();
        return count;
    }
    return 0;
}

```

This code shows the `getCountFromTable()` method in `AdminPageActivity`, which is used to retrieve the total number of entries in a specified database table. By executing a raw SQL query with `SELECT COUNT(*)`, this method returns the count of records in the provided `tableName`. If a valid result is found, the count is returned; otherwise, it defaults to zero. This method aids in gathering summary statistics for display in the admin dashboard.

5.1.17 ManageClassTypesActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_manage_class_types);

    db = new DatabaseHelper(this);

    lvClassTypes = findViewById(R.id.lvClassTypes);
    searchView = findViewById(R.id.searchView);
    btnAddClassType = findViewById(R.id.btnAddClassType);

    loadClassTypesFromDatabase();

    classTypeAdapter = new ClassTypeAdapter(this, classTypes, db);
    lvClassTypes.setAdapter(classTypeAdapter);

    btnAddClassType.setOnClickListener(v -> showClassTypeDialog(null));

    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        @Override
        public boolean onQueryTextSubmit(String query) {
            return false;
        }

        @Override
        public boolean onQueryTextChange(String newText) {
            classTypeAdapter.getFilter().filter(newText);
            return true;
        }
    });
}

```

This code illustrates the `onCreate()` method in `ManageClassTypesActivity`, which is responsible for setting up the interface for managing yoga class types. This method initializes the database helper, list view, search view, and add button. It loads class types from the database and sets up the `ClassTypeAdapter` to manage the display of class types. A `SearchView.OnQueryTextListener` is used to filter the list based on user input, allowing for easy searching of class types.

```

private void loadClassTypesFromDatabase() {
    classTypes = new ArrayList<>();
    Cursor cursor = db.getAllClassTypes();
}

```

```

if (cursor.getCount() == 0) {
    Toast.makeText(this, "No class types found", Toast.LENGTH_SHORT).show();
    return;
}
while (cursor.moveToNext()) {
    ClassType classType = new ClassType(
        cursor.getString(cursor.getColumnIndex("id")),
        cursor.getString(cursor.getColumnIndex("name")),
        cursor.getString(cursor.getColumnIndex("status"))
    );
    classTypes.add(classType);
}
cursor.close();
}

```

This code shows the `loadClassTypesFromDatabase()` method in `ManageClassTypesActivity`, responsible for loading all class types from the database. This method creates a cursor to retrieve class type data, checking if any results exist. If no class types are found, it displays a toast message indicating this. Otherwise, it iterates over each row, creating `ClassType` objects from the retrieved data and adding them to the `classTypes` list, which will be displayed in the activity.

```

public void showClassTypeDialog(ClassType classType) {
    AlertDialog.Builder builder = new AlertDialog.Builder(this);
    builder.setTitle(classType == null ? "Add Class Type" : "Edit Class Type");

    final EditText input = new EditText(this);
    input.setText(classType != null ? classType.getName() : "");
    builder.setView(input);

    builder.setPositiveButton("Save", (dialog, which) -> {
        String name = input.getText().toString().trim();
        if (name.isEmpty()) {
            Toast.makeText(this, "Name cannot be empty", Toast.LENGTH_SHORT).show();
        } else if (classType == null) {
            addClassType(name);
        } else {
            updateClassType(classType, name);
        }
    });
    builder.setNegativeButton("Cancel", null);
    builder.show();
}

```

This code shows the `showClassTypeDialog()` method in `ManageClassTypesActivity`, which displays an alert dialog for adding or editing a class type. If `classType` is null, the dialog title is set to "Add Class Type"; otherwise, it shows "Edit Class Type." The dialog includes a text input field for the class type name. If the name field is empty, a toast message warns the user. On clicking "Save," the class type is added or updated based on its existence.

```

private void addClassType(String name) {
    String id = java.util.UUID.randomUUID().toString();
    ClassType newClassType = new ClassType(id, name, "Pending");
    if (db.addClassType(newClassType)) {
        classTypes.add(newClassType);
        refreshPage();
        Toast.makeText(this, "Class Type added", Toast.LENGTH_SHORT).show();
    }
}

```

This code shows the `addClassType()` method in `ManageClassTypesActivity`, which is responsible for adding a new class type to the database. It generates a unique ID for the class type, assigns a "Pending" status, and saves it to the database. If the addition is successful, the new class type is added to the list and the page is refreshed. A toast notification then confirms the addition with the message "Class Type added."

```

private void updateClassType(ClassType classType, String name) {
    classType.setName(name);
    classType.setStatus("Pending");
    if (db.updateClassType(classType)) {
        refreshPage();
        Toast.makeText(this, "Class Type updated", Toast.LENGTH_SHORT).show();
    }
}

```

This code displays the `updateClassType()` method in `ManageClassTypesActivity`, which updates an existing class type with a new name and sets its status to "Pending". After modifying the class type, it checks if the update was successful. If so, the page is refreshed, and a toast notification confirms the update with the message "Class Type updated."

```
public void refreshPage() {
    loadClassTypesFromDatabase();
    classTypeAdapter.notifyDataSetChanged();
}
```

This code shows the `refreshPage()` method in `ManageClassTypesActivity`, which reloads the list of class types from the database and updates the adapter to reflect any changes. This ensures that the displayed data stays current after any modifications.

5.1.18 ClassTypeAdapter.java

```
public ClassTypeAdapter(@NonNull Context context, List<ClassType> classTypeList, DatabaseHelper dbHelper) {
    super(context, 0, classTypeList);
    this.context = context;
    this.classTypeList = new ArrayList<>(classTypeList);
    this.filteredClassTypeList = new ArrayList<>(classTypeList);
    this.dbHelper = dbHelper;
    this.firebaseHelper = new FirebaseHelper();
}

@Override
public int getCount() {
    return filteredClassTypeList.size();
}

@Override
public ClassType getItem(int position) {
    return filteredClassTypeList.get(position);
}
```

Figure shows the constructor and essential methods of the `ClassTypeAdapter` class, which manages the display of `ClassType` items within a list. The adapter takes in a list of class types, initializes both the full and filtered lists, and sets up database and Firebase helpers for future data operations. The `getCount()` and `getItem()` methods return the size and specific item of the filtered list, respectively, ensuring accurate display based on the current filter.

```
@NonNull
@Override
public View getView(int position, View convertView, @NonNull ViewGroup parent) {
    ClassType classType = getItem(position);

    if (convertView == null) {
        convertView = LayoutInflater.from(context).inflate(R.layout.item_class_type, parent, false);
    }

    TextView tvClassName = convertView.findViewById(R.id.tvClassName);
    TextView tvClassStatus = convertView.findViewById(R.id.tvClassStatus);
    ImageButton btnMoreOptions = convertView.findViewById(R.id.btnMoreOptions);

    if (classType != null) {
        tvClassName.setText("🔗 " + classType.getName());
        tvClassStatus.setText("Status: " + classType.getStatus());
        btnMoreOptions.setOnClickListener(v -> showPopupMenu(v, classType));
    }

    return convertView;
}
```

Figure displays the `getView()` method within the `ClassTypeAdapter` class, which generates and populates a view for each `ClassType` item in the list. If `convertView` is null, it inflates a new view based on the `item_class_type` layout. The method sets the class name and status in `TextView` elements, and attaches a click listener to an options button, which triggers a popup menu for additional actions related to the `ClassType`.

```

private void showPopupMenu(View view, ClassType classType) {
    PopupMenu popup = new PopupMenu(context, view);
    popup.getMenuInflater().inflate(R.menu.class_item_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        int itemId = item.getItemId();
        if (itemId == R.id.menu_update) {
            ((ManageClassTypesActivity) context).showClassTypeDialog(classType);

            ((ManageClassTypesActivity) context).refreshPage();
            return true;
        } else if (itemId == R.id.menu_delete) {
            new AlertDialog.Builder(context)
                .setTitle("Confirm Delete")
                .setMessage("Are you sure you want to delete this class type?")
                .setPositiveButton("Yes", (dialog, which) -> {

                    firebaseHelper.deleteClassTypeFromFirebase(classType.getId());
                    dbHelper.deleteClassType(classType.getId());

                    ((ManageClassTypesActivity) context).refreshPage();
                    Toast.makeText(context, "Class Type deleted", Toast.LENGTH_SHORT).show();
                })
                .setNegativeButton("No", null)
                .show();
            return true;
        }
        return false;
    });
    popup.show();
}

```

This code illustrates the `showPopupMenu()` method within the `ClassTypeAdapter` class. This method displays a popup menu for each `ClassType` item, offering options such as updating or deleting the class type. When the update option is selected, it opens a dialog to edit the class type. Selecting the delete option triggers a confirmation dialog; if confirmed, the method removes the class type from both Firebase and the local database, then refreshes the page and shows a deletion success message.

```

@Override
public Filter getFilter() {
    return new Filter() {
        @Override
        protected FilterResults performFiltering(CharSequence constraint) {
            List<ClassType> filteredResults = new ArrayList<>();
            String query = constraint.toString().toLowerCase().trim();

            if (query.isEmpty()) {
                filteredResults.addAll(classTypeList);
            } else {
                for (ClassType classType : classTypeList) {
                    if (classType.getName().toLowerCase().contains(query)) {
                        filteredResults.add(classType);
                    }
                }
            }

            FilterResults results = new FilterResults();
            results.values = filteredResults;
            results.count = filteredResults.size();
            return results;
        }

        @Override
        protected void publishResults(CharSequence constraint, FilterResults results) {
            filteredClassTypeList.clear();
            filteredClassTypeList.addAll((List<ClassType>) results.values);
            notifyDataSetChanged();
        }
    };
}

```

This code shows the `getFilter()` method in the `ClassTypeAdapter` class, which enables filtering functionality for class types based on user input. This method returns a new `Filter` object that defines how filtering is performed. In `performFiltering`, it matches class types whose names contain the search query. The `publishResults` method updates the filtered list and refreshes the display, enabling a dynamic search experience within the class types list.

5.1.19 ClassListActivity.java

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_class_list);

    listView = findViewById(R.id.listView);
    btnAddClass = findViewById(R.id.btnEnterYogaClass);
    searchView = findViewById(R.id.searchView);
    spinnerFilter = findViewById(R.id.spinnerFilter);

    yogaClassList = new ArrayList<>();
    databaseHelper = new DatabaseHelper(this);

    loadClassesFromDatabase();

    classAdapter = new ClassAdapter(this, yogaClassList, databaseHelper);
    listView.setAdapter(classAdapter);

    DataSyncHelper dataSyncHelper = new DataSyncHelper(this, databaseHelper);
    dataSyncHelper.syncDataFromFirestoreIfNeeded();

    btnAddClass.setOnClickListener(v -> {
        Intent intent = new Intent(ClassListActivity.this, EnterYogaDetailsActivity.class);
        startActivity(intent);
    });

    ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, getUniqueDaysOfWeek());
    spinnerAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinnerFilter.setAdapter(spinnerAdapter);

    spinnerFilter.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
        @Override
        public void onItemSelected(AdapterView<?> parentView, View selectedItemView, int position, long id) {
            String selectedDay = (String) spinnerFilter.getSelectedItem();
            if (selectedDay.equals("All")) {
                classAdapter.getFilter().filter("");
            } else {
                classAdapter.getFilter().filter(selectedDay);
            }
        }
        @Override
        public void onNothingSelected(AdapterView<?> parentView) {}
    });

    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        @Override
        public boolean onQueryTextSubmit(String query) {
            return false;
        }

        @Override
        public boolean onQueryTextChange(String newText) {
            classAdapter.getFilter().filter(newText);
            return true;
        }
    });

    BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
    bottomNavigationView.setSelectedItemId(R.id.navigation_class);

    bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
        int id = item.getItemId();

        if (id == R.id.navigation_home) {
            Intent intent = new Intent(ClassListActivity.this, AdminPageActivity.class);
            startActivity(intent);
            return true;
        } else if (id == R.id.navigation_class) {
            return true;
        } else if (id == R.id.navigation_classinstance) {
            Intent intent = new Intent(ClassListActivity.this, ClassInstanceListActivity.class);
            startActivity(intent);
            return true;
        } else if (id == R.id.navigation_booking) {
            Intent intent = new Intent(ClassListActivity.this, BookingListActivity.class);
            startActivity(intent);
            return true;
        } else if (id == R.id.navigation_account) {
            Intent intent = new Intent(ClassListActivity.this, AccountListActivity.class);
            startActivity(intent);
            return true;
        }
        return false;
    });
}
```

This code illustrates the `onCreate()` method within `ClassListActivity`, responsible for initializing and displaying a list of yoga classes in the app. This method sets up UI components such as the `ListView` to show class entries, the Add Class button, `SearchView` for filtering by class name, and a dropdown filter for day selection. It also integrates the `DataSyncHelper` to synchronize data from Firebase if needed, ensuring the displayed list is up-to-date. Additionally, a bottom navigation setup allows users to navigate between main sections, like class instances, bookings, and accounts.

```
private List<String> getUniqueDaysOfWeek() {
    Set<String> daySet = new HashSet<>();
    for (YogaClass yogaClass : yogaClassList) {
        daySet.add(yogaClass.getDayOfWeek());
    }
    List<String> dayList = new ArrayList<>(daySet);
    dayList.add(0, "All");
    return dayList;
}
```

This code shows the `getUniqueDaysOfWeek()` method in `ClassListActivity`, which retrieves a list of unique days when yoga classes are scheduled. By iterating through the `yogaClassList`, it collects distinct `dayOfWeek` values using a `HashSet` to avoid duplicates. This list is then converted to an `ArrayList`, with an additional "All" option appended for comprehensive filtering. This method enables users to filter classes by specific days.

```
private void loadClassesFromDatabase() {
    Cursor cursor = databaseHelper.getAllYogaClasses();
    if (cursor == null || cursor.getCount() == 0) {
        Toast.makeText(this, "No classes found", Toast.LENGTH_SHORT).show();
        return;
    }
    while (cursor.moveToNext()) {
        YogaClass yogaClass = new YogaClass(
            cursor.getString(0),
            cursor.getString(1),
            cursor.getString(2),
            cursor.getInt(3),
            cursor.getInt(4),
            cursor.getDouble(5),
            cursor.getString(6),
            cursor.getString(7),
            cursor.getString(8)
        );
        yogaClassList.add(yogaClass);
    }
    cursor.close();
}
```

This code displays the `loadClassesFromDatabase()` method in `ClassListActivity`, which loads yoga class data from the database. It queries all yoga classes using `databaseHelper.getAllYogaClasses()` and iterates through the results. For each entry, a new `YogaClass` object is instantiated with values retrieved from the cursor, then added to `yogaClassList`. If no data is found, a toast message "No classes found" is displayed. This method ensures that the app has an up-to-date list of available yoga classes.

5.1.20 ClassAdapter.java

```
public ClassAdapter(@NonNull Context context, List<YogaClass> yogaClassList, DatabaseHelper dbHelper) {
    super(context, 0, yogaClassList);
    this.context = context;
    this.yogaClassList = new ArrayList<>(yogaClassList);
    this.filteredYogaClassList = new ArrayList<>(yogaClassList);
    this.dbHelper = dbHelper;
    this.firebaseHelper = new FirebaseHelper();
}

@Override
public int getCount() {
    return filteredYogaClassList.size();
}

@Override
```

```
public YogaClass getItem(int position) {
    return filteredYogaClassList.get(position);
}
```

This code shows the constructor and essential methods of the ClassAdapter in the Yoga application. The ClassAdapter constructor initializes context, a list of yoga classes (yogaClassList), a filtered list (filteredYogaClassList), and database helpers (dbHelper and firebaseHelper). The getCount() method returns the number of items in filteredYogaClassList, and getItem(int position) retrieves the item at the specified position. This adapter facilitates displaying filtered yoga classes in the list view.

```
@NonNull
@Override
public View getView(int position, View convertView, @NonNull ViewGroup parent) {
    YogaClass yogaClass = getItem(position);

    if (convertView == null) {
        convertView = LayoutInflater.from(context).inflate(R.layout.item_class, parent, false);
    }

    TextView tvTypeOfClass = convertView.findViewById(R.id.tvTypeOfClass);
    TextView tvDay = convertView.findViewById(R.id.tvDay);
    TextView tvTime = convertView.findViewById(R.id.tvTime);
    TextView tvCapacity = convertView.findViewById(R.id.tvCapacity);
    TextView tvPrice = convertView.findViewById(R.id.tvPrice);
    TextView tvDescription = convertView.findViewById(R.id.tvDescription);
    TextView tvStatus = convertView.findViewById(R.id.tvStatus);
    ImageButton btnMoreOptions = convertView.findViewById(R.id.btnMoreOptions);

    if (yogaClass != null) {
        tvTypeOfClass.setText("💎 " + yogaClass.getTypeOfClass());
        tvDay.setText("📅 " + yogaClass.getDayOfWeek());
        tvCapacity.setText("👥 " + String.valueOf(yogaClass.getCapacity()));
        tvPrice.setText("💰 " + String.valueOf(yogaClass.getPrice()));
        tvDescription.setText("📖 " + (yogaClass.getDescription() == null ? "..." : yogaClass.getDescription()));
        tvStatus.setText("🚦 " + yogaClass.getStatus());

        String startTime = yogaClass.getTime();
        int duration = yogaClass.getDuration();

        String displayTime = calculateEndTime(startTime, duration);
        tvTime.setText("🕒 " + displayTime);

        btnMoreOptions.setOnClickListener(v -> showPopupMenu(v, yogaClass));
    }

    return convertView;
}
```

This code presents the getView() method within ClassAdapter, responsible for populating each item in the list with yoga class details. The method initializes views such as tvTypeOfClass, tvDay, tvTime, and others to display specific details like type, day, time, capacity, and price of the yoga class. It also sets up a popup menu for further options, utilizing data from the YogaClass object. This method ensures that each list item reflects the correct data for each yoga class.

```
private String calculateEndTime(String startTime, int duration) {
    try {
        SimpleDateFormat timeFormat = new SimpleDateFormat("hh:mm a", Locale.getDefault());
        Calendar calendar = Calendar.getInstance();
        SimpleDateFormat inputFormat = new SimpleDateFormat("HH:mm", Locale.getDefault());
        calendar.setTime(inputFormat.parse(startTime));
        Calendar endTime = (Calendar) calendar.clone();
        endTime.add(Calendar.MINUTE, duration);
        return timeFormat.format(calendar.getTime()) + " - " + timeFormat.format(endTime.getTime());
    } catch (Exception e) {
        e.printStackTrace();
        return "";
    }
}
```

This code displays the calculateEndTime() method in ClassAdapter, which calculates the end time of a yoga class based on its start time and duration. The method uses SimpleDateFormat to parse the start time, and a Calendar instance to add the duration in minutes, resulting in the end time.

```

@Override
public Filter getFilter() {
    if (yogaClassFilter == null) {
        yogaClassFilter = new YogaClassFilter();
    }
    return yogaClassFilter;
}

```

This code shows the `getFilter()` method in the `ClassAdapter` class. Figure checks if the `yogaClassFilter` instance is null and initializes it as a new `YogaClassFilter` if needed. The purpose of this method is to provide a filter that enables the adapter to filter yoga classes based on certain criteria, enhancing the user's ability to search and display relevant results.

```

private void showPopupMenu(View view, YogaClass yogaClass) {
    PopupMenu popup = new PopupMenu(context, view);
    popup.getMenuInflater().inflate(R.menu.class_item_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        int itemId = item.getItemId();
        if (itemId == R.id.menu_update) {
            Intent intent = new Intent(context, EnterYogaDetailsActivity.class);
            intent.putExtra("classId", yogaClass.getId());
            intent.putExtra("dayOfWeek", yogaClass.getDayOfWeek());
            intent.putExtra("time", yogaClass.getTime());
            intent.putExtra("capacity", yogaClass.getCapacity());
            intent.putExtra("duration", yogaClass.getDuration());
            intent.putExtra("price", yogaClass.getPrice());
            intent.putExtra("typeOfClass", yogaClass.getTypeOfClass());
            intent.putExtra("description", yogaClass.getDescription());
            intent.putExtra("isUpdate", true);
            context.startActivity(intent);
            return true;
        } else if (itemId == R.id.menu_delete) {
            boolean hasInstances = dbHelper.getClassInstancesByClassId(yogaClass.getId()).getCount() > 0;
            String message = hasInstances ?
                "Are you sure you want to delete the class containing these class instances?" :
                "Are you sure you want to delete this class?";
            new AlertDialog.Builder(context)
                .setTitle("Confirm Delete")
                .setMessage(message)
                .setPositiveButton("Yes", (dialog, which) -> {
                    dbHelper.deleteYogaClass(yogaClass.getId());
                    firebaseHelper.deleteYogaClassFromFirebase(yogaClass.getId());
                    if (hasInstances) {
                        dbHelper.deleteClassInstancesByClassId(yogaClass.getId());
                    }
                    yogaClassList.remove(yogaClass);
                    notifyDataSetChanged();
                    Toast.makeText(context, "Class deleted successfully!", Toast.LENGTH_SHORT).show();

                    Intent refreshIntent = new Intent(context, ClassListActivity.class);
                    refreshIntent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_NEW_TASK);
                    context.startActivity(refreshIntent);
                })
                .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
                .show();
            return true;
        } else if (itemId == R.id.menu_view_detail) {
            Intent intent = new Intent(context, YogaClassDetailsActivity.class);
            intent.putExtra("classId", yogaClass.getId());
            context.startActivity(intent);
            return true;
        }
        return false;
    });
    popup.show();
}

```

This code shows the `showPopupMenu()` method in `ClassAdapter`, which displays a popup menu with options to view details, update, or delete a yoga class. Selecting "view details" opens the `YogaClassDetailsActivity` for the selected class. Choosing "update" launches `EnterYogaDetailsActivity` with the class's details passed via intent. Selecting "delete" triggers a confirmation dialog that adapts its message based on whether associated class instances exist. Upon confirmation, the class and any related instances are deleted from the local database and Firebase, with a Toast message confirming the deletion.

```

private class YogaClassFilter extends Filter {
    @Override
    protected FilterResults performFiltering(CharSequence constraint) {
        FilterResults results = new FilterResults();
        if (constraint == null || constraint.length() == 0) {
            results.values = yogaClassList;
            results.count = yogaClassList.size();
        } else {
            String filterString = constraint.toString().toLowerCase();
            List<YogaClass> filteredList = new ArrayList<>();

            for (YogaClass yogaClass : yogaClassList) {
                String endTime = calculateEndTime(yogaClass.getTime(), yogaClass.getDuration());
                if (yogaClass.getDayOfWeek().toLowerCase().contains(filterString)
                    || yogaClass.getTime().toLowerCase().contains(filterString)
                    || endTime.toLowerCase().contains(filterString)
                    || String.valueOf(yogaClass.getCapacity()).contains(filterString)
                    || String.valueOf(yogaClass.getPrice()).contains(filterString)
                    || yogaClass.getTypeOfClass().toLowerCase().contains(filterString)
                    || (yogaClass.getDescription() != null && yogaClass.getDescription().toLowerCase().contains(filterString))) {
                    filteredList.add(yogaClass);
                }
            }
            results.values = filteredList;
            results.count = filteredList.size();
        }
        return results;
    }

    @Override
    protected void publishResults(CharSequence constraint, FilterResults results) {
        filteredYogaClassList = (List<YogaClass>) results.values;
        notifyDataSetChanged();
    }
}

```

This code displays the YogaClassFilter class in the ClassAdapter, specifically implementing two methods: performFiltering() and publishResults(). The performFiltering() method filters the yogaClassList based on a search string, checking multiple attributes like day of the week, time, capacity, and type. It returns matching yoga classes in the filteredList. The publishResults() method then updates the adapter's list and notifies the change, allowing relevant results to be displayed to the user.

5.1.21 YogaClassDetailsActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_yoga_class_details);

    tvTypeOfClass = findViewById(R.id.tvTypeOfClass);
    tvDayOfWeek = findViewById(R.id.tvDayOfWeek);
    tvTime = findViewById(R.id.tvTime);
    tvCapacity = findViewById(R.id.tvCapacity);
    tvPrice = findViewById(R.id.tvPrice);
    tvDescription = findViewById(R.id.tvDescription);
    tvStatus = findViewById(R.id.tvStatus);
    lvClassInstances = findViewById(R.id.lvClassInstances);

    dbHelper = new DatabaseHelper(this);

    String classId = getIntent().getStringExtra("classId");
    if (classId != null) {
        selectedYogaClass = dbHelper.getYogaClassById(classId);
        if (selectedYogaClass != null) {
            displayClassDetails();
            loadClassInstances(classId);
        }
    }
}

```

This method initializes the activity by setting up the user interface and retrieving data. It binds UI elements such as TextView and ListView components and sets up a DatabaseHelper instance for database operations. The classId is obtained from the intent, and if valid, details of the selected yoga class are displayed using displayClassDetails(), and associated class instances are loaded via loadClassInstances(classId).

```
private void displayClassDetails() {
    String endTime = calculateEndTime(selectedYogaClass.getTime(), selectedYogaClass.getDuration());
    tvTypeOfClass.setText(" 🏆 Type of Class: " + selectedYogaClass.getTypeOfClass());
    tvDayOfWeek.setText(" 📅 Day of Week: " + selectedYogaClass.getDayOfWeek());
    tvTime.setText(" 🕒 Time: " + endTime);
    tvCapacity.setText(" 🧑 Capacity: " + selectedYogaClass.getCapacity());
    tvPrice.setText(" 💰 Price: " + selectedYogaClass.getPrice());
    tvDescription.setText(" 📖 Description: " + selectedYogaClass.getDescription());
    tvStatus.setText(" 🏠 Status: " + selectedYogaClass.getStatus());
}
```

This method displays detailed information about the selected yoga class on the screen, such as the type of class, day of the week, capacity, price, description, and status. The start and end times of the class are calculated and displayed using the `calculateEndTime()` method.

```
private String calculateEndTime(String startTime, int duration) {
    try {
        SimpleDateFormat timeFormat = new SimpleDateFormat("HH:mm", Locale.getDefault());
        Calendar calendar = Calendar.getInstance();

        calendar.setTime(timeFormat.parse(startTime));

        calendar.add(Calendar.MINUTE, duration);

        String endTime = timeFormat.format(calendar.getTime());

        return startTime + " - " + endTime;
    } catch (Exception e) {
        e.printStackTrace();
        return startTime;
    }
}
```

This method calculates the end time of the yoga class based on the given start time and duration. It parses the `startTime` into a `Calendar` object, adds the specified duration, and formats the resulting end time as a string. If parsing fails, it returns the original start time.

```
private void loadClassInstances(String classId) {
    classInstanceList = new ArrayList<>();
    Cursor cursor = dbHelper.getClassInstancesByClassId(classId);
    if (cursor != null && cursor.moveToFirst()) {
        do {
            String id = cursor.getString(cursor.getColumnIndex("id"));
            String date = cursor.getString(cursor.getColumnIndex("date"));
            String instructor = cursor.getString(cursor.getColumnIndex("instructor"));
            String additionalComments = cursor.getString(cursor.getColumnIndex("additionalComments"));
            String status = cursor.getString(cursor.getColumnIndex("status"));

            classInstanceList.add(new ClassInstance(
                id,
                classId,
                date,
                instructor,
                additionalComments,
                status
            ));
        } while (cursor.moveToNext());
        cursor.close();
    }

    ClassInstanceListAdapter adapter = new ClassInstanceListAdapter(this, classInstanceList);
    lvClassInstances.setAdapter(adapter);
}
```

This method fetches all instances of the specified yoga class from the database. It creates `ClassInstance` objects using data retrieved from the database cursor and adds them to a list. Finally, it sets up a `ClassInstanceListAdapter` to display these instances in the `ListView`.

5.1.22 ClassInstanceListAdapter.java

```
public ClassInstanceListAdapter(Context context, List<ClassInstance> classInstanceList) {
    this.context = context;
    this.classInstanceList = classInstanceList;
    this.dbHelper = new DatabaseHelper(context);
}

@Override
public int getCount() {
```

```

        return classInstanceList.size();
    }

    @Override
    public Object getItem(int position) {
        return classInstanceList.get(position);
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    static class ViewHolder {
        TextView tvDate;
        TextView tvInstructor;
        TextView tvComments;
    }

```

This code defines the `ClassInstanceListAdapter` constructor, initializing context, a list of class instances, and a `DatabaseHelper`. The `getCount()`, `getItem()`, and `getItemId()` methods manage item count, retrieval, and ID based on position. The `ViewHolder` class optimizes view handling by storing references to `TextView` elements for efficient data binding.

```

@Override
public View getView(int position, View convertView, ViewGroup parent) {
    ViewHolder holder;
    if (convertView == null) {
        convertView = LayoutInflater.from(context).inflate(R.layout.list_item_class_instance, parent, false);
        holder = new ViewHolder();
        holder.tvDate = convertView.findViewById(R.id.tvInstanceDate);
        holder.tvInstructor = convertView.findViewById(R.id.tvInstanceInstructor);
        holder.tvComments = convertView.findViewById(R.id.tvInstanceComments);
        convertView.setTag(holder);
    } else {
        holder = (ViewHolder) convertView.getTag();
    }

    ClassInstance instance = classInstanceList.get(position);
    if (instance != null) {
        holder.tvDate.setText("📅 " + (instance.getDate() != null ? instance.getDate() : "N/A"));
        String instructorId = instance.getInstructorId();
        User instructor = dbHelper.getUserById(instructorId);
        String instructorName = (instructor != null && instructor.getFullName() != null) ? instructor.getFullName() :
"Unknown";

        holder.tvInstructor.setText("👤 Instructor: " + instructorName);
        holder.tvComments.setText("💬 Comments: " + (instance.getAdditionalComments() != null ?
instance.getAdditionalComments() : "No Comments"));
    }

    return convertView;
}

```

This code defines the `getView()` method for the `ClassInstanceListAdapter`, responsible for populating list items. It uses a `ViewHolder` pattern for efficient view recycling. If `convertView` is null, it inflates the layout, initializes `ViewHolder` components, and sets the tag. It binds data, displaying the date, instructor's name retrieved using the database helper, and additional comments for each `ClassInstance`. Missing data defaults to "N/A" or "Unknown."

5.1.23 EnterYogaDetailsActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_enter_yoga_details);
    db = new DatabaseHelper(this);
    firebaseHelper = new FirebaseHelper();

    tvHeading = findViewById(R.id.tvHeading);
    spDayOfWeek = findViewById(R.id.spDayOfWeek);
    npHour = findViewById(R.id.npHour);
    npMinute = findViewById(R.id.npMinute);
    etCapacity = findViewById(R.id.etCapacity);
    npDuration = findViewById(R.id.npDuration);
    etPrice = findViewById(R.id.etPrice);
    etDescription = findViewById(R.id.etDescription);
    spTypeOfClass = findViewById(R.id.spTypeOfClass);
    btnSaveYogaClass = findViewById(R.id.btnSaveYogaClass);
}

```

```

tvWifiStatus = findViewById(R.id.tvWifiStatus);

checkWifiStatus();
networkReceiver = new NetworkChangeReceiver(this);
IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
registerReceiver(networkReceiver, filter);
setupSpinnersAndPickers();

Intent intent = getIntent();
classId = intent.getStringExtra("classId");
if (classId != null) {
    hasClassInstances = checkIfClassInstancesExist(classId);
    tvHeading.setText("⌕ Update class");
    loadYogaClassForUpdate(intent);
}
btnSaveYogaClass.setOnClickListener(v -> {
    if (classId != null) {
        updateYogaClass();
    } else {
        saveYogaClass();
    }
});
}

```

This code shows the `onCreate()` method within `EnterYogaDetailsActivity`. This method initializes UI components, sets up network connectivity checks, and manages the logic for adding or updating yoga class details. It retrieves the `classId` to determine if a class is being edited or newly created, updating the header text accordingly. The `btnSaveYogaClass` button listener handles either saving a new class or updating an existing one based on the presence of `classId`.

```

private void checkWifiStatus() {
    ConnectivityManager cm = (ConnectivityManager) getSystemService(Context.CONNECTIVITY_SERVICE);
    NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
    if (activeNetwork != null && activeNetwork.getType() == ConnectivityManager.TYPE_WIFI) {
        tvWifiStatus.setText("WiFi: Connected");
    } else {
        tvWifiStatus.setText("WiFi: Not Connected");
    }
}

```

This code shows the `checkWifiStatus()` method in `EnterYogaDetailsActivity`. This method checks the device's Wi-Fi connectivity status. If a Wi-Fi connection is active, it sets the `tvWifiStatus` `TextView` to display "WiFi: Connected"; otherwise, it displays "WiFi: Not Connected". This provides users with real-time feedback on their connectivity status.

```

private void setupSpinnersAndPickers() {
    ArrayAdapter<CharSequence> dayOfWeekAdapter = ArrayAdapter.createFromResource(this,
        R.array.days_of_week, android.R.layout.simple_spinner_item);
    dayOfWeekAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spDayOfWeek.setAdapter(dayOfWeekAdapter);

    npHour.setMinValue(0);
    npHour.setMaxValue(23);
    npHour.setFormatter(i -> String.format("%02d", i));

    npMinute.setMinValue(0);
    npMinute.setMaxValue(59);
    npMinute.setFormatter(i -> String.format("%02d", i));

    int[] durationValues = {15, 30, 45, 60, 75, 90, 105, 120};
    npDuration.setMinValue(0);
    npDuration.setMaxValue(durationValues.length - 1);
    npDuration.setDisplayedValues(new String[]{"15", "30", "45", "60", "75", "90", "105", "120"});

    etCapacity.setInputType(android.text.InputType.TYPE_CLASS_NUMBER);
    etPrice.setInputType(android.text.InputType.TYPE_CLASS_NUMBER | android.text.InputType.TYPE_NUMBER_FLAG_DECIMAL);

    loadClassTypesIntoSpinner();
}

```

This code displays the `setupSpinnersAndPickers()` method in `EnterYogaDetailsActivity`. This method configures various input elements like the day of the week spinner and the hour, minute, and duration pickers. It sets ranges and formats for each picker to ensure that users can easily select valid times and

class durations. This setup enhances the user experience by providing structured and controlled input options.

```
private void loadClassTypesIntoSpinner() {
    Cursor cursor = db.getAllClassTypes();
    List<String> classTypeNames = new ArrayList<>();
    if (cursor.moveToFirst()) {
        do {
            classTypeNames.add(cursor.getString(cursor.getColumnIndex("name")));
        } while (cursor.moveToNext());
    }
    cursor.close();

    ArrayAdapter<String> classTypeAdapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, classTypeNames);
    classTypeAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spTypeOfClass.setAdapter(classTypeAdapter);
}
```

This code shows the `loadClassTypesIntoSpinner()` method in `EnterYogaDetailsActivity`. This method populates the class type spinner with values retrieved from the database. It retrieves class type names from the database, adds them to a list, and then uses an `ArrayAdapter` to display these names in a dropdown spinner, allowing users to select a specific class type easily.

```
private boolean checkIfClassInstancesExist(String classId) {
    Cursor cursor = db.getClassInstancesByClassId(classId);
    boolean exists = cursor != null && cursor.getCount() > 0;
    cursor.close();
    return exists;
}
```

This code shows the `checkIfClassInstancesExist()` method in `EnterYogaDetailsActivity`. This method checks if there are any instances associated with a specific class by querying the database with a `classId`. If instances are found, it returns true; otherwise, it returns false. This is useful for determining if a class has scheduled sessions before performing actions like updates or deletions.

```
private void loadYogaClassForUpdate(Intent intent) {
    String dayOfWeek = intent.getStringExtra("dayOfWeek");
    String time = intent.getStringExtra("time");
    int capacity = intent.getIntExtra("capacity", 0);
    int duration = intent.getIntExtra("duration", 0);
    double price = intent.getDoubleExtra("price", 0.0);
    String typeOfClass = intent.getStringExtra("typeOfClass");
    String description = intent.getStringExtra("description");

    ArrayAdapter<CharSequence> adapter = (ArrayAdapter<CharSequence>) spDayOfWeek.getAdapter();
    spDayOfWeek.setSelection(adapter.getPosition(dayOfWeek));

    if (time != null) {
        npHour.setValue(Integer.parseInt(time.split(":")[0]));
        npMinute.setValue(Integer.parseInt(time.split(":")[1]));
    }

    etCapacity.setText(String.valueOf(capacity));
    npDuration.setValue(findDurationIndex(duration));
    etPrice.setText(String.valueOf(price));
    spTypeOfClass.setSelection(getClassTypeIdIndex(typeOfClass));
    etDescription.setText(description);

    if (hasClassInstances) {
        disableFieldsExceptDescription();
    }
}
```

This code shows the `loadYogaClassForUpdate()` method in `EnterYogaDetailsActivity`. This method extracts data from the provided `Intent` to populate the fields for updating an existing yoga class. It sets values such as day, time, capacity, duration, price, type, and description in the relevant input fields. If class instances already exist for this class, certain fields are disabled to prevent unwanted modifications, enhancing data consistency.

```
private void disableFieldsExceptDescription() {
    spDayOfWeek.setEnabled(false);
    npHour.setEnabled(false);
}
```

```

npMinute.setEnabled(false);
etCapacity.setEnabled(false);
npDuration.setEnabled(false);
etPrice.setEnabled(false);
spTypeOfClass.setEnabled(false);
}

```

This code shows the `disableFieldsExceptDescription()` method in `EnterYogaDetailsActivity`. This method disables all input fields except for the description field. It is used to restrict edits to only the description while preventing modifications to other details like day, time, capacity, and price, which is especially useful when there are existing class instances associated with the class.

```

private int findDurationIndex(int duration) {
    int[] durationValues = {15, 30, 45, 60, 75, 90, 105, 120};
    for (int i = 0; i < durationValues.length; i++) {
        if (durationValues[i] == duration) {
            return i;
        }
    }
    return 0;
}

```

This code presents a helper function, `findDurationIndex(int duration)`, designed to find the position of a given duration value within a predefined array of acceptable durations. This function iterates through the `durationValues` array, searching for a match with the input duration. If found, it returns the index, otherwise it defaults to zero. This function is useful in UI contexts where a spinner or dropdown needs to pre-select the matching duration, ensuring the correct value is displayed for editing or reviewing.

```

private int getClassTypeIndex(String typeOfClass) {
    for (int i = 0; i < spTypeOfClass.getCount(); i++) {
        if (spTypeOfClass.getItemAtPosition(i).equals(typeOfClass)) {
            return i;
        }
    }
    return 0;
}

```

This code illustrates the `getClassTypeIndex(String typeOfClass)` function, which retrieves the index of a specified class type within a spinner. By iterating through all available items in the spinner, this function locates the position of `typeOfClass` and returns the index. If the specified class type is not found, it returns zero. This functionality is essential for setting the correct pre-selected item in the spinner when a class is being edited, ensuring that the current class type is visibly highlighted for the user.

```

private boolean checkIfClassExists(String dayOfWeek, String time, int capacity, int duration, double price, String typeOfClass, String description) {
    Cursor cursor = db.getYogaClass(dayOfWeek, time, capacity, duration, price, typeOfClass, description);
    return cursor != null && cursor.getCount() > 0;
}

```

This code displays the `checkIfClassExists` function, which checks if a class with specific attributes already exists in the database. It queries the database using attributes such as day of the week, time, capacity, duration, price, type, and description. If any matching records are found, it returns true, indicating that the class exists; otherwise, it returns false. This function is critical in preventing duplicate entries, ensuring data integrity by verifying the uniqueness of each class before saving.

```

private void saveYogaClass() {
    if (validateInputs()) {
        String id = UUID.randomUUID().toString();
        String dayOfWeek = spDayOfWeek.getSelectedItem().toString();
        String time = String.format("%02d:%02d", npHour.getValue(), npMinute.getValue());
        int capacity = Integer.parseInt(etCapacity.getText().toString());
        int duration = Integer.parseInt(npDuration.getDisplayedValues()[npDuration.getValue()]);
        double price = Double.parseDouble(etPrice.getText().toString());
        String typeOfClass = spTypeOfClass.getSelectedItem().toString();
        String description = etDescription.getText().toString();

        if (checkIfClassExists(dayOfWeek, time, capacity, duration, price, typeOfClass, description)) {
            Toast.makeText(this, "Class already exists!", Toast.LENGTH_SHORT).show();
        }
    }
}

```

```

    } else {
        YogaClass classModel = new YogaClass(id, dayOfWeek, time, capacity, duration, price, typeOfClass, description, "Pending");
        boolean result = db.addYogaClass(classModel);
        if (result) {
            Toast.makeText(this, "Class saved successfully!", Toast.LENGTH_SHORT).show();
            goToClassListActivity();
        } else {
            Toast.makeText(this, "Failed to save class", Toast.LENGTH_SHORT).show();
        }
    }
}
}
}

```

This code showcases the `saveYogaClass` function, responsible for saving a new yoga class after validating the input fields. It generates a unique ID and collects values from the input fields such as day, time, capacity, duration, and price. Before saving, it calls `checkIfClassExists` to ensure no duplicate class exists. If unique, it saves the class to the database and displays a confirmation message. This function enhances user experience by preventing duplicate entries and confirming successful saves.

```

private void updateYogaClass() {
    if (validateInputs()) {
        String description = etDescription.getText().toString();
        if (hasClassInstances()) {
            db.updateYogaClassDescription(classId, description, "Pending");
            Toast.makeText(this, "Description updated successfully!", Toast.LENGTH_SHORT).show();
        } else {
            String dayOfWeek = spDayOfWeek.getSelectedItem().toString();
            String time = String.format("%02d:%02d", npHour.getValue(), npMinute.getValue());
            int capacity = Integer.parseInt(etCapacity.getText().toString());
            int duration = Integer.parseInt(npDuration.getDisplayedValues()[npDuration.getValue()]);
            double price = Double.parseDouble(etPrice.getText().toString());
            String typeOfClass = spTypeOfClass.getSelectedItem().toString();

            YogaClass classModel = new YogaClass(classId, dayOfWeek, time, capacity, duration, price, typeOfClass, description, "Pending");
            boolean result = db.updateYogaClass(classModel);
            if (result) {
                Toast.makeText(this, "Class updated successfully!", Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this, "Failed to update class", Toast.LENGTH_SHORT).show();
            }
        }
        goToClassListActivity();
    }
}
}

```

This code illustrates the `updateYogaClass` function, which allows modification of an existing yoga class's details. If class instances exist, only the description can be updated; otherwise, all fields can be modified. This function ensures data consistency by restricting updates to critical fields only when necessary, preventing accidental changes to existing instances.

```

private boolean validateInputs() {
    if (!hasClassInstances()) {
        if (spDayOfWeek.getSelectedItem() == null || spDayOfWeek.getSelectedItem().toString().trim().isEmpty()) {
            Toast.makeText(this, "Please select a day of the week", Toast.LENGTH_SHORT).show();
            return false;
        }
        if (etCapacity.getText().toString().trim().isEmpty()) {
            Toast.makeText(this, "Please enter the class capacity", Toast.LENGTH_SHORT).show();
            return false;
        }
        if (etPrice.getText().toString().trim().isEmpty()) {
            Toast.makeText(this, "Please enter the class price", Toast.LENGTH_SHORT).show();
            return false;
        }
    }
    return true;
}
}

```

This code shows the `validateInputs` function, which verifies that essential input fields like day, capacity, and price are filled. If any of these fields are empty, a warning message is displayed, prompting the user to complete the required information. This validation ensures that no incomplete data is saved, maintaining data quality.

```
private void goToClassListActivity() {
    Intent intent = new Intent(this, ClassListActivity.class);
    intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_NEW_TASK);
    startActivity(intent);
    finish();
}
```

This code shows the `goToClassListActivity` function, which navigates the user back to the class list screen after saving or updating a class. The code uses intent flags to clear the activity stack, preventing users from returning to the edit screen when pressing back. This provides a smooth navigation experience.

5.1.24 ClassInstanceListActivity.java

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_class_instance_list);

    listView = findViewById(R.id.listViewClassInstances);
    btnAddClassInstance = findViewById(R.id.btnAddClassInstance);
    searchView = findViewById(R.id.searchView);
    spinnerDayOfWeekFilter = findViewById(R.id.spinnerDayOfWeekFilter);

    classInstanceList = new ArrayList<>();
    databaseHelper = new DatabaseHelper(this);

    loadYogaClasses();
    loadClassInstancesFromDatabase();

    classInstanceAdapter = new ClassInstanceAdapter(this, classInstanceList, databaseHelper, false);
    listView.setAdapter(classInstanceAdapter);

    DataSyncHelper dataSyncHelper = new DataSyncHelper(this, databaseHelper);
    dataSyncHelper.syncDataFromFirestoreIfNeeded();

    btnAddClassInstance.setOnClickListener(v -> {
        Intent intent = new Intent(ClassInstanceListActivity.this, EnterClassInstanceDetailsActivity.class);
        intent.putExtra("classId", "<classId>");
        startActivity(intent);
    });

    BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
    bottomNavigationView.setSelectedItemId(R.id.navigation_classinstance);

    bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
        int id = item.getItemId();

        if (id == R.id.navigation_home) {
            Intent intent = new Intent(ClassInstanceListActivity.this, AdminPageActivity.class);
            startActivity(intent);
            return true;
        } else if (id == R.id.navigation_class) {
            Intent intent = new Intent(ClassInstanceListActivity.this, ClassListActivity.class);
            startActivity(intent);
            return true;
        } else if (id == R.id.navigation_classinstance) {
            return true;
        } else if (id == R.id.navigation_booking) {
            Intent intent = new Intent(ClassInstanceListActivity.this, BookingListActivity.class);
            startActivity(intent);
            return true;
        } else if (id == R.id.navigation_account) {
            Intent intent = new Intent(ClassInstanceListActivity.this, AccountListActivity.class);
            startActivity(intent);
            return true;
        }

        return false;
    });

    ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, getUniqueDaysOfWeek());
    spinnerAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinnerDayOfWeekFilter.setAdapter(spinnerAdapter);

    spinnerDayOfWeekFilter.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
        @Override
        public void onItemSelected(AdapterView<?> parentView, View selectedItemView, int position, long id) {
            String selectedDay = (String) spinnerDayOfWeekFilter.getSelectedItem();
            classInstanceAdapter.filterByDayOfWeek(selectedDay);
        }
        @Override
        public void onNothingSelected(AdapterView<?> parentView) {}
    });
}
```

```

searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
    @Override
    public boolean onQueryTextSubmit(String query) {
        return false;
    }

    @Override
    public boolean onQueryTextChange(String newText) {
        classInstanceAdapter.getFilter().filter(newText);
        return true;
    }
});
}

```

This code shows the `onCreate()` method in the `ClassInstanceListActivity` class, where UI components like a list view for class instances, a search bar, and a day-of-week filter spinner are initialized. It sets up database and adapter connections, syncs data with Firebase, and includes navigation controls. The search bar filters displayed instances based on user input, while the spinner filters by selected days.

```

private List<String> getUniqueDaysOfWeek() {
    Set<String> daySet = new HashSet<>();
    for (YogaClass yogaClass : yogaClasses) {
        daySet.add(yogaClass.getDayOfWeek());
    }
    List<String> dayList = new ArrayList<>(daySet);
    dayList.add(0, "All");
    return dayList;
}

private void loadYogaClasses() {
    yogaClasses = new ArrayList<>();
    Cursor cursor = databaseHelper.getAllYogaClasses();
    while (cursor != null && cursor.moveToNext()) {
        yogaClasses.add(new YogaClass(
            cursor.getString(0),
            cursor.getString(1),
            cursor.getString(2),
            cursor.getInt(3),
            cursor.getInt(4),
            cursor.getDouble(5),
            cursor.getString(6),
            cursor.getString(7),
            cursor.getString(8)
        ));
    }
    if (cursor != null) {
        cursor.close();
    }
}

```

This code shows two methods: `getUniqueDaysOfWeek()` and `loadYogaClasses()`. The `loadYogaClasses()` method retrieves all yoga classes from the database, populating the `yogaClasses` list by creating `YogaClass` objects for each entry in the database cursor. Once loaded, `getUniqueDaysOfWeek()` extracts unique days of the week from the list of yoga classes, storing them in a set to ensure uniqueness, then converts the set to a list, adding "All" as the default filter option at the top.

```

private void loadClassInstancesFromDatabase() {
    Cursor cursor = databaseHelper.getAllClassInstances();
    if (cursor == null || cursor.getCount() == 0) {
        Toast.makeText(this, "No class instances found", Toast.LENGTH_SHORT).show();
        return;
    }
    while (cursor.moveToNext()) {
        ClassInstance classInstance = new ClassInstance(
            cursor.getString(0),
            cursor.getString(1),
            cursor.getString(2),
            cursor.getString(3),
            cursor.getString(4),
            cursor.getString(5)
        );
        classInstanceList.add(classInstance);
    }
    cursor.close();
}

```

This code shows the `loadClassInstancesFromDatabase()` method in `ClassInstanceListActivity`, which retrieves all class instance records from the database and adds them to a list. If no records are found, it displays a toast notification saying "No Class instances found". This method iterates through each database row, creates a `ClassInstance` object for each entry, and adds it to `classInstanceList` for display in the UI.

5.1.25 ClassInstanceAdapter.java

```
public ClassInstanceAdapter(@NonNull Context context, List<ClassInstance> classInstanceList, DatabaseHelper dbHelper, boolean isTeacherMode) {
    super(context, 0, classInstanceList);
    this.context = context;
    this.classInstanceList = new ArrayList<>();
    this.filteredClassInstanceList = new ArrayList<>();
    this.dbHelper = dbHelper;
    this.firebaseHelper = new FirebaseHelper();
    this.isTeacherMode = isTeacherMode;
}

@Override
public int getCount() {
    return filteredClassInstanceList.size();
}

@Override
public ClassInstance getItem(int position) {
    return filteredClassInstanceList.get(position);
}
```

This code illustrates the constructor and initial methods in the `ClassInstanceAdapter` class. Figure initializes the adapter with a context, list of class instances, and database helpers. It sets up separate lists for all class instances and filtered results, providing the ability to filter and display data dynamically.

```
@NonNull
@Override
public View getView(int position, View convertView, @NonNull ViewGroup parent) {
    ClassInstance classInstance = getItem(position);

    if (convertView == null) {
        convertView = LayoutInflater.from(context).inflate(R.layout.item_class_instance, parent, false);
    }

    TextView tvDate = convertView.findViewById(R.id.tvDate);
    TextView tvInstructor = convertView.findViewById(R.id.tvInstructor);
    TextView tvClassInfo = convertView.findViewById(R.id.tvClassInfo);
    TextView tvStatus = convertView.findViewById(R.id.tvStatus);
    TextView tvAdditionalComments = convertView.findViewById(R.id.tvAdditionalComments);
    ImageButton btnMoreOptions = convertView.findViewById(R.id.btnMoreOptions);

    if (classInstance != null) {
        tvDate.setText("📅 " + classInstance.getDate());
        tvStatus.setText("📌 " + classInstance.getStatus());
        tvAdditionalComments.setText("💬 " + classInstance.getAdditionalComments());
        YogaClass yogaClass = dbHelper.getYogaClassById(classInstance.getClassId());
        if (yogaClass != null) {
            tvClassInfo.setText("🧘 " + yogaClass.getTypeOfClass() + " - " + yogaClass.getDayOfWeek() + " - " + yogaClass.getTime());
        }
        User instructor = dbHelper.getUserById(classInstance.getInstructorId());
        if (instructor != null) {
            tvInstructor.setText("👤 " + instructor.getFullName());
        } else {
            tvInstructor.setText("👤 Instructor not found");
        }
        if (isTeacherMode) {
            btnMoreOptions.setVisibility(View.GONE);
            tvStatus.setVisibility(View.GONE);
        } else {
            btnMoreOptions.setVisibility(View.VISIBLE);
            btnMoreOptions.setOnClickListener(v -> showPopupMenu(v, classInstance));
        }
    }
    return convertView;
}
```

This code shows the `getView` method in the `ClassInstanceAdapter`, which populates each item in the list view with details from a `ClassInstance` object. This method formats the display of each instance, including

attributes like date, instructor, status, and additional comments. It also configures the visibility of options based on the user's role.

```
private void showPopupMenu(View view, ClassInstance classInstance) {
    PopupMenu popup = new PopupMenu(context, view);
    popup.getMenuInflater().inflate(R.menu.class_item_menu, popup.getMenu());
    popup.setOnMenuItemClickListener(item -> {
        int itemId = item.getItemId();
        if (itemId == R.id.menu_update) {
            Intent intent = new Intent(context, EnterClassInstanceDetailsActivity.class);
            intent.putExtra("instanceId", classInstance.getId());
            intent.putExtra("classId", classInstance.getClassId());
            intent.putExtra("date", classInstance.getDate());
            intent.putExtra("instructor", classInstance.getInstructor());
            intent.putExtra("additionalComments", classInstance.getAdditionalComments());
            intent.putExtra("isUpdate", true);
            context.startActivity(intent);
            return true;
        } else if (itemId == R.id.menu_delete) {
            new AlertDialog.Builder(context)
                .setTitle("Confirm Delete")
                .setMessage("Are you sure you want to delete this class instance?")
                .setPositiveButton("Yes", (dialog, which) -> {
                    dbHelper.deleteClassInstance(classInstance.getId());
                    firebaseHelper.deleteYogaClassInstancesFromFirebase(classInstance.getId());
                    classInstanceList.remove(classInstance);
                    filteredClassInstanceList.remove(classInstance);
                    notifyDataSetChanged();
                    Toast.makeText(context, "Class instance deleted successfully from SQLite and Firebase!", Toast.LENGTH_SHORT).show();
                })
                .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
                .show();
            return true;
        }
        return false;
    });
    popup.show();
}
```

This code demonstrates the showPopupMenu method, which displays a popup menu allowing users to update or delete a class instance. When deleting, it prompts for confirmation and removes the instance from both SQLite and Firebase if confirmed, ensuring data consistency across storage locations.

```
@Override
public Filter getFilter() {
    if (classInstanceFilter == null) {
        classInstanceFilter = new ClassInstanceFilter();
    }
    return classInstanceFilter;
}
```

This code illustrates the getFilter method, which initializes the filtering process in ClassInstanceAdapter. If no filter instance exists, it creates a new ClassInstanceFilter object, enabling search functionality to filter class instances based on specific criteria.

```
private class ClassInstanceFilter extends Filter {
    @Override
    protected FilterResults performFiltering(CharSequence constraint) {
        FilterResults results = new FilterResults();
        if (constraint == null || constraint.length() == 0) {
            results.values = classInstanceList;
            results.count = classInstanceList.size();
        } else {
            String filterString = constraint.toString().toLowerCase();
            List<ClassInstance> filteredList = new ArrayList<>();

            for (ClassInstance classInstance : classInstanceList) {
                YogaClass yogaClass = dbHelper.getYogaClassById(classInstance.getClassId());
                User instructor = dbHelper.getUserById(classInstance.getInstructor());

                if (classInstance.getDate().toLowerCase().contains(filterString)
                    || classInstance.getAdditionalComments().toLowerCase().contains(filterString)
                    || classInstance.getStatus().toLowerCase().contains(filterString)
                    || (yogaClass != null && (yogaClass.getTypeOfClass().toLowerCase().contains(filterString)
                    || yogaClass.getDayOfWeek().toLowerCase().contains(filterString)
                    || yogaClass.getTime().toLowerCase().contains(filterString)))
                    || (instructor != null && instructor.getFullName().toLowerCase().contains(filterString))) {
                    filteredList.add(classInstance);
                }
            }
            results.values = filteredList;
        }
    }
}
```

```

        results.count = filteredList.size();
    }
    return results;
}
@Override
protected void publishResults(CharSequence constraint, FilterResults results) {
    filteredClassInstanceList = (List<ClassInstance>) results.values;
    notifyDataSetChanged();
}
}

```

This code shows the `ClassInstanceFilter` class, responsible for filtering class instances based on the user's search input. It matches various fields such as date, comments, status, and instructor name, allowing dynamic display adjustments to only show relevant instances in the list view.

5.1.26 EnterClassInstanceDetailsActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_enter_class_instance_details);

    tvHeading = findViewById(R.id.tvHeading);
    spYogaClass = findViewById(R.id.spYogaClass);
    btnSelectDate = findViewById(R.id.btnSelectDate);
    spInstructor = findViewById(R.id.spInstructor);
    etAdditionalComments = findViewById(R.id.etAdditionalComments);
    btnSaveClassInstance = findViewById(R.id.btnSaveClassInstance);
    tvWifiStatusInstance = findViewById(R.id.tvWifiStatusInstance);

    db = new DatabaseHelper(this);
    firebaseHelper = new FirebaseHelper();

    loadYogaClassesToSpinner();
    loadInstructorsToSpinner();
    checkWifiStatus();

    networkReceiver = new NetworkChangeReceiver(this);
    IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
    registerReceiver(networkReceiver, filter);

    Intent intent = getIntent();
    instanceId = intent.getStringExtra("instanceId");

    if (instanceId != null) {
        String date = intent.getStringExtra("date");
        String instructor = intent.getStringExtra("instructor");
        String additionalComments = intent.getStringExtra("additionalComments");
        String classId = intent.getStringExtra("classId");

        btnSelectDate.setText(date);
        tvHeading.setText("Update Class Instance");
        etAdditionalComments.setText(additionalComments);

        for (int i = 0; i < spYogaClass.getCount(); i++) {
            ClassSpinnerItem item = (ClassSpinnerItem) spYogaClass.getItemAtPosition(i);
            if (item.getClassId().equals(classId)) {
                spYogaClass.setSelection(i);
                break;
            }
        }

        User user = db.getUserById(instructor);
        String fullName = user.getFullName();
        for (int i = 0; i < spInstructor.getCount(); i++) {
            String item = (String) spInstructor.getItemAtPosition(i);
            if (item.equals(fullName)) {
                spInstructor.setSelection(i);
                break;
            }
        }
        this.selectedDate = date;
        YogaClass yogaClass = db.getYogaClassById(classId);
        this.selectedDayOfWeek = yogaClass.getDayOfWeek();
    }

    btnSelectDate.setOnClickListener(v -> openDatePicker());
    btnSaveClassInstance.setOnClickListener(v -> {
        if (instanceId != null) {
            updateClassInstance();
        } else {
            saveClassInstance();
        }
    })
}

```

```

});
}

```

This code illustrates the `onCreate()` method in `EnterClassInstanceDetailsActivity`, which initializes the UI components and sets up necessary data for the activity. This method begins by binding UI elements like text views, spinners, and buttons. It then initializes instances of `DatabaseHelper` and `FirebaseHelper` to handle database operations. Next, the method loads data into the class and instructor spinners using helper functions. The code also includes a `NetworkChangeReceiver` to monitor connectivity, ensuring the network status is displayed. If `instanceId` is provided, it loads existing data, setting the UI fields accordingly. Lastly, the method assigns click listeners to handle date selection and save operations, differentiating between updating and creating a new instance.

```

private void checkWifiStatus() {
    ConnectivityManager cm = (ConnectivityManager) getSystemService(Context.CONNECTIVITY_SERVICE);
    NetworkInfo activeNetwork = cm.getActiveNetworkInfo();

    if (activeNetwork != null && activeNetwork.getType() == ConnectivityManager.TYPE_WIFI) {
        tvWifiStatusInstance.setText("WiFi: Connected");
    } else {
        tvWifiStatusInstance.setText("WiFi: Not Connected");
    }
}

```

This code shows the `checkWifiStatus()` method, which checks the current network connectivity status. If WiFi is connected, the `twWifiStatusInstance` `TextView` displays "WiFi: Connected." Otherwise, it displays "WiFi: Not Connected." This helps inform the user about the connectivity status directly within the application interface.

```

private void loadInstructorsToSpinner() {
    ArrayList<String> teacherList = new ArrayList<>();
    Cursor cursor = db.getUsersByRole("Teacher");

    if (cursor != null && cursor.getCount() > 0) {
        while (cursor.moveToNext()) {
            String teacherId = cursor.getString(cursor.getColumnIndex("id"));
            String fullName = cursor.getString(cursor.getColumnIndex("fullName"));

            teacherList.add(fullName);
            teacherIdMap.put(fullName, teacherId);
        }
        cursor.close();
    } else {
        Toast.makeText(this, "No teachers found in SQLite", Toast.LENGTH_SHORT).show();
    }
}

ArrayAdapter<String> adapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, teacherList);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
splInstructor.setAdapter(adapter);
}

```

This code shows the `loadInstructorsToSpinner()` method, which loads the list of instructors into a spinner (dropdown) from SQLite database. It queries users with the role "Teacher," then populates the spinner with their names. If no instructors are found, a toast message "No teachers found in SQLite" is displayed. This enables users to select an instructor when managing class instances.

```

private void loadYogaClassesToSpinner() {
    ArrayList<ClassSpinnerItem> yogaClassList = new ArrayList<>();
    HashMap<String, String> classDayOfWeekMap = new HashMap<>();
    Cursor cursor = db.getAllYogaClasses();

    if (cursor != null && cursor.getCount() > 0) {
        while (cursor.moveToNext()) {
            String classId = cursor.getString(cursor.getColumnIndex("id"));
            String typeOfClass = cursor.getString(cursor.getColumnIndex("typeOfClass"));
            String dayOfWeek = cursor.getString(cursor.getColumnIndex("dayOfWeek"));
            String time = cursor.getString(cursor.getColumnIndex("time"));
            int duration = cursor.getInt(cursor.getColumnIndex("duration"));

            String classInfo = typeOfClass + " - " + dayOfWeek + " - (" + time + " - " + getEndTime(time, duration) + ")";
            yogaClassList.add(new ClassSpinnerItem(classId, classInfo));
        }
    }
}

```

```

        classDayOfWeekMap.put(classId, dayOfWeek);
    }
} else {
    Toast.makeText(this, "No yoga classes found in SQLite", Toast.LENGTH_SHORT).show();
}

ArrayAdapter<ClassSpinnerItem> adapter = new ArrayAdapter<>(EnterClassInstanceDetailsActivity.this,
    android.R.layout.simple_spinner_item, yogaClassList);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spYogaClass.setAdapter(adapter);

if (cursor != null) {
    cursor.close();
}

spYogaClass.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parentView, View selectedItemView, int position, long id) {
        ClassSpinnerItem selectedClassItem = (ClassSpinnerItem) spYogaClass.getSelectedItem();
        String selectedClassId = selectedClassItem.getClassId();

        if (!selectedClassId.equals(currentSelectedClassId)) {
            selectedDayOfWeek = classDayOfWeekMap.get(selectedClassId);
            btnSelectDate.setText("Select Date");
            selectedDate = null;
            currentSelectedClassId = selectedClassId;
        }
    }
    @Override
    public void onNothingSelected(AdapterView<?> parentView) {
        selectedDayOfWeek = null;
    }
});

Intent intent = getIntent();

instanceId = intent.getStringExtra("instanceId");
if (instanceId != null) {
    String date = intent.getStringExtra("date");
    String classId = intent.getStringExtra("classId");

    btnSelectDate.setText(date);
    selectedDate = date;
    currentSelectedClassId = classId;
}
}

```

This code shows yoga classes from the SQLite database into a dropdown menu (spinner). Figure retrieves each class's details and formats them for display. If no classes are found, a toast message—"No yoga classes found in SQLite"—informs the user, helping them understand the database status. This functionality allows users to select a class from available options, enhancing the interactivity of the form.

```

private String getEndTime(String startTime, int duration) {
    String[] timeParts = startTime.split(":");
    int hour = Integer.parseInt(timeParts[0]);
    int minute = Integer.parseInt(timeParts[1]);
    int totalMinutes = (hour * 60) + minute + duration;
    int endHour = totalMinutes / 60;
    int endMinute = totalMinutes % 60;

    return String.format("%02d:%02d", endHour % 24, endMinute);
}

```

In this code, the `getEndTime()` function calculates the end time of a class based on its start time and duration. The function splits the `startTime` into hours and minutes, converts them to integers, and then calculates the total end time by adding the duration. The resulting hours and minutes are formatted and returned as a string in "HH" format. This helps display the end time of each class instance accurately based on its duration, improving the user's scheduling and time management.

```

private void openDatePicker() {
    Calendar calendar = Calendar.getInstance();
    int year = calendar.get(Calendar.YEAR);
    int month = calendar.get(Calendar.MONTH);
    int day = calendar.get(Calendar.DAY_OF_MONTH);

    DatePickerDialog datePickerDialog = new DatePickerDialog(this, (view, year1, month1, dayOfMonth) -> {
        Calendar selectedDate = Calendar.getInstance();
        selectedDate.set(year1, month1, dayOfMonth);
        String dayOfWeek = getDayOfWeek(selectedDate.get(Calendar.DAY_OF_WEEK));
    }, year, month, day);
}

```

```

selectedCalendar.set(year1, month1, dayOfMonth);

if (dayOfWeek.equals(selectedDayOfWeek) && selectedDate.after(Calendar.getInstance())) {
    selectedDate.set(year1, month1, dayOfMonth);
    btnSelectDate.setText(dayOfMonth + "/" + (month1 + 1) + "/" + year1);
    this.selectedDate = dayOfMonth + "/" + (month1 + 1) + "/" + year1;
} else {
    Toast.makeText(this, "Please select a valid " + selectedDayOfWeek + " that is not in the past.", Toast.LENGTH_SHORT).show();
}

}, year, month, day);

datePickerDialog.show();
}

```

In this code, the `openDatePicker()` function displays a date picker dialog to allow the user to select a date. The selected date must match the specified `selectedDayOfWeek` and cannot be a past date. If the selected date meets these criteria, it is set as the chosen date and displayed on the button. Otherwise, a Toast message informs the user to "Please select a valid [day] that is not in the past." This ensures that users select an appropriate date for scheduling class instances.

```

private String getDayOfWeek(int dayOfWeek) {
    switch (dayOfWeek) {
        case Calendar.SUNDAY:
            return "Sunday";
        case Calendar.MONDAY:
            return "Monday";
        case Calendar.TUESDAY:
            return "Tuesday";
        case Calendar.WEDNESDAY:
            return "Wednesday";
        case Calendar.THURSDAY:
            return "Thursday";
        case Calendar.FRIDAY:
            return "Friday";
        case Calendar.SATURDAY:
            return "Saturday";
        default:
            return "";
    }
}

```

In this code, the `getDayOfWeek()` function converts an integer `dayOfWeek` value (from Calendar constants) to its corresponding day name in English. This method uses a switch statement to match each integer value with a specific day, returning the day as a string (e.g., "Sunday" for `Calendar.SUNDAY`). If no match is found, it defaults to an empty string. This function aids in displaying the day of the week as text rather than as an integer value.

```

private boolean isInstructorAvailable(String instructorId, String date, String newStartTime, String newEndTime) {
    List<ClassInstance> classInstances = db.getClassInstancesByInstructorId(instructorId);

    for (ClassInstance instance : classInstances) {
        YogaClass yogaClass = db.getYogaClassById(instance.getClassId());
        if (yogaClass == null) continue;

        String instanceStartTime = yogaClass.getTime();
        String instanceEndTime = getEndTime(instanceStartTime, yogaClass.getDuration());

        if (date.equals(instance.getDate()) && isTimeOverlap(newStartTime, newEndTime, instanceStartTime, instanceEndTime)) {
            return false;
        }
    }
    return true;
}

private boolean isTimeOverlap(String start1, String end1, String start2, String end2) {
    return (start1.compareTo(end2) < 0 && end1.compareTo(start2) > 0);
}

```

In Figure, the `isInstructorAvailable()` function checks if a specified instructor is available at a given date and time. It retrieves all class instances associated with the instructor using `instructorId` and iterates through them. For each class instance, it calculates the start and end times and compares them with the new time

range (newStartTime to newEndTime). The function uses isTimeOverlap() to verify if the time ranges overlap. If an overlap is detected on the same date, it returns false to indicate the instructor's unavailability; otherwise, it returns true.

```
private void saveClassInstance() {
    String id = UUID.randomUUID().toString();
    ClassSpinnerItem selectedClassItem = (ClassSpinnerItem) spYogaClass.getSelectedItem();
    String classId = selectedClassItem.getClassId();
    String instructorName = spInstructor.getSelectedItem().toString();
    String instructorId = teacherIdMap.get(instructorName);
    String additionalComments = etAdditionalComments.getText().toString();

    YogaClass yogaClass = db.getYogaClassById(classId);
    String startTime = yogaClass.getTime();
    String endTime = getEndTime(startTime, yogaClass.getDuration());

    if (selectedDate == null || instructorId.isEmpty()) {
        Toast.makeText(this, "Date and Instructor are required!", Toast.LENGTH_SHORT).show();
        return;
    }

    if (!isInstructorAvailable(instructorId, selectedDate, startTime, endTime)) {
        Toast.makeText(this, "Instructor is not available for this time slot.", Toast.LENGTH_SHORT).show();
        return;
    }

    ClassInstance classInstance = new ClassInstance(id, classId, selectedDate, instructorId, additionalComments, "Pending");
    boolean result = db.addClassInstance(classInstance);

    if (result) {
        Toast.makeText(this, "Class instance saved to SQLite", Toast.LENGTH_SHORT).show();
        Intent intent = new Intent(EnterClassInstanceDetailsActivity.this, ClassInstanceListActivity.class);
        startActivity(intent);
        finish();
    } else {
        Toast.makeText(this, "Failed to save class instance to SQLite", Toast.LENGTH_SHORT).show();
    }
}
```

In this code, the saveClassInstance() function saves a new class instance to the SQLite database. It retrieves the selected yoga class and instructor, along with the specified date and time. The function first checks if both date and instructor fields are filled. If either is empty, it displays a toast message, "Date and Instructor are required!". It then verifies instructor availability for the selected time using isInstructorAvailable(). If unavailable, a message, "Instructor is not available for this time slot," is shown. Upon successful validation, the instance is saved to the database, and a toast confirms "Class instance saved to SQLite."

```
private void updateClassInstance() {
    ClassSpinnerItem selectedClassItem = (ClassSpinnerItem) spYogaClass.getSelectedItem();

    String newClassId = selectedClassItem.getClassId();
    String instructorName = spInstructor.getSelectedItem().toString();
    String instructorId = teacherIdMap.get(instructorName);

    String additionalComments = etAdditionalComments.getText().toString();

    YogaClass yogaClass = db.getYogaClassById(newClassId);
    String startTime = yogaClass.getTime();
    String endTime = getEndTime(startTime, yogaClass.getDuration());

    if (selectedDate == null || instructorId.isEmpty()) {
        Toast.makeText(this, "Date and Instructor are required!", Toast.LENGTH_SHORT).show();
        return;
    }

    if (!isInstructorAvailable(instructorId, selectedDate, startTime, endTime)) {
        Toast.makeText(this, "Instructor is not available for this time slot.", Toast.LENGTH_SHORT).show();
        return;
    }

    ClassInstance classInstance = new ClassInstance(instanceId, newClassId, selectedDate, instructorId, additionalComments, "Pending");
    boolean result = db.updateClassInstance(classInstance);
    if (result) {
        Toast.makeText(this, "Class instance updated in SQLite", Toast.LENGTH_SHORT).show();

        Intent intent = new Intent(EnterClassInstanceDetailsActivity.this, ClassInstanceListActivity.class);
        startActivity(intent);
    }
}
```

```

        finish();
    } else {
        Toast.makeText(this, "Failed to update class instance in SQLite", Toast.LENGTH_SHORT).show();
    }
}

```

In this code, the `updateClassInstance()` function updates an existing class instance in the SQLite database. It retrieves the selected class, instructor, and additional details. The function checks if both the date and instructor fields are filled, displaying "Date and Instructor are required!" if either is empty. It then verifies the instructor's availability for the selected time slot using `isInstructorAvailable()`. If the instructor is not available, a message "Instructor is not available for this time slot." appears. Upon successful validation, the function updates the instance in the database, confirming with "Class instance updated in SQLite."

```

@Override
protected void onStart() {
    super.onStart();
    registerReceiver(networkReceiver, new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION));
}

@Override
protected void onStop() {
    super.onStop();
    unregisterReceiver(networkReceiver);
}

```

This code shows the `onStart()` and `onStop()` methods, where network connectivity is managed. In `onStart()`, `registerReceiver()` is called to register `networkReceiver` with a filter for connectivity changes, allowing the app to monitor network status when the activity is visible. In `onStop()`, `unregisterReceiver()` is used to remove `networkReceiver`, stopping network monitoring when the activity is no longer visible. This ensures efficient use of system resources by only monitoring connectivity when necessary.

5.1.27 AccountListActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_account_list);

    listView = findViewById(R.id.listView);
    searchView = findViewById(R.id.searchView);
    spinnerFilter = findViewById(R.id.spinnerFilter);
    btnAddAccount = findViewById(R.id.btnAddAccount);

    databaseHelper = new DatabaseHelper(this);
    userList = new ArrayList<>();

    DataSyncHelper dataSyncHelper = new DataSyncHelper(this, databaseHelper);
    dataSyncHelper.syncDataFromFirebaseIfNeeded();
    loadUsersFromDatabase();
    loadRolesToSpinner();

    accountAdapter = new AccountAdapter(this, userList, databaseHelper);
    listView.setAdapter(accountAdapter);

    btnAddAccount.setOnClickListener(v -> {
        Intent intent = new Intent(AccountListActivity.this, AddAccountActivity.class);
        startActivity(intent);
    });

    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        @Override
        public boolean onQueryTextSubmit(String query) {
            return false;
        }

        @Override
        public boolean onQueryTextChange(String newText) {
            accountAdapter.getFilter().filter(newText);
            return true;
        }
    });

    spinnerFilter.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int position, long id) {

```

```

        String selectedRole = spinnerFilter.getSelectedItem().toString();
        accountAdapter.filterByRole(selectedRole);
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) {}
});

BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
bottomNavigationView.setSelectedItemId(R.id.navigation_account);

bottomNavigationView.setOnNavigationItemSelectedListener(item -> {
    int id = item.getItemId();

    if (id == R.id.navigation_home) {
        Intent intent = new Intent(AccountListActivity.this, AdminPageActivity.class);
        startActivity(intent);
        return true;
    } else if (id == R.id.navigation_class) {
        Intent intent = new Intent(AccountListActivity.this, ClassListActivity.class);
        startActivity(intent);
        return true;
    } else if (id == R.id.navigation_classinstance) {
        Intent intent = new Intent(AccountListActivity.this, ClassInstanceListActivity.class);
        startActivity(intent);
        return true;
    } else if (id == R.id.navigation_booking) {
        Intent intent = new Intent(AccountListActivity.this, BookingListActivity.class);
        startActivity(intent);
        return true;
    } else if (id == R.id.navigation_account) {
        return true;
    }
    return false;
});
}

```

This code shows the `onCreate()` method in `AccountListActivity`, where essential UI components are initialized, including `listView`, `searchView`, and `spinnerFilter`. `Figure` is used to set up data synchronization with `Firebase` and to load user data and roles for display. The `SearchView` provides a filter function, allowing users to search through accounts in real-time, and `spinnerFilter` helps filter accounts by specific roles. Additionally, a bottom navigation menu is configured for easy access to different sections of the application.

```

private void loadUsersFromDatabase() {
    Cursor cursor = databaseHelper.getAllUsers();

    if (cursor == null || cursor.getCount() == 0) {
        Toast.makeText(this, "No users found", Toast.LENGTH_SHORT).show();
        return;
    }

    while (cursor.moveToNext()) {
        User user = new User(
            cursor.getString(cursor.getColumnIndex("id")),
            cursor.getString(cursor.getColumnIndex("email")),
            cursor.getString(cursor.getColumnIndex("fullName")),
            cursor.getString(cursor.getColumnIndex("phoneNumber")),
            cursor.getString(cursor.getColumnIndex("password")),
            cursor.getString(cursor.getColumnIndex("status")),
            cursor.getString(cursor.getColumnIndex("role"))
        );
        userList.add(user);
    }
    cursor.close();
}

```

This code shows the `loadUsersFromDatabase()` method in `AccountListActivity`. This method retrieves user data from the `SQLite` database using a cursor. If no users are found, a toast notification with the message "No users found" is displayed. The method then iterates through each record, extracting user details such as `id`, `email`, `fullName`, `phoneNumber`, `password`, `status`, and `role`, and adds each user to the `userList`. `Figure` is essential for loading and managing user data within the activity.

```

private void loadRolesToSpinner() {
    Set<String> roleSet = new HashSet<>();
    for (User user : userList) {

```

```

        roleSet.add(user.getRole());
    }
    ArrayList<String> roles = new ArrayList<>(roleSet);
    roles.add(0, "All");
    ArrayAdapter<String> adapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, roles);
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinnerFilter.setAdapter(adapter);
}

```

This code displays the `loadRolesToSpinner()` method in `AccountListActivity`. This method collects unique user roles from `userList` and stores them in a `HashSet` to ensure uniqueness. It then converts the set into an `ArrayList` and adds an "All" option at the beginning of the list for filtering purposes. An `ArrayAdapter` is created with this role list and set to display in `spinnerFilter`. This method enables users to filter accounts by role in the activity.

5.1.28 AccountAdapter.java

```

public AccountAdapter(@NonNull Context context, List<User> userList, DatabaseHelper dbHelper) {
    super(context, 0, userList);
    this.context = context;
    this.originalUserList = new ArrayList<>(userList);
    this.filteredUserList = new ArrayList<>(userList);
    this.dbHelper = dbHelper;
    this.firebaseHelper = new FirebaseHelper();
}

@NonNull
@Override
public View getView(int position, View convertView, @NonNull ViewGroup parent) {
    User user = filteredUserList.get(position);

    if (convertView == null) {
        convertView = LayoutInflater.from(context).inflate(R.layout.item_account, parent, false);
    }

    TextView tvFullName = convertView.findViewById(R.id.tvFullName);
    TextView tvEmail = convertView.findViewById(R.id.tvEmail);
    TextView tvPhone = convertView.findViewById(R.id.tvPhone);
    TextView tvStatus = convertView.findViewById(R.id.tvStatus);
    ImageButton btnMoreOptions = convertView.findViewById(R.id.btnMoreOptions);
    TextView tvRole = convertView.findViewById(R.id.tvRole);

    if (user != null) {
        tvFullName.setText("👤 " + user.getFullName());
        tvEmail.setText("✉️ " + user.getEmail());
        tvPhone.setText("📞 " + user.getPhoneNumber());
        tvStatus.setText("🕒 " + user.getStatus());
        tvRole.setText("👤 " + user.getRole());
        btnMoreOptions.setOnClickListener(v -> showPopupMenu(v, user));
    }

    return convertView;
}

```

This code shows the `getView()` method in the `AccountAdapter` class. This method is responsible for rendering each user item in the list by inflating the `item_account` layout if the `convertView` is null. It retrieves user information like full name, email, phone number, status, and role from the `filteredUserList` and displays it in corresponding `TextView` elements. Additionally, it assigns an `OnClickListener` to the `btnMoreOptions` button to handle more actions for each user. This method ensures that each user's data is correctly displayed in the list view.

```

@Override
public int getCount() { return filteredUserList.size(); }
@Override
public User getItem(int position) { return filteredUserList.get(position); }
@Override
public long getItemId(int position) {
    return position;
}

@Override
public Filter getFilter() {
    return new Filter() {
        @Override

```

```

protected FilterResults performFiltering(CharSequence constraint) {
    List<User> filteredResults = new ArrayList<>();
    String query = constraint.toString().toLowerCase().trim();

    if (query.isEmpty()) {
        filteredResults.addAll(originalUserList);
    } else {
        for (User user : originalUserList) {
            if (user.getFullName().toLowerCase().contains(query) ||
                user.getEmail().toLowerCase().contains(query) ||
                user.getPhoneNumber().contains(query) ||
                user.getRole().toLowerCase().contains(query)) {
                filteredResults.add(user);
            }
        }
    }
    FilterResults results = new FilterResults();
    results.values = filteredResults;
    return results;
}

@Override
protected void publishResults(CharSequence constraint, FilterResults results) {
    filteredUserList.clear();
    filteredUserList.addAll((List<User>) results.values);
    notifyDataSetChanged();
}
};
}

```

This code displays the `getFilter()` method in the `AccountAdapter` class, which allows for filtering users based on a query. The `performFiltering()` method processes the query to filter the `originalUserList`, checking if each user's full name, email, phone number, or role contains the query string. If the query is empty, it displays all users. The `publishResults()` method updates `filteredUserList` with the filtered results and refreshes the view to show the changes. This filtering mechanism helps users quickly find specific accounts based on input criteria.

```

public void filterByRole(String role) {
    filteredUserList.clear();
    if (role.equals("All")) {
        filteredUserList.addAll(originalUserList);
    } else {
        for (User user : originalUserList) {
            if (user.getRole().equals(role)) {
                filteredUserList.add(user);
            }
        }
    }
    notifyDataSetChanged();
}
}

```

This code demonstrates the `filterByRole()` method within the `AccountAdapter` class. This method filters the `originalUserList` based on the specified role parameter. If the role is "All," it includes all users in `filteredUserList`; otherwise, it adds only the users whose role matches the given role parameter. After filtering, it calls `notifyDataSetChanged()` to refresh the displayed list, allowing users to view accounts according to selected roles easily.

```

private void showPopupMenu(View view, User user) {
    PopupMenu popup = new PopupMenu(context, view);
    popup.getMenuInflater().inflate(R.menu.class_item_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        int itemId = item.getItemId();
        if (itemId == R.id.menu_update) {
            Intent intent = new Intent(context, AddAccountActivity.class);
            intent.putExtra("userId", user.getId());
            intent.putExtra("fullName", user.getFullName());
            intent.putExtra("email", user.getEmail());
            intent.putExtra("phoneNumber", user.getPhoneNumber());
            intent.putExtra("password", user.getPassword());
            intent.putExtra("isUpdate", true);
            context.startActivity(intent);
            return true;
        } else if (itemId == R.id.menu_delete) {
            new AlertDialog.Builder(context)
                .setTitle("Confirm Delete")

```

```

        .setMessage("Are you sure you want to delete this account?")
        .setPositiveButton("Yes", (dialog, which) -> {
            dbHelper.deleteUser(user.getId());
            firebaseHelper.deleteUserFromFirebase(user.getId());

            originalUserList.remove(user);
            filteredUserList.remove(user);
            notifyDataSetChanged();

            Toast.makeText(context, "Account deleted successfully from SQLite and Firebase!", Toast.LENGTH_SHORT).show();
        })
        .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
        .show();
    return true;
}
return false;
});
popup.show();
}

```

This code illustrates the `showPopupMenu()` method within the `AccountAdapter` class. This method generates a popup menu with options to update or delete a user account. If "update" is selected, an intent is created to open `AddAccountActivity` with user details passed as extras, allowing modifications. If "delete" is chosen, an alert dialog prompts for confirmation. Upon confirmation, the account is deleted from both SQLite and Firebase, removed from local lists, and a toast message confirms successful deletion.

5.1.29 AddAccountActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_add_account);

    tvHeading = findViewById(R.id.tvHeading);
    etFullName = findViewById(R.id.etFullName);
    etEmail = findViewById(R.id.etEmail);
    etPhoneNumber = findViewById(R.id.etPhoneNumber);
    spRole = findViewById(R.id.spRole);
    etPassword = findViewById(R.id.etPassword);
    etConfirmPassword = findViewById(R.id.etConfirmPassword);
    btnAddAccount = findViewById(R.id.btnAddAccount);
    tvWifiStatusAccount = findViewById(R.id.tvWifiStatusAccount);

    ArrayAdapter<CharSequence> roleAdapter = ArrayAdapter.createFromResource(this,
        R.array.roles_array, android.R.layout.simple_spinner_item);
    roleAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spRole.setAdapter(roleAdapter);

    databaseHelper = new DatabaseHelper(this);
    firebaseHelper = new FirebaseHelper();

    checkWifiStatus();

    networkReceiver = new NetworkChangeReceiver(this);
    IntentFilter filter = new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION);
    registerReceiver(networkReceiver, filter);

    Intent intent = getIntent();
    userId = intent.getStringExtra("userId");
    if (userId != null) {
        loadUserForUpdate(userId);
        tvHeading.setText("✎ Update Account");
        btnAddAccount.setText("Update Account");
        etPassword.setVisibility(View.GONE);
        etConfirmPassword.setVisibility(View.GONE);
    }

    btnAddAccount.setOnClickListener(v -> {
        String fullName = etFullName.getText().toString();
        String email = etEmail.getText().toString();
        String phoneNumber = etPhoneNumber.getText().toString();
        String password = etPassword.getText().toString().trim();
        String confirmPassword = etConfirmPassword.getText().toString();
        String role = spRole.getSelectedItem().toString();

        if (TextUtils.isEmpty(fullName) || TextUtils.isEmpty(email) || TextUtils.isEmpty(phoneNumber)) {
            Toast.makeText(AddAccountActivity.this, "Please fill in all fields", Toast.LENGTH_SHORT).show();
            return;
        }

        if (userId != null) {
            updateAccount(fullName, email, phoneNumber, role);
        }
    });
}

```

```

    } else {
        if (databaseHelper.isEmailExists(email)) {
            Toast.makeText(AddAccountActivity.this, "Email already exists!", Toast.LENGTH_SHORT).show();
            return;
        }
        if (password.length() < 8) {
            Toast.makeText(AddAccountActivity.this, "Password must be at least 8 characters", Toast.LENGTH_SHORT).show();
            return;
        }
        if (TextUtils.isEmpty(password)){
            Toast.makeText(AddAccountActivity.this, "Please enter the password", Toast.LENGTH_SHORT).show();
            return;
        }
        if (!password.equals(confirmPassword)) {
            Toast.makeText(AddAccountActivity.this, "Passwords do not match", Toast.LENGTH_SHORT).show();
            return;
        }

        String salt = generateSalt();
        String saltedHashedPassword = databaseHelper.hashPasswordWithSalt(password, salt);
        User newUser = new User(UUID.randomUUID().toString(), email, fullName, phoneNumber, saltedHashedPassword, "Pending", role);

        boolean result = databaseHelper.addUser(newUser);

        if (result) {
            Toast.makeText(AddAccountActivity.this, "Account added successfully", Toast.LENGTH_SHORT).show();
            Intent intentToAccountList = new Intent(AddAccountActivity.this, AccountListActivity.class);
            startActivity(intentToAccountList);
            finish();
        } else {
            Toast.makeText(AddAccountActivity.this, "Failed to add account", Toast.LENGTH_SHORT).show();
        }
    }
});
}

```

This code illustrates the onCreate method within the AddAccountActivity class. Figure initializes the UI elements used for account creation or updating, such as input fields for full name, email, phone number, password, and role selection. The method also checks for an existing user ID to determine if the user is editing an existing account or creating a new one. Toast notifications provide user feedback on form validation, such as filling required fields or password requirements, ensuring a smooth user experience. If the operation succeeds, the user is redirected back to the AccountListActivity with a success message.

```

private void loadUserForUpdate(String userId) {
    User user = databaseHelper.getUserById(userId);
    if (user != null) {
        etFullName.setText(user.getFullName());
        etEmail.setText(user.getEmail());
        etPhoneNumber.setText(user.getPhoneNumber());
        originalEmail = user.getEmail();

        if ("Teacher".equals(user.getRole()) && hasClassInstances(userId)) {
            spRole.setEnabled(false);
            Toast.makeText(this, "This account is assigned to a class instance, role cannot be changed.", Toast.LENGTH_SHORT).show();
        }
    }
}

```

This code shows the loadUserForUpdate method in the AccountAdapter. This method populates the account update form with the user's information, such as full name, email, and phone number, based on the provided user ID. Additionally, if the user has a role of "Teacher" and is assigned to any class instances, the role selection spinner is disabled, preventing changes to the user's role. A Toast notification informs the user that the role cannot be modified for this account due to its association with class instances.

```

private boolean hasClassInstances(String userId) {
    Cursor cursor = databaseHelper.getClassInstancesByUserId(userId);
    boolean hasInstances = cursor != null && cursor.getCount() > 0;
    if (cursor != null) {
        cursor.close();
    }
    return hasInstances;
}

```

This code illustrates the `hasClassInstances` method within the `AccountAdapter`. This method checks if a user, identified by the given user ID, is associated with any class instances in the database. It retrieves a cursor pointing to the class instances linked to the user. If the cursor is non-null and contains records, the method returns true, indicating that the user has assigned class instances. Otherwise, it returns false. This method assists in determining whether the user's role can be modified.

```
private void updateAccount(String fullName, String email, String phoneNumber, String role) {
    if (!originalEmail.equals(email)) {
        if (databaseHelper.isEmailExists(email)) {
            Toast.makeText(AddAccountActivity.this, "Email already exists!", Toast.LENGTH_SHORT).show();
            return;
        }
    }

    User updatedUser = new User(userId, email, fullName, phoneNumber, databaseHelper.getUserById(userId).getPassword(), "Pending", role);
    boolean result = databaseHelper.updateUser(updatedUser);

    if (result) {
        Toast.makeText(this, "Account updated successfully", Toast.LENGTH_SHORT).show();
        Intent intentToAccountList = new Intent(AddAccountActivity.this, AccountListActivity.class);
        startActivity(intentToAccountList);
        finish();
    } else {
        Toast.makeText(this, "Failed to update account", Toast.LENGTH_SHORT).show();
    }
}
```

This code shows the `updateAccount` method in the `AccountAdapter`. This method is used to update a user's account details, including full name, email, phone number, and role. First, it checks if the email provided is different from the original email; if so, it validates that the new email does not already exist in the database. Upon successful validation, the method updates the user's information in the database. If the update succeeds, a toast message confirms the success and redirects the user to the `AccountListActivity`; otherwise, an error message is displayed.

```
private void checkWifiStatus() {
    ConnectivityManager cm = (ConnectivityManager) getSystemService(CONNECTIVITY_SERVICE);
    NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
    if (activeNetwork != null && activeNetwork.getType() == ConnectivityManager.TYPE_WIFI) {
        tvWifiStatusAccount.setText("WiFi: Connected");
    } else {
        tvWifiStatusAccount.setText("WiFi: Not Connected");
    }
}
```

This code shows the `checkWifiStatus` method in the `AccountAdapter`. This method is used to verify the Wi-Fi connection status. It accesses the system's connectivity service to check if the current active network is of type Wi-Fi. If the device is connected to Wi-Fi, it sets the `twWifiStatusAccount` `TextView` to display "WiFi: Connected"; otherwise, it displays "WiFi: Not Connected." This feedback helps users understand their connectivity status directly within the app.

```
private String generateSalt() {
    SecureRandom sr = new SecureRandom();
    byte[] salt = new byte[16];
    sr.nextBytes(salt);
    return Base64.getEncoder().encodeToString(salt);
}

@Override
protected void onStart() {
    super.onStart();
    registerReceiver(networkReceiver, new IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION));
}

@Override
protected void onStop() {
    super.onStop();
    unregisterReceiver(networkReceiver);
}
```

This code shows the `generateSalt`, `onStart`, and `onStop` methods in `AccountAdapter`. The `generateSalt` method creates a random salt using `SecureRandom` for password hashing, enhancing security by encoding the salt in Base64 format. The `onStart` and `onStop` methods manage the `networkReceiver`, registering it to monitor connectivity changes when the activity starts and unregistering it upon stopping, ensuring efficient resource management.

5.1.30 BookingListActivity.java

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_booking_list);

    listView = findViewById(R.id.listView);
    searchView = findViewById(R.id.searchView);
    spinnerFilter = findViewById(R.id.spinnerFilter);
    bookingList = new ArrayList<>();
    databaseHelper = new DatabaseHelper(this);
    DataSyncHelper dataSyncHelper = new DataSyncHelper(this, databaseHelper);
    dataSyncHelper.syncDataFromFirebaseIfNeeded();
    loadBookingsFromDatabase();

    bottomNavigationView = findViewById(R.id.bottom_navigation);
    bottomNavigationView.setSelectedItemId(R.id.navigation_booking);
    bottomNavigationView.setOnNavigationItemSelectedListener(new BottomNavigationView.OnNavigationItemSelectedListener() {
        @Override
        public boolean onNavigationItemSelected(@NonNull MenuItem item) {
            int itemId = item.getItemId();
            if (itemId == R.id.navigation_home) {
                startActivity(new Intent(getApplicationContext(), AdminPageActivity.class));
                overridePendingTransition(0, 0);
                return true;
            } else if (itemId == R.id.navigation_class) {
                startActivity(new Intent(getApplicationContext(), ClassListActivity.class));
                overridePendingTransition(0, 0);
                return true;
            } else if (itemId == R.id.navigation_classinstance) {
                startActivity(new Intent(getApplicationContext(), ClassInstanceListActivity.class));
                overridePendingTransition(0, 0);
                return true;
            } else if (itemId == R.id.navigation_booking) {
                return true;
            } else if (itemId == R.id.navigation_account) {
                startActivity(new Intent(getApplicationContext(), AccountListActivity.class));
                overridePendingTransition(0, 0);
                return true;
            }
            return false;
        }
    });

    bookingAdapter = new BookingAdapter(this, bookingList, databaseHelper);
    listView.setAdapter(bookingAdapter);

    ArrayAdapter<CharSequence> spinnerAdapter = ArrayAdapter.createFromResource(this,
        R.array.status_array, android.R.layout.simple_spinner_item);
    spinnerAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    spinnerFilter.setAdapter(spinnerAdapter);

    spinnerFilter.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parentView, View selectedItemView, int position, long id) {
            String selectedStatus = (String) spinnerFilter.getSelectedItem();
            bookingAdapter.filterByStatus(selectedStatus);
        }
        @Override
        public void onNothingSelected(AdapterView<?> parentView) {}
    });

    searchView.setOnQueryTextListener(new SearchView.OnQueryTextListener() {
        @Override
        public boolean onQueryTextSubmit(String query) {
            return false;
        }

        @Override
        public boolean onQueryTextChange(String newText) {
            bookingAdapter.getFilter().filter(newText);
            return true;
        }
    });
}
```

This code shows the `onCreate()` method in `BookingListActivity`, which initializes components for managing booking records. The code configures a `bottomNavigationView` for seamless navigation across different sections of the app. It also sets up `spinnerFilter` and `searchView` listeners for filtering bookings by status and performing text-based searches. The method provides feedback through `Toast` messages, improving user interaction by ensuring that users can easily filter, search, and navigate the booking list.

```
private void loadBookingsFromDatabase() {
    Cursor cursor = databaseHelper.getAllBookings();

    if (cursor == null || cursor.getCount() == 0) {
        Toast.makeText(this, "No bookings found", Toast.LENGTH_SHORT).show();
        return;
    }

    while (cursor.moveToNext()) {
        String id = cursor.getString(cursor.getColumnIndexOrThrow("id"));
        String status = cursor.getString(cursor.getColumnIndexOrThrow("status"));
        boolean canCancel = cursor.getInt(cursor.getColumnIndexOrThrow("canCancel")) == 1;
        int totalPrice = cursor.getInt(cursor.getColumnIndexOrThrow("totalPrice"));
        String userId = cursor.getString(cursor.getColumnIndexOrThrow("userId"));

        String classInstancesJson = cursor.getString(cursor.getColumnIndexOrThrow("classInstances"));
        List<ClassInstanceInfo> classInstanceInfoList = new Gson().fromJson(
            classInstancesJson,
            new TypeToken<List<ClassInstanceInfo>>() {}.getType()
        );

        Booking booking = new Booking(id, status, canCancel, totalPrice, userId, classInstanceInfoList);
        bookingList.add(booking);
    }

    cursor.close();
}
```

This code shows the `loadBookingsFromDatabase` method in `BookingListActivity`. This method retrieves all booking records from the database and populates them into `bookingList`. If no bookings are found, it displays a `Toast` message saying "No bookings found". For each booking, fields such as `id`, `status`, `canCancel`, `totalPrice`, and `userId` are extracted. Additionally, the method parses JSON data for class instances into a list using `Gson`. This approach ensures that all bookings are loaded and ready for further operations within the app.

5.1.31 BookingAdapter.java

```
public BookingAdapter(Context context, List<Booking> bookingList, DatabaseHelper dbHelper) {
    super(context, 0, bookingList);
    this.context = context;
    this.bookingList = new ArrayList<>(bookingList);
    this.filteredBookingList = new ArrayList<>(bookingList);
    this.dbHelper = dbHelper;
}

@Override
public int getCount() {
    return filteredBookingList.size();
}

@Override
public Booking getItem(int position) {
    return filteredBookingList.get(position);
}
```

This code shows the constructor, `getCount`, and `getItem` methods in `BookingAdapter`. The constructor initializes the adapter with a context, booking list, and a filtered list, which is a copy of the original list. The `getCount` and `getItem` methods provide access to the size and individual items of the filtered list, supporting the adapter's functionality in managing booking data.

```
@Override
public View getView(int position, View convertView, ViewGroup parent) {
    Booking booking = getItem(position);
```

```

if (convertView == null) {
    convertView = LayoutInflater.from(context).inflate(R.layout.item_booking, parent, false);
}

TextView tvUserName = convertView.findViewById(R.id.userName);
TextView tvTotalPrice = convertView.findViewById(R.id.totalPrice);
TextView tvStatus = convertView.findViewById(R.id.status);
TextView tvYogaClassCount = convertView.findViewById(R.id.yogaClassCount);
TextView tvClassCount = convertView.findViewById(R.id.classCount);
ImageButton btnMoreOptions = convertView.findViewById(R.id.btnMoreOptionsBooking);

User user = dbHelper.getUserById(booking.getUserId());
if (user != null) {
    tvUserName.setText("👤 " + user.getEmail());
} else {
    tvUserName.setText("👤 Unknown User");
}

tvTotalPrice.setText("💰 Total: $" + booking.getTotalPrice());
tvStatus.setText("📌 Status: " + booking.getStatus());

Set<String> uniqueYogaClassIds = new HashSet<>();
int classInstanceCount = 0;

for (ClassInstanceInfo instance : booking.getClassInstances()) {
    if (instance != null) {
        uniqueYogaClassIds.add(instance.getClassId());
        classInstanceCount++;
    }
}
tvYogaClassCount.setText("Yoga Classes: " + uniqueYogaClassIds.size());
tvClassCount.setText("Class Instances: " + classInstanceCount);
btnMoreOptions.setOnClickListener(v -> showPopupMenu(v, booking));

return convertView;
}

```

This code shows the `getView` method in `BookingAdapter`, which is responsible for setting up each item's view in the list. This method retrieves booking details, including user information, total price, status, and counts of unique yoga classes, displaying them in the respective text views.

```

@Override
public Filter getFilter() {
    if (bookingFilter == null) {
        bookingFilter = new BookingFilter();
    }
    return bookingFilter;
}

```

This code shows the `getFilter` method in `BookingAdapter`. This method provides a filter object that will allow the booking list to be filtered based on specified criteria, enabling users to refine their search results within the adapter.

```

public void filterByStatus(String status) {
    if (status.equals("All")) {
        filteredBookingList = new ArrayList<>(bookingList);
    } else {
        filteredBookingList.clear();
        for (Booking booking : bookingList) {
            if (booking.getStatus().equalsIgnoreCase(status)) {
                filteredBookingList.add(booking);
            }
        }
    }
    notifyDataSetChanged();
}

```

This code shows the `filterByStatus` method in `BookingAdapter`, which filters the booking list by the specified status. This method updates the filtered list to include only bookings matching the given status, refreshing the display based on the selected criteria.

```

private void showPopupMenu(View view, Booking booking) {
    PopupMenu popup = new PopupMenu(context, view);
    popup.getMenuInflater().inflate(R.menu.booking_item_menu, popup.getMenu());

    popup.setOnMenuItemClickListener(item -> {
        if (item.getItemId() == R.id.menu_view_detail) {
            Intent intent = new Intent(context, BookingDetailsActivity.class);
            intent.putExtra("bookingId", booking.getId());
        }
    });
}

```

```

        context.startActivity(intent);
        return true;
    }
    return false;
});

popup.show();
}

```

This code shows the showPopupMenu method in BookingAdapter. This method displays a pop-up menu with actions, such as viewing booking details. It allows the user to select options related to each booking item, enhancing the interaction within the list.

```

private class BookingFilter extends Filter {
    @Override
    protected FilterResults performFiltering(CharSequence constraint) {
        FilterResults results = new FilterResults();
        if (constraint == null || constraint.length() == 0) {
            results.values = bookingList;
            results.count = bookingList.size();
        } else {
            String filterString = constraint.toString().toLowerCase();
            List<Booking> filteredList = new ArrayList<>();

            for (Booking booking : bookingList) {
                User user = dbHelper.getUserById(booking.getUserId());
                if ((user != null && user.getEmail().toLowerCase().contains(filterString)) ||
                    String.valueOf(booking.getTotalPrice()).contains(filterString)) {
                    filteredList.add(booking);
                }
            }

            results.values = filteredList;
            results.count = filteredList.size();
        }
        return results;
    }

    @Override
    protected void publishResults(CharSequence constraint, FilterResults results) {
        filteredBookingList = (List<Booking>) results.values;
        notifyDataSetChanged();
    }
}

```

This code shows the BookingFilter class in BookingAdapter, which implements custom filtering logic for bookings. This class filters bookings based on user-provided constraints, such as user's email or total price, supporting dynamic search capabilities.

5.1.32 BookingDetailsActivity.java

```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_booking_details);

    tvFullName = findViewById(R.id.tvFullName);
    tvTotalPrice = findViewById(R.id.tvTotalPrice);
    tvStatus = findViewById(R.id.tvStatus);
    lvClassInstances = findViewById(R.id.lvClassInstances);
    btnAccept = findViewById(R.id.btnAccept);
    btnReject = findViewById(R.id.btnReject);

    dbHelper = new DatabaseHelper(this);
    firebaseHelper = new FirebaseHelper();

    Intent intent = getIntent();
    String bookingId = intent.getStringExtra("bookingId");

    if (bookingId != null) {
        booking = dbHelper.getBookingById(bookingId);
        if (booking != null) {
            User user = dbHelper.getUserById(booking.getUserId());
            if (user != null) {
                tvFullName.setText("👤 Full Name: " + user.getFullName());
            } else {
                tvFullName.setText("👤 Full Name: Unknown");
            }
        }

        tvTotalPrice.setText("💰 Total Price: $" + booking.getTotalPrice());
        tvStatus.setText("📄 Status: " + booking.getStatus());
    }
}

```

```

loadClassInstances(booking);

if (!"Pending".equals(booking.getStatus())) {
    btnAccept.setVisibility(View.GONE);
    btnReject.setVisibility(View.GONE);
} else {
    setUpButtons();
}
}
}
}
}

```

This code shows the onCreate method in BookingDetailsActivity, which initializes various UI elements and retrieves booking information using bookingId. If a booking is found, it displays details such as the user's full name, total price, and booking status. If the booking status is not "Pending," the accept and reject buttons are hidden, ensuring only pending bookings are actionable.

```

private void setUpButtons() {
    btnAccept.setOnClickListener(v -> {
        if (booking != null) {
            new AlertDialog.Builder(BookingDetailsActivity.this)
                .setTitle("Confirm Booking")
                .setMessage("Are you sure you want to confirm this booking?")
                .setPositiveButton("Yes", (dialog, which) -> {
                    booking.setStatus("Confirmed");
                    booking.setCanCancel(true);
                    dbHelper.updateBooking(booking);
                    Cursor cursor = dbHelper.getAllBookings();
                    if (cursor != null && cursor.moveToFirst()) {
                        do {
                            String bookingId = cursor.getString(cursor.getColumnIndex("id"));
                            String status = cursor.getString(cursor.getColumnIndex("status"));
                            boolean canCancel = cursor.getInt(cursor.getColumnIndex("canCancel")) == 1;
                            firebaseHelper.updateBookingStatusAndCancel(bookingId, status, canCancel);
                        } while (cursor.moveToNext());
                        cursor.close();
                    }
                    tvStatus.setText("Status: Confirmed");
                    btnAccept.setVisibility(View.GONE);
                    btnReject.setVisibility(View.GONE);
                    Toast.makeText(BookingDetailsActivity.this, "Booking Confirmed", Toast.LENGTH_SHORT).show();
                    Intent intent = new Intent(BookingDetailsActivity.this, BookingListActivity.class);
                    startActivity(intent);
                    finish();
                })
                .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
                .show();
        }
    });

    btnReject.setOnClickListener(v -> {
        if (booking != null) {
            new AlertDialog.Builder(BookingDetailsActivity.this)
                .setTitle("Reject Booking")
                .setMessage("Are you sure you want to reject this booking?")
                .setPositiveButton("Yes", (dialog, which) -> {
                    booking.setStatus("Rejected");
                    booking.setCanCancel(false);
                    dbHelper.updateBooking(booking);
                    Cursor cursor = dbHelper.getAllBookings();
                    if (cursor != null && cursor.moveToFirst()) {
                        do {
                            String bookingId = cursor.getString(cursor.getColumnIndex("id"));
                            String status = cursor.getString(cursor.getColumnIndex("status"));
                            boolean canCancel = cursor.getInt(cursor.getColumnIndex("canCancel")) == 1;

                            firebaseHelper.updateBookingStatusAndCancel(bookingId, status, canCancel);
                        } while (cursor.moveToNext());
                        cursor.close();
                    }
                    tvStatus.setText("Status: Rejected");
                    btnAccept.setVisibility(View.GONE);
                    btnReject.setVisibility(View.GONE);
                    Toast.makeText(BookingDetailsActivity.this, "Booking Rejected", Toast.LENGTH_SHORT).show();
                    Intent intent = new Intent(BookingDetailsActivity.this, BookingListActivity.class);
                    startActivity(intent);
                    finish();
                })
                .setNegativeButton("No", (dialog, which) -> dialog.dismiss())
                .show();
        }
    });
}
}
}
}

```

This code illustrates the `setUpButtons` method in `BookingDetailsActivity`, covering both the accept and reject button functionalities. For accepting a booking, it displays a confirmation dialog; if the user confirms, it updates the booking status to "Confirmed," enables cancellation, updates the database, and shows a Toast message, "Booking Confirmed." Conversely, the reject button presents a rejection confirmation dialog. Upon confirmation, it sets the booking status to "Rejected," disables cancellation, updates the database, and displays the Toast message, "Booking Rejected," notifying the user of the completed action.

```
private void loadClassInstances(Booking booking) {
    yogaClassList = new ArrayList<>();
    classInstanceMap = new HashMap<>();
    List<ClassInstanceInfo> classInstanceInfos = booking.getClassInstances();
    for (ClassInstanceInfo classInstanceInfo : classInstanceInfos) {
        String classId = classInstanceInfo.getClassId();
        YogaClass yogaClass = dbHelper.getYogaClassById(classId);

        if (yogaClass != null) {
            if (!classInstanceMap.containsKey(classId)) {
                yogaClassList.add(yogaClass);
                classInstanceMap.put(classId, new ArrayList<>());
            }
            ClassInstance classInstance = dbHelper.getClassInstanceById(classInstanceInfo.getClassInstanceId());
            if (classInstance != null) {
                classInstanceMap.get(classId).add(classInstance);
            }
        }
    }
    YogaClassAdapter adapter = new YogaClassAdapter(this, yogaClassList, classInstanceMap, dbHelper);
    lvClassInstances.setAdapter(adapter);
}
```

This code presents the `loadClassInstances` method in `BookingDetailsActivity`, which retrieves and displays all class instances associated with a booking. It organizes instances by class ID, gathers relevant data from the database, and populates the `lvClassInstances` `ListView`, enabling users to view all sessions within the booking.

5.1.33 YogaClassAdapter.java

```
public YogaClassAdapter(Context context, List<YogaClass> yogaClassList, Map<String, List<ClassInstance>> classInstanceMap, DatabaseHelper dbHelper) {
    this.context = context;
    this.yogaClassList = yogaClassList;
    this.classInstanceMap = classInstanceMap;
    this.dbHelper = dbHelper;
}

@Override
public int getCount() {
    return yogaClassList.size();
}

@Override
public Object getItem(int position) {
    return yogaClassList.get(position);
}

@Override
public long getItemId(int position) {
    return position;
}
```

This code shows the constructor and fundamental methods in `YogaClassAdapter`, setting up the context, lists of yoga classes, and database helper. The methods `getCount`, `getItem`, and `getItemId` provide essential data for item count, retrieval, and identification, ensuring smooth data handling within the adapter.

```
@Override
public View getView(int position, View convertView, ViewGroup parent) {
    YogaClass yogaClass = yogaClassList.get(position);

    if (convertView == null) {
        convertView = LayoutInflater.from(context).inflate(R.layout.item_yoga_class, parent, false);
    }

    TextView tvYogaClassInfo = convertView.findViewById(R.id.tvYogaClassInfo);
    TextView tvClassInstances = convertView.findViewById(R.id.tvClassInstances);
}
```

```

tvYogaClassInfo.setText(yogaClass.getTypeOfClass() + " - " + yogaClass.getDayOfWeek()
    + " (" + yogaClass.getTime() + " - " + getEndTime(yogaClass.getTime(), yogaClass.getDuration()) + ")");

List<ClassInstance> classInstances = classInstanceMap.get(yogaClass.getId());
StringBuilder classInstanceInfo = new StringBuilder();
if (classInstances != null) {
    for (ClassInstance classInstance : classInstances) {
        User instructor = dbHelper.getUserById(classInstance.getInstructor());
        String instructorName = (instructor != null) ? instructor.getFullName() : "Unknown Instructor";
        classInstanceInfo.append("🧘 ").append(classInstance.getDate())
            .append(" - 🕒 Instructor: ").append(instructorName).append("\n");
    }
}
if (classInstanceInfo.length() > 0) {
    tvClassInstances.setText(classInstanceInfo.toString());
} else {
    tvClassInstances.setText("No class instances available");
}
return convertView;
}

```

This code illustrates the `getView` method in `YogaClassAdapter`, responsible for populating the list item view with yoga class information. The method dynamically fills fields such as type, schedule, and instructor name, defaulting to "Unknown Instructor" if not found, and displays a message if no class instances are available.

```

private String getEndTime(String startTime, int duration) {
    String[] timeParts = startTime.split(":");
    int hour = Integer.parseInt(timeParts[0]);
    int minute = Integer.parseInt(timeParts[1]);

    int totalMinutes = (hour * 60) + minute + duration;
    int endHour = totalMinutes / 60;
    int endMinute = totalMinutes % 60;
    return String.format("%02d:%02d", endHour % 24, endMinute);
}

```

This code shows the `getEndTime` method in `YogaClassAdapter`, which calculates the end time of a yoga session based on start time and duration. This method splits the start time into hours and minutes, performs arithmetic calculations, and formats the result into a readable time format.

5.2 Customer Application

5.2.1 Booking.cs

```
namespace yogaapp_xamarin.Models
{
    public class Booking
    {
        public string id { get; set; } = string.Empty;
        public List<ClassInstanceInfo> classInstances { get; set; } = new List<ClassInstanceInfo>();
        public double totalPrice { get; set; }
        public string status { get; set; } = string.Empty;
        public string userId { get; set; } = string.Empty;
        public bool canCancel { get; set; } = false;
    }
    public class ClassInstanceInfo
    {
        public string classId { get; set; }
        public string classInstanceId { get; set; }
    }
}
```

This code shows the `Booking` and `ClassInstanceInfo` classes within `Booking.cs` in the .NET MAUI Xamarin application. The `Booking` class holds properties related to booking details, including `id`, `status`, `totalPrice`, `userId`, and `canCancel`, along with a list of `ClassInstanceInfo` objects representing the specific sessions in the booking. The `ClassInstanceInfo` class contains `classId` and `classInstanceId` properties, which identify the specific yoga class and session instance. This structure helps manage booking and session details systematically.

5.2.2 Cart.cs

```
public static class Cart
{
    private static FirebaseService _firebaseService = new FirebaseService();
    public static ObservableCollection<ClassInstance> ClassInstancesInCart { get; private set; } = new
    ObservableCollection<ClassInstance>();

    public static async Task LoadCartFromFirebase(string userId)
    {
        var classInstanceIds = await _firebaseService.GetCartByUserIdAsync(userId);
        ClassInstancesInCart.Clear();

        foreach (var classInstanceId in classInstanceIds)
        {
            var classInstance = await _firebaseService.GetClassInstanceById(classInstanceId);
            if (classInstance != null)
            {
                ClassInstancesInCart.Add(classInstance);
            }
        }
    }

    public static async Task SaveCartToFirebase(string userId)
    {
        var classInstanceIds = ClassInstancesInCart.Select(ci => ci.Id).ToList();
        await _firebaseService.SaveCartAsync(userId, classInstanceIds);
    }
}
```

This code shows the `LoadCartFromFirebase` and `SaveCartToFirebase` methods in the `Cart` class. The `LoadCartFromFirebase` method asynchronously retrieves all cart items for a specified user from Firebase, clearing the existing `ClassInstancesInCart` collection before adding the fetched items. This ensures that the

cart contents reflect the latest data from Firebase. The `SaveCartToFirebase` method collects the IDs of items currently in `ClassInstancesInCart` and saves them to Firebase, providing data persistence across user sessions. These methods collectively enable reliable synchronization between the app's local cart and the Firebase database.

```
public static void AddToCart(List<ClassInstance> classInstances, string userId)
{
    foreach (var classInstance in classInstances)
    {
        if (!IsInCart(classInstance))
        {
            ClassInstancesInCart.Add(classInstance);
        }
    }

    _ = SaveCartToFirebase(userId);
}

public static double GetTotalPrice(List<YogaClass> yogaClasses)
{
    double totalPrice = 0;

    foreach (var classInstance in ClassInstancesInCart)
    {
        var yogaClass = yogaClasses.FirstOrDefault(yc => yc.Id == classInstance.ClassId);
        if (yogaClass != null)
        {
            totalPrice += yogaClass.Price;
        }
    }

    return totalPrice;
}
```

This code displays the `AddToCart` and `GetTotalPrice` methods in the `Cart.cs` file. The `AddToCart` method adds a list of `ClassInstance` items to the cart if they are not already present and immediately saves the updated cart to Firebase. The `GetTotalPrice` method calculates the total price of all items in the cart by matching each `ClassInstance` with its respective `YogaClass` price.

```
public static bool IsInCart(ClassInstance classInstance)
{
    return ClassInstancesInCart.Any(ci => ci.Id == classInstance.Id);
}

public static void ClearCart(string userId)
{
    ClassInstancesInCart.Clear();
    _ = SaveCartToFirebase(userId);
}

public static void RemoveClassInstance(ClassInstance classInstance, string userId)
{
    ClassInstancesInCart.Remove(classInstance);
    _ = SaveCartToFirebase(userId);
}
```

This code presents utility methods in `Cart.cs` for managing cart contents. The `IsInCart` method checks if a specific `ClassInstance` is already in the cart. The `ClearCart` method empties the cart and saves this state to

Firebase. The `RemoveClassInstance` method removes a specified `ClassInstance` from the cart and then updates Firebase, ensuring the cart remains synchronized.

5.2.3 ClassInstance.cs

```
public class ClassInstance : INotifyPropertyChanged
{
    public string Id { get; set; } = string.Empty;
    public string ClassId { get; set; } = string.Empty;
    public string Date { get; set; } = string.Empty;
    public string Instructor { get; set; } = string.Empty;
    public string AdditionalComments { get; set; } = string.Empty;
    public string Status { get; set; } = string.Empty;

    private bool _isSelectable = true;

    private bool _isSelected;
    public bool IsSelected
    {
        get => _isSelected;
        set
        {
            if (_isSelected != value)
            {
                _isSelected = value;
                OnPropertyChanged(nameof(IsSelected));
            }
        }
    }

    public bool IsSelectable
    {
        get => _isSelectable;
        set
        {
            if (_isSelectable != value)
            {
                _isSelectable = value;
                OnPropertyChanged(nameof(IsSelectable));
            }
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

This code presents the `ClassInstance` class implementing `INotifyPropertyChanged` in `ClassInstance.cs`. This class includes properties like `Id`, `ClassId`, `Date`, `Instructor`, and `Status`, essential for representing individual class sessions. It introduces `IsSelectable` and `IsSelected` with change notifications via `OnPropertyChanged`, allowing UI components to dynamically react to selection state changes, enhancing interactivity within the application.

5.2.4 User.cs

```
namespace yogaapp_xamarin.Models
{
    public class User
    {

```

```

    public string id { get; set; } = string.Empty;
    public string email { get; set; } = string.Empty;
    public string password { get; set; } = string.Empty;
    public string fullName { get; set; } = string.Empty;
    public string phoneNumber { get; set; } = string.Empty;
    public string role { get; set; } = string.Empty;
    public string status { get; set; } = string.Empty;
}
}

```

This code shows the User class in User.cs, which defines properties such as id, email, password, fullName, phoneNumber, role, and status. These properties represent key information about a user, supporting features like authentication, role-based access, and user management within the application.

5.2.5 YogaClass.cs

```

namespace yogaapp_xamarin.Models
{
    public class YogaClass
    {
        public string Id { get; set; }
        public int Capacity { get; set; }
        public string DayOfWeek { get; set; }
        public string Description { get; set; }
        public int Duration { get; set; }
        public double Price { get; set; }
        public string Status { get; set; }
        public string Time { get; set; }
        public string TypeOfClass { get; set; }
    }
}

```

This code shows the YogaClass class in YogaClass.cs, defining properties like Id, Capacity, DayOfWeek, Description, Duration, Price, Status, Time, and TypeOfClass. These properties encapsulate key details for managing yoga classes, such as scheduling, capacity, pricing, and class type, essential for organizing and displaying class information in the app.

5.2.6 FirebaseService.cs

```

private readonly string firebaseDatabaseUrl = "https://yogaapp-42561-default-rtdb.asia-southeast1.firebaseio.com/";

public async Task<List<ClassInstance>> GetClassInstancesByClassIdAsync(string yogaClassId)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/ClassInstances.json");

    var classInstances = JsonConvert.DeserializeObject<Dictionary<string, ClassInstance>>(response);
    return classInstances?.Values.Where(ci => ci.ClassId == yogaClassId).ToList() ?? new List<ClassInstance>();
}

public async Task CreateBooking(Booking booking)
{
    var httpClient = new HttpClient();
    var jsonData = JsonConvert.SerializeObject(booking);
    var content = new StringContent(jsonData, Encoding.UTF8, "application/json");

    await httpClient.PutAsync($"{firebaseDatabaseUrl}/Bookings/{booking.id}.json", content);
}

public async Task<YogaClass> GetYogaClassById(string yogaClassId)
{

```

```

var httpClient = new HttpClient();
var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/YogaClasses/{yogaClassId}.json");
return JsonConvert.DeserializeObject<YogaClass>(response);
}

```

This code shows two methods in the `FirestoreService` class. The `GetClassInstancesByClassIdAsync` method retrieves a list of `ClassInstance` objects associated with a specific yoga class ID from Firebase. It does this by making an asynchronous HTTP GET request to fetch data, deserializing the response JSON into a dictionary of `ClassInstance` objects, and filtering the list based on the provided yoga class ID. The `CreateBooking` method, on the other hand, is used to create or update a booking in Firebase. It serializes the `Booking` object into JSON format and uses an HTTP PUT request to save this data to the Firebase database, ensuring that booking details are stored or updated.

```

public async Task<ClassInstance> GetClassInstanceById(string classInstanceId)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/ClassInstances/{classInstanceId}.json");
    return JsonConvert.DeserializeObject<ClassInstance>(response);
}

public async Task<int> GetClassInstanceCountByClassIdAsync(string classId)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/ClassInstances.json");

    var classInstances = JsonConvert.DeserializeObject<Dictionary<string, ClassInstance>>(response);
    if (classInstances != null)
    {
        return classInstances.Values.Count(ci => ci.ClassId == classId);
    }
    return 0;
}

```

This code illustrates the `GetClassInstanceByIdAsync` and `GetClassInstanceCountByClassIdAsync` methods in `FirestoreService`. The `GetClassInstanceByIdAsync` method retrieves a specific `ClassInstance` from Firebase by its unique ID, deserializing the JSON response into a `ClassInstance` object, which enables the application to access detailed data about a particular class session. The `GetClassInstanceCountByClassIdAsync` method counts the number of `ClassInstance` records associated with a particular yoga class ID, allowing for efficient inventory or schedule management by counting filtered instances based on the class ID.

```

public async Task<List<Booking>> GetBookingsByUserIdAsync(string userId)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Bookings.json");
    if (string.IsNullOrEmpty(response))
    {
        return new List<Booking>();
    }
    var bookingsDictionary = JsonConvert.DeserializeObject<Dictionary<string, Booking>>(response);

    if (bookingsDictionary == null)
    {
        return new List<Booking>();
    }

    var userBookings = bookingsDictionary.Values
        .Where(b => b.userId == userId)

```

```

        .ToList();
        return userBookings;
    }

    public async Task<Booking> GetBookingByIdAsync(string bookingId)
    {
        var httpClient = new HttpClient();
        var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Bookings/{bookingId}.json");
        if (string.IsNullOrEmpty(response))
        {
            return null;
        }
        return JsonConvert.DeserializeObject<Booking>(response);
    }

```

This code shows the `GetBookingsByUserIdAsync` and `GetBookingByIdAsync` methods in the `FirestoreService` class. The `GetBookingsByUserIdAsync` method fetches all bookings for a specific user by making an asynchronous GET request to Firebase, deserializing the JSON response into a dictionary of `Booking` objects, and filtering based on the `userId`. This helps display all bookings associated with a particular user. The `GetBookingByIdAsync` method retrieves a single booking record by its unique booking ID, which is useful for viewing or managing specific booking details.

```

    public async Task<List<User>> GetAllUsersAsync()
    {
        var httpClient = new HttpClient();
        var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users.json");

        if (string.IsNullOrEmpty(response))
        {
            return new List<User>();
        }

        var usersDictionary = JsonConvert.DeserializeObject<Dictionary<string, User>>(response);
        return usersDictionary?.Values.ToList() ?? new List<User>();
    }

    public async Task CancelBookingAsync(string bookingId)
    {
        var httpClient = new HttpClient();
        await httpClient.DeleteAsync($"{firebaseDatabaseUrl}/Bookings/{bookingId}.json");
    }

    public async Task<List<YogaClass>> GetAllYogaClassesAsync()
    {
        var httpClient = new HttpClient();
        var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/YogaClasses.json");

        var yogaClasses = JsonConvert.DeserializeObject<Dictionary<string, YogaClass>>(response);
        return yogaClasses != null ? new List<YogaClass>(yogaClasses.Values) : new List<YogaClass>();
    }

```

This code displays the `GetAllUsersAsync`, `CancelBookingAsync`, and `GetAllYogaClassesAsync` methods in the `FirestoreService` class. `GetAllUsersAsync` retrieves a list of all users by deserializing the Firebase response into a dictionary of `User` objects, which is beneficial for administrative tasks. `CancelBookingAsync` removes a booking record from Firebase by its booking ID using a DELETE request, aiding in booking management. The `GetAllYogaClassesAsync` method fetches all yoga classes by making an asynchronous GET

request and deserializing the response into a list of YogaClass objects, helping the application load available classes.

```
public async Task UpdateBookingStatusAsync(Booking booking)
{
    var httpClient = new HttpClient();
    var jsonData = JsonConvert.SerializeObject(booking);
    var content = new StringContent(jsonData, Encoding.UTF8, "application/json");

    await httpClient.PutAsync($"{firebaseDatabaseUrl}/Bookings/{booking.id}.json", content);
}

public async Task<User> GetUserByIdAsync(string userId)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users/{userId}.json");

    return !string.IsNullOrEmpty(response) ? JsonConvert.DeserializeObject<User>(response) : null;
}

public async Task<List<string>> GetCartByUserIdAsync(string userId)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Carts/{userId}.json");

    if (string.IsNullOrEmpty(response))
    {
        return new List<string>();
    }

    return JsonConvert.DeserializeObject<List<string>>(response) ?? new List<string>();
}
```

This code displays three methods in the `FirestoreService` class. The `UpdateBookingStatusAsync` method updates a booking's status in Firebase by serializing the `Booking` object and sending it through an HTTP PUT request, aiding in managing booking statuses directly in the database. The `GetUserByIdAsync` method retrieves user data based on a provided `userId`, deserializing the JSON response into a `User` object, which allows for user information display and modifications. Lastly, the `GetCartByUserIdAsync` method fetches the cart items associated with a specific user by `userId`, converting the JSON response into a list of item IDs, essential for accurately displaying the user's cart contents in the application.

```
public async Task SaveCartAsync(string userId, List<string> classInstanceIds)
{
    var httpClient = new HttpClient();
    var jsonData = JsonConvert.SerializeObject(classInstanceIds);
    var content = new StringContent(jsonData, Encoding.UTF8, "application/json");

    await httpClient.PutAsync($"{firebaseDatabaseUrl}/Carts/{userId}.json", content);
}
```

This code shows the `SaveCartAsync` method in the `FirestoreService` class. This method saves a user's cart to Firebase by serializing a list of `classInstanceIds` into JSON format. It then sends this serialized data to Firebase using an HTTP PUT request, placing the JSON data under the user's specific `userId` in the "Carts" collection. This approach ensures that the cart is consistently saved and synchronized across devices, allowing users to retrieve their cart contents regardless of where they access the application.

5.2.7 App.xaml.cs

```
namespace yogaapp_xamarin
{
    public partial class App : Application
    {
        public App()
        {
            InitializeComponent();
            CheckIfUserIsLoggedIn();

        }
        public static void SaveLoginStatus(string userId)
        {
            Preferences.Set("UserId", userId);
        }
        private void CheckIfUserIsLoggedIn()
        {
            var savedUserId = Preferences.Get("UserId", string.Empty);

            if (!string.IsNullOrEmpty(savedUserId))
            {
                MainPage = new AppShell();
            }
            else
            {
                MainPage = new NavigationPage(new WelcomePage());
            }
        }
    }
}
```

This code shows the App.xaml.cs file, which includes methods for initializing the application and managing login status. The App constructor calls `InitializeComponent()` and `CheckIfUserIsLoggedIn()` to set up the UI and determine if the user is already logged in. The `SaveLoginStatus` method saves the user's login status by storing their `UserId` in `Preferences`. The `CheckIfUserIsLoggedIn` method retrieves this `UserId`; if it exists, it sets `MainPage` to `AppShell` (indicating the main app interface), otherwise, it directs to the `LoginPage` to prompt the user to log in.

5.2.8 AppShell.xaml.cs

```
public string FullName { get; set; }
public string Email { get; set; }

private readonly string firebaseDatabaseUrl = "https://yogaapp-42561-default-rtdb.asia-southeast1.firebaseio.com/app/";
public AppShell()
{
    InitializeComponent();
    LoadUserInfo();
}
private async void LoadUserInfo()
{
    try
    {
        var savedUserId = Preferences.Get("UserId", string.Empty);
        if (!string.IsNullOrEmpty(savedUserId))
        {
            var httpClient = new HttpClient();
            var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users/{savedUserId}.json");
            if (!string.IsNullOrEmpty(response))
```

```

    {
        var user = JsonConvert.DeserializeObject<User>(response);
        FullName = user.fullName;
        Email = user.email;
        BindingContext = this;
    }
}
}
catch (Exception ex){ Console.WriteLine($"Error retrieving user info: {ex.Message}"); }
}

```

This code illustrates the AppShell.xaml.cs constructor and the LoadUserInfo method. The constructor initializes the UI with InitializeComponent and invokes LoadUserInfo to fetch the logged-in user's data from Firebase. In LoadUserInfo, the method retrieves the user's ID from local preferences, requests user data from Firebase, and binds it to FullName and Email properties for display. Error handling is included to log any issues retrieving user information.

```

private async void OnLogoutClicked(object sender, EventArgs e)
{
    Preferences.Remove("UserId");
    Application.Current.MainPage = new NavigationPage(new LoginPage());
}

```

This code shows the OnLogoutClicked method in AppShell.xaml.cs. This method handles the logout functionality by removing the "UserId" from local preferences and redirecting the user to the LoginPage, effectively logging the user out and returning them to the login interface.

5.2.9 WelcomePage.xaml.cs

```

namespace yogaapp_xamarin.Pages
{
    public partial class WelcomePage : ContentPage
    {
        public WelcomePage()
        {
            InitializeComponent();
        }

        private async void LogInButton_Clicked(object sender, EventArgs e)
        {
            await Navigation.PushAsync(new LoginPage());
        }

        private async void SignUpButton_Clicked(object sender, EventArgs e)
        {
            await Navigation.PushAsync(new SignUpPage());
        }
    }
}

```

This code shows the WelcomePage.xaml.cs file, which defines the WelcomePage class inheriting from ContentPage. This page includes two button event handlers, LogInButton_Clicked and SignUpButton_Clicked, each navigating to a different page. The LogInButton_Clicked method directs the user to the LoginPage, while the SignUpButton_Clicked method takes them to the SignUpPage, providing easy access to both login and registration options from the welcome screen.

5.2.10 SignUpPage.xaml.cs

```

private readonly string firebaseDatabaseUrl = "https://yogaapp-42561-default-rtdb.asia-
southeast1.firebaseio.com/app/";
private string generatedOtp = "";

public SignUpPage()
{
    InitializeComponent();
}

```

This code shows the initialization of the `firebaseDatabaseUrl` constant and the constructor for the `SignUpPage` class in `SignUpPage.xaml.cs`. Figure sets up a reference URL for Firebase, which will be used for database interactions, and initializes the components for the sign-up page interface.

```

private async void OnSignUpClicked(object sender, EventArgs e)
{
    try
    {
        string email = emailEntry.Text;
        string password = passwordEntry.Text;
        string fullName = fullNameEntry.Text;
        string phoneNumber = phoneNumberEntry.Text;
        string role = "Customer";
        string status = "Uploaded";

        if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password)
            || string.IsNullOrEmpty(fullName)
            || string.IsNullOrEmpty(phoneNumber))
        {
            await DisplayAlert("Error", "Please fill in all fields", "OK");
            return;
        }
        if (await IsEmailExistsAsync(email))
        {
            await DisplayAlert("Error", "Email already exists!", "OK");
            return;
        }
        if (phoneNumber.Length < 10)
        {
            await DisplayAlert("Error", "The phone number must have at least 10 digits", "OK");
            return;
        }
        if (password.Length < 8)
        {
            await DisplayAlert("Error", "Password must be at least 8 characters", "OK");
            return;
        }
        generatedOtp = GenerateOtp();
        SendOtpEmail(email, generatedOtp);
        await ShowOtpPopup(email, password, fullName, phoneNumber, role, status);
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", $"An error occurred: {ex.Message}", "OK");
    }
}

private string GenerateOtp()
{
    var random = new Random();
    return random.Next(100000, 999999).ToString();
}

```

```
}
```

This code demonstrates the `OnSignUpClicked` method, which handles sign-up logic. It validates user input, checking if required fields are filled and meet certain conditions (e.g., minimum length). If validation passes, an OTP is generated and sent for confirmation. Errors are displayed using alerts if validation fails.

```
private void SendOtpEmail(string email, string otp)
{
    try
    {
        var smtpClient = new SmtpClient("smtp.gmail.com")
        {
            Port = 587,
            Credentials = new System.Net.NetworkCredential("universalyoga.gcc210040@gmail.com", "yrpzqmxkbsxpmubt"),
            EnableSsl = true,
        };

        var mailMessage = new MailMessage
        {
            From = new MailAddress("universalyoga.gcc210040@gmail.com"),
            Subject = "Your OTP Code",
            Body = $"Your OTP code is: {otp}",
            IsBodyHtml = false,
        };
        mailMessage.To.Add(email);

        smtpClient.Send(mailMessage);
    }
    catch (Exception ex)
    {
        DisplayAlert("Error", $"Failed to send OTP: {ex.Message}", "OK");
    }
}
```

This code depicts the `SendOtpEmail` method, which sends a generated OTP to the user's email for verification. It utilizes SMTP with Gmail settings to send an email containing the OTP. If an error occurs, it shows an alert indicating failure in sending the OTP.

```
private async Task ShowOtpPopup(string email, string password, string fullName, string phoneNumber, string role, string status)
{
    bool isOtpCorrect = false;
    bool hasAttempted = false;

    while (!isOtpCorrect)
    {
        string promptMessage = hasAttempted ? "Incorrect OTP. Please try again:" : "Enter the OTP sent to your email";
        string enteredOtp = await DisplayPromptAsync("OTP Verification", promptMessage, "OK", "Cancel");

        if (enteredOtp == null)
        {
            return;
        }

        if (enteredOtp == generatedOtp)
        {
            isOtpCorrect = true;
            await CompleteSignup(email, password, fullName, phoneNumber, role, status);
        }
        else
        {
            isOtpCorrect = false;
        }
    }
}
```

```

        hasAttempted = true;
    }
}
}

```

This code illustrates the ShowOtpPopup method, which prompts the user to enter the OTP sent to their email. If the entered OTP matches, the sign-up is completed; otherwise, the user is prompted to re-enter. This loop continues until a correct OTP is provided or the user cancels.

```

private async Task CompleteSignup(string email, string password, string fullName, string phoneNumber, string role, string status)
{
    string salt = GenerateSalt();
    string saltedHashedPassword = HashPasswordWithSalt(password, salt);

    var userId = Guid.NewGuid().ToString();

    var user = new User
    {
        id = userId,
        email = email,
        password = saltedHashedPassword,
        fullName = fullName,
        phoneNumber = phoneNumber,
        role = role,
        status = status
    };

    var result = await CreateUserAsync(user);

    if (result)
    {
        await DisplayAlert("Success", "Sign up successful", "OK");
        await Navigation.PopAsync();
    }
    else
    {
        await DisplayAlert("Error", "Unable to sign up", "OK");
    }
}

```

This code shows the CompleteSignup method, where final user information is saved to the database. Passwords are hashed with a generated salt for security, and user details are stored in Firebase. Success or failure messages are displayed accordingly.

```

private async Task<bool> IsEmailExistsAsync(string email)
{
    try
    {
        var httpClient = new HttpClient();
        var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users.json");

        var users = JsonConvert.DeserializeObject<Dictionary<string, User>>(response);
        if (users != null)
        {
            foreach (var user in users.Values)
            {
                if (user.email == email)
                {
                    return true;
                }
            }
        }
    }
}

```

```

    }
}
catch (Exception ex)
{
    await DisplayAlert("Error", $"An error occurred while checking email: {ex.Message}", "OK");
}
return false;
}

```

This code details the `IsEmailExistsAsync` method, which checks if the user's email already exists in the Firebase database. If a match is found, the method returns `true`; otherwise, it returns `false`. An alert is displayed if an error occurs during this process.

```

private async Task<bool> CreateUserAsync(User user)
{
    try
    {
        var httpClient = new HttpClient();
        var jsonData = JsonConvert.SerializeObject(user);
        var content = new StringContent(jsonData, Encoding.UTF8, "application/json");
        var response = await httpClient.PutAsync($"{firebaseDatabaseUrl}/Users/{user.id}.json", content);

        return response.IsSuccessStatusCode;
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", $"Unable to create user: {ex.Message}", "OK");
        return false;
    }
}

```

This code shows the `CreateUserAsync` method, which adds a new user to the Firebase database. The user's information is serialized to JSON format and sent to Firebase. If the operation succeeds, `true` is returned; otherwise, `false` is returned, and an error alert is displayed.

```

private string GenerateSalt()
{
    using (var rng = new RNGCryptoServiceProvider())
    {
        byte[] saltBytes = new byte[16];
        rng.GetBytes(saltBytes);
        return Convert.ToBase64String(saltBytes);
    }
}

private string HashPasswordWithSalt(string password, string salt)
{
    using (var sha256 = SHA256.Create())
    {
        var saltedPassword = Encoding.UTF8.GetBytes(salt + password);
        var hashBytes = sha256.ComputeHash(saltedPassword);

        var hashString = BitConverter.ToString(hashBytes).Replace("-", "").ToLower();

        return $"{salt}:{hashString}";
    }
}

private void OnSignInTapped(object sender, EventArgs e)
{
    Navigation.PopAsync();
}

```

```
}
```

Figure illustrates the `GenerateSalt` and `HashPasswordWithSalt` methods for password security. `GenerateSalt` creates a random salt, and `HashPasswordWithSalt` hashes the password with the salt using SHA-256. This method enhances security by making stored passwords harder to reverse-engineer.

5.2.11 LoginPage.xaml.cs

```
private readonly string firebaseDatabaseUrl = "https://yogaapp-42561-default-rtdb.asia-southeast1.firebaseio.com/app/";
private string generatedOtp = "";

public LoginPage()
{
    InitializeComponent();
}
```

This code illustrates the initialization of the `firebaseDatabaseUrl` constant and the constructor for the `LoginPage` class in `LoginPage.xaml.cs`. Figure sets up a constant URL for Firebase, which will serve as the reference path for all interactions with the Firebase Realtime Database throughout the app. The constructor, `LoginPage()`, initializes the user interface components by calling `InitializeComponent()`, which is essential for rendering the elements on the login screen.

```
private async void OnLoginClicked(object sender, EventArgs e)
{
    try
    {
        var email = emailEntry.Text;
        var password = passwordEntry.Text;

        var user = await GetUserByEmailAndPasswordAsync(email, password);

        if (user != null && user.role == "Customer")
        {
            await DisplayAlert("Success", "Login successful", "OK");

            App.SaveLoginStatus(user.id);

            Application.Current.MainPage = new AppShell();
        }
        else
        {
            await DisplayAlert("Error", "Incorrect email or password", "OK");
        }
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", "An error occurred: " + ex.Message, "OK");
    }
}
```

This code displays the `OnLoginClicked` method, which handles user login validation in `LoginPage.xaml.cs`. When the login button is clicked, this method retrieves the email and password input by the user, then calls the asynchronous `GetUserByEmailAndPasswordAsync` function to verify these credentials against Firebase data. If a user with a matching email and password hash exists, the method sets the `MainPage` to the app's main shell, allowing access. Otherwise, it displays an error message if the credentials are

incorrect. This method includes a try-catch block to handle exceptions, ensuring reliable execution and feedback.

```
private async Task<User?> GetUserByEmailAndPasswordAsync(string email, string enteredPassword)
{
    var httpClient = new HttpClient();
    var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users.json");

    var users = JsonConvert.DeserializeObject<Dictionary<string, User>>(response);
    if (users != null)
    {
        foreach (var user in users.Values)
        {
            if (user.email == email)
            {
                if (user.role != "Customer")
                {
                    return null;
                }

                var passwordParts = user.password.Split(':');
                if (passwordParts.Length == 2)
                {
                    var storedSalt = passwordParts[0];
                    var storedHash = passwordParts[1];

                    var enteredPasswordHash = HashPasswordWithSalt(enteredPassword, storedSalt);

                    if (enteredPasswordHash == $"{storedSalt}:{storedHash}")
                    {
                        return user;
                    }
                }
            }
        }
    }
    return null;
}
```

This code illustrates the `GetUserByEmailAndPasswordAsync` method, an asynchronous function that retrieves and verifies user credentials in Firebase. This method fetches the list of users, checks if an email matches, and splits the stored password into salt and hash parts. Using the `HashPasswordWithSalt` function, it verifies that the hash of the entered password matches the stored hash, ensuring secure authentication. If both checks are successful, it returns the user object; otherwise, it returns null, denying access.

```
private string HashPasswordWithSalt(string password, string salt)
{
    using (var sha256 = SHA256.Create())
    {
        var saltedPassword = Encoding.UTF8.GetBytes(salt + password);
        var hashBytes = sha256.ComputeHash(saltedPassword);

        var hashString = BitConverter.ToString(hashBytes).Replace("-", "").ToLower();
        return $"{salt}:{hashString}";
    }
}

private void OnSignUpClicked(object sender, EventArgs e)
{
    Navigation.PushAsync(new SignUpPage());
}
```

```
}
```

This code shows the `HashPasswordWithSalt` method, which takes a password and a salt value, then generates a secure hash using SHA-256. This method combines the password and salt, hashes the result, and returns the salted hash string. This hashing approach provides an additional layer of security, protecting user credentials from unauthorized access.

```
private async void OnForgotPasswordTapped(object sender, EventArgs e)
{
    try
    {
        string email = emailEntry.Text;

        if (string.IsNullOrEmpty(email))
        {
            await DisplayAlert("Error", "Please enter your email to reset password", "OK");
            return;
        }

        if (!await IsEmailExistsAsync(email))
        {
            await DisplayAlert("Error", "Email does not exist!", "OK");
            return;
        }

        generatedOtp = GenerateOtp();
        SendOtpEmail(email, generatedOtp);

        bool isOtpCorrect = false;
        bool hasAttempted = false;

        while (!isOtpCorrect)
        {
            string promptMessage = hasAttempted ? "Incorrect OTP. Please try again:" : "Enter the OTP sent to your email";
            string enteredOtp = await DisplayPromptAsync("OTP Verification", promptMessage, "OK", "Cancel");

            if (enteredOtp == null)
            {
                return;
            }

            if (enteredOtp == generatedOtp)
            {
                isOtpCorrect = true;
                await ShowResetPasswordPopup(email);
            }
            else
            {
                isOtpCorrect = false;
                hasAttempted = true;
            }
        }
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", $"An error occurred: {ex.Message}", "OK");
    }
}
```

This code depicts the `OnForgotPasswordTapped` method, which initiates the password reset process by verifying the user's email. If the email exists, the method generates a 6-digit OTP using `GenerateOtp` and

sends it via `SendOtpEmail`. The OTP is then used for secure verification, allowing users to reset their password if they verify it successfully.

```
private async Task ShowResetPasswordPopup(string email)
{
    bool isPasswordValid = false;

    while (!isPasswordValid)
    {
        string promptMessage = "Enter new password (min. 8 characters):";
        string newPassword = await DisplayPromptAsync("Reset Password", promptMessage, "OK", "Cancel");

        if (newPassword == null)
        {
            return;
        }

        if (newPassword.Length < 8)
        {
            await DisplayAlert("Error", "Password must be at least 8 characters", "OK");
            continue;
        }

        bool isConfirmationValid = false;
        while (!isConfirmationValid)
        {
            string confirmNewPassword = await DisplayPromptAsync("Reset Password", "Confirm new password:", "OK", "Cancel");

            if (confirmNewPassword == null)
            {
                return;
            }

            if (newPassword != confirmNewPassword)
            {
                await DisplayAlert("Error", "Passwords do not match", "OK");
            }
            else
            {
                {
                    isConfirmationValid = true;
                }
            }
        }

        isPasswordValid = true;
        await UpdatePasswordInFirebase(email, newPassword);
    }
}
```

This code illustrates the `ShowResetPasswordPopup` method, prompting the user to enter and confirm a new password. It checks if the new password meets length requirements and matches the confirmation. If both conditions are met, it proceeds to update the password in Firebase for secure reset.

```
private async Task UpdatePasswordInFirebase(string email, string newPassword)
{
    string salt = GenerateSalt();
    string saltedHashedPassword = HashPasswordWithSalt(newPassword, salt);

    try
    {
        var httpClient = new HttpClient();
        var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users.json");
    }
}
```

```

var users = JsonConvert.DeserializeObject<Dictionary<string, User>>(response);
if (users != null)
{
    foreach (var userEntry in users)
    {
        if (userEntry.Value.email == email)
        {
            userEntry.Value.password = saltedHashedPassword;
            var jsonData = JsonConvert.SerializeObject(userEntry.Value);
            var content = new StringContent(jsonData, Encoding.UTF8, "application/json");
            await httpClient.PutAsync($"{firebaseDatabaseUrl}/Users/{userEntry.Key}.json", content);
            await DisplayAlert("Success", "Password has been reset successfully", "OK");
            return;
        }
    }
}
catch (Exception ex)
{
    await DisplayAlert("Error", $"An error occurred while updating the password: {ex.Message}", "OK");
}
}

```

This code shows the `UpdatePasswordInFirebase` method, which securely updates a user's password in Firebase. The method generates a salt, hashes the new password, and replaces the old password with the hashed one in Firebase. This ensures that the password is securely stored and updated.

```

private async Task<bool> IsEmailExistsAsync(string email)
{
    try
    {
        var httpClient = new HttpClient();
        var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users.json");

        var users = JsonConvert.DeserializeObject<Dictionary<string, User>>(response);
        if (users != null)
        {
            foreach (var user in users.Values)
            {
                if (user.email == email)
                {
                    return true;
                }
            }
        }
    }
    catch (Exception ex)
    {
        await DisplayAlert("Error", $"An error occurred while checking email: {ex.Message}", "OK");
    }
    return false;
}

```

This code illustrates the `IsEmailExistsAsync` method, which verifies if a given email is already registered in Firebase. By retrieving and deserializing user data, it checks for the presence of the email to prevent duplicate accounts and enforce unique user identification.

```

private string GenerateOtp()
{
    var random = new Random();
    return random.Next(100000, 999999).ToString();
}

```

```

private void SendOtpEmail(string email, string otp)
{
    try
    {
        var smtpClient = new SmtpClient("smtp.gmail.com")
        {
            Port = 587,
            Credentials = new System.Net.NetworkCredential("universalyoga.gcc210040@gmail.com", "yrpzqmxkbsxpmubt"),
            EnableSsl = true,
        };

        var mailMessage = new MailMessage
        {
            From = new MailAddress("universalyoga.gcc210040@gmail.com"),
            Subject = "Your OTP Code",
            Body = $"Your OTP code is: {otp}",
            IsBodyHtml = false,
        };
        mailMessage.To.Add(email);

        smtpClient.Send(mailMessage);
    }
    catch (Exception ex)
    {
        DisplayAlert("Error", $"Failed to send OTP: {ex.Message}", "OK");
    }
}

```

This code illustrates two methods: `GenerateOtp` and `SendOtpEmail`. The `GenerateOtp` method generates a random six-digit OTP, using the `Random` class to create a string between 100000 and 999999, which ensures a six-digit number. This OTP is then passed to `SendOtpEmail`, which configures an SMTP client to send the OTP via email. `SendOtpEmail` sets up the SMTP client with the Gmail server and user credentials, composes a message containing the OTP, and attempts to send it to the specified email. If any error occurs during the email process, it is caught and displayed in an alert, ensuring the user is informed if OTP delivery fails.

```

private string GenerateSalt()
{
    using (var rng = new RNGCryptoServiceProvider())
    {
        byte[] saltBytes = new byte[16];
        rng.GetBytes(saltBytes);
        return Convert.ToBase64String(saltBytes);
    }
}

```

This code shows the `GenerateSalt` method in `LoginPage.xaml.cs`. This method generates a cryptographic salt for password hashing. It uses the `RNGCryptoServiceProvider` class to create a secure, random 16-byte array (`saltBytes`). This array is then converted to a base64 string, which is returned as the salt. The generated salt enhances security by adding randomness to hashed passwords, making it more resistant to attacks such as rainbow table lookups.

5.2.12 MainPage.xaml.cs

```
private readonly string firebaseDatabaseUrl = "https://yogaapp-42561-default-rtdb.asia-southeast1.firebaseio.com/";
private FirebaseService _firebaseService;

public ObservableCollection<YogaClass> NewYogaClasses { get; set; }
public ObservableCollection<Booking> LatestBookings { get; set; }

private int _yogaClassCount;
public int YogaClassCount
{
    get => _yogaClassCount;
    set
    {
        if (_yogaClassCount != value)
        {
            _yogaClassCount = value;
            OnPropertyChanged(nameof(YogaClassCount));
        }
    }
}

public MainPage()
{
    InitializeComponent();
    _firebaseService = new FirebaseService();
    NewYogaClasses = new ObservableCollection<YogaClass>();
    LatestBookings = new ObservableCollection<Booking>();
    BindingContext = this;
}
```

This code shows the initialization of `firebaseDatabaseUrl` and the creation of observable collections `NewYogaClasses` and `LatestBookings` in `MainPage.xaml.cs`. These observable collections are used to display dynamic lists of yoga classes and recent bookings on the main page, enabling real-time updates.

```
private async void OnStartButtonClicked(object sender, EventArgs e)
{
    await Navigation.PushAsync(new YogaClassListPage());
}

protected override async void OnAppearing()
{
    base.OnAppearing();

    var user = await GetCurrentUser();
    if (user != null)
    {
        Console.WriteLine($"User is signed in. Email: {user.email}");
    }
    else
    {
        Console.WriteLine("No user is signed in.");
        Application.Current.MainPage = new NavigationPage(new LoginPage());
    }

    await LoadNewYogaClasses();
    await LoadLatestBookings();
    await LoadYogaClassCount();
}
```

This code illustrates the `OnStartButtonClicked` and `OnAppearing` methods in `MainPage.xaml.cs`. `OnStartButtonClicked` navigates to the yoga class list page, while `OnAppearing` checks if a user is signed in. If

no user is signed in, it redirects to the login page; otherwise, it loads recent yoga classes, bookings, and the yoga class count.

```
private async Task<User?> GetCurrentUser()
{
    try
    {
        var savedUserId = Preferences.Get("UserId", string.Empty);
        if (!string.IsNullOrEmpty(savedUserId))
        {
            var httpClient = new HttpClient();
            var response = await httpClient.GetStringAsync($"{firebaseDatabaseUrl}/Users/{savedUserId}.json");

            if (!string.IsNullOrEmpty(response))
            {
                return JsonConvert.DeserializeObject<User>(response);
            }
        }
        return null;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error retrieving user: {ex.Message}");
        return null;
    }
}
```

This code presents the `GetCurrentUser` method in `MainPage.xaml.cs`. This method retrieves the current user's information from Firebase based on the saved user ID. It helps ensure that user-specific data is available for subsequent operations in the app.

```
private async Task LoadNewYogaClasses()
{
    var classes = await _firebaseService.GetAllYogaClassesAsync();
    NewYogaClasses.Clear();

    foreach (var yogaClass in classes.Take(2))
    {
        if (TimeSpan.TryParse(yogaClass.Time, out TimeSpan startTime))
        {
            var endTime = startTime.Add(TimeSpan.FromMinutes(yogaClass.Duration));
            yogaClass.Time = $"{startTime:hh\\:\\:mm} - {endTime:hh\\:\\:mm}";
        }
        NewYogaClasses.Add(yogaClass);
    }
}

private async Task LoadLatestBookings()
{
    var userId = Preferences.Get("UserId", string.Empty);
    var bookings = await _firebaseService.GetBookingsByUserIdAsync(userId);
    LatestBookings.Clear();
    foreach (var booking in bookings.Take(2))
    {
        LatestBookings.Add(booking);
    }
}

private async Task LoadYogaClassCount()
{
    var classes = await _firebaseService.GetAllYogaClassesAsync();
    YogaClassCount = classes.Count;
}
```

```
}
```

This code displays the `LoadNewYogaClasses`, `LoadLatestBookings`, and `LoadYogaClassCount` methods in `MainPage.xaml.cs`. These methods fetch the latest yoga classes, bookings, and the total count of yoga classes from Firebase, updating the respective observable collections for real-time display on the main page.

5.2.13 YogaClassListPage.xaml.cs

```
private readonly FirebaseService _firebaseService = new FirebaseService();
private List<YogaClass> _allYogaClasses = new List<YogaClass>();

public YogaClassListPage()
{
    InitializeComponent();
    LoadYogaClasses();
}
```

This code shows the initialization of the Firebase service and a list of `YogaClass` objects in `YogaClassListPage.xaml.cs`. Figure prepares the Firebase service and initializes an empty list to store all Yoga classes, setting up the necessary components for data retrieval and display.

```
private async void LoadYogaClasses()
{
    _allYogaClasses = await _firebaseService.GetAllYogaClassesAsync();

    var yogaClassWithInstanceCounts = new Dictionary<YogaClass, int>();

    foreach (var yogaClass in _allYogaClasses)
    {
        var startTime = TimeSpan.Parse(yogaClass.Time);
        var duration = TimeSpan.FromMinutes(yogaClass.Duration);
        var endTime = startTime + duration;

        if (endTime.TotalMinutes >= 24 * 60)
        {
            endTime = endTime - TimeSpan.FromDays(1);
        }

        yogaClass.Time = $"{startTime:hh\\:\\:mm} - {endTime:hh\\:\\:mm}";

        var classInstanceCount = await _firebaseService.GetClassInstanceCountByClassIdAsync(yogaClass.Id);

        yogaClassWithInstanceCounts[yogaClass] = classInstanceCount;
    }
    yogaClassListView.ItemsSource = _allYogaClasses;
}
```

This code demonstrates the `LoadYogaClasses` method, which retrieves yoga classes from Firebase, calculates start and end times for each class, and adds instance counts to the dictionary. This method provides the necessary data formatting and retrieves real-time data to display in the UI.

```
private void OnSearchOptionChanged(object sender, CheckedChangedEventArgs e)
{
    if (dayOfWeekRadio.IsChecked)
    {
        dayOfWeekScrollView.IsVisible = true;
        timeOfDaySearchBar.IsVisible = false;
    }
    else if (timeOfDayRadio.IsChecked)
    {
        dayOfWeekScrollView.IsVisible = false;
    }
}
```

```

        timeOfDaySearchBar.IsVisible = true;
    }

    yogaClassListView.ItemsSource = _allYogaClasses;
}

```

This code illustrates the `OnSearchOptionChanged` method, which controls the visibility of search filters based on user selection. Figure dynamically adjusts the UI to show or hide specific search options, improving user experience for filtering yoga classes.

```

private void OnDayOfWeekCheckedChanged(object sender, CheckedChangedEventArgs e)
{
    var selectedDays = new List<string>();

    if (mondayCheckBox.IsChecked) selectedDays.Add("Monday");
    if (tuesdayCheckBox.IsChecked) selectedDays.Add("Tuesday");
    if (wednesdayCheckBox.IsChecked) selectedDays.Add("Wednesday");
    if (thursdayCheckBox.IsChecked) selectedDays.Add("Thursday");
    if (fridayCheckBox.IsChecked) selectedDays.Add("Friday");
    if (saturdayCheckBox.IsChecked) selectedDays.Add("Saturday");
    if (sundayCheckBox.IsChecked) selectedDays.Add("Sunday");

    if (!selectedDays.Any())
    {
        yogaClassListView.ItemsSource = _allYogaClasses;
    }
    else
    {
        var filteredClasses = _allYogaClasses
            .Where(c => selectedDays.Contains(c.DayOfWeek))
            .ToList();

        yogaClassListView.ItemsSource = filteredClasses;
    }
}

```

This code presents the `OnDayOfWeekCheckedChanged` method, filtering yoga classes based on selected days. This method collects selected days and displays only those classes that match the specified days, enabling targeted searches by day of the week.

```

private void OnTimeOfDaySearchChanged(object sender, TextChangedEventArgs e)
{
    var searchText = e.NewTextValue?.ToLower();

    if (string.IsNullOrEmpty(searchText))
    {
        yogaClassListView.ItemsSource = _allYogaClasses;
    }
    else
    {
        var filteredClasses = _allYogaClasses
            .Where(c => c.Time.ToLower().Contains(searchText))
            .ToList();

        yogaClassListView.ItemsSource = filteredClasses;
    }
}

```

```
}
```

This code shows the `OnTimeOfDaySearchChanged` method, enabling users to search yoga classes based on time input. This method filters the class list by checking if the class times contain the entered search text, providing a time-based filtering option.

```
private async void OnYogaClassSelected(object sender, SelectedItemChangedEventArgs e)
{
    if (e.SelectedItem == null) return;

    var selectedYogaClass = e.SelectedItem as YogaClass;

    if (selectedYogaClass != null)
    {
        await Navigation.PushAsync(new YogaClassDetailsPage(selectedYogaClass));
    }
    yogaClassListView.SelectedItem = null;
}
```

This code depicts the `OnYogaClassSelected` method, which navigates the user to the details page of a selected yoga class. This method allows users to view more information about a specific yoga class by selecting it from the list.

5.2.14 YogaClassDetailsPage.xaml.cs

```
public partial class YogaClassDetailsPage : ContentPage
{
    private YogaClass _yogaClass;

    public YogaClassDetailsPage(YogaClass yogaClass)
    {
        InitializeComponent();
        _yogaClass = yogaClass;
        BindingContext = _yogaClass;
    }

    private async void OnBookingButtonClicked(object sender, EventArgs e)
    {
        await Navigation.PushAsync(new BookingYogaClassPage(_yogaClass));
    }
}
```

This code shows the setup of the `YogaClassDetailsPage` class, which initializes the details page for a selected yoga class. The constructor accepts a `YogaClass` object, sets the data context to this object, and provides a method `OnBookingButtonClicked` to navigate to the booking page. This setup allows users to view details of a specific yoga class and proceed to book it.

5.2.15 BookingYogaClassPage.xaml.cs

```
private YogaClass _yogaClass;
private List<ClassInstance> _classInstances;
private double _totalPrice;
private FirebaseService _firebaseService = new FirebaseService();

public double TotalPrice
{
    get => _totalPrice;
    set
    {
        if (_totalPrice != value)
```

```

        {
            _totalPrice = value;
            OnPropertyChanged(nameof(TotalPrice));
        }
    }
}

public BookingYogaClassPage(YogaClass yogaClass)
{
    InitializeComponent();
    _yogaClass = yogaClass;
    LoadClassInstances();
    BindingContext = this;
}

```

This code shows the initialization of variables and the constructor for `BookingYogaClassPage`, which receives a `YogaClass` object and initializes the component, sets the `_yogaClass` variable, and calls `LoadClassInstances` to populate the list of class instances associated with the selected yoga class. The `TotalPrice` property with `OnPropertyChanged` notification updates the total cost dynamically based on selected classes.

```

private async void LoadClassInstances()
{
    _classInstances = await _firebaseService.GetClassInstancesByClassIdAsync(_yogaClass.Id) ?? new List<ClassInstance>();
    _filteredInstances = new List<ClassInstance>(_classInstances);

    var today = DateTime.Now.Date;
    var filteredInstances = new List<ClassInstance>();
    var allUsers = await _firebaseService.GetAllUsersAsync();

    foreach (var classInstance in _classInstances)
    {
        if (DateTime.TryParseExact(classInstance.Date, new[] { "d/M/yyyy", "dd/MM/yyyy" }, CultureInfo.InvariantCulture,
            DateTimeStyles.None, out DateTime classDate) && classDate >= today)
        {
            var instructor = allUsers.FirstOrDefault(u => u.id == classInstance.Instructor);
            classInstance.Instructor = instructor?.fullName ?? "Unknown Instructor";
            filteredInstances.Add(classInstance);
        }
    }

    _classInstances = filteredInstances;
    _filteredInstances = new List<ClassInstance>(_classInstances);

    foreach (var classInstance in _classInstances)
    {
        classInstance.PropertyChanged += OnClassInstancePropertyChanged;
    }

    foreach (var classInstance in _classInstances)
    {
        if (Cart.IsInCart(classInstance))
        {
            classInstance.IsSelected = false;
            classInstance.IsSelectable = false;
        }
    }

    classInstancesListView.ItemsSource = _filteredInstances;
}

```

```
}
```

This code presents the `LoadClassInstances` method, which retrieves and filters class instances based on their date. Only upcoming sessions are loaded, and each instance is assigned an instructor. Invalid date formats are logged, and instances already in the cart are marked non-selectable. The filtered list is then set as the item source for display.

```
private void OnClassInstancePropertyChanged(object sender, PropertyChangedEventArgs e)
{
    if (e.PropertyName == nameof(ClassInstance.IsSelected))
    {
        TotalPrice = _classInstances.Where(ci => ci.IsSelected).Count() * _yogaClass.Price;
    }
}
```

```
private async void OnAddToCartClicked(object sender, EventArgs e)
{
    var selectedInstances = _classInstances.Where(ci => ci.IsSelected).ToList();

    if (selectedInstances.Any())
    {
        var userId = Preferences.Get("UserId", string.Empty);

        if (string.IsNullOrEmpty(userId))
        {
            await DisplayAlert("Error", "Unable to identify user. Please log in again.", "OK");
            return;
        }

        Cart.AddToCart(selectedInstances, userId);

        await DisplayAlert("Success", "Yoga class has been added to your cart!", "OK");
        await Navigation.PopAsync();
    }
    else
    {
        await DisplayAlert("Error", "You haven't selected any classes!", "OK");
    }
}
```

```
private async void OnClassInstanceTapped(object sender, ItemTappedEventArgs e)
{
    if (e.Item == null) return;

    var selectedInstance = e.Item as ClassInstance;
    if (selectedInstance != null)
    {
        await DisplayAlert("Class Instance Details",
            $"Date: {selectedInstance.Date}\n" +
            $"Instructor: {selectedInstance.Instructor}\n" +
            $"Comments: {selectedInstance.AdditionalComments}",
            "OK");
    }
    ((ListView)sender).SelectedItem = null;
}
```

This code illustrates three event handlers. `OnClassInstancePropertyChanged` updates `TotalPrice` when the selection changes. `OnAddToCartClicked` adds selected instances to the cart after verifying user login, and `OnClassInstanceTapped` displays detailed information about a selected instance in an alert.

```
private void OnDateSearchBarTextChanged(object sender, TextChangedEventArgs e)
```

```

{
    string searchText = e.NewTextValue?.Trim().ToLower() ?? string.Empty;

    if (string.IsNullOrEmpty(searchText))
    {
        _filteredInstances = new List<ClassInstance>(_classInstances);
    }
    else
    {
        _filteredInstances = _classInstances
            .Where(ci => ci.Date != null && ci.Date.ToLower().Contains(searchText))
            .ToList();
    }

    classInstancesListView.ItemsSource = _filteredInstances;
}

```

This code shows a handler that filters class instances by date based on user input in a search bar. It trims and converts the input to lowercase, then updates the displayed list to show only instances with matching dates, or resets to show all if the input is empty.

5.2.16 MyCartPage.xaml.cs

```

private FirebaseService _firebaseService = new FirebaseService();
private List<YogaClass> _allYogaClasses;
private List<User> _allUsers;

public double TotalPrice { get; set; }
public ObservableCollection<ClassInstance> ClassInstancesInCart { get; set; }

public MyCartPage()
{
    InitializeComponent();
    ClassInstancesInCart = Cart.ClassInstancesInCart;
    BindingContext = this;
}

protected override async void OnAppearing()
{
    base.OnAppearing();
    var userId = Preferences.Get("UserId", string.Empty);

    if (!string.IsNullOrEmpty(userId))
        await Cart.LoadCartFromFirebase(userId);

    if (_allYogaClasses == null || !_allYogaClasses.Any())
        await LoadData();

    LoadCartData();
}

```

This code shows the MyCartPage class constructor and OnAppearing method. The constructor initializes the components, binds ClassInstancesInCart from the Cart, and sets the BindingContext. The OnAppearing method retrieves the user ID, loads cart data from Firebase if the user ID is valid, and checks if _allYogaClasses is populated, triggering LoadData if necessary.

```

private async Task LoadData()
{
    _allYogaClasses = await _firebaseService.GetAllYogaClassesAsync();
    _allUsers = await _firebaseService.GetAllUsersAsync();
}

```

```

if (_allYogaClasses != null && _allYogaClasses.Any())
{
    CalculateTotalPrice();
    LoadCartData();
}
else
{
    await DisplayAlert("Error", "Failed to load yoga classes. Please try again.", "OK");
}
}

private void LoadCartData()
{
    foreach (var classInstance in Cart.ClassInstancesInCart)
    {
        var instructor = _allUsers.FirstOrDefault(u => u.id == classInstance.Instructor);
        classInstance.Instructor = instructor?.fullName ?? "Unknown Instructor";
        classInstance.IsSelected = false;
    }

    var groupedClassInstances = Cart.ClassInstancesInCart
        .GroupBy(ci => ci.ClassId)
        .Select(group => new GroupedClassInstance
        {
            ClassId = group.Key,
            TypeOfClass = GetTypeOfClass(group.Key),
            Price = GetPriceOfClass(group.Key),
            PriceText = $"Price per: {GetPriceOfClass(group.Key):C}",
            ClassInstances = new ObservableCollection<ClassInstance>(group.ToList())
        })
        .ToList();

    cartListView.ItemsSource = groupedClassInstances.Any() ? groupedClassInstances : null;

    CalculateTotalPrice();
}

```

This code illustrates the LoadData and LoadCartData methods. The LoadData method retrieves all yoga classes and user data from Firebase. If the classes are successfully loaded, it calls CalculateTotalPrice and LoadCartData to initialize and display the cart items. The LoadCartData method then processes the cart items by grouping them according to class ID, setting each class instance's instructor details, and formatting the price. Finally, the grouped cart items are set as the data source for display, ensuring an organized view of cart contents with calculated pricing.

```

private double GetPriceOfClass(string classId)
{
    var yogaClass = _allYogaClasses.FirstOrDefault(yc => yc.Id == classId);
    return yogaClass?.Price ?? 0;
}

public class GroupedClassInstance
{
    public string ClassId { get; set; }
    public string TypeOfClass { get; set; }
    public double Price { get; set; }
    public string PriceText { get; set; }
    public ObservableCollection<ClassInstance> ClassInstances { get; set; }
}

private void CalculateTotalPrice()

```

```

{
    TotalPrice = Cart.ClassInstancesInCart
        .Where(ci => ci.IsSelected)
        .Sum(ci => GetPriceOfClass(ci.ClassId));
    OnPropertyChanged(nameof(TotalPrice));
}

private string GetTypeOfClass(string classId)
{
    var yogaClass = _allYogaClasses.FirstOrDefault(yc => yc.Id == classId);
    return yogaClass?.TypeOfClass ?? "Unknown";
}

```

This code shows utility methods in MyCartPage. GetPriceOfClass and GetTypeOfClass fetch the price and type for a specific class ID, while CalculateTotalPrice calculates the total price of selected items in the cart.

```

private async void OnConfirmBookingClicked(object sender, EventArgs e)
{
    var userId = Preferences.Get("UserId", string.Empty);

    if (string.IsNullOrEmpty(userId))
    {
        await DisplayAlert("Warning", "Unable to identify user. Please log in again.", "OK");
        return;
    }

    if (!Cart.ClassInstancesInCart.Any(ci => ci.IsSelected))
    {
        await DisplayAlert("Warning", "Please select at least one class to confirm booking.", "OK");
        return;
    }

    var selectedInstances = Cart.ClassInstancesInCart
        .Where(ci => ci.IsSelected)
        .Select(ci => new ClassInstanceInfo
        {
            classId = ci.ClassId,
            classInstanceId = ci.Id
        }).ToList();

    var booking = new Booking
    {
        id = Guid.NewGuid().ToString(),
        classInstances = selectedInstances,
        totalPrice = TotalPrice,
        status = "Pending",
        userId = userId
    };

    await _firebaseService.CreateBooking(booking);

    foreach (var selected in Cart.ClassInstancesInCart.Where(ci => ci.IsSelected).ToList())
    {
        Cart.RemoveClassInstance(selected, userId);
    }

    LoadCartData();
    await DisplayAlert("Success", "Booking confirmed!", "OK");
    await Navigation.PopAsync();
}

```

```
}
```

This code displays the `OnConfirmBookingClicked` method. This method confirms a booking by checking the user ID and selected items, creating a `Booking` instance with selected classes, and storing it in `Firestore`. Selected items are then removed from the cart.

```
private async void OnRemoveClassInstanceClicked(object sender, EventArgs e)
{
    var button = sender as ImageButton;
    var classInstance = button?.CommandParameter as ClassInstance;

    if (classInstance != null)
    {
        bool isConfirmed = await DisplayAlert("Confirm Delete", "Are you sure you want to remove this class instance from the cart?", "Yes", "No");

        if (isConfirmed)
        {
            var userId = Preferences.Get("UserId", string.Empty);
            Cart.RemoveClassInstance(classInstance, userId);
            LoadCartData();
        }
    }
}

private void OnCheckBoxCheckedChanged(object sender, CheckedChangedEventArgs e)
{
    CalculateTotalPrice();
}
```

This code shows the `OnRemoveClassInstanceClicked` and `OnCheckBoxCheckedChanged` methods. `OnRemoveClassInstanceClicked` removes a class from the cart after user confirmation, while `OnCheckBoxCheckedChanged` updates the total price when an item's selection state changes.

5.2.17 UserBookingsPage.xaml.cs

```
private System.Timers.Timer _refreshTimer;
private FirebaseService _firebaseService = new FirebaseService();

public UserBookingsPage()
{
    InitializeComponent();
}

protected override void OnAppearing()
{
    base.OnAppearing();
    _ = LoadUserBookings();

    _refreshTimer = new System.Timers.Timer(10000);
    _refreshTimer.Elapsed += (sender, e) =>
    {
        Device.BeginInvokeOnMainThread(async () => await LoadUserBookings());
    };
    _refreshTimer.Start();
}

protected override void OnDisappearing()
{
    base.OnDisappearing();

    if (_refreshTimer != null)
```

```

    {
        _refreshTimer.Stop();
        _refreshTimer.Dispose();
    }
}

```

This code shows the initialization of the `UserBookingsPage` class and its `OnAppearing` and `OnDisappearing` methods. The `OnAppearing` method initializes a timer set to refresh every 10 seconds, calling `LoadUserBookings` to keep booking data up-to-date. The `OnDisappearing` method stops and disposes of the timer when the page is no longer visible.

```

private async Task LoadUserBookings()
{
    var userId = Preferences.Get("UserId", string.Empty);

    if (string.IsNullOrEmpty(userId))
    {
        await DisplayAlert("Error", "Unable to identify user. Please log in again.", "OK");
        return;
    }

    var bookings = await _firebaseService.GetBookingsByUserIdAsync(userId);

    foreach (var booking in bookings)
    {
        booking.canCancel = booking.status != "Confirmed" && booking.status != "Cancelled" && booking.status != "Rejected";
    }

    bookingsListView.ItemsSource = null;
    bookingsListView.ItemsSource = bookings;
}

private async Task CancelBooking(Booking booking)
{
    booking.status = "Cancelled";
    await _firebaseService.UpdateBookingStatusAsync(booking);
}

```

This code displays the `LoadUserBookings` and `CancelBooking` methods. `LoadUserBookings` retrieves and displays the user's bookings, with conditions to check if a booking is eligible for cancellation. The `CancelBooking` method updates the booking's status to "Cancelled" in Firebase when cancellation conditions are met.

```

private async void OnCancelBookingClicked(object sender, EventArgs e)
{
    var button = sender as Button;
    var booking = button?.BindingContext as Booking;

    if (booking != null && booking.canCancel)
    {
        var latestBooking = await _firebaseService.GetBookingByIdAsync(booking.id);

        if (latestBooking.status != "Pending")
        {
            await DisplayAlert("Unable to Cancel", "This booking can no longer be cancelled because its status is no longer 'Pending'. Please reload to update status.", "OK");
            return;
        }
    }
}

```

```

        bool confirm = await DisplayAlert("Confirm Cancellation", "Are you sure you want to cancel this booking?", "Yes",
"No");
        if (confirm)
        {
            await CancelBooking(booking);
            await DisplayAlert("Success", "Booking cancelled successfully!", "OK");
            await LoadUserBookings();
        }
    }
}

private async void OnRefreshClicked(object sender, EventArgs e)
{
    await LoadUserBookings();
    await DisplayAlert("Success", "Page reloaded successfully!", "OK");
}

private async void OnBookingTapped(object sender, ItemTappedEventArgs e)
{
    if (e.Item is Booking selectedBooking)
    {
        await Navigation.PushAsync(new BookingDetailsPage(selectedBooking));
    }

    bookingsListView.SelectedItem = null;
}

```

This code shows additional functions for handling booking actions. The `OnCancelBookingClicked` method confirms cancellation eligibility and prompts the user for confirmation. The `OnRefreshClicked` refreshes the booking list, while `OnBookingTapped` navigates to the booking details page upon tapping a booking.

5.2.18 BookingDetailsPage.xaml.cs

```

public partial class BookingDetailsPage : ContentPage
{
    private readonly FirebaseService _firebaseService = new FirebaseService();

    public BookingDetailsPage(Booking selectedBooking)
    {
        InitializeComponent();
        BindingContext = new BookingDetailsViewModel(selectedBooking, _firebaseService);
    }
}

```

This code shows the initialization of the `BookingDetailsPage` class in `BookingDetailsPage.xaml.cs`. The constructor accepts a `selectedBooking` parameter and sets up the page's binding context to a new instance of `BookingDetailsViewModel`, passing in the selected booking and a Firebase service instance for data handling.

```

public class BookingDetailsViewModel : INotifyPropertyChanged
{
    private string _fullName;
    public string FullName
    {
        get => _fullName;
        set
        {
            if (_fullName != value)
            {

```

```

        _fullName = value;
        OnPropertyChanged(nameof(FullName));
    }
}

private double _totalPrice;
public double TotalPrice
{
    get => _totalPrice;
    set
    {
        if (_totalPrice != value)
        {
            _totalPrice = value;
            OnPropertyChanged(nameof(TotalPrice));
        }
    }
}

private string _status;
public string Status
{
    get => _status;
    set
    {
        if (_status != value)
        {
            _status = value;
            OnPropertyChanged(nameof(Status));
        }
    }
}

public ObservableCollection<GroupedClass> GroupedClasses { get; set; }
public BookingDetailsViewModel(Booking booking, FirebaseService firebaseService)
{
    GroupedClasses = new ObservableCollection<GroupedClass>();
    TotalPrice = booking.totalPrice;
    Status = booking.status;
    LoadBookingDetails(booking, firebaseService);
}
private async void LoadBookingDetails(Booking booking, FirebaseService firebaseService)
{
    var user = await firebaseService.GetUserByIdAsync(booking.userId);
    FullName = user?.fullName ?? "Unknown User";

    var allUsers = await firebaseService.GetAllUsersAsync();

    foreach (var classInstanceInfo in booking.classInstances)
    {
        var yogaClass = await firebaseService.GetYogaClassById(classInstanceInfo.classId);

        if (TimeSpan.TryParse(yogaClass.Time, out var startTime))
        {
            var endTime = startTime.Add(TimeSpan.FromMinutes(yogaClass.Duration));

            var existingGroup = GroupedClasses.FirstOrDefault(g => g.YogaClassId == yogaClass.Id);

            var classInstance = await firebaseService.GetClassInstanceId(classInstanceInfo.classInstanceId);
            var instructor = allUsers.FirstOrDefault(u => u.id == classInstance.Instructor);
            var classInstanceDisplay = new ClassInstanceDisplay
            {
                Date = classInstance.Date,
                Instructor = instructor?.fullName ?? "Unknown Instructor"
            }
        }
    }
}

```

```

};

if (existingGroup == null)
{
    GroupedClasses.Add(new GroupedClass
    {
        YogaClassId = yogaClass.Id,
        YogaClassTitle = $"{yogaClass.TypeOfClass} - {yogaClass.DayOfWeek} ({startTime:hh\\:\\:mm} -
{endTime:hh\\:\\:mm})",
        ClassInstances = new ObservableCollection<ClassInstanceDisplay> { classInstanceDisplay }
    });
}
else
{
    existingGroup.ClassInstances.Add(classInstanceDisplay);
}
}
}

public event PropertyChangedEventHandler PropertyChanged;
protected virtual void OnPropertyChanged(string propertyName)
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
}

```

This code illustrates the `BookingDetailsViewModel` class in `BookingDetailsPage.xaml.cs`, which implements `INotifyPropertyChanged` to manage data-binding for booking details. The properties `FullName`, `TotalPrice`, and `Status` each utilize private fields and notify updates through `OnPropertyChanged`, allowing the UI to reflect changes dynamically. The constructor accepts a `Booking` object and a `FirestoreService` instance, initializing the `GroupedClasses` collection and invoking `LoadBookingDetails`. The `LoadBookingDetails` method retrieves and formats booking information, grouping class instances with their details, such as dates and instructors, for display.

```

public class GroupedClass : INotifyPropertyChanged
{
    public string YogaClassId { get; set; }

    private string _yogaClassTitle;
    public string YogaClassTitle
    {
        get => _yogaClassTitle;
        set
        {
            if (_yogaClassTitle != value)
            {
                _yogaClassTitle = value;
                OnPropertyChanged(nameof(YogaClassTitle));
            }
        }
    }

    public ObservableCollection<ClassInstanceDisplay> ClassInstances { get; set; }

    public event PropertyChangedEventHandler PropertyChanged;
    protected virtual void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}

```

```
public class ClassInstanceDisplay
{
    public string Date { get; set; }
    public string Instructor { get; set; }
}
```

This code shows the `GroupedClass` and `ClassInstanceDisplay` classes in `BookingDetailsPage.xaml.cs`. The `GroupedClass` class implements `INotifyPropertyChanged` to manage data binding for properties such as `YogaClassId`, `YogaClassTitle`, and `ClassInstances`, a collection of `ClassInstanceDisplay` objects. The `OnPropertyChanged` method is used to notify the UI of property changes. The `ClassInstanceDisplay` class represents individual class instances with properties like `Date` and `Instructor`, which hold the details of each yoga session for display purposes.

6. SECTION 6 - TESTING

6.1 Admin Application

Table 3. Test case for admin application

ID	Test Case	Function	Input Data	Expected Results	Actual Results	Status
T01	Verify that the Log In button is clickable and navigates to the Login screen.	Navigation	Tap Log In button	The app should navigate to the Login screen.	The app navigates to the Login screen.	Pass
T02	Verify that the Sign Up button is clickable and navigates to the Sign Up screen.	Navigation	Tap Sign Up button	The app should navigate to the Sign Up screen.	The app navigates to the Sign Up screen.	Pass
T03	Verify that the user can log in successfully with correct email and password.	Login	Email: "admin@example.com", Password: "correctpassword"	The user is navigated to AdminPageActivity if they are an admin or TeacherPageActivity if they are a teacher.	The user is navigated to AdminPageActivity or TeacherPageActivity based on their role.	Pass
T04	Verify that the login fails with an incorrect email or password, and an error message is displayed.	Login	Email: "admin@example.com", Password: "wrongpassword"	The app should display a toast message: "Incorrect email or password".	The app displays a toast message: "Incorrect email or password".	Pass
T05	Verify that the login fails if the email or password fields are empty, and an error message is	Login	Email: "", Password: ""	The app should display a toast message: "Please enter complete information".	The app displays a toast message: "Please enter complete information".	Pass

	displayed.					
T06	Verify that clicking on the "Sign Up" text navigates the user to the SignUpActivity.	Navigation	Tap "Sign Up" text	The app should navigate to SignUpActivity.	navigate to SignUpActivity. The app navigates to Pass SignUpActivity.	Pass
T07	Verify that the user can sign up successfully with valid inputs for all fields.	Sign Up	Full Name: "John Doe", Phone Number: "123456789", Email: "john@example.com", Password: "password123", Confirm Password: "password123", Role: "Admin"	The app displays a toast message: "Sign up successful" and navigates to LoginActivity	The app displays a toast message: "Sign up successful" and navigates to LoginActivity.	Pass
T08	Verify that the sign-up fails if the email already exists in the database, and an error message is displayed.	Sign Up	Full Name: "John Doe", Phone Number: "123456789", Email: "existing@example.com", Password: "password123", Confirm Password: "password123", Role: "Admin"	The app displays a toast message: "Email already exists!".	The app displays a toast message: "Email already exists!".	Pass
T09	Verify that the sign-up fails if	Sign Up	Full Name: "John Doe",	The app displays a toast message:	The app displays a toast message:	Pass

	the password is less than 8 characters, and an error message is displayed.		Phone Number: "123456789", Email: "john@example.com", Password: "short", Confirm Password: "short", Role: "Admin"	"Password must be at least 8 characters".	"Password must be at least 8 characters".	
T10	Verify that the sign-up fails if the password and confirm password do not match, and an error message is displayed.	Sign Up	Full Name: "John Doe", Phone Number: "123456789", Email: "john@example.com", Password: "password123", Confirm Password: "differentpassword", Role: "Admin"	The app displays a toast message: "Passwords do not match".	The app displays a toast message: "Passwords do not match".	Pass
T11	Verify that clicking on the "Sign In" text navigates the user to the LoginActivity.	Navigation	Tap "Sign In" text	The app should navigate to LoginActivity.	The app navigates to LoginActivity.	Pass
T12	Verify that clicking the "View Yoga Class" button navigates the user to the	Admin Page	Tap "View Yoga Class" button	The app should navigate to ClassListActivity.	The app navigates to ClassListActivity.	Pass

	ClassListActivity.					
T13	Verify that clicking the "View Class Instance" button navigates the user to the ClassInstanceListActivity.	Admin Page	Tap "View Class Instance" button	The app should navigate to ClassInstanceListActivity.	The app navigates to ClassInstanceListActivity.	Pass
T14	Verify that clicking the "View Accounts" button navigates the user to the AccountListActivity.	Admin Page	Tap "View Accounts" button	The app should navigate to AccountListActivity.	The app navigates to AccountListActivity.	Pass
T15	Verify that clicking the "View Bookings" button navigates the user to the BookingListActivity.	Admin Page	Tap "View Bookings" button	The app should navigate to BookingListActivity.	The app navigates to BookingListActivity.	Pass
T16	Verify that clicking the "Logout" option in the settings menu logs the user out and navigates to LoginActivity.	Admin Page	Tap "Logout" in settings	The app should navigate to LoginActivity with a clear task flag, logging the user out.	The app navigates to LoginActivity, clearing previous activities from the back stack.	Pass
T17	Verify that	Admin Page	Tap "Reset Data" in	The app should display a	The app displays a confirmation	Pass

	clicking the "Reset Data" option in the settings menu displays a confirmation dialog for data reset.		settings	confirmation dialog with options to confirm or cancel the data reset action.	dialog with "Yes" and "No" options.	
T18	Verify that confirming data reset deletes all data except users from both SQLite and Firebase databases.	Admin Page	Confirm data reset	The app should delete all data from tables class_type, bookings, yoga_classes, class_instances, excluding users.	All data from specified tables is deleted in both SQLite and Firebase, with a toast: "All data reset except Users".	Pass
T19	Verify that clicking the "Manage Class Types" option in the settings menu navigates the user to ManageClassTypesActivity.	Admin Page	Tap "Manage Class Types" in settings	The app should navigate to ManageClassTypesActivity.	The app navigates to ManageClassTypesActivity.	Pass
T20	Verify that all classes are loaded successfully from the database.	Load Classes	Open ClassListActivity	All yoga classes stored in the database should be displayed in the list view.	All yoga classes are displayed in the list view.	Pass
T21	Verify that clicking the "Add Class" button navigates the user to EnterYogaDetails	Yoga class	Tap "Add Class" button	The app should navigate to EnterYogaDetailsActivity.	The app navigates to EnterYogaDetailsActivity.	Pass

	Activity.					
T22	Verify that the user can filter the class list by day using the spinner.	Yoga class	Select a day (e.g., "Monday")	The class list should display only classes scheduled on the selected day.	The class list displays only classes scheduled on "Monday".	Pass
T23	Verify that the user can search classes by keyword in the search view.	Yoga class	Enter search term (e.g., "Yoga")	The class list should display only classes matching the search term.	The class list displays only classes matching "Yoga".	Pass
T24	Verify that clicking the "Update" option in the class item's menu navigates to EnterYogaDetails Activity with class details for editing.	Yoga class	Tap "Update" in item menu	The app should navigate to EnterYogaDetailsActivity with class details populated for editing.	The app navigates to EnterYogaDetailsActivity with class details populated for editing.	Pass
T25	Verify that clicking the "Delete" option in the class item's menu displays a confirmation dialog and deletes the class upon confirmation.	Yoga class	Tap "Delete" in item menu	The app should display a confirmation dialog, and upon confirmation, delete the class from the list and database.	The app displays a confirmation dialog, and upon confirmation, deletes the class.	Pass
T26	Verify that the search functionality works.	Yoga class	Search "Yoga"	The class list only shows classes of the "Yoga" type.	The class list only shows classes that match the search criteria.	Pass

T27	Verify that clicking on "Home" in the bottom navigation navigates the user to AdminPageActivity.	Bottom Navigation	Tap "Home"	The app should navigate to AdminPageActivity.	The app navigates to AdminPageActivity .	Pass
T28	Verify that clicking on "Class Instance" in the bottom navigation navigates the user to ClassInstanceList Activity.	Bottom Navigation	Tap "Class Instance"	The app should navigate to ClassInstanceListActivity.	The app navigates to ClassInstanceListActivity.	Pass
T29	Verify that clicking on "Booking" in the bottom navigation navigates the user to BookingListActivity.	Bottom Navigation	Tap "Booking"	The app should navigate to BookingListActivity.	The app navigates to BookingListActivity.	Pass
T30	Verify that clicking on "Account" in the bottom navigation navigates the user to AccountListActivity.	Bottom Navigation	Tap "Account"	The app should navigate to AccountListActivity.	The app navigates to AccountListActivity.	Pass

T31	Verify that a new yoga class can be saved successfully with valid input data.	Save Yoga Class	All fields filled with valid data	The app should display a toast message: "Class saved successfully!" and navigate to ClassListActivity	The app displays a toast message: "Class saved successfully!" and navigates to ClassListActivity.	Pass
T32	Verify that an existing yoga class can be updated successfully with valid input data.	Update Yoga Class	All fields filled with updated data	The app should display a toast message: "Class updated successfully!" and navigate to ClassListActivity.	The app displays a toast message: "Class updated successfully!" and navigates to ClassListActivity.	Pass
T33	Verify that only the description field is editable if the class has existing instances.	Update Yoga Class	Class has instances	Only the description field should be enabled for editing, and all other fields should be disabled.	Only the description field is enabled; other fields are disabled.	Pass
T34	Verify that duplicate yoga classes cannot be saved, and an error message is displayed.	Save Yoga Class	Duplicate class details	The app should display a toast message: "Class already exists!".	The app displays a toast message: "Class already exists!".	Pass
T35	Verify that a "Wi-Fi: Connected" status is displayed when connected to Wi-Fi.	Wi-Fi Status Check	Device connected to Wi-Fi	The Wi-Fi status should display "Wi-Fi: Connected".	The Wi-Fi status displays "Wi-Fi: Connected".	Pass
T36	Verify that a "Wi-Fi: Not Connected" status is displayed when not connected to Wi-Fi.	Wi-Fi Status Check	Device not connected to Wi-Fi	The Wi-Fi status should display "Wi-Fi: Not Connected".	The Wi-Fi status displays "Wi-Fi: Not Connected".	Pass

	status is displayed when not connected to Wi-Fi.			Connected".		
T37	Verify that after saving a class, the app navigates back to ClassListActivity.	Navigation	Tap "Save" button	The app should navigate to ClassListActivity after saving the class successfully.	The app navigates to ClassListActivity after saving the class.	Pass
T38	Verify that the update mode is initiated with pre-filled data when editing an existing class.	Load Class for Update	Open activity with classId	The app should display "Update class" in the heading and pre-fill all fields with the class's data.	The app displays "Update class" and pre-fills fields with class data.	Pass
T39	Verify that all class instances are loaded successfully from the database.	Load Class Instances	Open ClassInstanceList Activity	All class instances stored in the database should be displayed in the list view.		Pass
T40	Verify that clicking the "Add Instance" button navigates to EnterClassInstanceDetailsActivity with the classId parameter.	Navigation	Tap "Add Instance" button	The app should navigate to EnterClassInstanceDetailsActivity with classId passed as an extra.	The app navigates to EnterClassInstanceDetailsActivity with classId as an extra.	Pass
T41	Verify that searching by a keyword filters class instances based on date,	Search Filter	Enter search keyword	The list view should update to display only instances matching the	The list view displays only matching class instances.	Pass

	instructor, or additional comments.			search keyword.		
T42	Verify that clicking the update option for a class instance navigates to EnterClassInstanceDetailsActivity for editing.	Update Class Instance	Select "Update" in options menu	The app should navigate to EnterClassInstanceDetailsActivity with current details of the instance.	The app navigates to EnterClassInstanceDetailsActivity with pre-filled details for editing.	Pass
T43	Verify that clicking the delete option removes the class instance from the database and Firebase and updates the list view.	Delete Class Instance	Select "Delete" in options menu	The class instance should be removed from the list view and both SQLite and Firebase databases.	The class instance is removed from the list view and both databases.	Pass
T44	Verify that all available yoga classes are loaded into the class spinner.	Load Classes	Open EnterClassInstanceDetailsActivity	All yoga classes should be listed in the spYogaClass spinner.	All classes are listed in spinner.	Pass
T45	Verify that all available instructors are loaded into the instructor spinner.	Load Instructors	Open EnterClassInstanceDetailsActivity	All instructors should be listed in the spInstructor spinner.	All instructors are listed in spinner.	Pass
T46	Verify that selecting a different class in the spinner updates the	Class Selection	Select a different class in spYogaClass	The selected day of the week should be updated based on the selected class.	Day of the week updates correctly.	Pass

	selected day of the week for date selection.					
T47	Verify that selecting a date opens the date picker dialog.	Open DatePicker	Tap "Select Date" button	The DatePicker dialog should open for date selection.	DatePicker dialog opens as expected.	Pass
T48	Verify that selecting an invalid date (not matching the class day of the week) shows an error message.	Date Validation	Select invalid date in DatePicker	The app should display an error message: "Please select a valid [day] that is not in the past."	Error message displayed for invalid date.	Pass
T49	Verify that the selected date is displayed correctly on the "Select Date" button after choosing a valid date.	Date Display	Select a valid date	The selected date should be displayed on the "Select Date" button in DD/MM/YYYY format.	Date displays correctly on button.	Pass
T50	Verify that attempting to save without selecting a date shows an error message.	Save Validation	Leave date unselected and click "Save"	The app should display an error message indicating that the date is required.	Error message displayed for missing date.	Pass
T51	Verify that attempting to save without selecting an instructor shows an error message.	Save Validation	Leave instructor unselected and click "Save"	The app should display an error message indicating that the instructor is required.	Error message displayed for missing instructor.	Pass
T52	Verify that	Conflict	Select	The app should	Error message	Pass

	selecting an instructor with a conflicting schedule displays a warning and prevents saving the instance.	Check	conflicting instructor and date	display an error message indicating that the instructor is not available for the chosen time slot.	displayed for scheduling conflict.	
T53	Verify that saving a new class instance adds the instance to the SQLite database when all fields are filled correctly.	Save Class Instance	Fill out form and click "Save"	The class instance should be saved to SQLite, and a success message should be displayed.	Class instance saved successfully in SQLite.	Pass
T54	Verify that updating an existing class instance modifies the instance details in the database.	Update Class Instance	Edit details and click "Save"	The class instance details should be updated in SQLite, and a success message should be displayed.	Class instance updated successfully in SQLite.	Pass
T55	Verify that the "Save Class Instance" button displays the updated label text "Update Class Instance" when editing an existing instance.	Update Mode Display	Open activity with existing instance data	The "Save Class Instance" button text should change to "Update Class Instance" when editing an existing instance.	Button text updates correctly for edit mode.	Pass
T56	Verify that the end time is	Time Calculatio	Select a class with known	The end time should be	End time calculated	Pass

	calculated correctly based on the selected class's start time and duration.	n	start time/duration	calculated correctly and used for checking scheduling conflicts.	accurately.	
T57	Verify that all bookings are loaded successfully from the database and displayed in the list view.	Load Bookings	Open BookingListActivity	All bookings stored in the database should be displayed in the list view.	All bookings are displayed in the list view.	Pass
T58	Verify that selecting a filter option in the spinner updates the list view to display only bookings with the selected status.	Filter by Status	Select a status in spinnerFilter	The list view should update to display only bookings matching the selected status.	List view updates based on selected status.	Pass
T59	Verify that entering a search keyword filters the list of bookings based on matching user name or total price.	Search Filter	Enter a search keyword in searchView	The list view should display only bookings with user names or total prices matching the search keyword.	List view displays matching bookings only.	Pass
T60	Verify that selecting "View Details" in the options menu for a booking	Options Menu	Tap "View Details" in options menu	The app should navigate to BookingDetailsActivity with the correct booking	App navigates to details with correct ID.	Pass

	item opens the booking details screen.			ID, displaying all details related to the selected booking.		
T61	Verify that bookings are displayed with the correct user name, total price, status, number of unique yoga classes, and class instances.	Display Booking Info	Open BookingListActivity	Each booking item should display the user name, total price, status, unique yoga class count, and class instance count correctly based on the booking data.	Booking info displays correctly.	Pass
T62	Verify that the booking details (user's full name, total price, and status) are displayed correctly based on the booking ID passed to BookingDetailsActivity.	Display Booking Details	Open BookingDetailsActivity with a booking ID	The screen should display the full name, total price, and status of the booking associated with the provided booking ID.	Details are displayed correctly.	Pass
T63	Verify that the list of class instances for the booking is displayed correctly in the list view.	Display Class Instances	Open BookingDetailsActivity with a booking ID	The list view should display all class instances associated with the booking, including the correct class information.	Class instances are displayed correctly.	Pass
T64	Verify that clicking the "Accept" button	Accept Booking	Tap "Accept" button	The booking status should be updated to	Status updated, buttons hidden.	Pass

	updates the booking status to "Confirmed" and makes the booking cancelable.			"Confirmed," canCancel set to true, and status displayed as "Confirmed." The "Accept" and "Reject" buttons should disappear.		
T65	Verify that clicking the "Reject" button updates the booking status to "Rejected" and prevents cancellation.	Reject Booking	Tap "Reject" button	The booking status should be updated to "Rejected," canCancel set to false, and status displayed as "Rejected." The "Accept" and "Reject" buttons should disappear.	Status updated, buttons hidden.	Pass
T66	Verify that attempting to accept or reject a booking displays a confirmation dialog.	Confirm Dialog	Tap "Accept" or "Reject" button	A confirmation dialog should appear, asking the user to confirm the action.	Confirmation dialog displayed.	Pass
T67	Verify that each class instance is correctly grouped and displayed under its associated yoga class in the list view.	Group Class Instances	Open BookingDetailsActivity	Class instances should be displayed grouped under their associated yoga classes, with correct class information displayed for each group.	Instances grouped correctly.	Pass
T68	Verify that	Update	Tap "Accept"	The booking	Status synced to	Pass

	confirming a booking updates the status and cancel option in both SQLite and Firebase databases.	Booking in Firebase	button	status in Firebase should match the status in SQLite, with canCancel set to true.	Firebase.	
T69	Verify that rejecting a booking updates the status and cancel option in both SQLite and Firebase databases.	Update Booking in Firebase	Tap "Reject" button	The booking status in Firebase should match the status in SQLite, with canCancel set to false.	Status synced to Firebase.	Pass
T70	Verify that all user accounts are loaded successfully from the database and displayed in the list view.	Load Users	Open AccountListActivity	All users stored in the database should be displayed in the list view.	Users are displayed correctly.	Pass
T71	Verify that clicking the "Add Account" button navigates to AddAccountActivity.	Add Account Navigation	Tap "Add Account" button	The app should navigate to AddAccountActivity.	Navigates to AddAccountActivity.	Pass
T72	Verify that searching by a keyword filters user accounts based on name,	Search Filter	Enter search keyword	The list view should update to display only accounts that match the search	Displays matching accounts.	Pass

	email, phone, or role.			keyword in either name, email, phone, or role.		
T73	Verify that selecting a role from the spinner filters accounts based on that role.	Role Filter	Select a role from the spinner	The list view should update to display only accounts with the selected role.	Displays accounts with selected role.	Pass
T74	Verify that clicking on the update option in the popup menu of a user opens AddAccountActivity with pre-filled user details.	Update Account	Tap update option in popup menu	The app should open AddAccountActivity with the selected user's details pre-filled, allowing for account editing.	Opens AddAccountActivity with details.	Pass
T75	Verify that clicking on the delete option in the popup menu of a user removes the user from SQLite and Firebase databases.	Delete Account	Tap delete option in popup menu	The selected user account should be deleted from both SQLite and Firebase, and the list view should update to remove the deleted account.	User deleted from databases.	Pass
T76	Verify that a confirmation dialog appears when attempting to delete a user account.	Delete Confirmation	Tap delete option in popup menu	A confirmation dialog should appear, asking the user to confirm or cancel the deletion of the account.	Confirmation dialog displayed.	Pass
T77	Verify that all	Field	Leave a field	Toast message	Correct message	Pass

	fields are filled before creating a new account.	Validation	empty	"Please fill in all fields" should be displayed if any required field is empty.	displayed.	
T78	Verify that email uniqueness is checked before creating a new account.	Email Uniqueness Check	Use an existing email	Toast message "Email already exists!" should be displayed if the email is already used by another account.	Correct message displayed.	Pass
T79	Verify that the password meets the minimum length requirement of 8 characters.	Password Length Check	Enter a password with <8 characters	Toast message "Password must be at least 8 characters" should be displayed.	Correct message displayed.	Pass
T80	Verify that the password and confirm password fields match.	Password Match Check	Enter different values in two fields	Toast message "Passwords do not match" should be displayed if the password and confirm password do not match.	Correct message displayed.	Pass
T81	Verify that a new account is successfully created when all fields are valid.	Create Account	Enter all valid fields	A new account should be added to the database, a success message displayed, and the user should navigate to AccountListActivity.	Account created successfully.	Pass
T82	Verify that an existing account is	Update Account	Update details for an existing account	The account should be updated in the database, a	Account updated successfully.	Pass

	successfully updated when userId is provided and fields are valid.			success message displayed, and the user should navigate to AccountListActivity.		
T83	Verify that a user with the role "Teacher" cannot change their role if they are assigned to a class instance.	Role Change Restriction	Update role of a teacher with assignments	The role spinner should be disabled, and a toast message should display "This account is assigned to a class instance, role cannot be changed."	Role change restriction works.	Pass
T84	Verify that AddAccountActivity loads existing user details correctly when userId is provided for editing.	Load User Details	Open AddAccountActivity with userId	The fields should be pre-filled with the existing user's details, the heading should change to "Update Account", and password fields should be hidden.	User details loaded correctly.	Pass
T85	Verify that a salt is generated and used in the password hash when creating a new account.	Password Salting	Enter a password	Password should be hashed with a unique salt, and the salted hash should be stored in the database.	Password salting works as expected.	Pass
T86	Verify that pressing the "Update Account" button updates user	Account Synchronization	Update account with Wi-Fi connected	User information should be updated in SQLite, and if connected to Wi-Fi, Firebase should also be	Updates to both databases.	Pass

	information in both SQLite and Firebase if connected to Wi-Fi.			updated with the new user details.		
T87	Verify that the confirm password field is ignored during updates, as password change is not required for updates.	Password Field Ignorance	Update an account without password fields	The confirm password field should not be considered during an account update, as the password field is hidden and update only involves non-password details.	Works as expected.	Pass
T88	Verify that the list of class types loads successfully from the database.	Load Class Types	Open ManageClassTypesActivity	All class types stored in the database should be displayed in the list view.	Class types load successfully	Pass
T89	Verify that the "Add Class Type" button opens the dialog to create a new class type.	Open Add Dialog	Tap "Add Class Type" button	A dialog should open with an empty input field for the class type name and a "Save" button to add the new class type.	Dialog opens as expected.	Pass
T90	Verify that a class type cannot be added if the name field is empty.	Validation	Leave name field empty	A toast message "Name cannot be empty" should be displayed if the user tries to add a class type without a name.	Correct message displayed.	Pass
T91	Verify that a new class type	Add Class Type	Enter a valid class type name	The new class type should be	Class type added successfully.	Pass

	is added successfully when a name is provided.			added to the database, displayed in the list, and a success toast message should be shown.		
T92	Verify that the update dialog opens with the selected class type's name pre-filled.	Open Update Dialog	Tap "Edit" on an existing class	The dialog should open with the name field pre-filled with the selected class type's name and a "Save" button to update the class type.	Update dialog opens correctly.	Pass
T93	Verify that an existing class type is updated successfully when a new name is provided.	Update Class Type	Modify the class type name	The class type should be updated in the database with the new name, and a success message should be shown.	Class type updated successfully.	Pass
T94	Verify that the search functionality filters class types based on the entered query.	Search Filter	Enter a search query	The list should update to display only class types matching the search query.	Search filter works correctly.	Pass
T95	Verify that clicking the delete option removes the selected class type after confirmation.	Delete Class Type	Tap "Delete" on an existing class	An alert dialog should ask for confirmation. Upon confirmation, the class type should be deleted from the database and	Class type deleted successfully.	Pass

				the list updated.		
T96	Verify that deleting a class type synchronizes the deletion to Firebase if connected to the internet.	Firebase Sync	Delete a class type	The class type should be deleted from Firebase if there is an internet connection, and a success toast message should be displayed.	Sync works as expected.	Pass
T97	Verify that attempting to save an updated class type with an empty name displays an error message.	Update Validation	Clear the name in the update dialog	A toast message "Name cannot be empty" should be displayed if the user tries to save an updated class type without a name.	Correct message displayed.	Pass
T98	Verify that class instances are synced from Firebase to SQLite if there are no records in SQLite.	Sync Class Instances	No class instances in SQLite	All class instances should be fetched from Firebase and added to SQLite if Wi-Fi is connected.	Data syncs successfully.	Pass
T99	Verify that yoga classes are synced from Firebase to SQLite if there are no records in SQLite.	Sync Yoga Classes	No yoga classes in SQLite	All yoga classes should be fetched from Firebase and added to SQLite if Wi-Fi is connected.	Data syncs successfully.	Pass
T100	Verify that user data is synced from Firebase to SQLite,	Sync Users	Wi-Fi connected	User data in Firebase should be synced to SQLite, with	Data syncs successfully.	Pass

	updating existing users and adding new users.			existing users updated and new users added.		
T101	Verify that booking data is synced from Firebase to SQLite, updating existing bookings and adding new ones.	Sync Bookings	Wi-Fi connected	Booking data in Firebase should be synced to SQLite, with existing bookings updated and new bookings added.	Data syncs successfully.	Pass
T102	Verify that class types are synced from Firebase to SQLite if there are no records in SQLite.	Sync Class Types	No class types in SQLite	All class types should be fetched from Firebase and added to SQLite if Wi-Fi is connected.	Data syncs successfully.	Pass
T103	Verify that the sync operation is skipped for all data types if Wi-Fi is not connected.	Wi-Fi Check	No Wi-Fi connection	No data should be fetched from Firebase; log message should indicate sync skipped due to lack of Wi-Fi connection	Sync operation is skipped.	Pass
T104	Verify that syncClassInstances uploads all pending class instances to Firebase when Wi-Fi is connected.	syncClassInstances	Wi-Fi connected	All class instances with status "Pending" in SQLite should be uploaded to Firebase and their statuses updated to "Uploaded" in	Class instances synced.	Pass

				SQLite.		
T105	Verify that syncYogaClass uploads all pending yoga classes to Firebase when Wi-Fi is connected.	syncYogaClass	Wi-Fi connected	All yoga classes with status "Pending" in SQLite should be uploaded to Firebase and their statuses updated to "Uploaded" in SQLite.	Yoga classes synced.	Pass
T106	Verify that syncUsers uploads all pending users to Firebase when Wi-Fi is connected.	syncUsers	Wi-Fi connected	All users with status "Pending" in SQLite should be uploaded to Firebase and their statuses updated to "Uploaded" in SQLite.	Users synced.	Pass
T107	Verify that syncClassTypes uploads all pending class types to Firebase when Wi-Fi is connected.	syncClassTypes	Wi-Fi connected	All class types with status "Pending" in SQLite should be uploaded to Firebase and their statuses updated to "Uploaded" in SQLite.	Class types synced.	Pass
T108	Verify that data sync continues to run every 30 seconds if Wi-Fi remains connected.	startSyncTask	Wi-Fi connected	Data sync task should be triggered every 30 seconds, syncing pending data to Firebase and updating their statuses.	Data sync runs as expected.	Pass
T109	Verify that data sync is paused	onReceive	Wi-Fi disconnected	No data should be uploaded to	Sync paused message appears.	Pass

	when Wi-Fi is disconnected.			Firebase; a toast message should indicate that sync is paused		
--	-----------------------------	--	--	---	--	--

6.2 Customer Application

Table 4. Test case for customer application

ID	Test Case	Function	Input Data	Expected Results	Actual Results	Status
T01	Verify the "Log In" button navigates to LoginPage	Navigation	When "Log In" button is clicked	User should be navigated to LoginPage	The app navigates to the LoginPage.	Pass
T02	Verify that the "Sign Up" button is clickable and navigates to SignUpPage.	Navigation	Tap "Sign Up" button	The app should navigate to the SignUpPage	The app navigates to the SignUpPage.	Pass
T03	Verify that the user can successfully log in with valid credentials.	OnLoginClicked	Valid email and password	User should be logged in and navigate to AppShell.	User is logged in and navigates to AppShell.	Pass
T04	Verify that the user receives an error message with incorrect email or password.	OnLoginClicked	Invalid email or password	Display "Incorrect email or password" message.	Displays "Incorrect email or password" message.	Pass
T06	Verify that "Forgot Password" functionality prompts for OTP and allows password reset if OTP is correct.	OnForgotPasswordTapped	Tap "Forgot Password"	Prompt for OTP, then allow password reset if OTP matches.	OTP prompt displayed, and password reset allowed after correct OTP.	Pass

T07	Verify that "Forgot Password" shows an error if OTP is incorrect.	OnForgotPasswordTapped	Enter incorrect OTP	Display "Incorrect OTP" message.	Displays "Incorrect OTP" message.	Pass
T08	Verify that user is asked to enter a new password with a minimum length of 8 characters during reset.	ShowResetPasswordPopup	Correct OTP entered	Prompt to enter a new password with a minimum of 8 characters.	Prompt displayed for new password with minimum length requirement.	Pass
T09	Verify that password reset fails if new password and confirmation do not match.	ShowResetPasswordPopup	Mismatched new and confirm password	Display "Passwords do not match" message.	Display "Passwords do not match" message.	Pass
T10	Verify that password reset is successful when new password meets all criteria and matches confirmation.	UpdatePasswordInFirebase	Valid new password and confirmation	Display "Password has been reset successfully" message.	Displays "Password has been reset successfully" message.	Pass
T11	Verify that user is prevented from resetting password if email does not exist.	IsEmailExistsAsync	Non-existing email in the database	Display "Email does not exist!" message.	Displays "Email does not exist!" message.	Pass
T12	Verify that OTP email is sent	SendOtpEmail	Valid email	OTP email should be sent	OTP email sent successfully.	Pass

	successfully to the user's registered email address.			successfully.		
T13	Verify all required fields are validated when empty	OnSignUp Clicked	Leave all fields empty and tap 'Sign Up Now'	Display an error alert 'Please fill in all fields'	Alert displays as expected	Pass
T14	Verify email uniqueness validation	OnSignUp Clicked	Enter an existing email and valid details in other fields	Display an error alert 'Email already exists!'	Alert displays as expected	Pass
T15	Verify phone number length validation	OnSignUp Clicked	Enter a phone number with less than 10 digits	Display an error alert 'The phone number must have at least 10 digits'	Alert displays as expected	Pass
T16	Verify password length validation	OnSignUp Clicked	Enter a password with less than 8 characters	Display an error alert 'Password must be at least 8 characters'	Alert displays as expected	Pass
T17	Verify OTP generation and email sending on sign-up	OnSignUp Clicked	Enter valid details in all fields	OTP is generated and sent to the provided email	OTP email is received	Pass
T18	Verify OTP validation during sign-up	ShowOtpPop	Enter incorrect OTP and then correct OTP	Displays 'Incorrect OTP' prompt, then proceeds upon correct OTP entry	Works as expected	Pass
T19	Verify user is added to Firebase on successful sign-up	CompleteSignup	Enter correct OTP and confirm password	User details are saved in Firebase and 'Sign up successful' message displays	User is created in Firebase, alert displays as expected	Pass
T20	Verify password	CompleteSignup	Enter valid password	Password is hashed and salted	Hashed password is stored in	Pass

	hashing and salting during sign-up			before saving in Firebase	Firebase	
T21	Verify navigation to 'Sign In' page on tapping 'Already have an account? Sign In'	OnSignInTapped	Tap 'Sign In' link	Navigates to LoginPage	Navigates as expected	Pass
T22	Verify that all yoga classes are loaded successfully	LoadYogaClasses	Open YogaClassListPage	The list should display all yoga classes with updated start and end times, and instance counts	The list displays all classes as expected	Pass
T22	Verify search by day of the week option functionality	OnSearchOptionChanged	Select 'Search by Day of the Week'	The day-of-week checkboxes should become visible, and the search bar should be hidden	The day-of-week checkboxes are visible	Pass
T23	Verify search by time of day option functionality	OnSearchOptionChanged	Select 'Search by Time of Day'	The search bar should become visible, and the day-of-week checkboxes should be hidden	The search bar is visible	Pass
T24	Verify filtering yoga classes by specific days of the week	OnDayOfWeekCheckedChanged	Select days of the week checkboxes	The list should display only the classes matching the selected days	Only classes matching selected days shown	Pass
T25	Verify filtering yoga classes by time of day	OnTimeOfDaySearchChanged	Enter time (e.g., '09:00')	The list should display only the classes matching the entered time of day	Only classes matching entered time shown	Pass

T26	Verify navigation to YogaClassDetailsPage when a class is selected	OnYogaClassSelected	Select a class from the list	The app should navigate to YogaClassDetailsPage displaying details of the selected yoga class	Navigation to details page works correctly	Pass
T27	Verify that the yoga class details page displays all relevant information	YogaClassDetailsPage	Navigate to YogaClassDetailsPage with a valid YogaClass object	Yoga class details such as type, description, price, time, capacity, and day should be displayed correctly	Yoga class details display as expected	Pass
T28	Verify the "Book Yoga Class" button navigates to the booking page	OnBookingButtonClicked	Tap "Book Yoga Class" button	The app navigates to the booking page with the selected yoga class information	App navigates to booking page with yoga class details	Pass
T29	Verify that the 'Start Shopping Journey' button navigates to the Yoga class list page	OnStartButtonClicked	Tap 'Start' button	The app navigates to the Yoga class list page	App navigates to Yoga class list page	Pass
T30	Verify that the New Yoga Classes section displays the latest two yoga classes	LoadNewYogaClasses	Load yoga classes data	The New Yoga Classes section should display the latest two yoga classes with correct details	New Yoga Classes section displays correct data	Pass
T31	Verify that the Latest Bookings section displays the user's latest two bookings	LoadLatestBookings	Load booking data from Firebase	The Latest Bookings section should display the user's latest two bookings with accurate details	Latest Bookings section displays correct booking information	Pass

T32	Verify the correct count of yoga classes is displayed in 'Start Shopping Journey' section	LoadYogaClassCount	Count of yoga classes	The count of yoga classes should be displayed correctly in the 'Start Shopping Journey' section	Yoga class count displays accurately	Pass
T33	Verify that the list of available class instances loads successfully	Data Load	Open `BookingYogaClassPage`	The list of available class instances should load and display with details like date, instructor, and comments for each instance	The list loads and displays as expected	Pass
T34	Verify that selecting a class instance updates the total price	Selection	Select a class instance	The total price should be calculated based on the selected class instance(s) price	Total price updates correctly	Pass
T35	Verify that unselecting a class instance updates the total price	Selection	Unselect a previously selected class	The total price should decrease by the deselected class instance's price	Total price updates correctly	Pass
T36	Verify that tapping on a class instance displays its details	Navigation	Tap on a class instance	A dialog should display showing instance details like date, instructor, and comments	Dialog shows instance details	Pass
T37	Verify that 'Add to Cart' without any selection displays an	Validation	Click 'Add to Cart' with no selection	An error message should appear, indicating that no classes were	Error message displayed	Pass

	error			selected		
T38	Verify that selected class instances are added to the cart on 'Add to Cart'	Cart Addition	Select instances and click 'Add to Cart'	A success message should appear, and the selected instances should be added to the cart	Instances added to cart successfully	Pass
T39	Verify that class instances already in the cart are disabled from selection	Selection Restriction	Open `BookingYogaClassPage`	Instances already in the cart should be disabled for selection in the `BookingYogaClassPage`	Instances in the cart are disabled	Pass
T40	Verify that the cart list loads with the correct items from the Firebase database when the page appears.	LoadCartData	Page Load	The cart should display the correct list of yoga classes previously added by the user.	The cart displays the correct list of yoga classes as per the Firebase database.	Pass
T41	Verify the 'Total Price' displays correctly based on selected classes.	CalculateTotalPrice	Select/Deselect classes	The total price should update accurately, reflecting only the selected classes.	The total price updates accurately when classes are selected or deselected.	Pass
T42	Verify that the 'Confirm Booking' button is disabled if no classes are selected for booking.	OnConfirmBookingClicked	No classes selected for booking	The booking confirmation should not proceed, and a warning should be shown prompting to select a class.	A warning message appears, preventing booking when no class is selected.	Pass
T43	Verify that a	OnConfirm	Classes	A success message	A success	Pass

	success message displays upon confirming a booking.	BookingClicked	selected and booking confirmed	should display, indicating the booking is confirmed.	message appears confirming the booking.	
T44	Verify that the 'Remove' button removes the selected class instance from the cart.	OnRemoveClassInstanceClicked	Remove button clicked on a class instance	The selected class instance should be removed from the cart, and the total price should update accordingly.	The selected class instance is removed, and the total price updates as expected.	Pass
T45	Verify that the 'Remove' button shows a confirmation dialog before removing a class instance.	OnRemoveClassInstanceClicked	Click on Remove button	A confirmation dialog should appear, asking if the user wants to remove the class instance.	A confirmation dialog appears as expected.	Pass
T46	Verify the correct instructor's name is displayed for each class instance in the cart.	LoadCartData	Load cart items	Each class instance should display the correct instructor's name fetched from Firebase.	Instructor names display accurately as fetched from Firebase.	Pass
T47	Verify that selecting and deselecting the checkbox of class instances updates the total price correctly.	OnCheckboxCheckedChanged	Select/Deselect checkbox	The total price should update based on whether a class instance is selected or deselected.	The total price updates correctly when the checkbox is selected or deselected.	Pass
T48	Verify the bookings list	LoadUserBookings	Logged-in user's ID	Bookings list should load,	Bookings list loaded	Pass

	loads successfully for the logged-in user.			displaying each booking's status, price, and number of class instances.	successfully	
T49	Verify the refresh button reloads the bookings list.	OnRefresh Clicked	Tap refresh button	Bookings list should reload, updating any changes in booking status or other details.	Bookings list reloaded successfully	Pass
T50	Verify that the user can cancel a booking with status 'Pending'.	OnCancelBookingClicked	Select a booking with 'Pending' status and tap 'Cancel'	The booking status should change to 'Cancelled', and the booking should no longer be cancellable.	Booking status changed to 'Cancelled'	Pass
T51	Verify that the user cannot cancel a booking with status 'Confirmed'.	OnCancelBookingClicked	Select a booking with 'Confirmed' status and tap 'Cancel'	An alert should display indicating the booking can no longer be cancelled, and no changes should be made to the booking.	Alert displayed as expected; booking not cancelled	Pass
T52	Verify that tapping on a booking navigates to the booking details page.	OnBookingTapped	Select a booking from the list	User should be navigated to BookingDetailsPage, displaying detailed information for the selected booking.	Navigation to BookingDetailsPage successful	Pass
T53	Verify automatic refreshing every 10	_refreshTimer.Elapsed	Wait for 10 seconds with bookings status changed	Bookings list should refresh automatically, showing updated	Bookings list auto-refreshed with updated status	Pass

	seconds updates the bookings list.		externally	status for any modified bookings.		
T54	Verify that the booking details page loads with correct booking information	Load Booking Details	Select a booking and navigate to BookingDetails Page	The page should display the user's full name, total price, booking status, and associated yoga classes with their respective details.	The page displays the user's full name, total price, booking status, and associated yoga classes with their respective details as expected.	Pass
T55	Verify that each yoga class shows the correct class title and instances	Display Grouped Classes	Grouped yoga classes are displayed	Each yoga class group displays the class title along with associated class instances, including the date and instructor name.	The yoga class groups display the class title along with associated class instances, including the date and instructor name as expected.	Pass
T56	Verify that class instances display correctly when users search for class instances by date	Search Class Instances	Enter date, month, year in the search bar	Class instances will be filtered and show class instances whose day, month, or year matches the search query.	The class instance should be filtered and show class instances whose day, month or year matches the search query.	Pass

7. REFERENCE

Nielsen, J., 1994. Usability Engineering. San Diego: Academic Press.